



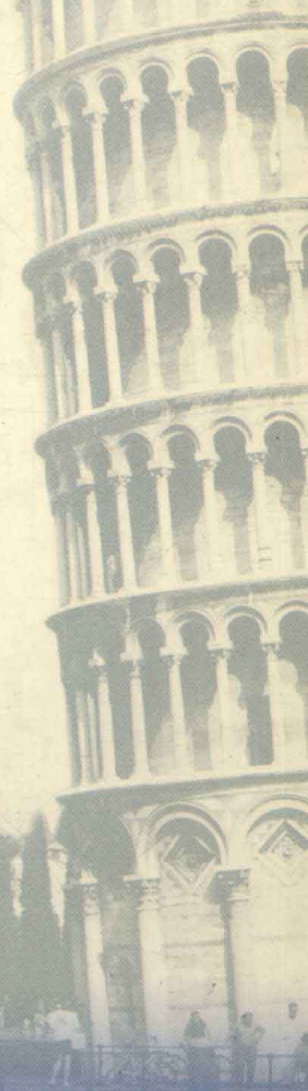
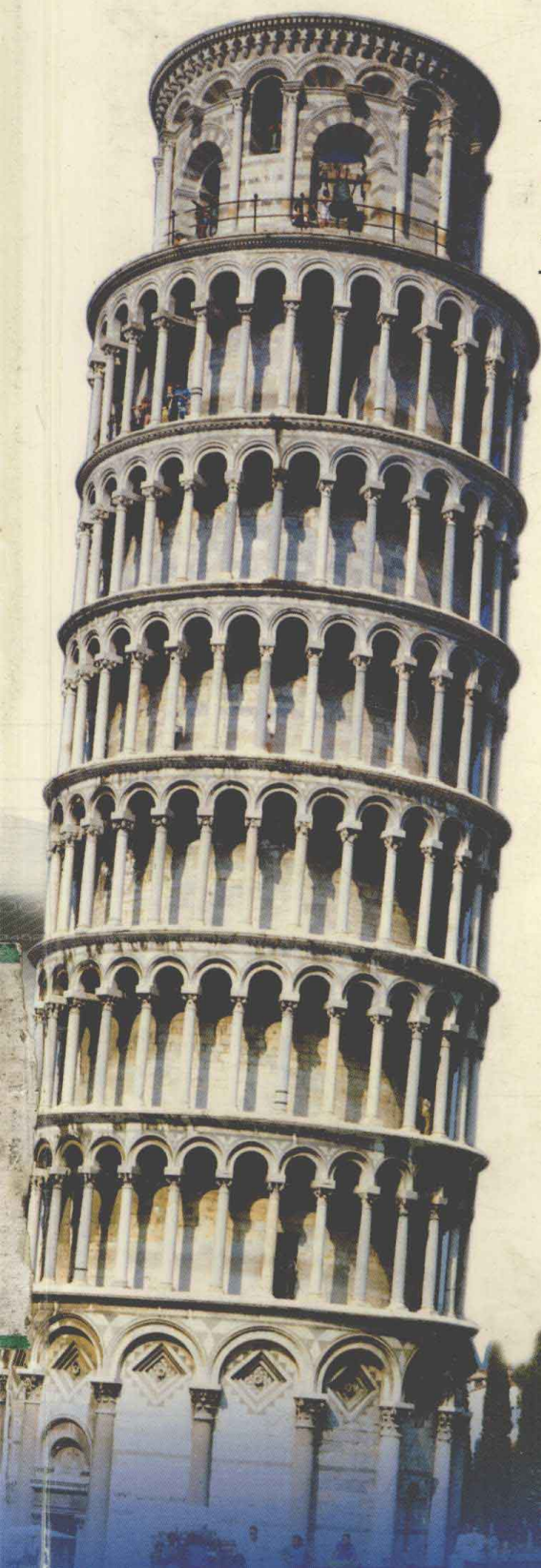
Visual C++

V i s u a l C + +

课程设计

案例精选与编程指导

主编 = 陈清华 朱红
东南大学出版社



计算机编程语言课程设计丛书

- Visual Basic课程设计——题目精选
- Visual FoxPro课程设计——项目案例精选
- Visual C++课程设计——案例精选与编程指导
- C语言课程设计——案例精选与编程指导



附赠光盘提供全套丛书题目的样例程序和所需资源，以及部分程序源代码。

V i s u a l C +

策划编辑：张 煦

文字编辑：张爱华

ISBN 7-81089-275-4



9 787810 892759 >

ISBN 7-81089-275-4

TP·33 总定价：105.00元
(共4本，每本附赠光盘)

计算机编程语言课程设计丛书

Visual C++ 课程设计

——案例精选与编程指导

陈清华 朱 红 主编

东南大学出版社
· 南 京 ·

内 容 提 要

学习程序设计语言的主要目的是能够利用简洁的语句编写出高效、完整的实用程序。本书正是为了满足广大读者的这一实际需要而编写的。书中精选出大量涉及各个领域的优秀案例,使读者通过阅读、分析优秀程序以及在样例程序的引导下编写程序等手段,掌握编写大规模实用程序的方法和技巧。

本书共分8部分,内容包括域名服务器的仿真程序样例及样例详解,经典有趣的游戏程序,简单实用的数学问题求解程序,清晰易懂的矩阵及向量处理程序,实用有趣、可扩充的小型管理程序,获取及利用时间和日期的实用程序,有趣、可扩充的多媒体应用程序,有一定难度的数据结构应用程序。书中精选的程序实用性强,语言简练,注释通俗易懂。程序组织结构及对问题的处理方法多种多样,覆盖面广,对读者编写大型实用程序有一定的借鉴作用。

本书是专门为大专院校理工科学生进行C++程序设计语言课程设计而编写的教材,对广大的计算机编程爱好者而言也是一本难得的实用参考书。

图书在版编目(CIP)数据

Visual C++课程设计——案例精选与编程指导 / 陈清华,
朱红主编. —南京:东南大学出版社,2003.6

(计算机编程语言课程设计丛书)

ISBN 7-81089-275-4

I. V... II. ①陈... ②朱... III. C语言—程序设计
IV. TP312

中国版本图书馆 CIP 数据核字(2003)第 041923 号

东南大学出版社出版发行

(南京四牌楼2号 邮编 210096)

出版人:宋增民

江苏省新华书店经销 南京玉河印刷厂印刷

开本:787 mm×1092 mm 1/16 印张:21.25 字数:544千字

2003年6月第1版 2003年6月第1次印刷 印数:1—5000册

总定价:105.00元(全套共4本,每本附赠光盘)

(凡因印装质量问题,可直接向发行科调换。电话:025-3795801,网址:press.seu.edu.cn)

计算机编程语言课程设计丛书

编委会名单

主 任:李文忠

副主任:朱 敏 陈清华

编 委:(以姓氏笔画为序)

丁晟春	王必友	王姗姗	王 栋
冯继生	卢雪松	叶锡君	李文忠
江国华	朱 敏	朱敦铭	邵定宏
宗 平	林 庆	陈利群	陈清华
张基温	周 群	张 煦	顾元刚
郑成增	施 诚	赵明生	唐 全
秦 军	徐逢辰	夏 斐	傅德胜

丛书总序

过去我国高等院校的非计算机专业大学生的计算机基础教育是分为三个层次进行的,即文化基础教学、技术基础教学、专业基础教学。由于除计算机专业外的理、工、农、医、文等各类专业十分复杂,同时各类专业对学生的知识结构要求不一,对掌握计算机知识技能的要求相差也很大,即使同是工科类但由于专业不一样,要求亦不同,如电子类专业和机械类专业要求就不一样。尽管如此,各专业基本上都设有“计算机文化基础”的教学内容,在这类课程中大都设置了不同的计算机语言课,如:BASIC、FORTRAN、PASCAL 及 C 语言等。根据许多语言教师和学生们的反映,学生们学过了程序设计语言,但却不会进行程序设计,当然更谈不上开发应用软件了。而由于信息技术的发展,学生毕业后工作中很需要程序设计的技能。

新世纪对高校计算机教育提出了更高的要求,一是初、高中开设了信息技术课,今后进入大学的新生具有较好的计算机基础知识;二是信息技术的发展对大学生的计算机应用能力提出了更高的要求,因此,“程序设计”课程必须改变过去学生学了程序设计语言不会进行程序设计的状况。全国高校计算机基础教育研究会 2000 年深圳学术年会经过充分讨论一致认为:理工科大学生应当学习程序设计,以具有一定的软件开发能力。

我们认为,对大学生程序设计能力的培养是十分重要的,抓住了这一要害问题,以培养学生程序设计能力为主线,通过安排一个大作业,就能带动学生对程序设计的技巧、系统分析和数据结构、数据库等许多知识的综合应用,可以大大提高学生的分析思维能力,为毕业设计和以后工作需要打下一个良好的基础。

我们江苏省高校计算机基础教学工作委员会通过调查研究,召集了东南大学、南京理工大学、南京航空航天大学、河海大学、江南大学、南京师范大学、南京邮电学院、中国药科大学、南京工业大学、南京农业大学、江苏大学、解放军理工大学、江苏教育学院、南京气象学院、南通师范学院;扬州大学、南京中医药大学、常州工学院、沙洲工学院等院校多年从事计算机基础教学的第一线骨干教师们进行了多次研讨,综合了大家的宝贵意见,认为当前急需编写并出版一套“计算机编程语言课程设计系列教材”,以满足教学之需要。为保证本套书之质量,我们成立了丛书编委会,确定了以南京理工大学陈清华老师为主编的编写组。此套书采用开放式滚动出版的办法,即通过实践不断吸收广大师生的意见,充实新的内容,吐故纳新,努力打造精品,以取得较好的教学效果,热诚欢迎广大师生提出宝贵意见。

李文忠

2003 年 5 月 24 日

前 言

学了汉语的字、词、成语及语法,你未必能写出好的文章。但是优秀的文章读多了,你就能写出好的文章。而写出好文章的人,未必都是从文学院科班毕业的。

如果你不会游泳,即使请来国家游泳队的总教练,在岸上将蛙泳的每个动作要领都讲解得清清楚楚,你也把每个动作铭记在心,做得准确无误,而你正要跃跃欲试时,将你扔到深水里,你一定会被淹死。

本书主要目的是通过引导读者在阅读、分析、理解精选的优秀样例程序的基础上,学会一步一步设计完整程序的基本方法。提高读者用 C++ 程序设计语言来解决问题的能力,进而掌握编写高效、完整、面向对象的实用程序的设计方法,及编写大规模实用程序的方法和技巧。

本书中的优秀样例程序是从近 10 万行程序代码中精心挑选出来的。全书共分 8 部分,内容包括域名服务器的仿真程序样例及样例详解,6 个经典有趣的游戏程序,7 个简单实用的数学问题求解程序,6 个清晰易懂的矩阵及向量处理程序,5 个实用有趣的小型管理程序,2 个简单实用的时间和日期处理程序,5 个多媒体应用程序,以及 6 个实用但有一定难度的数据结构应用程序。

书中精选的程序实用性强,语言简练,注释通俗易懂。程序组织结构及对问题的处理方法多种多样,覆盖面广,对读者编写大型实用程序有一定的借鉴作用。

如果能把本书中的样例程序研读一遍,你就知道别人是如何写出优秀的程序的。如果你能在读懂三个以上样例程序的前提下,并按照题目中提出的“课程设计要求”去做,且能很好完成,你已经成为编程高手,这将使你在工作中受益无穷。

本书是专门为大专院校理工科学生进行 C++ 程序设计语言课程设计而编写的教材,对广大的计算机编程爱好者也是一本难得的实用参考书。

本书主编为陈清华和朱红,主要参编人员有朱近、靳丛、刘永、戴益民、王庆宝、刘明、钱芸生、赵琦、张峰、闫玉德。

本书能够正式出版,首先要感谢“计算机编程语言课程设计丛书编委会”的所有教授及专家,特别是东南大学的朱敏教授、南京邮电学院的秦军教授、中国药科大学的唐全教授以及南京航空航天大学的王珊珊教授,更要感谢南京理工大学张岳新教授、李晓梅、梅景春、陆汉栋、李涛、宗士增等相关领导,他们为本书的前期策划及编写提出了很多宝贵意见。

本书是编者总结多年来从事计算机教学的经验,结合实际科研开发的成果,几经修订编写成的。但由于时间仓促,书中难免有不当之处,恳请读者不吝指正。

陈清华

Cqh608@yahoo.com

2003 年 4 月于南京理工大学

使 用 说 明

本书可以用作“Visual C++课程设计”的教材,也可以用作“Visual C++程序设计”课程大作业及上机编程参考书。如果作为课程设计教材使用,以下使用说明可供参考。

一、各学校根据课程进度和学时数来安排学生选题。每个设计程序按难易程度被分为 A 或 B 两个等级,原则上 B 级题为单选题,A 级题可合做。建议由学生自由选题,但原则上不能重复。如果是二人合做一题,则教师要给学生规定好各自的工作,保证学生课程设计的不重复。所附光盘中“补充部分”文件夹中的题也可选做。

二、原则上学生在进行课程设计时应独立完成,以培养其分析问题和解决问题的能力。教师可以安排一定的时间答疑。

三、要求学生必须利用一定的时间(可以利用假期)针对所选题目进行分析、注释。分析清楚程序主要完成的功能,如果有类,注释清楚类的功能、类中的数据变量用途、类中的函数定义;注释清楚每个函数的功能、函数的参数、函数的返回值含义;注释清楚整个程序中,各代码段之间的前后运行关系、函数的前后调用关系;注释清楚程序中的每一行语句的具体含义。

四、学生在完成课程设计的基本功能要求之后,可根据各自的兴趣对所选设计题目的功能进行扩充,对程序结构进行改写。

五、本书和光盘上提供的代码程序仅作参考。所附光盘中为每个题目提供了一个代码程序。在进行课程设计时,可将包含设计题目的文件夹复制到硬盘中,去掉所有文件的只读属性,如果是第 1,2,3,4,5,6,8 部分的题目建议在 VC6.0 环境下编译运行,如果是第 7 部分的题目建议在 Turbo C++ 2.0 以上环境下编译及运行。

六、编程时,要求变量名、函数名、类名规范,语句嵌套采用缩进格式,尽量采用面向对象的设计方法。请参见附录 1 和附录 2。

七、课程设计结束时,学生应提交完成的所有源程序及相关文件。同时撰写《课程设计报告》,其中主要包括以下内容:对题目的分析、注释,所增加功能模块的设计及设计中遇到的主要问题及解决方法,设计中尚存的不足,对设计的感想和心得体会等。

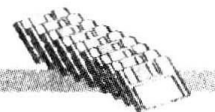
八、教师在考核学生的设计时,应增加面试过程。这样可以全面准确地了解学生对基础知识的掌握和对设计题目所做的工作,同时杜绝作弊现象的发生。因为题目的难易程度差别较大,第 8 部分设计题目相对较难,考核时应予以充分考虑。

目 录

1 样例及样例解答	(1)
样例 域名服务器的仿真程序	(2)
样例解答 域名服务器的仿真程序	(9)
2 游戏程序	(29)
第1题 扑克牌游戏	(30)
第2题 TICTACTOE 游戏	(38)
第3题 一个21点的扑克牌游戏	(44)
第4题 猜单词游戏	(52)
第5题 福彩游戏	(65)
第6题 对抗游戏	(70)
3 数学问题求解程序	(75)
第1题 计算器	(76)
第2题 分数计算器	(82)
第3题 制式转换	(92)
第4题 八皇后问题	(99)
第5题 求解方程	(105)
第6题 高斯—赛德尔迭代法	(115)
第7题 字符串操作	(122)
4 矩阵及向量处理程序	(133)
第1题 矩阵旋转反射	(134)
第2题 矩阵乘法计算	(137)
第3题 矩阵数乘	(138)
第4题 矩阵转置	(149)
第5题 矩阵加法	(150)
第6题 矩阵乘法验证	(151)
5 小型管理程序	(153)
第1题 工资管理	(154)
第2题 物品库存管理	(160)
第3题 股票交易系统	(168)
第4题 个人财政支出管理	(182)
第5题 电话簿管理	(188)
6 时间和日期处理程序	(201)
第1题 时间和日期	(202)

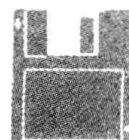
Visual C++

第 2 题 日期格式处理·····	(227)
7 多媒体应用程序 ·····	(241)
第 1 题 屏保程序·····	(242)
第 2 题 牛顿法图解方程·····	(248)
第 3 题 图形运动程序·····	(251)
第 4 题 特殊图像效果·····	(255)
第 5 题 窗口菜单显示·····	(262)
8 数据结构应用程序 ·····	(269)
第 1 题 矩阵链表堆栈·····	(270)
第 2 题 双向链表·····	(281)
第 3 题 二叉树后序遍历·····	(293)
第 4 题 二叉树操作·····	(303)
第 5 题 模板实现排行榜管理·····	(315)
第 6 题 模板实现电话簿管理·····	(323)
附录 1 变量命名应遵守的原则 ·····	(329)
附录 2 VC++ 编程应遵守的原则 ·····	(330)
参考文献·····	(332)



1

样例及样例解答



样例 域名服务器的仿真程序

1.1.1 程序功能简介

一个域名服务器的仿真(模拟)程序,其中包括数据的输入、查询和删除功能,要求在该程序的基础上添加一些功能,使程序的功能更为全面,操作更加方便。

1.1.2 课程设计要求

(1) 本程序每个功能只能完成一次操作,若进行同样的操作需退回到主菜单中方可进行,增加可以多次进行同样操作的功能。

(2) 每次程序执行完后数据不能保存下来,应该将输入的数据存放在一个文件中,在每个数据记录格式中添加时间单元,存入添加数据时的系统时间,并可以根据不同的时间范围(某时间之前、某时间之后、某段时间之间)来查找域名。

(3) 将数据保存在文件中,增加打开原有文件和建立新文件的功能。

(4) 增加修改原有数据的功能,在修改数据时,应将新旧数据同时在屏幕上显示,并询问是否以新数据替换旧数据,按“确认”键后再将数据替换,否则放弃。

1.1.3 课程设计说明

(1) 本程序以链表的方式存储数据,基本结构比较合理,不需修改,应该在原有结构的基础上增加一些函数。

(2) 评定难易级别:A 级。

1.1.4 关键源代码注解

```
#include <stdlib.h>
#include <time.h>
#include <iostream.h>
#include <ctype.h>
#include <string.h>
#include <fstream.h>
#include <stdio.h>

struct webaddress //地址簿格式
{
    char www[5];
    char name[9];
    char adm[5];
    char country[4];
    webaddress * next; //增加指针变量,作为链表进行操作
};
```

```

webaddress * head=NULL;           //全局变量,链表头
void Initial(void)
{
    cout<<" * * * * * \n";
    cout<<" 欢迎使用域名服务器查程序 \n";
    cout<<" 请选择以下功能 \n";
    cout<<" \n";
    cout<<" 1. 查询数据 \n";
    cout<<" 2. 插入数据\n";
    cout<<" 3. 删除数据\n";
    cout<<" 4. 打印全部数据\n";
    cout<<" 5. 退出\n";
    cout<<"\n";
    cout<<" * * * * * \n";
}

void Fill(webaddress * pt1,char * TempChar)
{
    int i,j;
    int dot=0;
    static char str[10];
    for(j=0;TempChar[j]! ='\0';j++)
    {
        TempChar[j]=tolower(TempChar[j]);    //将输入的字符串变成小写
        if(TempChar[j]=='.')
            dot++;        //统计数
    }
    if(dot==0)
    {
        cout<<"数据格式错误,未插入数据! \n";
        return;
    }
    for(i=0;i<4;i++)
        str[i]=TempChar[i];
    str[i]='\0';
    if(strcmp(str,"www.")==0)
    {
        strcpy(pt1->www,str);
        dot--;
        j=i;
    }
    else

```



```

    {
        j=0;
        strcpy(pt1->www,"");
    }
    for(i=0;TempChar[j]! ='.&&TempChar[j]! =\0;i++,j++)
    {
        str[i]=TempChar[j];
    }
    str[i]=\0;
    j=j+1;
    dot--;
    strcpy(pt1->name,str);
    for(i=0;TempChar[j]! ='.&&TempChar[j]! =\0;i++,j++)
    {
        str[i]=TempChar[j];
    }
    str[i]=\0;
    j=j+1;
    strcpy(pt1->adm,str);
    for(i=0;TempChar[j]! =\0&&dot;i++,j++)
        str[i]=TempChar[j];
    str[i]=\0;
    strcpy(pt1->country,str);
    return;
}

void Insert(webaddress * pt)
{
    webaddress * pt1,* pt2;
    pt1=pt2=head;
    if (head == NULL )
    {
        head = pt;
        pt->next = 0;
        return ;
    }
    if (strcmp( head->name, pt->name)>=0 )
    {
        pt->next=head;
        head=pt;
        return;
    }
    pt2=pt1 = head;
    while ( pt2->next && strcmp(pt2->name, pt->name)<0 )

```

```

{
    pt1 = pt2 ;
    pt2 = pt2->next;
}
if (strcmp( pt2->name, pt->name)<0 )
{
    pt2->next = pt;
    pt->next = 0;
}
else
{
    pt->next = pt2;
    pt1->next = pt;
}
return ;
}

void Print(webaddress * pt)
{
    cout<<pt->www<<pt->name<<". "<<pt->adm<<". "<<pt->country<<endl;
}

void LookUpData(void)
{
    webaddress * pt;
    char webname[10];
    cout<<"输入域名的名称\n";
    cin>>webname;
    pt = head;
    while(pt)
    {
        if(strcmp(webname,pt->name)==0)
        {
            cout<<"查找出的域名为:\n";
            Print(pt);
            break;
        }
        pt=pt->next;
    }
    if(pt==NULL)
        cout<<"数据文件中没有该地址\n";
    cin.get();
    cin.get();
}

```

```
        return;
    }

    void InsertData(void)
    {
        webaddress * pt;
        char Tempchar[22];
        cout<<"请输入要插入的域名\n";
        cin.get();
        cin.getline(Tempchar,22);
        pt=new webaddress;
        Fill(pt,Tempchar);
        Insert(pt);
        return;
    }

    void Delete(char * Tempchar)
    {
        webaddress * p1, * p2;
        char choice;
        if ( head == NULL )
        {
            cout <<"数据表为空,无域名可删! \n";
            return ;
        }
        if ( strcmp(head->name,Tempchar)==0 )
        {
            p1=head;
            cout<<"存在,该域名为:\n";
            Print(p1);
            cout<<"请确定是否删除该域名(y/n)? \n";
            cin>>choice;
            if(choice=='Y' || choice=='y')
            {
                head = head->next;
                delete p1;
                cout <<"删除了该域名! \n";
            }
            else
            {
                cout<<"未删除域名! \n";
            }
        }
    }
```



```

else
{
    p2=p1 = head;
    while (strcmp(p2->name,Tempchar) !=0 && p2->next != NULL )
    {
        p1 = p2 ;
        p2 = p2->next;
    }
    if (strcmp( p2->name,Tempchar) ==0 )
    {
        cout<<"存在,该域名为:\n";
        Print(p2);
        cout<<"请确定是否删除该域名(y/n)? \n";
        cin>>choice;
        if(choice=='Y' || choice=='y')
        {
            p1->next = p2->next;
            delete p2;
            cout <<"删除了该域名! \n";
        }
        else
        {
            cout<<"未删除域名! \n";
        }
    }
    else cout <<"数据表上没有找到要删除的域名! \n";
}

return;
}

void DeleteData(void)
{
    char Tempchar[22];
    cout<<"请输入要删除的域名的名称(第二个字段)\n";
    cin.get();
    cin.getline(Tempchar,22);
    Delete(Tempchar);
    cin.get();
    cin.get();
    return;
}

```



```
void PrintData(void)
{
    webaddress * pt;
    pt = head;
    if(head != 0)
        cout<<"数据表中的域名有:\n";
    while(pt)
    {
        Print(pt);
        pt = pt->next;
    }
}

void main(void)
{
    int choice;
    do
    {
        Initial();
        cin>>choice;
        switch(choice)
        {
            case 1:
                LookUpData();    break;
            case 2:
                InsertData();    break;
            case 3:
                DeleteData();    break;
            case 4:
                PrintData();    getchar();break;
            case 5:
                break;
            default:
                cout<<"选择错误! \n";
        }
    } while(choice != 5);
}
```



样例解答 域名服务器的仿真程序

1.2.1 课程设计的解答说明

本程序以链表的方式存储数据,基本结构比较合理,不需修改,在原有结构的基础上增加一些函数。

域名数据的基本格式是 www.name.address.country,但一些其他的格式也应可以识别如:name.address,即没有 www 和 country。

tm 为 time.h 中定义的数据结构,内容如下:

```
struct tm
{
    int tm_sec;    //秒数 0~59
    int tm_min;    //分钟数 0~59
    int tm_hour;   //小时 0~23
    int tm_mday;   //月份日期 1~31
    int tm_mon;    //月份 0~11
    int tm_year;   //年份 1900~2000
    int tm_wday;   //星期日期 0~6
    int tm_yday;   //年份日期 0~365
    int tm_isdst;  //白天的标志位
};
```

在 1.2.4 课程设计的源代码实现中未使用类。如要将此样例作为设计题目布置给学生,可以要求设计如下:

- (1) 将程序改写成类。
- (2) 扩充仿真程序的功能:仿真域名解析的过程。

1.2.2 课程设计的任务分解

要完成 1.1.2 课程设计要求(1),应在程序的各个功能处理函数中(如添加、删除等)增加循环语句,只有不选择继续操作时才退出。

要完成 1.1.2 课程设计要求(2)和(3)中与文件有关的操作,可以设计一个函数(如程序中的 SaveFile 函数),完成将输入的数据及每次程序执行完后的数据保存到一个文件中,增加打开原有文件和建立新文件的功能。

要完成 1.1.2 课程设计要求(2)中与时间有关的操作,可以设计 LookUp_time_between, LookUp_time_before, LookUp_time_after 和 LookUp_time 函数来分别查找域名。

① LookUp_time_between 函数:查询的域名在某两个时间之间,首先输入这两个时间,然后再依次查找链表中满足条件的数据,并进行显示。

② LookUp_time_before 函数:查询的域名在某时间之前输入,首先输入这个时间,然后再查找链表中满足条件的数据,并进行显示。

③ LookUp_time_after 函数:要查询的域名在某时间之后输入,首先输入这个时间,然后

再查找链表中满足条件的数据,并进行显示。

要完成 1.1.2 课程设计要求(4)可以设计 Modify 及 ModifyData 函数。

① Modify 函数:修改链表中域名机构名为字符串 TempChar 的节点数据,首先要求用户输入域名,然后在链表中查询满足条件的数据,将新旧数据显示的同时询问是否要求替换,当确认替换时,先删除原数据,再添加新数据。

② ModifyData 函数:是 Modify 函数的辅助函数,在其中实施具体的操作。

1.2.3 课程设计的新增函数

(1) 函数 void Fill(webaddress * pt1, char * TempChar, ifstream infile);将 TempChar 中的字符串写入 pt1 所指向的结构体单元;将数据文件中的年份、月份、日期、小时、分钟,分别复制进链表节点中相应的区域。

(2) 函数 void OpenFile(void):打开数据文件,将数据文件中的数据输入到链表中。

(3) 函数 void LookUp_name(void):按域名的机构名查询符合条件的链表数据。

(4) 函数 int CompareTime(webaddress * pt, int * time):将链表中节点数据中的时间与整型数组中的时间进行一一比较,当节点中的时间在数组中的时间之后,函数返回 1,否则返回 0。

(5) 函数 void LookUp_time_between(void):要查询的域名在某两个时间之间,首先输入这两个时间,然后再依次查找链表中满足条件的数据,并进行显示。

(6) 函数 void LookUp_time_before(void):要查询的域名在某时间之前输入,首先输入这个时间,然后再查找链表中满足条件的数据,并进行显示。

(7) 函数 void LookUp_time_after(void):要查询的域名在某时间之后输入,首先输入这个时间,然后再查找链表中满足条件的数据,并进行显示。

(8) 函数 void LookUp_time(void):按输入域名的时间进行查找。

(9) 函数 void Modify(char * Tempchar):修改链表中域名机构名为字符串 Tempchar 的节点数据。

(10) 函数 void ModifyData(void):修改链表中域名机构名为字符串 Tempchar 的节点数据,是 Modify(void)的辅助函数。

(11) 函数 void SaveFile(void):将链表中所有节点的数据都保存在数据文件中。

1.2.4 课程设计的源代码实现及详细注解

```
#include <stdlib.h>
#include <time.h>
#include <iostream.h>
#include <ctype.h>
#include <string.h>
#include <fstream.h>
#include <stdio.h>
```

```
struct webaddress    //结构体:地址簿格式
{
```

```

char www[5];          //存放字符串"www. ",若省略该字符串,则为空
char name[9];         //存放机构名,排序等均以此为据
char adm[5];          //存放网络名,不可省略
char country[4];      //存放国家名,可省略
int year;             //以下为注册域名或修改域名的时间:年
int month;            //月
int day;              //日
int hour;             //时
int minute;           //分
webaddress * next;    //指针变量,作为链表进行操作,指向下一节点
};
webaddress * head=NULL; //全局变量,链表头
tm * CurrentTime;
time_t time_date; // time_t 里面放的是星期、月份、日期、时间及年份等数据
void Initial(void)    //初始化程序,输出开始菜单选项
{
    cout<<" * * * * * \n";
    cout<<" 欢迎使用域名服务器查程序 \n";
    cout<<" 请选择以下功能 \n";
    cout<<" \n";
    cout<<" 1. 从数据文件中读入数据 \n";
    cout<<" 2. 查询数据 \n";
    cout<<" 3. 插入数据\n";
    cout<<" 4. 删除数据\n";
    cout<<" 5. 修改原数据 \n";
    cout<<" 6. 打印全部数据\n";
    cout<<" 7. 保存数据文件 \n";
    cout<<" 8. 退出\n";
    cout<<"\n";
    cout<<" * * * * * \n";
}

// Fill 函数:将 TempChar 字符串中的数据复制进节点指针 pt1 所指向的结构体空间,形成节点数据
void Fill(webaddress * pt1,char * TempChar)
{
    int i,j;
    int dot=0;          //代表域名中小数点的个数
    static char str[10]; //暂时存放域名的各个部分
    for(j=0;TempChar[j]! ='\0';j++)
    {
        TempChar[j]=tolower(TempChar[j]); //将输入的字符串变成小写
        if(TempChar[j]=='.')
            dot++;          //统计小数点的个数,以判断域名省略了几个部分
    }
}

```

```

    }
    if(dot==0) //没有小数点,不是域名的形式,该数据不能插入链表
    {
        cout<<"数据格式错误,未插入数据!\n";
        return;
    }
    for(i=0;i<4;i++) //将字符串的前四个字符复制进 TempChar,以确定是否为第一部分"www."
        str[i]=TempChar[i];
    str[i]='\0'; //str 数组用\0结束,可以用字符串处理函数操作
    if(strcmp(str,"www.")==0) //str 字符串中的内容与"www."比较
    {
        strcpy(ptl->www,str); //相同,说明域名的第一部分为"www.",将其复制进链表节点的对应部分
        dot--; //小数点的位置数减 1,因为已经处理完一部分
        j=i; //下面对原字符串 TempChar 的处理从第 4 个字符开始
    }
    else //不相同,说明域名的第一部分为机构名
    {
        j=0; //下面对原字符串 TempChar 的处理仍从第一个字符开始
        strcpy(ptl->www,""); //将链表节点对应"www."的部分赋为空
    }

    //以下处理域名的第二部分
    for(i=0;TempChar[j]!='.' && TempChar[j]!='\0';i++,j++)
    {
        str[i]=TempChar[j]; //将原字符串从 j 开始到下一个小数点的所有字符复制进 str 中
    }
    str[i]='\0'; //同样,str 以\0结束,可以用字符串处理函数操作
    j=j+1; //字符串中已处理过的字符序号加 1,以跳过域名间作间隔的小数点
    dot--; //处理过第二部分,小数点位置数减 1
    strcpy(ptl->name,str); //将域名的第二部分复制进链表节点中的相应区域

    //以下处理域名的第三部分:网络名
    for(i=0;TempChar[j]!='.' && TempChar[j]!='\0';i++,j++)
    {
        str[i]=TempChar[j]; //将原字符串从 j 开始到下一个小数点(存在第四部分)或到字符串结束(域
        //名省略了第四部分)的所有字符复制到 str 中
    }
    str[i]='\0'; //同样,str 以\0结束,可以用字符串处理函数操作
    j=j+1; //将原字符串处理过的序号加 1,跳过小数点
    strcpy(ptl->adm,str); //将第三部分复制进链表节点相应的区域
    for(i=0;TempChar[j]!='.' && TempChar[j]!='\0';i++,j++)
        str[i]=TempChar[j]; //最后处理第四部分(如果存在),不存在为空
    str[i]='\0'; //同样,str 以\0结束,可以用字符串处理函数操作

```

```
strcpy(ptl->country, str); //将第三部分复制进链表节点相应的区域
time_date=time(0); //time()函数是取得自格林威治时间 1970 年 1 月 1 日 00:00
//至当前时间所经过的秒数
CurrentTime=localtime(&time_date);
//localtime()函数是将类型为 time_t 的时间值,放入类型为 tm 的结构体变量中
ptl->year=CurrentTime->tm_year+1900; //将当前时间变为标准形式
ptl->month=CurrentTime->tm_mon+1; //取出月份赋给类中的私有数据 Month,tm_mon 是从
//0 开始,表示 12 月份依次为 0~11,故真实月份需加 1
ptl->day=CurrentTime->tm_mday; //取出当前时间的天数放入结构体的节点中
ptl->hour=CurrentTime->tm_hour; //取出当前时间的小时数放入结构体的节点中
ptl->minute=CurrentTime->tm_min; //取出当前时间的分钟数放入结构体的节点中
return;
}
```

// 新增 Fill 函数:将 TempChar 中的字符串写入 ptl 所指向的结构体单元;将数据文件中的年份、月份、日期、
// 小时、分钟,复制进链表节点中相应的区域

```
void Fill(webaddress * ptl,char * TempChar,ifstream infile)
{
    int i,j;
    int dot=0; //存放 TempChar 字符串中的小数点数,以判断域名的形式有几个部分
    static char str[10]; //暂时存放字符串的单元
    for(j=0;TempChar[j]!='\0';j++)
    {
        TempChar[j]=tolower(TempChar[j]); //将输入的字符串变成小写
        if(TempChar[j]=='.')
            dot++; //统计小数点数
    }
    if(dot==0) //如果数据中没有小数点,则该数据不是域名
    {
        cout<<"数据格式错误,未插入数据!\n";
        return;
    }
    for(i=0;i<4;i++)
        str[i]=TempChar[i]; //将字符串中前 4 个字符复制进 str,以判断是否为"www."
    str[i]='\0'; //str 数组用'\0'结束,可以用字符串处理函数操作
    if(strcmp(str,"www.")==0) //str 字符串中的内容与"www."比较
    {
        strcpy(ptl->www,str); //相同,说明域名的第一部分为"www.",将其复制进链表节点的对应部分
        dot--; //小数点的位置数减 1,因为已经处理完一部分
        j=i; //下面对原字符串 TempChar 的处理从第 4 个字符开始
    }
    else //不相同,说明域名的第一部分为机构名
    {

```

```

j=0;    //下面对原字符串 TempChar 的处理仍从第一个字符开始
strcpy(pt1->www,""); //将链表节点对应"www."的部分赋为空
}
;
//以下处理域名的第二部分
for(i=0;TempChar[j]! ='.&&TempChar[j]! =\0;i++,j++)
{
    str[i]=TempChar[j];    //将原字符串从j开始到下一个小数点的所有字符复制进 str 中
}
str[i]=\0;                //同样,str 以\0结束,可以用字符串处理函数操作
j=j+1;                    //字符串中已处理过的字符序号加1,以跳过域名间作间隔的小数点
dot--;                     //处理过第二部分,小数位置数减1
strcpy(pt1->name,str);    //将域名的第二部分复制进链表节点中的相应区域
;
//以下处理域名的第三部分:网络名
for(i=0;TempChar[j]! ='.&&TempChar[j]! =\0;i++,j++)
{
    str[i]=TempChar[j]; //将原字符串从j开始到下一个小数点(存在第四部分)或到字符串结束
                        //(域名省略了第四部分)的所有字符复制到 str 中
}
str[i]=\0;                //同样,str 以\0结束,可以用字符串处理函数操作
j=j+1;                    //将原字符串处理过的序号加1,跳过小数点
strcpy(pt1->adm,str);      //将第三部分复制进链表节点相应的区域
for(i=0;TempChar[j]! =\0&&dot;i++,j++)
    str[i]=TempChar[j];    //最后处理第四部分(如果存在),不存在为空
str[i]=\0;
strcpy(pt1->country,str);
infile>>pt1->year;        //将数据文件中的年份复制进链表节点中相应的区域
infile>>pt1->month;        //复制月份
infile>>pt1->day;          //复制日期
infile>>pt1->hour;         //复制小时
infile>>pt1->minute;       //复制分钟
infile.getline(TempChar,20); //读入下一行
return;
}

//Insert 函数:将节点指针所指向的数据排序插入链表中
void Insert(webaddress * pt)
{
    webaddress * pt1,* pt2; //节点指针
    pt1=pt2=head;          //指向链表头
    if (head == NULL) //当链表头为空,即链表内没有节点,此时插入的节点应为链表内的惟一的节点
    {

```

```

        head = pt;          //插入的节点作为链表中的惟一的节点,
        pt->next = 0;        //也是最后的节点,所以指针部分指向空
        return ;
    }

    if (strcmp( head->name, pt->name)>=0 ) //如果原链表不为空,新插入的节点按机构名进行排序
        //插入,当新插入节点的机构名小于头节点时,应使新插入的节点作为头节点
    {
        pt->next=head;      //新插入节点的尾部连接原头节点
        head=pt;           //将新插入节点作为头节点
        return;
    }

    pt2=pt1 = head;        //若新插入节点应插在头节点之后,这时要在链表中寻找插入点的位置
    while ( pt2->next && strcmp(pt2->name, pt->name)<0 ) //当新插入节点大于 pt2 所指向的
    //节点时,pt2 应指向下一节点,继续寻找插入位置,而 pt1 跟在 pt2 后面,方便找到插入点后进行操作
    {
        pt1 = pt2;          //pt1 跟在 pt2 后面
        pt2 = pt2->next;      //pt2 继续寻找下一节点
    }

    if (strcmp(pt2->name,pt->name)<0)          //当 pt2 指向链表尾部且新插入的节点依然大于 pt2 所
        //指向的节点(链表尾节点),此时,应将新插入的节点作为链表尾节点
    {
        pt2->next = pt;      //pt2 的尾部与新节点连接
        pt->next = 0;         //新节点的尾部指向空,即为链表尾节点
    }

    else //当新插入的节点小于 pt2 所指向的节点时,新节点应插入在 pt1 和 pt2 之间
    {
        pt->next = pt2;      //新插入节点的尾部连接 pt2
        pt1->next = pt;       //pt1 的尾部连接新插入的节点
    }

    return ;
}

//新增 OpenFile 函数:打开数据文件,将数据文件中的数据输入到链表中
void OpenFile(void)
{
    ifstream infile;          //定义文件对象
    webaddress * pt1, * pt2;   //定义域名类型的链表指针
    pt2=head;                 //pt2 指向链表头指针
    char FileName[20];         //存放从键盘输入的数据文件名
    char Tempchar[22];         //存放从数据文件读入的一条域名记录
    cout<<"请输入数据文件名: ";
    cin>>FileName;            //从键盘输入数据文件名
    infile.open(FileName);     //打开该数据文件
}

```

```

if(! infile)                //打开文件发生错误,程序退出
{
    cout<<"打开文件错误! \n";
    exit(0);
}
while(pt2&&pt2->next) //当链表不为空,即内存中存在域名链表,数据文件中的域名数据将接在
                        //原内存的链表后
{
    pt2=pt2->next;    //pt2 指向链表的最后一个节点
}
while(infile.getline(Tempchar,22)) //从数据文件中读取一行数据进 Tempchar 单元,
                                    //数据未读完时循环读入
{
    pt1=new webaddress;           //开辟一新节点
    Fill(pt1,Tempchar,infile);    //将 Tempchar 中的数据写进节点 pt1
    Insert(pt1);                  //将此数据排序插入链表中
}
infile.close();                  //关闭数据文件
return;
}

//Print 函数:显示链表指针指向的节点中的全部数据,依次为域名及输入数据的时间
void Print(webaddress * pt)
{
    cout<<pt->www<<pt->name<<'. '<<pt->adm<<'. '<<pt->country<<'\t'<<
    pt->year<<'- '<<pt->month<<'- '<<pt->day<<'\t'<<pt->hour<<': '<<pt->
    minute<<endl;
}

// Initial_lookup 函数:显示查询域名的子菜单选项
void Initial_lookup(void)
{
    cout<<"欢迎使用域名查询功能! \n";
    cout<<"请选择查询条件\n";
    cout<<"\n";
    cout<<"1. 按域名检索查询\n";
    cout<<"2. 按数据建立时间查询\n";
    cout<<"3. 退出\n";
    return;
}

//新增 LookUp_name 函数:按域名的机构名查询符合条件的链表数据

```

```
void LookUp_name(void)
```

```
{
    webaddress * pt;    //链表指针,从链表头依次指向链表各节点
    char webname[10];   //存放从键盘输入的要查找的域名机构名
    cout<<"输入域名的名称\n";
    cin>>webname;       //输入字符串进入 webname
    pt=head;            //链表指针指向链表头
    while(pt)           //指针依次向后移动,当指针未到链表尾时循环
    {
        if(strcmp(webname,pt->name)==0) //指针指向的域名符合查找条件
        {
            cout<<"查找出的域名为:\n";    //显示链表中的全部域名
            Print(pt);                      //显示链表中的一个节点的数据
            break;
        }
        pt=pt->next; //指针所指向的节点不符合查找条件,指针顺着链表节点向后移动
    }
    if(pt==NULL) //退出循环后指针为空,表示指针查找完整个节点,并未找到符合条件的节点
        cout<<"数据文件中没有该地址\n";
    cin.get();    //显示屏幕暂停,任按一键继续
    cin.get();
    return;
}
```

// 新增 CompareTime 函数:将链表中节点数据中的时间与整型数组中的时间进行一一比较,当节点中的时间
//在数组中的时间之后,函数返回 1,否则返回 0

```
int CompareTime(webaddress * pt,int * time)
```

```
{
    if(pt->year > time[0]) //比较年
        return 1;
    else if(pt->year < time[0])
        return 0;
    if(pt->month > time[1]) //比较月
        return 1;
    else if(pt->month < time[1])
        return 0;
    if(pt->day > time[2]) //比较日期
        return 1;
    else if(pt->day < time[2])
        return 0;
    if(pt->hour > time[3]) //比较小时
        return 1;
    else if(pt->hour < time[3])
```



```

        return 0;
    if(pt->minute > time[4]) //比较分钟
        return 1;
    else if(pt->minute < time[4])
        return 0;
    return 1;
}

```

//新增 LookUp_time_between, LookUp_time_before, LookUp_time_after 和 LookUp_time 函数完成功能:
 //任务(2)中可以根据不同的时间范围(某时间之前、某时间之后、某段时间之间)来查找数据;
 // 新增 LookUp_time_between 函数:要查询的域名在某两个时间之间,首先输入这两个时间,然后再依次查
 //找链表中满足条件的数据,进行显示

```
void LookUp_time_between(void)
```

```
{
```

```
    int a[5], b[5];
```

```
    cout<<"请输入具体的时间段(第一时间>第二时间)\n";
```

```
    cout<<"请输入第一时间年份: ";
```

```
    cin>>a[0];    // 输入年
```

```
    cout<<"请输入第一时间月份: ";
```

```
    cin>>a[1];    // 输入月
```

```
    cout<<"请输入第一时间日期: ";
```

```
    cin>>a[2];    // 输入日期
```

```
    cout<<"请输入第一时间小时: ";
```

```
    cin>>a[3];    // 输入小时
```

```
    cout<<"请输入第一时间分钟: ";
```

```
    cin>>a[4];    // 输入分钟
```

```
    cout<<"请输入第二时间年份: ";
```

```
    cin>>b[0];    // 输入年
```

```
    cout<<"请输入第二时间月份: ";
```

```
    cin>>b[1];    // 输入月份
```

```
    cout<<"请输入第二时间日期: ";
```

```
    cin>>b[2];    // 输入日期
```

```
    cout<<"请输入第二时间小时: ";
```

```
    cin>>b[3];    // 输入小时
```

```
    cout<<"请输入第二时间分钟: ";
```

```
    cin>>b[4];    // 输入分钟
```

```
    webaddress * pt=head;    //链表指针,指向链表头
```

```
    while(pt)    //指针依次向后移动,当指针未到链表尾时
```

```
{
```

```
    if(CompareTime(pt,a)==0&&CompareTime(pt,b)==1)
```

```
        //当指针所指向的数据符合时间条件时调用函数,即在第一时间之后,第二时间之前
```

```
        Print(pt);    //显示这一数据
```

```
        pt=pt->next;    //指针指向下一节点
```



```

    }
    return;
}

// 新增 LookUp_time_before 函数:要查询的域名在某时间之前输入,这时,首先输入这个时间,然后再查找
//链表中满足条件的数据,一一显示
void LookUp_time_before(void)
{
    int a[5];    //整型数组,依次存放输入的时间年、月、日、时、分(均为整型值)
    cout<<"请输入具体的时间\n";
    cout<<"请输入年份: ";
    cin>>a[0];    //输入年
    cout<<"请输入月份: ";
    cin>>a[1];    //输入月
    cout<<"请输入日期: ";
    cin>>a[2];    //输入日期
    cout<<"请输入小时: ";
    cin>>a[3];    //输入小时
    cout<<"请输入分钟: ";
    cin>>a[4];    //输入分钟
    webaddress * pt=head;    //链表指针,指向链表头
    while(pt)    //指针依次向后移动,当指针未到链表尾时
    {
        if(CompareTime(pt,a)==0)    // 当指针所指向的数据符合时间条件时调用函数
            Print(pt);    //显示这一数据
        pt=pt->next;    //指针指向下一节点
    }
    return;
}

// 新增 LookUp_time_after 函数:要查询的域名在某时间之后输入,这时,首先输入这个时间,然后再查找链
//表中满足条件的数据,一一显示
void LookUp_time_after(void)
{
    int a[5];    //整型数组,依次存放输入的时间年、月、日、时、分(均为整型值)
    cout<<"请输入具体的时间\n";
    cout<<"请输入年份: ";
    cin>>a[0];    //年;
    cout<<"请输入月份: ";
    cin>>a[1];    //月;
    cout<<"请输入日期: ";
    cin>>a[2];    //日
    cout<<"请输入小时: ";

```

```

cin>>a[3];    //时
cout<<"请输入分钟: ";
cin>>a[4];    //分
webaddress * pt=head;    //链表指针,指向链表头
while(pt)    //指针依次向后移动,当指针未到链表尾时循环
{
    if(CompareTime(pt,a)==1) //当指针所指向的数据符合时间条件时调用函数
        Print(pt);    //显示这一数据
    pt=pt->next;    //指针指向下一节点
}
return;
}

```

// 新增 LookUp_time 函数:按输入域名的时间进行查找

```
void LookUp_time(void)
```

```

{
    int choice; //子菜单选项变量,以下为按时间查找的各种可能情况,供用户选择
    cout<<"请选择正确的时间段\n";
    cout<<"1. 要查询的域名在某时间之后\n";
    cout<<"2. 要查询的域名在某时间之前\n";
    cout<<"3. 要查询的域名在某两时间之间\n";
    cout<<"4. 退出\n";
    cout<<"\n";
    cin>>choice; //键入选项
    switch(choice)
    {
        case 1:
            LookUp_time_after();    //要查询的域名在某时间之后
            break;
        case 2:
            LookUp_time_before();    //要查询的域名在某时间之前
            break;
        case 3:
            LookUp_time_between();    //要查询的域名在某两个时间之间
            break;
        case 4:
            break;
        default:
            cout<<"选择错误,退出!";
    }
    return;
}

```

// LookUpData 函数:查找某个域名

```
void LookUpData(void)
{
    int choice;        //选择子菜单选项的变量
    do
    {
        Initial_lookup();    //显示子菜单
        cin>>choice;        //键入选择
        switch(choice)
        {
            case 1:
                LookUp_name();    //按域名的机构名查询
                break;
            case 2:
                LookUp_time();    //按输入域名的时间查询
                break;
            case 3:
                break;            //退出
            default:
                cout<<"输入错误,请重新输入\n";
        }
    } while (choice!=3);
}
```

// InsertData 函数:将键盘键入的一行数据排序插入到链表中

```
void InsertData(void)
{
    webaddress * pt;    //节点指针,若输入的数据合法,则将数据放入该指针所指向的结构体空间中
    char choice;        //是否继续操作的选择变量
    char Tempchar[22]; //字符数组,存放从键盘输入的域名数据
    do
    {
        cout<<"请输入要插入的域名\n";
        cin.get();        //将输入缓冲区清空
        cin.getline(Tempchar,22);    //接受键盘输入的字符串
        pt=new webaddress;    //用节点指针开辟空间,以存放输入的数据
        Fill(pt, Tempchar);    //调用函数,将字符串中的数据以标准形式放入到节点中
        Insert(pt);        //将该节点排序插入链表中
        cout<<"是否继续插入数据(y/n)? \n";
        cin>>choice;        //接受选择变量,以确定是否继续插入
    } while(choice=='y' || choice=='Y'); //继续插入,重复循环
    return;
}
```

// Delete 函数:将域名机构名为字符串 Tempchar 的节点从链表中删除

```
void Delete(char * Tempchar)
```

```
{
    webaddress * p1, * p2;    //节点指针变量
    char choice;              //该变量为键入的确认选择
    if ( head == NULL )      //链表头指针为空,即链表中无数据,不需删除,程序直接返回
    {
        cout << "数据表为空,无域名可删! \n"; //输出提示信息
        return ;
    }
    if ( strcmp(head->name, Tempchar) == 0 ) //当链表头指针即是要删除的域名时,将头指针后的指针
                                              //作为头指针,然后将该节点删除
    {
        p1 = head;                // p1 指向头指针
        cout << "存在该域名为: \n";
        Print(p1);                //输出 p1 中的数据内容
        cout << "请确定是否删除该域名(y/n)? \n";
        cin >> choice;            //从键盘输入信息,确定是否真要删除该节点
        if (choice == 'Y' || choice == 'y') //当输入 Y 或 y 时,确定要删除该节点,以下是删除操作
        {
            head = head->next;    //将链表的头指针移后一个节点
            delete p1;            //删除原头指针所在的节点
            cout << "删除了该域名! \n"; //输出提示信息
        }
        else                    //若选择不删除该节点
        {
            cout << "未删除域名! \n"; //输出提示信息
        }
    }
    else //若链表头节点不满足删除条件,则 p2 指针顺着链表寻找满足删除条件的节点
    {
        p2 = p1 = head;        //首先, p1, p2 从头指针开始沿链表查找
        while ( strcmp(p2->name, Tempchar) != 0 && p2->next != NULL )
            //查找过程是循环过程,当没有找到符合条件的节点且没有查找完整个链表时,循环继续
        {
            p1 = p2;            //p1 跟在 p2 后面
            p2 = p2->next;      //p2 接着查找下一个节点
        }
        if ( strcmp( p2->name, Tempchar) == 0 ) //找到要删除的节点正是 p2 所指向的节点
        {
            cout << "存在该域名为: \n"; //输出该节点的数据信息
            Print(p2);
            cout << "请确定是否删除该域名(y/n)? \n"; //要求用户确认一下是否真的要删除
            cin >> choice; //输入用户的选择
        }
    }
}
```



```

    if(choice=='Y' || choice=='y')                //如果确认为真的要删除
    {
        p1->next = p2->next; //将 p2 指向节点的下一个节点的尾部连接到 p1 所指向的
                               //节点,将 p2 所指向的节点从链表中断开
        delete p2;           //删除 p2 所指向的节点
        cout <<"删除了该域名! \n";
    }
    else                                           //如果用户选择不删除
    {
        cout<<"未删除域名! \n"; //输出提示信息
    }
}

else cout<<"数据表上没有找到要删除的域名! \n";/* 如果查找完链表中的所有节点未找到符
合条件的节点,这时输出提示信息 */

}

return;
}

// DeleteData 函数:根据机构名删除域名
void DeleteData(void)
{
    char Tempchar[22];                          //存放键入的要删除的域名机构名
    cout<<"请输入要删除的域名的名称(第二个字段)\n";
    cin.get();                                  //清空输入缓冲区
    cin.getline(Tempchar,22);                  //接受字符串
    Delete(Tempchar);                           //调用函数,删除符合要求的域名
    cin.get();                                  //暂停,任按一键继续
    cin.get();
    return;
}

/* 新增 Modify 函数:修改链表中域名机构名为字符串 Tempchar 的节点数据,输出原节点数据内容 */
void Modify(char * Tempchar)
{
    webaddress * pt;                            //节点指针,指向修改后的节点数据
    webaddress * p1, * p2;                      //节点指针,用来在链表中寻找符合条件的节点
    char choice;                                //确认是否替换原节点的选择变量
    char webname[22];                           //接受从键盘输入的新修改的域名数据
    if ( head == NULL ) {                      //头指针为空,即链表中没有数据,不需修改
        cout <<"数据表为空,无域名可修改! \n"; //输出提示信息
        return ;
    }
}

```

```

if ( strcmp(head->name,Tempchar)==0 )    //当要修改的节点就是头节点时
{
    p1=head;                            //p1 指向头节点
    cout<<"存在该域名为:\n";            //输出头节点数据
    Print(p1);
    cout<<"请输入修改的域名:\n";        //输入欲修改的域名数据
    cin>>webname;                        //将新域名数据放入 webname
    pt=new webaddress;                   //用 pt 指针在内存中生成一个节点
    Fill(pt,webname);                     //将欲修改的域名数据放入该节点中
    cout<<"请确定是否替换原数据(y/n)? \n";
    cin>>choice;                          //输入选择,以确认是否真的用新域名替换旧域名
    if(choice=='Y'||choice=='y')          /* 选择确认替换,以下进行替换,即将旧的节点从链表中删除,
                                           然后再将新的节点插入链表中,这样,链表依然是顺序存放 */
    {
        p1=head;                        //p1 指向头节点
        head=head->next;                 //头节点移到下一个节点
        delete p1;                       //删除 p1 所指向的节点(原头节点)
        Insert(pt);                      //将新键入的节点插入到原链表中
        cout<<"修改了该域名! \n";        //输出提示信息
    }
    else                                  //如果选择不替换
    {
        cout<<"未修改域名! \n";          //输出提示信息,不操作,直接退出
    }
}
else                                     /* 如果欲替换的域名不是头节点,则用指针顺着链表查找满足要求的节点 */
{
    p2=p1 = head;                        //从链表头指针开始
    while (strcmp(p2->name,Tempchar) !=0 && p2->next != NULL )
//当没有查找到满足要求的节点且没有查找完链表时,循环查找
    {
        p1 = p2 ;                        //p1 跟在 p2 后面
        p2 = p2->next;                    //p2 继续顺着链表查找
    }
    if (strcmp( p2->name,Tempchar) ==0 )  /* 当 p2 指向的节点满足查找要求时,首先显示该
                                           节点数据,再确认是否真的要替换 */
    {
        cout<<"存在该域名为:\n";
        Print(p2);                       //输出原节点数据内容
        cout<<"请输入修改的域名:\n";    //输入欲替换的域名
        cin>>webname;
        pt=new webaddress;                /* 用 pt 指针在内存中开辟空间,存放欲替换的节点数据 */
        Fill(pt,webname);                 //将键入的数据转换成节点形式
    }
}

```

```

cout<<"请确定是否替换原数据(y/n)? \n";
cin>>choice;           //输入确认信息
if(choice=='Y' || choice=='y')    //如果确认真的替换,首先将旧的节点删去,
                                //再将新的节点插入到原链表中
{
    p1->next = p2->next; /* 将 p2 指向下一个节点的尾部连接到 p1 所指向的节点,
                                将 p2 所指向的节点从链表中断开 */
    delete p2;           //删除 p2 所指向的节点
    Insert(pt);           //将欲替换的节点插入链表
    cout <<"修改了该域名! \n"; //输出提示信息
}
else                       //如果确认不做替换
{
    cout<<"未修改域名! \n";    //输出提示信息
}
}

else cout<<"数据表上没有找到要修改的域名! \n"; /* 如果查找完整个链表仍没有满
                                足要求的域名,则输出提示信息 */
}

return;
}

// ModifyData 函数:修改链表中域名机构名为字符串 Tempchar 的节点数据
void ModifyData(void)
{
    char Tempchar[22];           //字符数组,存放要修改域名的机构名
    cout<<"请输入要修改的域名的名称(第二个字段)\n";
    cin>>Tempchar;               //从键盘输入机构名
    Modify(Tempchar);           //调用函数,将链表中符合条件的域名显示并修改
    cin.get();                  //任按一键继续
    cin.get();
    return;
}

// PrintData 函数:输出链表中所有节点的数据信息
void PrintData(void)
{
    webaddress * pt;             //节点指针,依次指向链表中的各个节点
    pt= head;                   //从链表头开始显示
    if(head!=0)                 //如果链表不为空,即链表中存在数据,依次显示
        cout<<"数据表中的域名有:\n";
    while(pt)                   //当未显示到链表尾部时,循环显示
    {

```



```

Print(pt);                //显示一个节点数据
pt=pt->next;              //指针指向链表的下一节点
}
}

// SaveFile 函数:将链表中所有节点的数据都保存在数据文件中
void SaveFile(void)
{
    ofstream outfile;      //输出文件对象
    char Filename[20];     //存放从键盘中输入的文件名
    cout<<"请输入数据文件名:\n"; //根据需要选择不同的文件
    cin>>Filename;         //接受文件名
    outfile.open(Filename); //以输出的方式打开数据文件
    if(! outfile)          //打开文件错误,输出提示信息,退出程序
    {
        cout<<"打开文件错误!\n";
        exit(0);
    }

    webaddress * pt;       //节点指针,指向链表中的各个节点
    pt= head;              //从链表头开始向数据文件中输入数据
    while(pt)              //当指针没有找到链表尾时
    {
        if(pt->www )       //如果该节点的域名数据存在第一部分 www.,则输出
            outfile<<pt->www;
        outfile<<pt->name<<'. '<<pt->adm; //接着输出第二、第三部分:机构名、网络名
        if(pt->country )   //如果域名存在第四部分(国名),则输出
            outfile<<'. '<<pt->country<<endl;
        outfile<<pt->year <<\r<<pt->month <<\r<<pt->day<<\r
            <<pt->hour<<\r<<pt->minute<<endl; //输出键入域名的时间数据
        pt=pt->next;       //顺着链表依次指向下一节点
    }

    cout<<"文件存储完毕\n";
}

void main(void)
{
    int choice;
    do
    {
        Initial();        //输出菜单选项
        cin>>choice;       //用户从键盘输入选择
        switch(choice)
        {

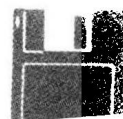
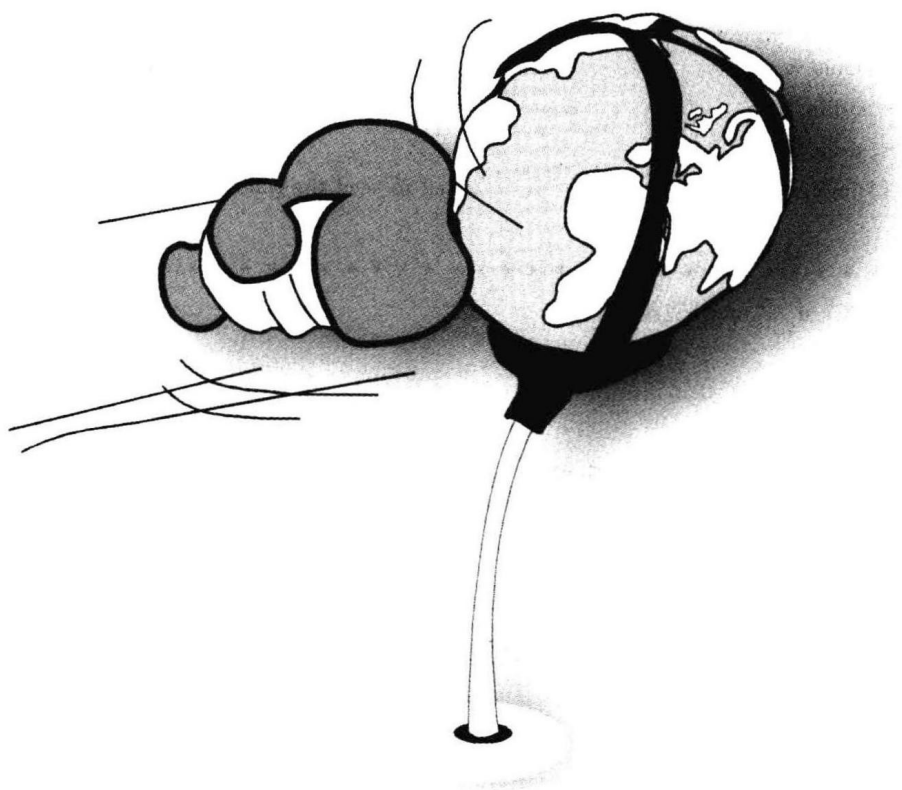
```

```

case 1:
    OpenFile();          //打开数据文件
    break;
case 2:
    LookUpData();        //查找数据
    break;
case 3:
    InsertData();        //插入数据
    break;
case 4:
    DeleteData();        //删除数据
    break;
case 5:
    ModifyData();        //修改数据
    break;
case 6:
    PrintData();         //显示链表中的所有数据
    getchar();break;
case 7:
    SaveFile();          //将链表中的节点数据保存在数据文件中
    break;
case 8:
    break;               //退出
default:
    cout<<"选择错误! \n";
}
}while(choice!=8);      //当选择为 8 时退出
}
    
```




游戏程序



第 1 题 扑克牌游戏

2.1.1 程序功能简介

创建一副扑克,并完成洗牌等操作。

2.1.2 课程设计要求

- (1) 创建一副扑克,显示一副扑克,洗牌,依次分发给 4 个人并显示。
- (2) 删去一张扑克后,余牌显示。
- (3) 增加大小王。

2.1.3 课程设计说明

- (1) 可增加函数 SendDeck(),可用一个二维数组来表示 4 个玩家。
- (2) 评定难易级别:A 级。

2.1.4 关键源代码注解

```
// * This Program was written entirely by the author Frank Vokoun.
```

```
/* * * * * */ * * * * * preprocessor directives * * * * *
//
#include <iostream.h>
#include <string.h>
#include <conio.h>
#include <stdlib.h>
#include <stdio.h>
#include <time.h> // used to seed the random number generator
//
/* * * * * * */
class Playing_Card //扑克类
{
private:
    int m_Value;
    char m_Face[3]; //扑克的数字大小
    char m_Suit; //扑克的花色(黑、红、梅、方)
public:
    Playing_Card(); // CONSTRUCTOR
    void showcard(); // Displays an object of class Playing_Card
    void Card_Make(int);
};
```

```

class Deck    //一副扑克(52 张)
{
private:
    Playing_Card m_cardarray[52];    // Object array of class Playing_Card, Cardarray—扑克牌数组
    int m_lastdelt;

public:
    Deck();    // CONSTRUCTOR initializes empty Deck object
    void MakeDeck()    // makes a complete object of class Deck
    void Deal_One();    // Deals a card from top of the deck
    void ShowDeck();    // Displays a Deck object to the screen
    void DeckShuff(int);    // "Shuffles" a Deck object for int times, shuffle—洗牌
    void Remove_Card();    // Removes used card from Deck
};

// * * * * * prototypes * * * * *

void Program_Init();    // Program information
int main();
void Center_Text(char []);    // Centers text
int get_number();    // Gets a integer from user
char Get_Key();    // Gets a char from user, pauses program
void DeckMakeDriver();    // Tests Several Functions including Makedeck
int getRandInt(int min, int max);    // Random number generator
void prog_close();    // pauses before program termination

// * * * * * Main * * * * *

int main()
{
    srand( (unsigned)time( NULL ) );    // Seeds GetRandInt
    int Card_Number = 0;
    Program_Init();    // Showd title etc.
    DeckMakeDriver();    // The Main Function Driver Tests Deck and
                        // Playing_Card classes
    prog_close();    // pauses screen output
    return 1;
}

Playing_Card::Playing_Card()    // CONSTRUCTOR Constructor replaces default constructor
{
    int i;
    for(i=1; i<=3; )    // inits string variable to blanks
    {
        m_Face[i] = ' ';
    }
}

```

```

        i++;
    }
    m_Suit = ''; // inits char variable to blank
    m_Value = 0;
}

```

```
void Program_Init()
```

```

{
    Center_Text("Lab #5");
    cout << endl;
    Center_Text("A Deck Driver");
    cout << endl << "\n" ;
    Center_Text("By Frank Vokoun");
    cout << endl << "\n" ;
    Center_Text("(C) 2001, Frank Vokoun");
    cout << endl << "\n" ;
    Center_Text("Hit the <Return> key to Continue. .");
    cin.get();
}

```

```
char Get_Key()
```

```

{
    char x;
    x = cin.get();
    cout << endl;
    return x;
}

```

```
void Playing_Card::showcard()
```

```

{
    cout << " ";
    cout << m_Face ;
    cout.width(1);
    cout << m_Suit;
    cout << " ";
}

```

```
void Center_Text(char ShellText[80])
```

```

{
    int length;
    int center;
    length= strlen(ShellText);
}

```

★ ★ ★ ★ ★

```
center = (80 - length)/2;
for(;center!=0;center--)
{
    cputs(" ");
}
cputs(ShellText);
}
```

```
int get_number()
{
    int Input_Integer = 0;
    Center_Text("Please enter an integer between 0 and 51. 52 to quit.");
    cout << endl;
    cin >> Input_Integer;
    return Input_Integer;
}
```

```
void Playing_Card::Card_Make(int num)
{
```

```
    int i = 0;
    char j;
    int face_num = num % 13;
```

```
    switch(face_num) // Assigns a Face value for string cards
```

```
    {
    case 0: strcpy(m_Face, " A");break;
    case 9: strcpy(m_Face, "10");break;
    case 10: strcpy(m_Face, " J");break;
    case 11: strcpy(m_Face, " Q");break;
    case 12: strcpy(m_Face, " K");break;
    default;
```

```
        j = char(face_num + 49); // Fills string with number mod 13
```

```
        if(i<3)
```

```
        {
            m_Face[i] = ' '; i++;
            m_Face[i] = j; i++;
            m_Face[i] = NULL;
            break;
        }
```

```
    }
```

```
    if(num <= 12)m_Suit = 6; // assigns suit use ascii values for
```




```

if(num >12 && num <= 25)m_Suit = 3;           //card symbols
if(num >25 && num <= 38)m_Suit = 5;
if(num >38 && num <= 51)m_Suit = 4;

if(face_num <= 9)m_Value = face_num + 1;      // Card value's Ace = 1
if(face_num >= 10)m_Value = 10;
}

```

```
void DeckMakeDriver()
```

```

{
    Deck deck_1;
    deck_1.MakeDeck();           // Creates the Deck
    deck_1.ShowDeck();           // Displays deck to screen
    Get_Key();                   // Pauses Program
    deck_1.DeckShuff(250);       // Shuffles the deck 250 times
    deck_1.ShowDeck();           // Displays deck to screen
    cout << endl << endl << endl;
    char y;
    do
    {
        deck_1.Deal_One();
        cout << endl;
        cout << endl << endl << "Y to Deal One more N to quit"<< endl;
        cin >> y;
        y = toupper(y);
    }while(y == 'Y');
}

```

```
void Deck::MakeDeck()           // creates a full deck not a construct
```

```

{
    m_lastdelt = 51;             // Set Deck to empty
    while(m_lastdelt > -1)       // iterate until deck is full
    {
        m_cardarray[m_lastdelt].Card_Make(m_lastdelt); // call card make for every card object
        m_lastdelt--;           // inside the deck object ie cardarray
    }
}

```

```
void Deck::ShowDeck()
```

```

{
    int index = m_lastdelt + 1;
    int newline = 0;
    Center_Text("Current Deck of Cards from top to bottom");
}

```

```

    cout << endl;
    while(index<=51)
    {
        if(newline % 11 == 0) cout << endl;
        m_cardarray[index].showcard();           // calls showcard for every card in
        newline++;                                // the cardarray object
        index++;
    }
}

int getRandInt(int min, int max)
// 1. set a num = to random number generator.
// 2. make num = num % (max - min + 1) + min.
// 3. Return the num
{
    int numToReturn;
    numToReturn = rand();
    numToReturn = numToReturn % (max - min + 1) + min;
    return numToReturn;
}

void Deck::DeckShuff( int times)    //洗牌次数
{
    int x, split;    //split 是分开成两部分的位置,如上部分、下部分
    Center_Text("Shuffling Deck");
    cout << endl;
    for(x=0;x<=times;x++)            // iterate input number of times
    {
        split = getRandInt(20,35);    // Get split location
        Deck topdeck;                 // Create 2 new unfilled decks
        Deck bottomdeck;
        int i;
        int bottomdeckindex = 1;
        int topdeckindex = 1;
        for(i=0;i<=split - 1;)
        {
            topdeck.m_cardarray[topdeckindex] = this->m_cardarray[i];
            topdeckindex++;
            i++;
        }
        for(i=(split);i< 52;)          // move remaining cards to bottom deck
        {
            bottomdeck.m_cardarray[bottomdeckindex] = this->m_cardarray[i];

```

```

        bottomdeckindex++;
        i++;
    }
    int deckoutindex = 0;                // set deck to fill's index to zero
    int numcardstomovetop;
    int numcardstomovebottom;
    int j;
    int h = 0;
        bottomdeckindex = 52 - split;    // set index to num cards in bottom
    topdeckindex = split;                // set index to num cards in top
    while(deckoutindex <= 51)
    {
        numcardstomovetop = getRandInt(2,7);
                                //从上部分抽取的张数,是2~7之间的随机数
        numcardstomovebottom = getRandInt(2,7);
        // Move a random number of cards(2-7)from bottomdeck to original deck
        for(j=0;j <=numcardstomovebottom;j++)
        {
            if(bottomdeckindex > 0)        // check for available cards
            {
                this->m_cardarray[deckoutindex] = bottomdeck.m_cardarray[bottomdeckindex];
                deckoutindex++;
                bottomdeckindex--;
            }
            for(h=0;h<=numcardstomovetop;h++)
                // Move a random number of cards (2 to 7) from topdeck to original deck
            {
                if((topdeckindex > 0) && (deckoutindex <=52))
                    // check for available cards and slots
                {
                    this->m_cardarray[deckoutindex]=topdeck.m_cardarray[topdeckindex];
                    deckoutindex++;
                    topdeckindex--;
                }
            }
        }
    }
    this->m_lastdelt = -1;    // Return a complete shuffled deck
}

void prog_close()                // Pauses Program so the last output can be seen
{

```

```

        cout << endl << endl;
        cout << " Hit the <Return> key to Continue. ." << endl;
        cout << endl << endl;
        Get_Key(); // Necessary for clear input
        cin.get();
    }

    void Deck::Remove_Card() // Remove card delt from Deck
    {
        m_cardarray[m_lastdelt] = Playing_Card(); // reinit's used card prevents mistakes
    }

    void Deck::Deal_One()
    //      1. Check if lastdealt = 51 if so Error
    //      2. Increment last dealt
    //      3. Return card[last dealt]
    //
    {
        if(m_lastdelt != 51) // Checks for available cards
        {
            m_lastdelt++;
            cout.width(5);
            cout << "    Card delt";
            m_cardarray[m_lastdelt].showcard();
        }
        else
        {
            cout << "Out of range Error";
            prog_close();
        }
    }
}

Deck::Deck() // CONSTRUCTOR replaces default
{
    int lastdelt = 0;
    int i;
    for(i=0; i<=51; i++)
    {
        m_cardarray[i] = Playing_Card(); // calls constructor class Playing_Card
    } // for every Playing_Card object in
    // the class Deck's array
}

```

第 2 题 TICTACTOE 游戏

2.2.1 程序功能简介

该程序是 TICTACTOE 游戏,棋盘为 3×3 ,3 个棋子连起来即为赢。输入参数时,若输入的数字超出边界或在已有棋子的地方再输入,则提示出错,要求重新输入。

2.2.2 课程设计要求

- (1) 将程序改写成类。
- (2) 将棋盘扩充成 4×4 。
- (3) 统计输赢结果并打印输出。
- (4) 选做:使游戏更生动。(输赢条件相同)

2.2.3 课程设计说明

- (1) 棋盘可用数组来定义,如 `Board[4][4] = {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0}`。
- (2) 题目要求把源程序封装在类中,需要编写一个类来封装所有相关函数和变量。同时对 `main` 主函数进行改写,以完成题目的要求。
- (3) 评定难易级别:B 级。

2.2.4 关键源代码注解

```
#include <iostream. h>
#include <stdlib. h>

void PrintBoard();           // game board
void PrintInfo();           // some info about the rules of the game is printed
void PrintStats(int, int, int); // prints the stats of the match
void IfError(int&, int&);     // When invalid input is entered, prints the error
void ChoiceOfChar(char&);    // Asks the users if they want to play again
void PromptTurnO(int&, int&); // Prompts the first user for data
void PromptTurnX(int&, int&); // Second user has to input coordinates
char board[3][3];           // array definition
int main()
{
    int ROW;                 // elements of 2 dimensional array
    int COLUMN;
    int FirstPlayer;         // number of winning games for first player
    int SecondPlayer;
    int Draws;               // number of draws in a game
    FirstPlayer = 0;
    SecondPlayer = 0;
```

```

Draws = 0;
char choice;
choice = 'Y';           // Initial value is Y in order to initialize the main loop
PrintInfo();
while(choice == 'Y')    // In case the players want to play again
{
    // this is the main loop
    for (ROW = 0; ROW < 3; ROW++)    // these forloops are used to process the array
    for (COLUMN = 0; COLUMN < 3; COLUMN++)    // by row and column
        board[ROW][COLUMN] = '';    // blank characters as initial values
    int x;    // (x,y) coordinates entered by the users
    int y;
    int SpotsOnBoard;    // this is the sum of spots available on the game board
    SpotsOnBoard = 0;
    while ( SpotsOnBoard <= 9)    // because there is only 9 characters available on the board
    {
        if (SpotsOnBoard == 0)
        {
            PrintBoard();    // This function draws initially the blank game board
            // so it is easier for the user to pick the x and y values

            PromptTurnO(x, y);

            If Error(x, y);    // If the user's input is invalid, this function prints
                                // the error message and prompts for correction

            board[x - 1][y - 1] = 'O';    // actually the array starts from 0, NOT from 1
            SpotsOnBoard++;    // (as entered by the user)
                                // so the program have to "fix" the input
            PrintBoard();    // If 3 coordintates are O's in one of 8 possible
                                // winning combinations, then O wins
            if (board[0][0] == 'O' && board[1][1] == 'O' && board[2][2] == 'O')
            {
                cout << " O player wins the game" << endl;
                FirstPlayer++;
                break;
            }
            else if (board[2][0] == 'O' && board[1][1] == 'O' && board[0][2] == 'O')
            {
                cout << "O player wins the game" << endl;
                FirstPlayer++;
                break;
            }
            else if (board[0][0] == 'O' && board[1][0] == 'O' && board[2][0] == 'O')
            {
                cout << "O player wins the game" << endl;

```

```

    FirstPlayer++;
    break;
}
else if (board[0][1] == 'O' && board[1][1] == 'O' && board[2][1] == 'O')
{
    cout << "O player wins the game" << endl;
    FirstPlayer++;
    break;
}
else if (board[0][2] == 'O' && board[1][2] == 'O' && board[2][2] == 'O')
{
    cout << "O player wins the game" << endl;
    FirstPlayer++;
    break;
}
else if (board[0][0] == 'O' && board[0][1] == 'O' && board[0][2] == 'O')
{
    cout << "O player wins the game" << endl;
    FirstPlayer++;
    break;
}
else if (board[1][0] == 'O' && board[1][1] == 'O' && board[1][2] == 'O')
{
    cout << "O player wins the game" << endl;
    FirstPlayer++;
    break;
}
else if (board[2][0] == 'O' && board[2][1] == 'O' && board[2][2] == 'O')
{
    cout << "O player wins the game" << endl;
    FirstPlayer++;    // counts the number of won games
    break;            // break statement is used to make sure that only one possibility
}                    // will be processed
else if (SpotsOnBoard == 9)
{
    cout << "Draw!" << endl;
    Draws++;
    break;            // Without this brake statement it was not working properly
}

PromptTurnX(x, y);
IfError(x, y);
board[x - 1][y - 1] = 'X';

```



```
SpotsOnBoard++;
PrintBoard();
if (board[0][0] == 'X' && board[1][1] == 'X' && board[2][2] == 'X')
{
    cout << "X player is victorious" << endl;
    SecondPlayer++;
}
else if (board[2][0] == 'X' && board[1][1] == 'X' && board[0][2] == 'X')
{
    cout << "X player is victorious" << endl;
    SecondPlayer++;
}
else if (board[0][0] == 'X' && board[1][0] == 'X' && board[2][0] == 'X')
{
    cout << "X player is victorious" << endl;
    SecondPlayer++;
}
else if (board[0][1] == 'X' && board[1][1] == 'X' && board[2][1] == 'X')
{
    cout << "X player is victorious" << endl;
    SecondPlayer++;
}
else if (board[0][2] == 'X' && board[1][2] == 'X' && board[2][2] == 'X')
{
    cout << "X player is victorious" << endl;
    SecondPlayer++;
}
else if (board[0][0] == 'X' && board[0][1] == 'X' && board[0][2] == 'X')
{
    cout << "X player is victorious" << endl;
    SecondPlayer++;
}
else if (board[1][0] == 'X' && board[1][1] == 'X' && board[1][2] == 'X')
{
    cout << "X player is victorious" << endl;
    SecondPlayer++;
}
else if (board[2][0] == 'X' && board[2][1] == 'X' && board[2][2] == 'X')
{
    cout << "X player is victorious" << endl;
    SecondPlayer++;
}
}
```



```

    ChoiceOfChar(choice);
}

PrintStats(FirstPlayer, SecondPlayer, Draws);
system("PAUSE");
return 0;
}

void PrintBoard()
// Post : The Game board is printed to help with choosing the coordintates
// Either X or O will be printed on the board
// Board is updated while the game progresses
{
    cout << endl;
    cout << "  1    2    3 " << endl;
    cout << "1 " << board[0][0] << " | " << board[0][1] << " | " << board[0][2] << endl;
    cout << " ---|---|--- " << endl;
    cout << "2 " << board[1][0] << " | " << board[1][1] << " | " << board[1][2] << endl;
    cout << " ---|---|--- " << endl;
    cout << "3 " << board[2][0] << " | " << board[2][1] << " | " << board[2][2] << endl;
    cout << endl;
    cout << endl;
}

void PrintInfo()
// Post : Some info about the rules of the game is printed
{
    cout << "First player is represented by O, second by X. " << endl;
    cout << "Enter the coordinates for Rows and Columns" << endl;
    cout << "Strike enter when Done. " << endl;
    cout << endl;
}

void PrintStats(int FirstPlayer, int SecondPlayer, int Draws)
// Pre : Players have to decide to exit the game
// Post : Returns the number of won games and draws
{
    cout << "The O player have won " << FirstPlayer << " times" << endl;
    cout << "The X player have won " << SecondPlayer << " times" << endl;
    cout << "There was " << Draws << " draws" << endl;
    cout << "Thanks for using my program" << endl;
}

void IfError(int& x, int& y)

```



```
// Pre : User entered invalid data, program doesn't know where to put them
// Post : Error message is printed, user is prompted to enter the right input
{
    while (x > 3 || x < 1 || y > 3 || y < 1 || ('O' == board[x - 1][y - 1]
        || 'X' == board[x - 1][y - 1]))
    {
        cout << "This coordinate is not allowed, try again" << endl;
        cout << "row: ";
        cin >> x;
        cout << "column: ";
        cin >> y;
    }
}
```

```
void ChoiceOfChar(char& choice)
```

```
// Pre : Entire program has been executed
```

```
// Post : User is given a chance to play again
```

```
{
    cout << endl;
    cout << " Press CAPITAL Y if you want to play again. " << endl;
    cout << " Otherwise, press any other letter key to exit the loop. " << endl;
    cin >> choice;
}
```

```
void PromptTurnO(int& x, int& y)
```

```
// Post : First player enters the coordinates of a spot on a game board
```

```
{
    cout << "Turn of the first player (O), enter the coordinates" << endl;
    cout << "Row: ";
    cin >> x;
    cout << "Column: ";
    cin >> y;
}
```

```
void PromptTurnX(int& x, int& y)
```

```
// Post : Second player enters the coordinates of a spot on a game board
```

```
{
    cout << "Turn of the second player (X), enter the coordinates" << endl;
    cout << "Row: ";
    cin >> x;
    cout << "Column: ";
    cin >> y;
}
```

第3题 一个21点的扑克牌游戏

2.3.1 程序功能简介

一个21点的扑克牌游戏,玩家一共可以要5张牌,但如果牌的点数超过21,则爆了,自动出局;在不超过21点的情况下,玩家与庄家比牌的点数大小,大者为赢。

2.3.2 课程设计要求

(1) 将所有的函数与相应的数据封装到类中,并改写主程序,使程序简化。

(2) 程序中一共有13张扑克牌可以随机抽取,大于10的点数为10,现要求大于10的点数全部为半点。

(3) 要求庄家永远不会暴庄,即庄家可以查看下一张牌,若大于21点,则拒绝,当然,在游戏规则上不能泄露这点秘密。

(4) 高级玩家可以查看下一张牌,即按H键,然后按要求输入密码,密码正确可以看牌,并决定是否要牌。

(5) 每次要牌后可以设定赔率,即如果开始下的是10元钱的注,如果牌好,你可以要求再下注,当然不能超过你所有的总钱数。

(6) 将界面的提示说明改成中文界面,界面的解释详细友好,可以吸引更多的玩家。

2.3.3 课程设计说明

(1) 题目要求把源程序封装在类中,需要编写一个类来封装所有相关函数和变量,同时对main主函数进行改写,以完成题目的要求。

(2) 题目要求把抽到大于10点的牌的点数设为半点,只需设计随机取下一张牌的成员函数时使用if语句判定牌的点数是否大于10,大于10的牌对其重新赋值为0.5即可。

(3) 题目要求庄家永远不会暴庄,庄家取下一张牌之前先判定取牌后总点数是否会大于21点,大于的话就拒绝取下一张,小于时就继续取牌,在函数中首先随机产生下一张牌,接着用条件语句判定原有点数与新牌的点数之和是否大于21点,然后决定是否取牌。

(4) 题目要求超级玩家在输入密码后可以查看下一张牌,只需设计一成员函数,判定玩家是否继续要牌之前加入对玩家是否输入H键的判定,如果有输入则调用为处理玩家输入密码而单独编写的成员函数,在该函数中,首先判定玩家的密码输入是否正确,不正确则返回0,正确则返回1。

(5) 题目要求玩家可以根据手上的牌决定是否下注,只需设计一成员函数,检查玩家的赌注累计不超过他所拥有的总资本。

(6) 评定难易级别:B级。

2.3.4 关键源代码注解

```
//wait.h begin  
#include <time.h>
```

```

    ★ ★ ★ ★ ★
#include <stdio.h>
void wait(int milli)
{
    clock_t start;
    start=clock();
    while( ( clock()-start ) < milli )
    ;
}
//wait.h end
//utility.h begin
//Isaac Shaffer
//Utility Libray
// void Pause(),int GetInt(int L,int H),void signature(), int random(long hi, long lo)

#include<iostream.h>
#include<stdlib.h>
#include<conio.h>
#include<time.h>

//=====
void pause()
//displays a message and waits for the user to hit a key
{
    cout<<"Press Any Key To Continue"<<endl;
    getch();
}

//=====
int GetInt(int L=0,int H=100)
{
    int ent;
    cout<<"Please Enter A Number Between "<<L<<" and "<<H<<endl;
    cin>>ent;
    while((ent<L)|| (ent>H))
    {
        cout<<"Error"<<endl;
        cout<<"Value must be between "<<L<<" and "<<H<<endl;
        cin>>ent;
    }
    return(ent);
}

//=====
void sign()

```

```

//Displays ISAAC SHAFFER
{
    cout<<"This Program Was Written By Isaac Shaffer"<<endl;
}

//=====

int random(long hi,long lo)
//This Program Finds A Random Number Between Hi and Low
{
    int ran;
    srand((unsigned)time(NULL));
    ran=rand()%(hi-(lo-1))+lo;
    return(ran);
}

//utility. h end
//ISAAC SHAFFER      3-2-2000
//BLACKJACK GAME WITH FUNCTIONS
//THIS PROGRAM IS MENT TO BE A SIMPLE BLACK JACK GAME

#include<iostream. h>
#include<stdlib. h>
#include "utility. h"
#include  "wait. h"
//fuctions
void replay(char &ans);      //allows replay of game
void hit(int &total);        //take another card
void deal(int &player,int &cpu,int &playerturns,int &cputurns); //deals hand
void BET(int &bet,int &money); //takes bet
void print(int wins,int lose,int draw,int money); //prints final score
void rules(); //shows rules
void results(int player,int cpu,int bet,int &money,int &draw,int &win,int &lose);
//finds winner

int main()
{
    int player=0,cpu=0,win=0,lose=0,draw=0,playerturns=0,cputurns=0,money=0,bet;
    char ans;
    system("cls"); //执行系统命令 cls
    rules();
    cout<<"\t\t\tWOULD YOU LIKE TO PLAY : ";
    cin>>ans;
    if((ans == 'y')||(ans=='Y')) //checks to see they want to play
    {
        cout<<"\t\t\tI WILL LET YOU START WITH $ 100"<<endl; //starts u with money
    }
}

```

```

    ★ ★ ★ ★ ★
    money=100;
    cout<<"\t\t\t";
    pause();
}
else
{
    return(0);
}
do{
    system("cls");
    if(money<0) //sees if your broke
    {
        cout<<"Your broke"<<endl; //sees if your broke
        return(0);
    }
    BET(bet,money); //calls bet fuction
    deal(player,cpu,playerturns,cputurns); //Deals teh cards
do{
    cout<<"\t\t\tWould You Like To Hit or Stay :"; //ask user if the want a hit
    cin>>ans;
    if((ans=='h')||(ans=='H'))
    {
        playerturns++; //adds one to players total cards
        if(playerturns>5) //sees if player can take a card
        {
            cout<<"\t\t\tYou Can't Have more than 5 cards";
        }
    }

    if((playerturns<6)&&(ans=='h')) //sees if the coumputer can take a card
    {
        cout<<endl;
        hit(player); //calls hit function
    }
} while((ans=='h')||(ans=='H')); //Takes value for hit or stay
for(;(cpu<16)&&(cputurns<6);cputurns++) //Tells the cpu wither to take a card or not
{
    cout<<endl;
    cout<<"\t\t\tThe Dealer Takes A Card"<<endl;
    hit(cpu); //calls hit function
}
cout<<endl;

```

```

    cout<<endl;
    cout<<"\t\t\tThe Dealer Has A Total:"<<cpu<<endl; //displays dealers total and then yours
    cout<<"\t\t\tYou Have A Total Of:"<<player<<endl;
    cout<<endl;
    results(player,cpu,bet,money,draw,win,lose); //send data to function to tell if you won our not
    replay(ans);
} while((ans=='y')||(ans=='Y'));
print(win,lose,draw,money);
cout<<endl;
cout<<"\t\t\t\t";
return(0);
}

//-----
void rules()
//This Function Displays the rules for Black Jack
{
    cout<<"\t\t WELCOME TO ISAAC SHAFFER'S BLACK JACK GAME"<<endl;
    cout<<"\t\t\t HERE ARE SOME SIMPLE RULES"<<endl;
    cout<<"\t\t\t1: You Can only have a max of 5 cards. "<<endl;
    cout<<"\t\t\t2: If you bust you automatically lose. "<<endl;
    cout<<"\t\t\t3: If you win you win double what you bet. "<<endl;
    cout<<"\t\t\t4: The Dealer stops at or after 16. "<<endl;
    cout<<"\t\t\t\t";
    pause();
}

//-----
void BET(int &bet,int &money)
//This Function finds out how much money they wish to bet;
{
    system("cls");
    cout<<"\t\t\tYOU HAVE : $ "<<money<<endl;
    cout<<"\t\t\tHow Much Do You Wish To Bet: ";
    cin>>bet;
    if(bet<0)
    {
        bet=bet*-1;
    }
    money=money-bet;
}

//-----
void deal(int &player,int &cpu,int &playerturns,int &cputurns)
//This Function Deals The hands
{

```



```

int playercard1,playercard2,cpucard1,cpucard2;
playerturns=2;
cputurns=2;
playercard1=random(13,1);
cout<<"\n\t\t\tDEALING HAND"<<endl;
wait(350);
playercard2=random(13,1);
wait(150);
cpucard1=random(13,1);

wait(350);
cpucard2=random(13,1);
if(playercard1>=10)
{
    playercard1=10;
}
if(playercard2>=10)
{
    playercard2=10;
}
if(cpucard1>=10)
{
    cpucard1=10;
}
if(cpucard2>=10)
{
    cpucard2=10;
}

player=playercard1+playercard2;
cpu=cpucard1+cpucard2;
cout<<"\t\t\tYOU HAVE a total of : "<<player<<endl;
cout<<"["<<playercard1<<"]";
cout<<"["<<playercard2<<"]";
cout<<endl;
cout<<"\t\t\tThe DEALER HAS A "<<cpucard1<<" SHOWING"<<endl;
cout<<endl;
cout<<"[ * ] "<<"["<<cpucard1<<"]";
}
//-----
void hit(int &total)
//This fuction is to deal a card and add it to the total
{
    int card;

```




```

        card=random(13,1);
        if(card>=10)
        {
            card=10;
        }
        total=total+card;
        cout<<"\t\t\tThe card is a : "<<card<<endl;
        cout<<"\t\t\tTotal is : "<<total<<endl;
    }
    //-----
    void results(int player,int cpu,int bet,int &money,int &draw,int &win,int &lose)
    //this function finds who wins
    {
        if(cpu==player)
        {
            cout<<"\t\t\tIT WAS A DRAW HOUSE WINS"<<endl;
            draw++;
        }
        if(player>21)
        {
            cout<<"\t\t\tYou Bust"<<endl;
            lose++;
        }
        else
        {
            if(cpu<player)
            {
                cout<<"\n\t\t\tYOU WIN";
                money=money+(bet*2);
                win++;
            }
        }
        if(cpu>21)
        {
            cout<<"\t\t\tDealer Bust"<<endl;

            if(player<21)
            {
                cout<<"\n\t\t\tYOU WIN";
                win++;
                money=money+(bet*2);
            }
        }
    }
}

```



```
else
{
    if(cpu>player)
    {
        cout<<"\t\t\tYOU LOSE"<<endl;
        lose++;
    }
}

//-----
void replay(char &ans)
{
    cout<<"\n\t\t\tWould You Like To Play This Game Again :";
    cin>>ans;
}

//-----
void print(int wins,int lose,int draw,int money)
{
    cout<<"\t\t\tWINS : "<<wins<<endl;
    cout<<"\t\t\tLOSE : "<<lose<<endl;
    cout<<"\t\t\tDRAWS : "<<draw<<endl;
    cout<<"\t\t\tMONEY : "<<money<<endl;
}
```

第 4 题 猜单词游戏

2.4.1 程序功能简介

一个猜单词游戏。由计算机或一方给出一个单词,另一方根据单词的个数猜出单词。

2.4.2 课程设计要求

(1) 根据所猜单词中非重复字母个数,给出 20% 的提示字母(任意),显示于单词中的正确位置。

(2) 完善界面,使程序运行的界面友好,以使用户操作,增加提示信息,保证每种程序的功能都能良好地运行。

(3) 可自行设计扩充本程序功能。

(4) 在主程序中,设计对各函数功能的测试程序。

2.4.3 课程设计说明

(1) 主程序提示:选择 1 则进入人人对战,选择 2 则进入人机对战,选择 3 退出。设计人人对战函数,人机对战函数。

(2) 设计用来判断猜题者所输入的字符是不是已经输入过的函数。

(3) 设计用来判断猜题者所输入的字母是不是正确单词 word 的组成部分的函数。

(4) 设计用来从文件 words.txt 中随机抽取一个单词的函数。

(5) 设计根据猜测次数相应的画一个人的头像,增加游戏的的可看性的函数。

(6) 评定难易级别:A 级。

2.4.4 关键源代码注解

```
// file: draw.h
// Counter class definition
#ifndef DRAW_H           // Check for class redefinition
#define DRAW_H
class draw
{
public:
    // Constructor
    draw();
    // Member functions
    void rope();           // Rope
    void head();           // Head
    void neck();           // Neck
    void leftarm();        // Left arm
    void rightarm();       // Right arm
```

```
        ★ ★ ★ ★ ★
void waisttop();    // Waist top
void leftleg();     // Left leg
void rightleg();    // Right leg
};

#endif // DRAW_H

// file: draw.cpp
// Draw class implementation
#include "draw.h"
#include <iostream>
using namespace std;
// Constructor
draw::draw(){}
// Functions
void draw::rope()    // rope
{    cout << "  |";  }
void draw::head()    // head
{    cout << "  O"; }
// neck
void draw::neck()
{    cout << "  |";  }
// left arm
void draw::leftarm()
{    cout << " /";   }
// right arm
void draw::rightarm()
{    cout << " \\";   }
// top waist
void draw::waisttop()
{    cout << " { }"; }
// left leg
void draw::leftleg()
{    cout << " /";   }
// right leg
void draw::rightleg()
{    cout << " \\";   }
// file: hangman.cpp
// name: Kyle Burmark  professor: M. Gelotte  date: 2/26/02
// Description: This is a computer version of the game hangman, the user
// can play against a 2nd player or the computer, and he can enter how many guesses he gets
// Start of program Header file definitions

#include <iostream>                // Standard input/output
```

```

#include <string>                // String manipulation
#include <cctype>                // Character manipulation and testing
#include <fstream>              // File stream
#include <cstdlib>              // Used for random function
#include <time.h>                // Used for better random number
#include "draw.h"                // Draws hangman

using namespace std;
// Function declarations
void instruction(int& choice);    // Gives instructions and gets choice
void usergame(int i);           // Plays 2nd user game
void compgame(int i);           // Plays against computer
void test(string word, char letter, int& numwrong, string& temp, int i);
/* Tests current letter and replaces starred word */
void lchosen(char letter, string& letterchosen, int& check, int& chosencounter, int i);
// Checks current letter and adds it to letters chosen output if not entered already
void rnd(string& word, int i);    // Gets random word from file
void drawman(int numguess, int numwrong, int i); // Draws hangman
inline istream& Flush(istream& stream); // Flushes cin stream

// Start of main
int main()
{
    int i = 0;                // Counter variable for loops
    int exit = 0;             // Main loop exit variable
    int choice;               // Users inputed choice for type of game or to exit
    // Main control loop
    do {                      // while exit != 1
        system("cls");        // 执行系统命令清屏
        instruction(choice);  // Give instructions
        switch(choice)
        {
            case 1:
                usergame(i);    // Calls user game
                break;
            case 2:
                compgame(i);    // Calls computer game
                break;
            case 3:
                cout << "Goodbye" << endl; // Exits
                exit = 1;
                break;
            default:

```



```

    cerr << "Invalid choice - try again" << endl;    // Invalid choice error
}
} while (exit != 1);
// End main loop
system("pause");    // 执行系统命令: 暂停
return 0;
}
// End main
// Subprograms
//-----
// Instructions subprogram

void instruction(int& choice)
{
    cout << "          - Hangman -" << endl << endl;
    cout << "          Created by Kyle Burmark" << endl << endl;
    cout << " * * * * * " << endl;
    cout << endl;
    cout << " Enter -1- to play against user" << endl;
    cout << " Enter -2- to play against computer" << endl;
    cout << " Enter -3- to quit" << endl;
    cout << endl;
    cout << " * * * * * " << endl << endl;
    cout << "Choice: ";    // Get user choice
    cin >> choice;
    // Bad data type check
    while (! cin)
    {
        cerr << "Invalid character" << endl
            << "Enter again - choice: ";
        Flush(cin);
        cin >> choice;
    }
    // End check
    system("cls");
}
// End instruction
// User game subprogram
void usergame(int i)
{
    int numguess = 0;    // Number of guesses player gets
    int numwrong = 0;    // Number of wrong guesses player has so far
    int check;    // A variable to test whether to check the letter entered in the test function

```

```

int wordcheck;    // A variable to check whether the user has entered an invalid word
int end = 0;      // A variable to test what to output to the user and to exit the usergame loop
int chosencounter = 0;

// A counter to tell the replace function where to put the letter chosen in the letterchosen string
char letter;      // User inputted letter
string word;      // Word user is trying to guess
string temp;      // Updated output word user sees
string letterchosen = "          ";

// Defines length of letterchosen string
do {              // while user puts in guesses outside of allowed range
    cout << "How many chances does the person have (4 - 10): ";
    cin >> numguess;
} while (numguess < 4 || numguess > 10);
cout << "Enter word 2nd user: ";
cin >> word;
do {              // while user inputs bad word
    wordcheck = 0;
    for (int i = 0; i < word.length(); i++)
    {
        if (! isalpha(word.at(i)))
        {
            wordcheck = 1;
        }
    }
    if (wordcheck == 1)
    {
        cout << "Invalid — Enter word again: ";
        cin >> word;
    }
} while (wordcheck == 1);
temp = word;      // Sets temp string length to word string length
// Replace temp with stars loop
for (i = 0; i < word.length(); i++)
{
    temp.replace(i, 1, 1, '*');
}
// End loop
system("cls");
// Main game loop
do {
    drawman(numguess, numwrong, i);
    // Tests if user guessed word
    if (word == temp)

```



```

    cout << endl << endl;
    cout << "You guessed it [ " << word << " ]" << endl << endl;
    system("pause");
    end = 1;
}

// Tests if user failed to guess word
if (numwrong == numguess)
{
    cout << endl << endl;
    cout << "You failed" << endl << endl;
    system("pause");
    end = 2;
}

// Play while above conditions aren't true
if (end == 0)
{
    cout << endl << endl << endl;
    cout << "Letters chosen: " << letterchosen << endl;
    cout << endl << endl << endl;
    cout << "Guesses left: " << numguess - numwrong << endl << endl;
    cout << "      " << temp << endl << endl;
    cout << "Letter: ";
    cin >> letter;
    // Test for bad letter
    while (! isalpha(letter))
    {
        Flush(cin);
        cout << "Not a letter — enter letter: ";
        cin >> letter;
    }
    // End test
    lchosen(letter, letterchosen, check, chosencounter, i);
    // Test whether to check letter against correct word string
    if (check == 0)    { test(word, letter, numwrong, temp, i);    }
    else    {    ;    }
    // End test
    system("cls");
}

// End game
system("cls");
} while(end != 1 && end != 2);
// End main game loop

```



```

// Tests end variable to see what to display
if (end == 2)
{
    cout << "Correct word was [ " << word << " ]" << endl << endl;
    system("pause");
}

if (end == 1)    {    cout << " ";    }
system("cls");
}

// End user game
// Start computer game
// See user game for variable and loop information
void compgame(int i)
{
    int numguess = 0;
    int numwrong = 0;
    int check;
    int end = 0;
    int chosencounter = 0;
    char letter;
    string word;
    string temp;
    string letterchosen = "          ";
    do {
        cout << "How many chances do you want (4 - 10): ";
        cin >> numguess;
    } while (numguess < 4 || numguess > 10);
    rnd(word, i);    // Gets random word
    temp = word;
    for (i = 0; i < word.length(); i++)    {    temp.replace(i, 1, 1, '*');    }
    system("cls");
    do {
        drawman(numguess, numwrong, i);
        if (word == temp)
        {
            cout << endl << endl;
            cout << "You guessed it [ " << word << " ]" << endl << endl;
            system("pause");
            end = 1;
        }
        if (numwrong == numguess)
        {
            cout << endl << endl;

```

```

    ★ ★ ★ ★ ★
    cout << "You failed" << endl << endl;
    system("pause");
    end = 2;
}
if (end == 0)
{
    cout << endl << endl << endl;
    cout << "Letters chosen: " << letterchosen << endl;
    cout << endl << endl << endl;
    cout << "Guesses left: " << numguess - numwrong << endl << endl;
    cout << "      " << temp << endl << endl;
    cout << "Letter: ";
    cin >> letter;
    while (! isalpha(letter))
    {
        Flush(cin);
        cout << "Not a letter — enter letter: ";
        cin >> letter;
    }

    lchosen(letter, letterchosen, check, chosencounter, i);
    if (check == 0)
    {
        test(word, letter, numwrong, temp, i);
    }
    else {      ;      }
    system("cls");
}
system("cls");
} while(end != 1 && end != 2);
if (end == 2)
    cout << "Correct word was [ " << word << " ]" << endl << endl;
    system("pause");
if (end == 1)    cout << endl;
system("cls");
}

// End computer game
// Checks current letter chosen by user
void lchosen(char letter, string& letterchosen, int& check, int& chosencounter, int i)
{
    check = 0;
    // Check if letter is equal to any letter in letterchosen string loop
    for (i = 0; i < letterchosen.length(); i++)

```

```

    {
        if (letter == letterchosen.at(i)) {check = 1; }
    }
    // End loop
    // Test if letter is already chosen
    if (check == 1)
    {
        cout << endl;
        cout << "Letter already chosen" << endl;
        system("pause");
    }
    // Puts letter in string if not chosen
    else
    {
        letterchosen.replace(chosencounter, 1, 1, letter);
        chosencounter++;
    }
}
// End chosen function
// Tests whether letter is in word string
void test(string word, char letter, int& numwrong, string& temp, int i)
{
    int check2 = 0; // Checks whether letter is in word string, = 1 if it is
    // Check for letter match loop
    for (i = 0; i < word.length(); i++)
    {
        if (letter == word.at(i))
        {
            temp.replace(i, 1, 1, letter);
            check2 = 1;
        }
    }
    // End loop
    // Displays "wrong letter" if user inputed bad letter
    if (check2 == 0)
    {
        cout << endl;
        cout << "Wrong letter" << endl;
        system("pause");
        numwrong++;
    }
}
// End test function

```

```

// Gets random word
void rnd(string& word, int i)
{
    int x;                // Random number to pass to word file loop
    ifstream ins;          // File stream
    srand(time(NULL));     // Gets better random number based on time
    x = rand()%100;        // Gets number 0 - 99
    ins.open("words.txt"); // Opens random word file
    // Tests for bad file and gets word if not a bad file
    if (ins.fail())
    {
        cerr << "Words.txt is not in same folder as hangman.exe, " << endl
              << "put in correct file and run again and make sure it's " << endl
              << "called words.txt" << endl;
        system("pause");
        main();
    }
    else
    {
        for (i = 0; i < (x + 1); i++) {      getline(ins, word);      }
    }
    // End test
    ins.close(); // Close file
}

// End random function
// Draws hangman
void drawman(int numguess, int numwrong, int i)
{
    draw d; // Data object
    // Draw loop based on number of guesses and number of wrong guesses
    for (i = 0; i <= numwrong; i++)
    {
        if (numguess == 4)
        {
            switch(i)
            {
                case 1: d.rope();    cout << endl;    d.head();
                        cout << endl;    d.neck();    break;
                case 2:    cout << endl;    d.leftarm();
                        d.rightarm();    break;
                case 3: cout << endl; d.waisttop();    break;
                case 4: cout << endl;    d.leftleg();    d.rightleg();
                        cout << endl << endl << "Dead" << endl;    break;
            }
        }
    }
}
```

```

    }
}
else if (numguess == 5)
{
    switch(i)
    {
        case 1:    d. rope();    break;
        case 2:    cout << endl; d. head(); cout << endl; d. neck(); break;
        case 3:    cout << endl; d. leftarm(); d. rightarm(); break;
        case 4:    cout << endl; d. waisttop(); break;
        case 5:    cout << endl; d. leftleg(); d. rightleg();
                    cout << endl << endl << "Dead" << endl; break;
    }
}
else if (numguess == 6)
{
    switch(i)
    {
        case 1:    d. rope();    break;
        case 2:    cout << endl; d. head(); cout << endl; d. neck(); break;
        case 3:    cout << endl; d. leftarm(); d. rightarm(); break;
        case 4:    cout << endl; d. waisttop(); break;
        case 5:    cout << endl; d. leftleg(); break;
        case 6:    d. rightleg(); cout << endl << "Dead" << endl; break;
    }
}
else if (numguess == 7)
{
    switch(i)
    {
        case 1:    d. rope();    break;
        case 2:    cout << endl; d. head(); cout << endl; d. neck(); break;
        case 3:    cout << endl; d. leftarm(); break;
        case 4:    d. rightarm(); break;
        case 5:    cout << endl; d. waisttop(); break;
        case 6:    cout << endl; d. leftleg(); break;
        case 7:    d. rightleg(); cout << endl << "Dead" << endl; break;
    }
}
else if (numguess == 8)
{
    switch(i)
    {

```

✧ ✧ ✧ ✧ ✧

```

case 1:    d. rope();    break;
case 2:    cout << endl;    d. head();    break;
case 3:    cout << endl;    d. neck();    break;
case 4:    cout << endl;    d. leftarm();    break;
case 5:    d. rightarm();    break;
case 6:    cout << endl;    d. waisttop();    break;
case 7:    cout << endl;    d. leftleg();    break;
case 8:    d. rightleg();    cout << endl << "Dead" << endl;    break;
}
}

else if (numguess == 9)
{
    switch(i)
    {
case 1:    d. rope();    break;
case 2:    cout << endl;    d. head();    break;
case 3:    cout << endl;    d. neck();    break;
case 4:    cout << endl;    d. leftarm();    break;
case 5:    d. rightarm();    break;
case 6:    cout << endl;    d. waisttop();    break;
case 7:    break;
case 8:    cout << endl;    d. leftleg();    break;
case 9:    d. rightleg();    cout << endl << "Dead" << endl;    break;
    }
}

else if (numguess == 10)
{
    switch(i)
    {
case 1:    d. rope();    break;
case 2:    cout << endl;    d. head();    break;
case 3:    cout << endl;    d. neck();    break;
case 4:    cout << endl;    d. leftarm();    break;
case 5:    d. rightarm();    break;
case 6:    cout << endl;    d. waisttop();    break;
case 7:    break;
case 8:    cout << endl;    d. leftleg();    break;
case 9:    d. rightleg();
cout << "One last chance. What do you want on your tombstone?";    break;
case 10:    cout << endl << "Dead" << endl;    break;
    }
}
}

```



```
// End draw loop
}
// End draw function
// Flushes cin stream
inline istream& Flush(istream& stream)
{
    stream.clear();
    int chars_to_skip = stream.rdbuf() ->in_avail();
    return stream.ignore(chars_to_skip);
}
// End flush function
// End subprograms
// End program
```

第 5 题 福彩游戏

2.5.1 程序功能简介

这是一个类似于福彩的游戏程序。游戏者输入自己的名字和吉祥数字,计算机由此初始化随机发生器,然后游戏者选定游戏难易程度,由此确定数字范围,游戏者输入范围内的 6 个数字,程序判断输入是否有效。若无效,发出错误信息并退出;若有效,则和计算机产生的该范围之内的 6 个数字比较,若全部匹配(不考虑顺序),则游戏者赢,否则输。

2.5.2 课程设计要求

- (1) 此游戏者不可能赢,分析其原因,并修改程序使游戏者有可能赢。
- (2) 扩充游戏规则,让游戏者可选,并设计出相应的函数。
- (3) 可以选择游戏的难易程度(输入 h 为难,输入 e 为易),并设计出相应的函数。
- (4) 将所有的函数与相应的数据封装到类中。
- (5) 改写主程序,使程序简化。

2.5.3 课程设计说明

- (1) 在计算输赢的总和中漏加了 `truematch[2]`,使 `total` 的值必小于 6,所以游戏者必输。
- (2) 程序定义时 `num` 只有 4 个元素,但是使用时却出现了 `num[5]`,会发生溢出,导致错误。
- (3) 修改后在求 `total` 的式子中加入了 `truematch[2]`,并将 `num[5]` 改为 `PowerBall`。
- (4) 函数 `srand()` 是随机函数。
- (5) 评定难易级别:B 级。

2.5.4 关键源代码注解

```
//brought to you by the Red Deer College Code Junkies.
```

```
//included libraries necessary at runtime
```

```
#include <stdlib. h>
```

```
#include <time. h>
```

```
#include <stdio. h>
```

```
#include <cmath>
```

```
#include <iomanip. h>
```

```
#include <iostream. h>
```

```
//calling the various functions used in the program
```

```
int isMatch(int test, int cp_num[] );
```

```
int getgamenbr();
```

```
int menu();
```

```
int j();
```



```

int main()
{

    //defining the variables used in the program
    char name[20], Reply2;
    int seed, PowerBall, total, play=1;
    int truematch[6];
    int menu();

    cout<<"//-----\n";
    cout<<" * Welcome to PowerBall! * \n";
    cout<<"-----\n";
    cout<<"          -by Chris Grose\n\n";

    /* getting user input for name. Will store in an array, and will be called throughout the program */
    cout<<"Please enter your name [1 to 20 characters]: ";
    cin.getline (name,20);

    //welcoming the user with a personal touch!
    cout<<"\n Welcome "<<name<<"! Please enter your lucky number for seed: ";
    cin>>seed;

    /* setting the random variable when generating random numbers [depending on what seed the user enters] */
    srand(seed);          // srand(seed)函数根据 seed 的值(称为种子)产生随机数
    do
    {
        //users inputted whiteball numbers
        int num [5];
        //computers inputted whiteball numbers
        int cp_num [6];

        cout <<"\n"<<name<<" , please enter your white ball numbers.\n";
        //user input, declared
        cout <<"\n";
        //getting five inputs for the users whiteball numbers. stored in the array called 'num'
        int j;
        for (j=0; j<5; j++)
        {
            cout<<"Enter Ball # "<<j+1<<": ";
            while (! (cin >> num[j]))
            {
                //error messages if the user tries to enter something invalid when prompted
            }
        }
    }
}

```

```

cin.clear();
while (! (cin.get() != '\n'))
    continue;
cout << "Sorry, that is an invalid entry. Please enter a number between 1 and 48. Thank you! \n";
}

if ((num[j]<1))
{
    cout << "Sorry, you have entered a number less than 0. Please enter a number between 1 and 48. \n ";
    j--;
}

if ((num[j]>48))
{
    cout << "Sorry, you have entered a number greater than 48. Please enter a number between 1
and 48. \n";
    j--;
}
}

cout << "\n";
//getting the users choice for the powerball
cout << name << ", please enter your PowerBall number: ";
cin >> PowerBall;

/* getting random whiteball numbers for the computers choices [calling from the 'getgamenbr' function] */
cp_num[0]= getgamenbr();
cp_num[1]= getgamenbr();
cp_num[2]= getgamenbr();
cp_num[3]= getgamenbr();
cp_num[4]= getgamenbr();
cp_num[5]= getgamenbr();

/* calculating if the computers whiteball choices match the users choices and displays 0 if there is no
match, and 1 if there is a match */
truematch[0]= isMatch(num[0],cp_num);
truematch[1]= isMatch(num[1],cp_num);
truematch[2]= isMatch(num[2],cp_num);
truematch[3]= isMatch(num[3],cp_num);
truematch[4]= isMatch(num[4],cp_num);
truematch[5]= isMatch(num[5],cp_num);

// displaying the results of the game in a visually appealing manner

```

```

cout<<"\n * * * * * \n";
cout <<"\nPlay # "<<play;
cout<<"\n * * * * * \n";
cout <<"\n";
//the computers whiteball numbers
cout <<"The computers picks were: "<<cp_num[0]<<"    "<<cp_num[1]
    <<"    "<<cp_num[2]<<"    "<<cp_num[3]<<"    "<<cp_num[4]
    <<"    "<<"PB"<<"    "<<cp_num[5]<<"\n";

cout <<"-----\n";
//the users whiteball numbers
cout <<name<<" , your picks were:  "<<num[0]<<"    "<<num[1]
    <<"    "<<num[2]<<"    "<<num[3]<<"    "<<num[4]<<"    "<<"PB
"<<"    "<<"PowerBall"<<"\n";
cout <<"-----\n";
//showing if any of the numbers [computers vs users] match
cout <<name<<" , your matches were: "<<truematch[0]<<"    "<<truematch[1]<<"    "
<< truematch[2]<<"    "<<truematch[3]<<"    "
    <<truematch[4]<<"    "<<"PB"<<"    "<<truematch[5]<<"\n";
cout <<"-----\n";
//adds up total matches
total=(truematch[0] + truematch[1] + truematch[3] + truematch[4] + truematch[5]);
//displays how many numbers matched
cout <<"\nTotal Matches: "<<total<<"\n\n";
//showing the appropriate message depending on how many numbers were matched [if all 6
//match, you win. otherwise, you lose!]
if (total==6)
{
    cout <<"Congratulations "<<name<<"! You won the jackpot!!";
}
else
    cout <<"Sorry "<<name<<" , you did not win the jackpot!";
cout<<"\n * * * * * \n";
//asking the user if he/she would like to play the game again
cout<<name<<" , would you like to play the game again? [y or n]"<<endl;
cin>> Reply2;
play=play + 1;
//exiting the program if user response is 'n' to play again
}while (Reply2=='y' || Reply2=='Y');
cout<<"\nThank you for playing PowerBall "<<name<<"! :-)"<<endl;
    exit (1);
return 0;
}

```



```
int getgamenbr()
{
    //function to return a random number between 1 and 48
    return (rand() % 48) + 1;
}

int isMatch(int test, int cp_num[] )
{
    //calculating if there are any matches
    int match = 0;
    //loop to test all numbers
    for (int i=0; i<5; i++)
        if (test == cp_num[i]) match = 1;
    cout << "\n";
    return match;
}

int j()
{
    int n = 0;

    if (j == j) n=1;
    cout<< "\n";
    cout<< "Only enter a number once. ";
    return j();
}
```

第 6 题 对抗游戏

2.6.1 程序功能简介

一个对抗游戏,根据随机产生的数字决定双方的输赢。

2.6.2 课程设计要求

(1) 当双方进行对抗时,玩家可根据自身的牌点决定赢牌的把握,例如拿到 A,则把握增大,反映在分数上每次输赢不是 1 分。

(2) 将双方牌点的比较扩大为两张牌点之和,若出现一组对子,则对子牌点大于任何单牌,若双方均为对子,则比较牌点之和。

(3) 要求自行设计不少于 5 种以上的对抗规则,用于游戏中。

(4) 在主程序中,设计程序运行的友好界面,以使用户操作,并增加提示信息。

2.6.3 课程设计说明

(1) 花色比较的顺序是:红桃>方块>梅花>黑桃。

(2) 首先根据牌点比较,如果产生一对对子,则对子牌点大于任何单牌。

(3) 如果双方都是对子,则比较牌点,大的获胜,若牌点相同,则比较各牌花色。

(4) 如果双方都不是对子,则看哪方是顺子,若都有顺子,则比较最大牌值,牌大者胜。

(5) 如果双方都不是对子,若都有顺子,且最大牌值相等,则依次比较两者花色最大点数。

(6) 如果双方都不是对子,也没有顺子,则比较牌点之和。

(7) 如果双方都不是对子,也没有顺子,且牌点之和相等,则依次比较牌点,大者获胜。

(8) 如果双方都不是对子,也没有顺子,且牌点之和,最大牌点相等,则依次比较花色大小。

(9) 如果双方牌点与花色完全相同时,为平局。

(10) 评定难易级别:B 级。

2.6.4 关键源代码注解

```
#include <iostream.h>
#include <stdlib.h>
#include <time.h>
#define MAX 13      //牌的最大点为 13
class War
{
    int plyr, comp;    //plyr 是玩家的牌点, comp 是计算机的牌点
    static int Score;  //玩家的分数,当分数为 0 或 50 时游戏结束
    static int Loop;   //出牌次数,游戏结束时给出总的出牌次数
public:
    int Player();      //返回玩家的牌点
```



```

int Computer();    //返回计算机的牌点
void Initialize();  //给出玩家和计算机的牌点
int CheckScore();  //返回玩家分数
void GameOver();
War();             //缺省构造函数,初始化对象
static int Increase();    //分数增加
static int Decrease();    //分数减少
static int Loops();       //出牌次数增加
};

int War::Score = 26;      //初始分数
int War::Loop = 0;        //比赛轮数
int War::Increase()       //增分,每轮赢一分
{
    ++Score;
    return Score;
}

int War::Decrease()       //减分,每轮输一分
{
    --Score;
    return Score;
}

int War::Loops()          //双方出牌一次,为一轮
{
    Loop++;
    return Loop;
}

War::War()                //构造函数,初始双方牌点为 0
{
    plyr, comp = 0;
}

int War::Player()         //返回玩家牌点
{
    return plyr;
}

int War::Computer()       //返回计算机的牌点
{
    return comp;
}

void War::Initialize()     //随机产生双方牌点
{
    srand ( time (NULL) );
    plyr = rand()%MAX+2;    //牌点为 2~14,14 点为 A

```

```

        comp = rand()%MAX+2;
    }

    int War::CheckScore()    //返回玩家分数
    {

        return Score;
    }

    void War::GameOver()    //游戏结束时的相关信息
    {

        cout << endl << "游戏结束! 谢谢参与." << endl;
        cout << "你用了 " << War::Loops() << " 轮." << endl;
    }

    void main()
    {

        War Play;
        int x;
        cout << endl << "准备开始!" << endl << endl;

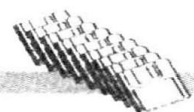
        Play.Initialize();    //随机产生双方牌点
        Play.Loops();        //轮次增加

        cout << "你的牌是一张";
        if(Play.Player() == 11)
            cout << "J" << endl;
        else if(Play.Player() == 12)
            cout << "Q" << endl;
        else if (Play.Player() == 13)
            cout << "K" << endl;
        else if (Play.Player() == 14)
            cout << "A" << endl;
        else
            cout << Play.Player() << endl;
        cout << "计算机有一张";
        if(Play.Computer() == 11)
            cout << "J" << endl;
        else if(Play.Computer() == 12)
            cout << "Q" << endl;
        else if (Play.Computer() == 13)
            cout << "K" << endl;
        else if (Play.Computer() == 14)
            cout << "A" << endl;
        else
            cout << Play.Computer() << endl;
        if (Play.Player() > Play.Computer())    //判断双方牌点的大小,决定分数的增减

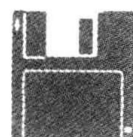
```



```
    Play.Increase();
else if (Play.Player() < Play.Computer())
    Play.Decrease();
else
    cout << "势均力敌!" << endl;
Play.CheckScore();    //返回分数
cout << endl << "你的分数是：" << Play.CheckScore() << endl;
switch (Play.CheckScore())
{
    case 0:
    case 52:
        Play.GameOver(); break;    //分数为 0 或 52 则游戏结束
    default:
        cout << endl << "继续 1 [退出 0]" << endl;
        //选择游戏是否继续进行
        cin >> x;
        if (x == 1)    main();    //若是递归调用主函数
        //否则退出系统
        break;
}
```

3 数学问题求解程序



第 1 题 计算器

3.1.1 程序功能简介

实现计算器的功能。用户根据程序提示,输入数字,选择要进行的运算,可以进行加、减、乘、除等运算。

3.1.2 课程设计要求

- (1) 完善计算器程序,改为可对实数操作。
- (2) 完善程序,改为多个操作数基本四则运算,遇到 0 为止。
- (3) 增加函数,完成四则混合运算,增加相应的主菜单选项。
- (4) 添加语句,使四则运算具有测试功能。
- (5) 可扩充其功能。

3.1.3 课程设计说明

- (1) 设计一面向对象的类,含有加、减、乘、除运算及开平方等功能的函数。
- (2) 设计加、减、乘、除运算及求平方根的函数。
- (3) 设计实现四则混合运算的函数。
- (4) 设计测试函数。
- (5) 评定难易级别:B 级。

3.1.4 关键源代码注解

```
/* This is a simple OOP Calculator that i made because i wanted to learn how to use classes so i made an OOP
   calc. anyways i was gonna use inline asm for the add, sub, div, mult functions */
#include <process.h>
#include <iostream.h>
#include <conio.h>
#include <stdlib.h>
#include <math.h>
int number1,number2,answer,val;
class oopcalc {
public:
void calcadd();
void calesub();
void calediv();
void calcemult();
void calcfartocel();
void calcceltofar();
void calcsroot();
```

```

void exitprog();
void menu();
void badinput();
private:
int add(int x, int y);
int sub(int x, int y);
int div(int x, int y);
int mult(int x, int y);
int fartocel(int x);
int celtofar(int x);
int sqroot(int x);
};
void oopcalc::calcadd()
{
cout << "The Add Function\n";
cout << "First number: ";
cin >> number1;
cout << "Second number: ";
cin >> number2;
answer = add(number1, number2);
cout << number1 << " + " << number2 << " = " << answer << endl;
cout << "Press any key to continue\n";
getch();
menu();
}
void oopcalc::calcsb()
{
cout << "The Subtract Function\n";
cout << "First number: ";
cin >> number1;
cout << "Second number: ";
cin >> number2;
answer = sub(number1, number2);
cout << number1 << " - " << number2 << " = " << answer << endl;
cout << "Press any key to continue\n";
getch();
menu();
}
void oopcalc::calcdv()
{
cout << "The Divide Function\n";
cout << "First number: ";
cin >> number1;

```

```

    cout << "Second number: ";
    cin >> number2;
    answer = div(number1, number2);
    cout << number1 << " / " << number2 << " = " << answer << endl;
    cout << "Press any key to continue\n";
    getch();
    menu();
}

void oopcalc::calcmult()
{
    cout << "The Multiply Function\n";
    cout << "First number: ";
    cin >> number1;
    cout << "Second number: ";
    cin >> number2;
    answer = mult(number1, number2);
    cout << number1 << " x " << number2 << " = " << answer << endl;
    cout << "Press any key to continue\n";
    getch();
    menu();
}

void oopcalc::calc fartocel()           // 计算 Farenheit 到 Celsius
{
    cout << "The Farenheit to Celsius Function\n";
    cout << "Enter a tempature in Farenheit: ";
    cin >> number1;
    answer = fartocel(number1);
    cout << "The tempature in Celsius is " << answer << endl;
    cout << "Press any key to continue\n";
    getch();
    menu();
}

void oopcalc::calcceltofar()           // 计算 Celsius 到 Farenheit
{
    cout << "The Celsius to Farenheit Function\n";
    cout << "Enter a tempature in Celsius: ";
    cin >> number1;
    answer = celtofar(number1);
    cout << "The tempature in Farenheit is " << answer << endl;
    cout << "Press any key to continue\n";
    getch();
    menu();
}

```



```

void oopcalc::calcsroot()
{
    cout << "The Square Root Function\n";
    cout << "First number: ";
    cin >> number1;
    answer = sqrt(number1);
    cout << "The square root of " << number1 << " is " << answer << endl;
    cout << "Press any key to continue\n";
    getch();
    menu();
}

void oopcalc::exitprog()
{
    exit(-1);
}

void oopcalc::menu()
{
    char input;
    oopcalc a;
    system("cls"); //执行系统命令:cls—清屏
    cout << "=====MENU=====\\n";
    cout << "1: Add two numbers\\n";
    cout << "2: Subtract two numbers\\n";
    cout << "3: Divide two numbers\\n";
    cout << "4: Multiply two numbers\\n";
    cout << "5: Convert Farenheit to Celsius\\n";
    cout << "6: Convert Celsius to Farenheit\\n";
    cout << "7: Find the square root of a number\\n";
    cout << "8: Exit this program\\n";
    cout << "Choice: ";
    cin >> input;
    cout << "=====\\n";
    switch (input)
    {
        case '1':    a. calcadd();
                    break;
        case '2':    a. calcsbub();
                    break;
        case '3':    a. calcddiv();
                    break;
        case '4':    a. calcmult();
                    break;
        case '5':    a. calcfartocel();
    }
}

```



```

        break;
    case '6':    a. calcceltofar();
        break;
    case '7':    a. calcsroot();
        break;
    case '8':    a. exitprog();
        break;
    default:    a. badinput();
}
}
void oopcalc::badinput()
{
    cout << "BAD INPUT! \n";
    cout << "Press any key to continue\n";
    getch();
    menu();
}
int oopcalc::add(int x, int y)
{
    val = x + y;
    return val;
}
int oopcalc::sub(int x, int y)
{
    val = x - y;
    return val;
}
int oopcalc::div(int x, int y)
{
    val = x / y;
    return val;
}
int oopcalc::mult(int x, int y)
{
    val = x * y;
    return val;
}
int oopcalc::fartocel(int x)
{
    int cel = ((x - 32) * 5) / 9;
    return cel;
}
int oopcalc::celtofar(int x)

```

✧ ✧ ✧ ✧ ✧

```
{
int f;
f = x * 9 / 5 + 32;
return f;
}
int oopcalc::sqroot(int x)
{
int g = sqrt(x);
return g;
}
void main()
{
oopcalc s;
s.menu();
}
```


第2题 分数计算器

3.2.1 程序功能简介

分数计算及测试程序,具有对输入的运算进行“+,-,*,/,^”的功能。

3.2.2 课程设计要求

- (1) 将主程序设计成一个分数计算器。
- (2) 可完成“+,-,*,/,^”等基本运算。
- (3) 可将分数化为十进制小数和带分数。
- (4) 可执行“+,-,*,/,^”(含括号的)四则运算。

3.2.3 课程设计说明

- (1) 编写程序时,建议用两个数组分别来存放相邻的输入数据。
- (2) 每个运算符的前后可能出现的其他运算符,应该作相应的处理。
- (3) 四则运算要符合一定的格式,例如: $1/2+2/3*3/4+(1/2+2/3*3/4)+1/2^2/1\#$
其中约定:① 符号‘#’用于表示输入结束
② 符号‘^’为乘方符号。
- (4) 评定难易级别:A级。

3.2.4 关键源代码注解

```
/* Rick Woods   Last edited: 2/14/02   Fractions class — header file — 分数类头文件 */

#ifndef _TGR_FRACTIONS_H
#define _TGR_FRACTIONS_H

#include <iostream.h>
#include <conio.h>
#include <stdlib.h>
#include <math.h>
// #include <bool.h>
#include <string.h>
class fraction    //分数类定义
{
public:
    fraction();    //constructors
    fraction(int initn, int initd);
    //accessors
    int numerator() const;    //取分子,const 成员函数,不能修改数据成员
    int denominator() const;    //取分母
```

```

double decimal() const;      //将分数转换为对应的小数
fraction absval() const;     //分数的绝对值
//modifiers
void setnum(int newnum);     //设置分子
void setden(int newden);     //设置分母
void simplify();             //约分
void recip();                //分子分母转置
const fraction & operator+= (int rhs);
private:
    int n;
    int d;
};
//free (nonmember) functions
//miscellany
int GCF(int x, int y); //求 x,y 的最大公因子
//addition operators
fraction operator+ (const fraction &lhs, const fraction &rhs);
fraction operator+ (const fraction &lhs, int rhs);
//subtraction operators
fraction operator- (const fraction &lhs, const fraction &rhs);
fraction operator- (const fraction &lhs, int rhs);
//multiplication operators
fraction operator* (const fraction &lhs, const fraction &rhs);
fraction operator* (const fraction &lhs, int rhs);
//division operators
fraction operator/ (const fraction &lhs, const fraction &rhs);
fraction operator/ (const fraction &lhs, int rhs);
//power operator
fraction operator^ (const fraction &base, int exp); //^(幂)运算符重载
//comparison operators
bool operator== (const fraction &lhs, const fraction &rhs);
bool operator!= (const fraction &lhs, const fraction &rhs);
bool operator>= (const fraction &lhs, const fraction &rhs);
bool operator<= (const fraction &lhs, const fraction &rhs);
bool operator> (const fraction &lhs, const fraction &rhs);
bool operator< (const fraction &lhs, const fraction &rhs);
bool operator== (const fraction &lhs, int rhs);
bool operator!= (const fraction &lhs, int rhs);
bool operator>= (const fraction &lhs, int rhs);
bool operator<= (const fraction &lhs, int rhs);
bool operator> (const fraction &lhs, int rhs);
bool operator< (const fraction &lhs, int rhs);
// I/O operators/functions

```

```
ostream & operator<< (ostream &os, const fraction &f); //输出运算符重载
istream & operator>> (istream &is, fraction &f); //输入运算符重载
void PrintAsMixed(ostream &os, const fraction &f);
```

```
#endif
```

```
/* fraction.cpp
```

```
Rick Woods Last edited: 2/14/02
```

```
Fractions class — implementation file — 分数类执行文件开始 */
```

```
#include "fraction.h"
```

```
fraction::fraction() : n(0), d(1) {} //缺省构造函数(=0/1)
```

```
fraction::fraction(int initn, int initd) : n(initn), d(initd) {}
```

```
int fraction::numerator() const //取分子
```

```
{
```

```
    return n;
```

```
}
```

```
int fraction::denominator() const
```

```
{
```

```
    return d;
```

```
}
```

```
double fraction::decimal() const
```

```
{
```

```
    return double(n) / double(d);
```

```
}
```

```
fraction fraction::absval() const
```

```
{
```

```
    fraction returnValue(abs(n), abs(d));
```

```
    return returnValue;
```

```
}
```

```
void fraction::setnum(int newnum)
```

```
{
```

```
    n = newnum;
```

```
}
```

```
void fraction::setden(int newden)
```

```
{
```

```
    if(newden != 0)
```

```
        d = newden;
```

```
    else
```

```
{
```

```
        cout << "Error: Attempted to set denominator to zero! \n"
```

```
        << "Press any key to terminate";
```

```
        getch();
```

✧ ✧ ✧ ✧ ✧

```

    abort();
}
}
void fraction::simplify()
{
    int r = GCF(n, d);
    n /= r;
    d /= r;
}
void fraction::recip()
{
    int temp = n;
    n = d;
    d = temp;
}

const fraction & fraction::operator+= (int rhs)
{
    fraction newrhs(rhs, 1);
    return (*this + newrhs);
}

int GCF(int x, int y)    // 辗转相除, 求 x,y 的最大公约数
{
    int r = x%y;
    if(r==0)
        return y;
    else
        return GCF(y, r);
}

fraction operator+ (const fraction &lhs, const fraction &rhs)
{
    fraction sum;
    int newnum = (lhs.numerator() * rhs.denominator())
                + (rhs.numerator() * lhs.denominator());
    int newden = (lhs.denominator() * rhs.denominator());
    sum.setnum(newnum);
    sum.setden(newden);
    sum.simplify();
    return sum;
}

fraction operator+ (const fraction &lhs, int rhs)

```

```
{
    fraction newrhs(rhs, 1);
    return lhs + newrhs;
}

fraction operator- (const fraction &lhs, const fraction &rhs)
{
    fraction newrhs = rhs;
    newrhs.setnum(-rhs.numerator());
    return lhs + newrhs;
}

fraction operator- (const fraction &lhs, int rhs)
{
    fraction newrhs(rhs, -1);
    return lhs + newrhs;
}

fraction operator* (const fraction &lhs, const fraction &rhs)
{
    fraction product;

    int newnum = lhs.numerator() * rhs.numerator();
    int newden = lhs.denominator() * rhs.denominator();
    product.setnum(newnum);
    product.setden(newden);
    product.simplify();

    return product;
}

fraction operator* (const fraction &lhs, int rhs)
{
    fraction prod(lhs.numerator() * rhs, lhs.denominator());
    prod.simplify();
    return prod;
}

fraction operator/ (const fraction &lhs, const fraction &rhs)
{
    if(rhs == 0)
    {
        cout << "Error: Division by zero attempted! \n"
              << "Press any key to terminate";
    }
}
```

```

    getch();
    abort();
}

fraction newrhs = rhs;
    newrhs.recip();
    return lhs * newrhs;
}

fraction operator/ (const fraction &lhs, int rhs)
{
    if(rhs == 0)
    {
        cout << "Error: Division by zero attempted! \n"
            << "Press any key to terminate";
        getch();
        abort();
    }
    fraction quot(lhs.numerator(), lhs.denominator() * rhs);
    quot.simplify();
    return quot;
}

fraction operator^ (const fraction &base, int exp)
{
    fraction returnValue;
    returnValue.setnum(pow(base.numerator(), exp));
    returnValue.setden(pow(base.denominator(), exp));
    return returnValue;
}

bool operator== (const fraction &lhs, const fraction &rhs)
{
    return lhs.decimal() == rhs.decimal();
}

bool operator!= (const fraction &lhs, const fraction &rhs)
{
    return lhs.decimal() != rhs.decimal();
}

bool operator>= (const fraction &lhs, const fraction &rhs)
{

```

```
    return lhs.decimal() >= rhs.decimal();
}

bool operator<= (const fraction &lhs, const fraction &rhs)
{
    return lhs.decimal() <= rhs.decimal();
}

bool operator> (const fraction &lhs, const fraction &rhs)
{
    return lhs.decimal() > rhs.decimal();
}

bool operator< (const fraction &lhs, const fraction &rhs)
{
    return lhs.decimal() < rhs.decimal();
}

bool operator== (const fraction &lhs, int rhs)
{
    return lhs.decimal() == rhs;
}

bool operator!= (const fraction &lhs, int rhs)
{
    return lhs.decimal() != rhs;
}

bool operator>= (const fraction &lhs, int rhs)
{
    return lhs.decimal() >= rhs;
}

bool operator<= (const fraction &lhs, int rhs)
{
    return lhs.decimal() <= rhs;
}

bool operator> (const fraction &lhs, int rhs)
{
    return lhs.decimal() > rhs;
}
```

★★★★★

```
bool operator< (const fraction &lhs, int rhs)
```

```
{
    return lhs.decimal() == rhs;
}
```

```
ostream & operator<< (ostream &os, const fraction &f)
```

```
{
    fraction temp = f.absval();
    int numer = f.numerator();
    int denom = f.denominator();
    if((! ((numer > 0 && denom > 0)           // This extremely complex and seemingly
        || (numer < 0 && denom < 0)))        // useless if statement tests to see if
        && f != 0)                            // fraction f is negative
    os << '-';
    if(temp.numerator() == 0 || temp.denominator() == 1)
        os << temp.numerator();
    else
        os << temp.numerator() << '/' << temp.denominator();
    return os;
}
```

```
istream & operator>> (istream &is, fraction &f)    //is 是输入流, f 是待输入的分数
```

```
{
    int newnum, newden;
    char slash = '/';
    do
    {
        is >> newnum >> slash >> newden;
    } while(slash != '/');
    f.setnum(newnum);
    f.setden(newden);
    return is;
}
```

//将分数 f 按整数,真分数格式输出到流 os

```
void PrintAsMixed(ostream &os, const fraction &f)
```

```
{
    int wholePart = f.numerator() / f.denominator();
    fraction fracPart = f - wholePart;

    if(wholePart != 0)
        cout << wholePart << " ";
    if(fracPart != 0)
```



```

    cout << fracPart;
}

// Fractions class — implementation file——分数类执行文件结束

//分数计算测试程序
#include <iostream. h>
#include <conio. h>
#include "fraction. h"
int main()
{
    fraction f1;
    fraction f2; //定义两个分数 f1,f2
    int i;

    // clrscr(); //利用>>重载输入 f1,f2
    cout << "First fraction:\n";
    cin >> f1;
    cout << "\nSecond fraction:\n";
    cin >> f2;
    cout << "\nEnter an integer\n";
    cin >> i;

    f1. simplify();
    f2. simplify();
    //对两分数进行+,-,*,/运算
    fraction sum = f1 + f2;
    fraction diff = f1 - f2;
    fraction prod = f1 * f2;
    fraction quot;
    if(f2 != 0) quot = f1 / f2; //输出小数运算结果
    cout << f1 << " in decimal form is " << f1. decimal() << "\n";
    cout << f2 << " in decimal form is " << f2. decimal() << "\n\n";
    //利用<<重载输出运算结果
    cout << f1 << " + " << f2 << " = " << sum << " (" << sum. decimal() << ")\n";
    cout << f1 << " - " << f2 << " = " << diff << " (" << diff. decimal() << ")\n";
    cout << f1 << " * " << f2 << " = " << prod << " (" << prod. decimal() << ")\n";
    cout << f1 << " / " << f2 << " = ";
    if(f2 != 0) cout << quot << " (" << quot. decimal() << ")\n\n";
    else cout << "Undefined! \n\n"; //进行分数(f1)和整数(i)的四则运算并输出结果
    cout << f1 << " + " << i << " = " << f1 + i << "\n";
    cout << f1 << " - " << i << " = " << f1 - i << "\n";
    cout << f1 << " * " << i << " = " << f1 * i << "\n";
    cout << f1 << " / " << i << " = ";

```



```

if(i != 0) cout << f1 / i << "\n\n";
else cout << "Undefined! \n\n";
cout << "Boolean operator tests (1 = true, 0 = false)\n";
cout << f1 << " == " << f2 << ": " << (f1 == f2) << '\n';
cout << f1 << " != " << f2 << ": " << (f1 != f2) << '\n';
cout << f1 << " >= " << f2 << ": " << (f1 >= f2) << '\n';
cout << f1 << " <= " << f2 << ": " << (f1 <= f2) << '\n';
cout << f1 << " > " << f2 << ": " << (f1 > f2) << '\n';
cout << f1 << " < " << f2 << ": " << (f1 < f2) << "\n\n";
double dec1 = f1.decimal();
double dec2 = f2.decimal();
if(dec1 > 1 && (int(dec1) != dec1))
{
    cout << f1 << " as a mixed number is ";
    PrintAsMixed(cout, f1);
    cout << '\n';
}
if(dec2 > 1 && (int(dec2) != dec2))
{
    cout << f2 << " as a mixed number is ";
    PrintAsMixed(cout, f2);
}

    getch();
    return 0;
}

```

第 3 题 制式转换

3.3.1 程序功能简介

一个转换程序。从米制到标准制及从标准制到米制的变换程序。

3.3.2 课程设计要求

(1) 设计进制转换:从十进制到二进制,从十进制到八进制,从十进制到十六进制,从二进制到十进制,从八进制到十进制,从十六进制到十进制的转换程序。

(2) 设计时间转换:如北京时间对于其他世界主要城市间的时间变换。

(3) 设计汉化用户接口。

(4) 将所有的函数与相应的数据封装到类中,并改写主程序。

3.3.3 课程设计说明

(1) 设计从米制到标准制及从标准制到米制的变换的类。

(2) 设计时间类。

(3) 设计数制类。

(4) 评定难易级别:B级。

3.3.4 关键源代码注解

```
// Feel free to distribute this program to anyone. http://www.mcarterbrown.com
#include <iostream>
#include <stdlib.h>    // Really don't need this, this is compiler specific

using namespace std;
char choosetd;        // I'm declaring this a global variable for ease of use
float inch1;
float inch2;
float foot;
float yard;
float mile;
float millimeter;
float centimeter;
float meter1;
float meter2;
float kilometer;
int first;            // this is also a global variable that is used throughout
float firstfunction();
float secondfunction();
float thirdfunction();
```



```

float fourthfunction();
float fifthfunction();
float sixthfunction();
float seventhfunction();
float eighthfunction();
float ninthfunction();
float tenthfunction();

int main()                //The beginning of the program
{
    cout << " * ----- * \n";
    cout << "| This is a simple conversion program freely distributed by   |\n";
    cout << "| http://www. mcarterbrown. com                               |\n";
    cout << "| Thank you for using my conversion program!                         |\n";
    cout << " * ----- * \n";
    cout << " \n\n\n";
    cout << "To (q)uit any time type enter \"q\" \n\n";
    cout << "To convert from Standard Measurments to Metric enter \"1\": \n";
    cout << "To convert from Metric to Standard Measurments enter \"2\": \n";
    cin >> first;

    if (first == 1)        // If you enter the number 1 then this will happen
    {
        int choosetd;
        cout << "What would you like to convert? \n";
        cout << "(1)Inches to Millimeters: \n";
        cout << "(2)Inches to Centimeters: \n";
        cout << "(3)Feet to Meters: \n";
        cout << "(4)Yards to Meters: \n";
        cout << "(5)Miles to Kilometers: \n";
        cin >> choosetd;

        if (choosetd == 1)
        {
            cout << "\nEnter the number of inches you wish converted: \n";
            cin >> inch1;
            cout << "You entered " << inch1 << " inches, which equals "
            << firstfunction() << " millimeters. \n\n";
            system("PAUSE");
            // Everytime you see this and don't understand, don't worry it is compiler specific
            return main();
        }
    }
    else if (choosetd == 2)
    {
        cout << "\nEnter the number of inches you wish converted: \n";
    }
}

```

```

        cin >> inch2;
        cout << "You entered " << inch2 << " inches, which equals "
        << secondfunction() << " centimeters. \n\n";
        system("PAUSE"); //执行系统命令 PAUSE
        return main();
    }

    else if      (choosestd == 3)
    {
        cout << "\nEnter the number of feet you wish converted: \n";
        cin >> foot;
        cout << "You entered " << foot << " feet, which equals " << thirdfunction()
        << " meters. \n\n";
        system("PAUSE");
        return main();
    }

    else if      (choosestd == 4)
    {
        cout << "\nEnter the number of yards you wish converted: \n";
        cin >> yard;
        cout << "You entered " << yard << " yards, which equals "
        << fourthfunction() << " meters. \n\n";
        system("PAUSE");
        return main();
    }

    else if      (choosestd == 5)
    {
        cout << "\nEnter the number of miles you wish converted: \n";
        cin >> mile;

        cout << "You entered " << mile << " miles, which equals " << fifthfunction()
        << " kilometers. \n\n";
        system("PAUSE");
        return main();
    }

    else if ((choosestd != 1) || (choosestd != 2) || (choosestd != 3) || (choosestd != 4) || (choosestd != 5))
    {
        int x;
        cout << "\n\nThank you for using this program. \n\n";
        cin >> x;
        return 0;
    }
}

```

```

    else if (first == 2)           // If you enter number 2 then this will happen
    {
        int choosetd;
        cout << "What would you like to convert? \n";
        cout << "(1)Millimeters to Inches: \n";
        cout << "(2)Centimeters to Inches: \n";
        cout << "(3)Meters to Feet: \n";
        cout << "(4)Meters to Yards: \n";
        cout << "(5)Kilometers to Miles: \n";
        cin >> choosetd;
        if (choosetd == 1)
        {
            cout << "\nEnter the number of millimeters you wish converted: \n";
            cin >> millimeter;
            cout << "You entered " << millimeter << " millimeters, which equals "
            << sixthfunction() << " inches. \n\n";
            system("PAUSE");
            return main();
        }
        else if (choosetd == 2)
        {
            cout << "\nEnter the number of centimeters you wish converted: \n";
            cin >> centimeter;
            cout << "You entered " << centimeter << " centimeters, which equals "
            << seventhfunction() << " inches. \n\n";
            system("PAUSE");
            return main();
        }
        else if (choosetd == 3)
        {
            cout << "\nEnter the number of meters you wish converted: \n";
            cin >> meter1;
            cout << "You entered " << meter1 << " meters, which equals "
            << eighfunction() << " feet. \n\n";
            system("PAUSE");
            return main();
        }
        else if (choosetd == 4)
        {
            cout << "\nEnter the number of meters you wish converted: \n";
            cin >> meter2;
            cout << "You entered " << meter2 << " meters, which equals "
            << ninthfunction() << " yards. \n\n";

```

```

        system("PAUSE");
        return main();
    }

    else if (choosetd == 5)
    {
        cout << "\nEnter the number of kilometers you wish converted: \n";
        cin >> kilometer;
        cout << "You entered " << kilometer << " kilometers, which equals "
        << tenthfunction() << " miles. \n\n";
        system("PAUSE");
        return main();
    }

    else if ((choosetd != 1) || (choosetd != 2) || (choosetd != 3) || (choosetd != 4) || (choosetd != 5))
    {
        int x;
        cout << "\n\nThank you for using this program. \n\n";
        cin >> x;
        return 0;
    }

}

else if ((first != 1) || (first != 2)) // If you don't enter either then nothing
{
    int x;
    cout << "End Program. \n\n\nThis program was brought to you by
    \n          http://www.mcarterbrown.com\n\n\n";
    cout << "Thank you\n\n\n";
    cin >> x;
    return 0; // This will end the program
}

}

float firstfunction() // The start of all our funtions from here down
{
    double x;
    x = (inch1 * 25.4);
    return x;
}

float secondfunction()
{
    double x;
    x = (inch2 * 2.54);
    return x;
}

```



```
float thirdfunction()
{
    float x;
    x = (foot * .3048);
    return x;
}

float fourthfunction()
{
    float x;
    x = (yard * .9144);
    return x;
}

float fifthfunction()
{
    float x;
    x = (mile * 1.609);
    return x;
}

float sixthfunction()
{
    float x;
    x = (millimeter / 25.4);
    return x;
}

float seventhfunction()
{
    float x;
    x = (centimeter / 2.54);
    return x;
}

float eighthfunction()
{
    float x;
    x = (meter1 / .3048);
    return x;
}

float ninthfunction()
{
    float x;
    x = (meter2 / .9144);
    return x;
}

float tenthfunction()
```




```
{  
    float x;  
    x = (kilometer / 1.609);  
    return x;  
}
```



第 4 题 八皇后问题

3.4.1 程序功能简介

解决八皇后问题的程序。

3.4.2 课程设计要求

- (1) 增加函数,每输入一组解,暂停屏幕,显示“按任意键继续!”。
- (2) 完善程序,编程计算八皇后问题共有几种排列方案。
- (3) 增加输出,显示在第一个皇后确定后,共有几组排列。

3.4.3 课程设计说明

(1) 八皇后问题是一个古老而著名的问题,是回溯算法的典型例题。该问题是 19 世纪著名的数学家高斯 1850 年提出:在 8×8 格的国际象棋上摆放 8 个皇后,使其不能互相攻击,即任意两个皇后都不能处于同一行、同一列或同一斜线上,问有多少种摆法。高斯认为有 76 种方案。1854 年在柏林的象棋杂志上不同的作者发表了 40 种不同的解,后来有人用图论的方法解出 92 种结果。

(2) 当指示用户输入第一个皇后的位置时,行列数必须为 $[0, 7]$ 区间内的整数。

(3) 只要计算出第一个皇后放在任一相同列 $0 \sim 7$ 行 8 种位置的排列总数即为八皇后问题的全部排列方案数目。

(4) 评定难易级别:B 级。

3.4.4 关键源代码注解

```
// * * * * *
*                               Board. cpp                               *
*      BY Bobby Foerster   Created: 03/21/01   Modified: 03/28/01   *
* * * * *
#include <iostream. h>
#include "Board. h"

Board::Board()    //构造函数,初始化为空
{
    for(int i=0; i<8; i++)
    {
        Rows[i] = free;
        for(int j=0; j<8; j++)    board[i][j] = '.';
    }
    for(int k=0; k<15; k++)    DiagsLR[k] = DiagsRL[k] = free;
}
```

```
void Board::Print() //显示一组排列结果
```

```
{
    cout << endl;
    for(int i=0; i<8; i++)
    {
        for(int j=0; j<8; j++)
            cout << board[i][j] << " ";
        cout << endl;
    }
}
```

```
bool Board::isAttacked(int row, int col) //是否冲突,是返回 TRUE,否返回 FALSE
```

```
{
    int diagLR = col-row+7;
    int diagRL = row+col;
    if (Rows[row] == used || DiagsLR[diagLR] == used || DiagsRL[diagRL] == used)
        return true;
    return false;
}
```

```
void Board::PlaceQueen(int row, int col) //放皇后
```

```
{
    int diagLR = col-row+7; //左对角线元素
    int diagRL = row+col; //右对角线元素
    board[row][col] = 'Q';
    Rows[row] = used;
    DiagsLR[diagLR] = used;
    DiagsRL[diagRL] = used;
}
```

```
void Board::RemoveQueen(int row, int col) //移去皇后
```

```
{
    int diagLR = col-row+7;
    int diagRL = row+col;
    board[row][col] = '.';
    Rows[row] = free;
    DiagsLR[diagLR] = free;
    DiagsRL[diagRL] = free;
}
```

```
/*
 *
 * Board.h
 *
 * BY Bobby Foerster Created: 03/21/01 Modified: 03/28/01
 *
 */
```



```

enum States { used, free };
class Board    //一盘棋 8 * 8
{
private:
    char board[8][8];
    States Rows[8], DiagsLR[15], DiagsRL[15];
public:
    Board();
    ~Board(){}
    bool isAttacked(int, int);
    void Print();
    void PlaceQueen(int, int);
    void RemoveQueen(int, int);
};

/*****
*                               Stack.h                               *
*      BY Bobby Foerster   Created: 03/21/01   Modified: 03/28/01   *
*****/

const int SSize = 8;
typedef int SType;

class Stack {
private:
    SType data[SSize];
    int Top;
public:
    Stack() { Top = 0; }
    ~Stack() { }
    bool isEmpty() { return ! Top; }
    int Size() { return Top; }
    bool Push(SType);
    bool Pop(SType &);
    bool StackTop(SType &);
};

/*****
*                               Stack.cpp                               *
*      BY Bobby Foerster   Created: 03/21/01   Modified: 03/28/01   *
*****/

#include "Stack.h"

```



```
bool Stack::Push(SType d)
```

```
{
    if (Top == SSize)
        return false;
    else {
        data[Top] = d;
        Top++;
        return true;
    }
}
```

```
bool Stack::Pop(SType &d)
```

```
{
    if (! Top)
        return false;
    else {
        Top--;
        d = data[Top];
        return true;
    }
}
```

```
bool Stack::StackTop(SType &d)
```

```
{
    if (! Top)
        return false;
    else {
        d = data[Top-1];
        return true;
    }
}
```

```
/*
 *      CS2617 Assignment 3 --                               main.cpp      *
 *      BY Bobby Foerster   Created:  03/21/01   Modified: 03/28/01      *
 */
```

```
#include <iostream.h>
#include "Stack.h"
#include "Board.h"
```

```
int main(void)
```

```
{
```



```
Stack rowStack;
int qRow, qCol, col, row, attacked, exitLoop;
Board myBoard;

cout << "Enter the position of the first queen\nRow: " << flush;
cin >> qRow;
cout << "Column: " << flush;
cin >> qCol;          //第一个皇后位置

myBoard.PlaceQueen(qRow, qCol);
if (qCol == 0)
    col = 1;
else
    col = 0;
row = 0;

do
{
    while(row < 8) //超界
    {
        exitLoop = 0;
        if (! (attacked = myBoard.isAttacked(row, col))) //若无皇后,条件成立
        {
            myBoard.PlaceQueen(row, col);          //放皇后
            rowStack.Push(row);                     //入栈
            row = 0;
            col++;
            if (col == qCol)
                col++;
            exitLoop = 1;
        }
        if (exitLoop)
            break; //找到退出本层循环
        else row++; //到下一行
    }

    if (col == 8)
    {
        myBoard.Print();
        row = 8;
    }

    if (row == 8)
```



```
{  
    rowStack.Pop(row);  
    col--; //到前一列  
    if (col == qCol)  
        col--;  
    myBoard.RemoveQueen(row,col); //移去皇后  
    row++; //找下一个位置  
}  
  
} while(col >= 0); //回到 0 列  
return 1;  
}
```



第 5 题 求解方程

3.5.1 程序功能简介

求解方程 $ax^n + b \cdot x^{(n-1)} + \dots + cx + k$ 根的程序。

3.5.2 课程设计要求

- (1) 将 C 程序改成 C++ 程序。
- (2) 扩充其功能, 使该程序能求解幂 $n=6$ 或更高次幂。
- (3) 增加功能: 一进入程序, 首先显示汉化的命令提示, 以便用户使用该程序。
- (4) 将所有的函数与相应的数据封装到类中, 并改写主程序。
- (5) 测试程序, 并输出所有的实数和虚数解。

3.5.3 课程设计说明

(1) 根据方程的第一项与最后一项(常数项), 求解方程的一个根。具体方法是: k 是一个能整除第一项系数的整数, a 是一个能整除常数项的数, 将数 (a/k) 代入方程, 检验是否为方程的根, 如果是, 则把方程除以 $(x - a/k)$ 得到新方程。对新方程重复上步, 直到首项为 x^2 。

(2) 三次方程至少要有有一个有理数根, 四次至少两个, 五次至少三个, 依此类推。

(3) 三次以上的方程, 根据解对方程降次时, 得到的新方程系数将被强制转化成整数, 导致方程解的精度下降和局限性。

(4) 评定难易级别: A 级。

3.5.4 关键源代码注解

/* Basic theory:

1. # of roots = highest power
 2. All rational roots will be factors of k / factors of a in the general equation of $ax^n + bx^{(n-1)} + \dots + cx + k$, Quadratic formula—二次公式, rational—有理(的), irrational—无理(的)
 3. Quadratic formula can solve for irrational / imaginary roots (i dont know cubic / quartic formulas)
 4. Irrational roots and imaginaries come in pairs (quadratic formula guarantees this)
 5. Given polynomial $P(x) = ax^n + bx^{(n-1)} + \dots + cx + k$ and given that q is a root, $P(q) = 0$
- */

```
#include "stdio.h"
#include "conio.h"
#include "iostream.h"
#include "string.h"
#include "stdlib.h"
#include "ctype.h"
#include "math.h"
```



```
#include "process.h"
void quadraticequation(int,int,int,char *,char *);
void getterm(char *,char *,int&);
void getfactorsof(int *,int&,int);
void reduceeqbyfactor(char *,char *,float);
void gettermforpower(char *,char *,int);
int getcoeff(char *);
int getdegreeofterm(char *);
int remaindertheorem(char *,float);
int exp(int,int);
float forallfactors(int *,int *,int,int,char *,char *);
float exp(float,int);
float calcequval(char *,float);
float getcoeff_f(char *);
char terms[6][40];

void main()
{
    char * equ, * equ2, roots[5][40];
    int loc=0, current_rootnum=0, a=0, b=0, c=0, numterms;
    int deg, f_a[1000], f_k[1000], num_a, num_k;
    float result;
    equ=(char *)malloc(5000);
    equ2=(char *)malloc(5000);
```

```
printf("Quadratic, cubic, most quartic & some quintic solver\nBy Albert Chaulk\n\nDos & Don'ts\n\tUse ^n
for exponents\n\tDon't do x ^ n, put it like x^n\n\tDo not use x^1 or x^0. These really screw up the searches\
\n\tDon't use decimal or fractional coefficients\n\tCubics must have at least 1 rational root. Quartics, 2, and
quintics. 3\n\tTerms must be listed in decending order (highest power to lowest)\n\tTerms may be skipped
by not including them or using a coefficient of 0\n\n");
```

```
printf("Equation: ");
gets(equ);
if(strlen(equ)==0){
    printf("No equation! \n");
    return;
}
```

```
for(int i=0;i<6&&equ[loc]!=='&&equ[loc]!=='\0&&equ[loc]!=='\n';i++){
    getterm(equ,terms[i],loc);
    numterms=i;
    if((deg=getdegreeofterm(terms[0]))>5||deg<2){
        printf("Invalid term \"%s\" Exponent is outside range (2 to 5)\n",terms[0],deg);
        return;
```

```

}
switch(deg){//there are no breaks after 3-5 because after factoring, its 1 power smaller
case 5:
    getfactorsof(f_a,num_a,getdegreeofterm(terms[0]));
    c=0;
    for(i=1;i<numterms;i++){
        if(getdegreeofterm(terms[i])==0){
            c=getcoeff(terms[i]);
        }
    }
    getfactorsof(f_k,num_k,c);
    result=forallfactors(f_a,f_k,num_a,num_k,eq,roots[current_rootnum]);
    if(result<0.0f&&result>-0.05f){
        printf("Error! Couldn't divide any factors\n");
        return;
    }
    current_rootnum++;
    reduceeqbyfactor(eq,eq2,result);
    strcpy(eq,eq2);
    loc=0;
    for(i=0;i<6&&eq[loc]!=='&&eq[loc]!=='\0'&&eq[loc]!=='\n';i++){
        getterm(eq,terms[i],loc);
        numterms=i;
    }
case 4:
    getfactorsof(f_a,num_a,getdegreeofterm(terms[0]));
    c=0;
    for(i=1;i<numterms;i++){
        if(getdegreeofterm(terms[i])==0){
            c=getcoeff(terms[i]);
        }
    }
    getfactorsof(f_k,num_k,c);
    result=forallfactors(f_a,f_k,num_a,num_k,eq,roots[current_rootnum]);
    if(result<0.0f&&result>-0.05f){
        printf("Error! Couldn't divide any factors\n");
        return;
    }
    current_rootnum++;
    reduceeqbyfactor(eq,eq2,result);
    strcpy(eq,eq2);
    loc=0;
    for(i=0;i<6&&eq[loc]!=='&&eq[loc]!=='\0'&&eq[loc]!=='\n';i++){
        getterm(eq,terms[i],loc);
    }
}

```

```

        numterms=i;
    case 3:
        getfactorsof(f_a,num_a,getdegreeofterm(terms[0]));
        c=0;
        for(i=1;i<numterms;i++){
            if(getdegreeofterm(terms[i])==0){
                c=getcoeff(terms[i]);
            }
        }
        getfactorsof(f_k,num_k,c);
        if((result=forallfactors(f_a,f_k,num_a,num_k,eq,roots[current_rootnum]))==0.0f);
        if(result<0.0f&&result>-0.05f){
            printf("Error! Couldn't divide any factors\n");
            return;
        }
        current_rootnum++;
        reduceeqbyfactor(eq,eq2,result);
        strcpy(eq,eq2);
        loc=0;
        for(i=0;i<6&&eq[loc]!='&&eq[loc]!='\0&&eq[loc]!='\n';i++){
            getterm(eq,terms[i],loc);
            numterms=i;
        }
    case 2:
        b=c=0;
        for(i=1;i<numterms;i++){
            if(getdegreeofterm(terms[i])==1)
                b=getcoeff(terms[i]);
        }
        for(i=1;i<numterms;i++){
            if(getdegreeofterm(terms[i])==0)
                c=getcoeff(terms[i]);
        }
        quadraticequation(getcoeff(terms[0]),b,c,roots[current_rootnum],
            roots[current_rootnum+1]);
        current_rootnum+=2;
        break;
    }
    printf("Roots: ");
    for(i=0;i<current_rootnum;i++){
        printf("%s",roots[i]);
        if(i+1<current_rootnum)
            printf(", ");
    }
    printf("\n");
    system("pause");

```



```

}
void quadraticequation(int a,int b,int c,char * r1,char * r2)
{
    // (-b +- sqrt( b*b - 4*a*c )) / 2*a
    if(b*b-4*a*c<0){
        sprintf(r1,"(%d+i*sqrt(%d))/%d",b*-1,(b*b-4*a*c)*-1,2*a);
        sprintf(r2,"(%d-i*sqrt(%d))/%d",b*-1,(b*b-4*a*c)*-1,2*a);
    }
    else{
        if(sqrt(b*b-4*a*c)-(int)sqrt(b*b-4*a*c)==0){
            sprintf(r1,"%.3f",((float)((float)-b+(float)sqrt(b*b-4*a*c))/
            (float)(2*a)));
            sprintf(r2,"%.3f",((float)((float)-b-(float)sqrt(b*b-4*a*c))/
            (float)(2*a)));
        }
        else{
            sprintf(r1,"(%d+sqrt(%d))/%d",b*-1,b*b-4*a*c,2*a);
            sprintf(r2,"(%d-sqrt(%d))/%d",b*-1,b*b-4*a*c,2*a);
        }
    }
}

```

```

void getterm(char * src,char * dest,int &loc)

```

```

{
    int loc2=0;
    if(src[loc]=='-'){
        dest[loc2]='-';
        loc2++;
    }
    for(;src[loc+loc2]!=='+'&&src[loc+loc2]!=='-'&&src[loc+loc2]!=='\n'&&src[loc+loc2]!=='\0'
    &&src[loc+loc2]!=='=';loc2++){
        dest[loc2]=src[loc+loc2];
        dest[loc2]='\0';
        loc+=loc2;
    }
    if(src[loc]!=='-'&&src[loc]!=='\0')
        loc++; //+1 passes terminating character, but a - sign is part of a term
}

```

```

void getfactorsof(int * factor_list,int &num,int val)

```

```

{
    int fcount=0;
    num=0;
    val=abs(val);
    for(int i=1;i<=val;i++){
        if((float)((float)((float)val/(float)i)-(int)((float)val/(float)i))==0.0f)

```

```

        num++;
//factor_list=(int *)malloc(1000);
    for(i=1;i<=val;i++)
        if((float)((float)((float)val/(float)i)-(int)((float)val/(float)i))==0.0f){
            factor_list[fcount]=i;
            fcount++;
        }
}

int getcoeff(char * term)
{
    char * temp;
    int mult=1;
    temp=(char *)malloc(strlen(term));
    for(int i=0;term[i]!='';i++);    //bypass whitespace
    if(term[i]=='-'){
        mult=-1;
        i++;
    }
    for(;! isdigit(term[i])&&! isalpha(term[i]);i++);
    if(isalpha(term[i]))
        return mult;
    for(int j=0;isdigit(term[i+j]);j++)
        temp[j]=term[i+j];
    return atoi(temp) * mult;
}

float getcoeff_f(char * term)
{
    char * temp;
    int mult=1;
    if(term[0]=='\0')
        return 0.0f;
    temp=(char *)malloc(strlen(term));
    for(int i=0;term[i]!='';i++);    //bypass whitespace
    if(term[i]=='-'){
        mult=-1;
        i++;
    }
    for(;! isdigit(term[i])&&! isalpha(term[i]);i++);
    if(isalpha(term[i]))
        return (float)mult;
    for(int j=0;isdigit(term[i+j]);j++)
        temp[j]=term[i+j];
    return (float)atoi(temp) * (float)mult;
}

```



```

}
int getdegreeofterm(char * term)
{
    if(strchr(term,'x') != NULL){ //variable term
        if(strchr(term,'^') != NULL){ //exponent term
            for(int i=0;term[i] != '~';i++);
            //no error checking needed because ^ must be in the string
            i++;
            switch(term[i]){
                case '2': return 2;
                case '3': return 3;
                case '4': return 4;
                case '5': return 5;
            }
        }
        else
            return 1;
    }
    else
        return 0;
    return -1;
}

int remaindertheorem(char * equ,float root)
{
    //P(x) = ax^n + bx^(n-1) + ... + cx + k. If q is a root, P(q) = 0
    char terms[5][40];
    float val[5],totalval=0.0f;
    int loc=0;
    for(int i=0;i<6;&&equ[loc] != '='&&equ[loc] != '\0'&&equ[loc] != '\n';i++)
        getterm(equ,terms[i],loc);
    int numterms=i;
    for(i=0;i<numterms;i++){
        val[i]=calcequval(terms[i],root);
        totalval+=val[i];
    }
    if((totalval * 0.99f > -0.002f && totalval * 0.99f < 0.002f) return 0;
    else return 1;
}

int exp(int val,int e)
{
    int v2=val;
    if(e==0) return 1;
    for(int i=1;i<=e;i++) v2 *= val;
    return v2;
}

```

```

    }
    float exp(float val,int e)
    {
        float v2=val;
        if(e==0)    return 1.0f;
        for(int i=1;i<=e;i++)    v2 *= val;
    return v2;
    }
    float calcequval(char * equ,float val)
    {
        int coeff=getcoeff(equ);
        int power=getdegreeofterm(equ);
        return coeff * exp(val,power);
    }
    void reduceequbyfactor(char * base,char * newequ,float factor)
    {
        float synthterms[6]={0.0f,0.0f,0.0f,0.0f,0.0f,0.0f};    //synthetic division vars
        int newterms[5]={0,0,0,0,0};
        int j=0,olddegree;
        char temp[50]="",exp[10]="",tempterms[6][40];
        if(remaindertheorem(base,factor))    return;
        olddegree=getdegreeofterm(terms[0]);
        for(int i=/ * olddegree * /0;i<=olddegree;i++){
            gettermforpower(base,temp,i);
            synthterms[i]=getcoeff_f(temp);
        }
        newterms[4]=(int)synthterms[5];
        for(i=4;i>=1;i--)
            newterms[i-1]=(int)((float)(newterms[i] * factor+ synthterms[i]));
        for(i=4;newterms[i]==0;i--);
        strcpy(newequ,"");
        for(;i>=0;i--){
            if(newterms[i]!=0){
                switch(i){
                    case 0:
                        strcpy(exp,"");
                        break;
                    case 1:
                        strcpy(exp,"x");
                        break;
                    case 2:
                        strcpy(exp,"x^2");
                        break;

```

★★★★★

```

case 3:
    strcpy(exp, "x^3");
    break;
case 4:
    strcpy(exp, "x^4");
    break;
}
if(newterms[i] != 1){
    if(newterms[i] < 0)
        sprintf(tempters[i], "%d%s", newterms[i], exp);
    else{
        if(i < olddegree - 1){
            sprintf(tempters[i], "+ %d%s", newterms[i], exp);
        }
        else
            sprintf(tempters[i], "%d%s", newterms[i], exp);
    }
}
else{
    if(newterms[i] < 0)
        sprintf(tempters[i], "%s", exp);
    else{
        if(i < olddegree - 1){
            sprintf(tempters[i], "+ %s", exp);
        }
        else
            sprintf(tempters[i], "%s", exp);
    }
}
if(newterms[i] == 1 && exp[0] == '\0'){
    if(newterms[i] > 0)
        sprintf(tempters[i], "+ %d", newterms[i]);
    else
        sprintf(tempters[i], "%d", newterms[i]);
}
strcat(newequ, tempters[i]);
}
}
}

```

```

void gettermforpower(char * equ, char * dest, int power)
{
    for(int i=0; i<6; i++)

```



```

        if(getdegreeofterm(terms[i]) == power){
            strcpy(dest, terms[i]);
            return;
        }
    //if(i==6)
        strcpy(dest, "0");
    }

float forallfactors(int * a, int * k, int n_a, int n_k, char * equ, char * root)
{
    for(int i=0; i<n_a; i++){
        for(int j=0; j<n_k; j++){
            if(remaindertheorem(equ, (float)((float)k[j]/(float)a[i])) == 0){
                int r=k[j]/a[i];
                sprintf(root, "%d", r);
                return (float)r;
            }
            if(remaindertheorem(equ, (float)((float)k[j]/(float)a[i]) * -1.0f) == 0){
                int r=-k[j]/a[i];
                sprintf(root, "%d", r);
                return (float)r;
            }
        }
    }

    if(remaindertheorem(equ, 0.0f) == 0){
        sprintf(root, "0");
        return 0.0f;
    }

    return -0.0001f;    //value too small to be returned, used for errors
}

```

```

/* **** */
*
*          Class Matrix
*
* **** */
#include <iostream.h>
#include <math.h>
#include <conio.h>
#include <iomanip.h>

class matrix    //高斯—赛德尔矩阵算法类
{
friend void operator<<(ostream &,matrix &);

```

```

friend void operator>>(istream &,matrix &);
protected:
int row,column;
//'mat' is my actual matrix
double * * mat;
//size of 'variable' always =(column-1)
int varnum;
//contains the solution set
double * variable;
//counts number of iterations
int itercount;
//performs a single iteration on all the equations
//the argument is the relaxation coefficient
void iteration(double);
//checks the allowable error
bool epsilon(double *,double *,int,double);
public:
matrix(int,int);
//small funtion for asking the user for the number of rows and columns
static void initialize(int&,int&);
//prepares function for reduction
void rearrange();
//solves by gauss—seigel method,
//perfomes multiple iterations until a solution is found
//the argument is the relaxation—coefficient, and this is "past" to 'iteration'
void solve(double);
//prints the solution
void show_answer();
};
//END OF CLASS MATRIX
/* * * * * *
*      constructor——构造函数      *
* * * * *
matrix::matrix(int r=1, int c=1):row(r),column(c)
{
//creates my matrix dynamically, but there are still no values in the positions
mat=new double * [row];
    for(int i=0;i<row;i++)
    {
        mat[i]=new double[column];
    }
}
//end of constructor

```

```

/* * * * * *
* 矩阵行列设置。ROWS:线性方程的个数 COLUMNS:系数及常数的个数
* * * * *
void matrix::initialize(int &i,int &j)
{
cout<<"How many ROWS? >";
cin>>i;
cout<<"How many COLUMNS? >";
cin>>j;
//clrscr();
}

/* * * * * *
* 高斯—赛德尔矩阵变换函数 *
* * * * *
//rearranges the system so that it can be solved by gauss—seigel method
//must always rearrange before solving
void matrix::rearrange()
{
varnum=column-1;
//'variable' will contain the solution set
//they will get initialized with the first guess in the 'solve' function
variable=new double[varnum];
/* divides all terms by the desired variables
(the variable which is being solved for) coefficient * /
for(int i=0;i<row;i++)
{
double coefficient=mat[i][i];
for(int j=0;j<column;j++)
{
mat[i][j]/=coefficient;
}
}
//all variables except the diagonal are being brought to the other side
for( i=0;i<row;i++)
{
for(int j=0;j<column-1;j++)
{
mat[i][j] * =-1;
}
mat[i][i]=0;
}
}
//END OF REARRANGE FUNCTION

```

```

/* * * * * *
 *          iteration funtion——迭代函数          *
 * * * * * */
//performs a single iteration on the system, using passed assumed values
void matrix::iteration(double lambda)
{
//'last' is for the relaxation equation
double last;
    for(int i=0;i<varnum;i++)
    {
        last=variable[i];
        variable[i]=0;
        for(int j=0;j<varnum;j++)
        {
            variable[i] += mat[i][j] * variable[j];
        }
        variable[i] += mat[i][column-1];
        //new value after relaxation
        variable[i] = last + lambda * (variable[i] - last);
    }
}

/* * * * * *
 *          solve function——求解函数          *
 * * * * * */
void matrix::solve(double lambda)
{
//DECLARATIONS AND INITIALIZATIONS
//initializes first guess
    for (int i=0;i<varnum;i++)
    {
        variable[i]=0;
    }
    itercount=0;
    //this is the allowable error this value can be changed to suit the problem
    double criterion=0.0001;
    double * newest=new double[varnum];
    double * last=new double[varnum];
    for( i=0;i<varnum;i++)
    {
        newest[i]=variable[i];
    }
//END OF DECLARATIONS AND INITIALIZATIONS

```



```
//MAIN BODY OF THE FUNCTION STARTS HERE—
//while('condition not met'){perform another iteration}
do
{
for(int i=0;i<varnum;i++)
{
last[i]=newest[i];
}
//this is the most important part,
//everything else in this loop is only to check that the criterion is met
iteration(lambda);
for( i=0;i<varnum;i++)
{
newest[i]=variable[i];
}
itercount++;
}
while(epsilon(newest,last,varnum,criterion));
}

/* * * * * *
*          Epsilon Criterion—Epsilon 精度要求          *
* * * * * */
bool matrix::epsilon(double * newest,double * last,int size,double criterion)
{
for(int i=0;i<size;i++)
{
//if(it has not met the criterion)
if((fabs(newest[i]-last[i])/newest[i])>criterion)
{
//then (return 1)=(condition not met) and the loop is repeated
return 1;
}
}
}
//criterion has been met
return 0;
}

/* * * * * *
*  show answer function— 输出求解结果函数          *
* * * * * */
//‘solve’ function must be executed before ‘show_answer’ function
void matrix::show_answer()
{
for(int i=0;i<varnum;i++)
```

```

    {
        cout<<"X"<<(i+1)<<"="<<variable[i]<<endl;
    }

    cout<<endl<<endl<<"Number of Iterations="<<itercount<<endl;
}

/* * * * * *
 *      stream operators——矩阵输入、输出流重载函数      *
 * * * * * */
void operator<<(ostream& out, matrix& m)
{
    for (int i=0; i<m. row; i++)
    {
        for (int j=0; j<m. column; j++)
        {
            out<<m. mat[i][j]<<setw(10);
        }
        out<<endl<<endl;
    }
}

void operator>>(istream& in, matrix& m)
{
    for(int i=0; i<m. row; i++)
    {
        for(int j=0; j<m. column; j++)
        {
            cout<<"value of indice ["<<(i+1)<<"]["<<(j+1)<<"]";
            in>>m. mat[i][j];
        }
    }

    //clrscr();
}

//end of stream operators
/* * * * * *
 *      MAIN      *
 * * * * * */
void main()
{
    cout
    <<"This Program solves linear systems by the Gauss—Seigel Method"
    <<endl
    <<"Try to input the matrix so that there is Diagonal Dominance"
    <<endl<<endl;
    int i,j;

```

★★★★★

```
matrix::initialize(i,j);
matrix one(i,j);
cin>>one;
char X;
cout<<"Do you want to see the matrix?:(y/n)>";
cin>>X;
if(X=='y'){
    cout<<one;
    one.rearrange();
    double relax_coef;
    for(char x='y';x=='y' && x!='n';)
    {
        cout<<endl<<"Relaxation Coefficient(1=no relaxation):>";
        cin>>relax_coef;
        one.solve(relax_coef);
        one.show_answer();
        cout<<endl<<"try another Relaxation Coefficient? :(y/n)>";
        cin>>x;
    }
    cin.ignore(128,'\n');
    cin.ignore(128,'\n');
}
```


第 7 题 字符串操作

3.7.1 程序功能简介

通过运算符的重载,实现字符串的直接操作,如:去掉串首、串尾与子串相同的字符,子串在主串中的定位,用新子串更换主串中的指定子串等,所有功能已经过检验。

3.7.2 课程设计要求

- (1) 将主函数 main() 中的输出语句 printf() 改为 VC++ 中的输出语句。
- (2) 对程序进行修改,对功能进行补充与完整,并进行调试。
- (3) 程序中对字符串的处理函数均使用了字符数组 sAns,它占用了大量存储空间,试使用动态分配存储空间的方法改写其中的函数(至少一个),使之避免使用预先定义的长度很大的数组。
- (4) 在主函数 main() 中添加程序中定义的字串处理函数的测试语句(至少调用其中的 3 个函数),来验证字串处理函数的功能。

3.7.3 课程设计说明

- (1) 源程序中的 operater + 的处理有问题,功能与 += 重合,要改正。
- (2) 可以用动态分配存储空间的方法改写函数。
- (3) 评定难易级别:B 级。

3.7.4 关键源代码注解

```
//GFHString.h begin
//----- FString Class, Greg De Haas -- 27 March 2002 -----

#ifndef GFH_STRING    //条件编译起始行
#define GFH_STRING
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <stdarg.h>    //定义参数个数可变的函数时用到
#define MAX_FSTRING_LENGTH 25000    //用作数组长度

//Variant Class
class FString {        // 字符串类定义
private:
    char    * sString; //String data must be first (to look like string)
public:
    //Constructor
```



```

FString();
//Public Methods
FString Mid(const int nStart,const int nLength);
FString Left(const int nLength);
FString Right(const int nLength);
int InStr(const int nStart, const char * sSearch,const bool bCaseSensitive=true);
int InStrRev(const int nStart, const char * sSearch,const bool bCaseSensitive=true);
FString RTrim(const char sChar=' ');
FString LTrim(const char sChar=' ');
FString Trim(const char sChar=' ');
FString Replace(const char * sSearch, const char * sReplace);
FString Format(const char * sFormat, ...);
int Length();
//Various Operators
inline operator char * ();    //转换函数
inline FString operator+(const char * sAppend);
inline FString operator+(const FString sAppend);
inline char operator[](const int nIndex);
inline FString operator+=(char * val);
inline FString operator+=(FString val);
//Comparison Operators
inline int operator<(const char * val);
inline int operator<=(const char * val);
inline int operator>(const char * val);
inline int operator>=(const char * val);
inline int operator!=(const char * val);
inline int operator==(const char * val);
inline int operator<(const FString val);
inline int operator<=(const FString val);
inline int operator>(const FString val);
inline int operator>=(const FString val);
inline int operator!=(const FString val);
inline int operator==(const FString val);
//Assignment Operators
FString operator=(const char * val);
FString operator=(const FString val);
};

#endif    //条件编译结束行
//GFHString.h end
//GFSSString.h begin
//----- FString Class, Greg De Haas - 27 March 2002 -----
//This is a simple string class that can come in useful

```



```

//I've kept the naming conventions similar to VB for simplicity
//-----
#ifdef GFS_STRING
#define GFS_STRING
#include "GFHString.h"
#include <windows.h>
//下面是 GFHString.h 中说明的各函数的具体定义
//Constructor
FString::FString() { sString=""; }
//Comparisons
int FString::operator<(const char * val) { return(strcmp(sString,val) < 0); }
int FString::operator<=(const char * val) { return(strcmp(sString,val) <= 0); }
int FString::operator>(const char * val) { return(strcmp(sString,val) > 0); }
int FString::operator>=(const char * val) { return(strcmp(sString,val) >= 0); }
int FString::operator!=(const char * val) { return(strcmp(sString,val) != 0); }
int FString::operator==(const char * val) { return(strcmp(sString,val) == 0); }
int FString::operator<(const FString val) { return(strcmp(sString,val.sString) < 0); }
int FString::operator<=(const FString val) { return(strcmp(sString,val.sString) <= 0); }
int FString::operator>(const FString val) { return(strcmp(sString,val.sString) > 0); }
int FString::operator>=(const FString val) { return(strcmp(sString,val.sString) >= 0); }
int FString::operator!=(const FString val) { return(strcmp(sString,val.sString) != 0); }
int FString::operator==(const FString val) { return(strcmp(sString,val.sString) == 0); }
//Other operators
FString FString::operator+=(char * val) {
    char sAns[MAX_FSTRING_LENGTH]="";
    strcpy(sAns,sString);
    strcat(sAns,val);
    sString = _strdup(sAns);
    return(* this);
}
FString FString::operator+=(FString val) {
    char sAns[MAX_FSTRING_LENGTH]="";
    strcpy(sAns,sString);
    strcat(sAns,val.sString);
    sString = _strdup(sAns);
    return(* this);
}
char FString::operator[](const int nIndex) { //数组下标运算符重载
    if (nIndex <= 0) return(0);
    if (nIndex > (int)strlen(sString)) return(0); //判断下标是否越界
    return(sString[nIndex-1]);
}
FString::operator char * () { return(_strdup(sString)); }

```

★★★★★

```

FString FString::operator+(const char * sAppend) {
    char sAns[MAX_FSTRING_LENGTH]="";
    strcpy(sAns,sString);
    strcat(sAns,sAppend);
    sString = _strdup(sAns);
    return( * this);
}

FString FString::operator+(const FString sAppend) {
    char sAns[MAX_FSTRING_LENGTH]="";
    strcpy(sAns,sString);
    strcat(sAns,sAppend.sString);
    sString = _strdup(sAns);
    return( * this);
}

FString FString::operator=(const char * val) {
    sString = _strdup(val); return( * this);
}

FString FString::operator=(FString val) {
    sString = _strdup(val.sString); return( * this);
}

//Trim both left + right characters——串首串尾整理
FString FString::Trim(char sChar) {
    return(this->RTrim(sChar).LTrim());    //注意此处的用法
}

//Trim right characters——串尾整理,即去掉串尾与 sChar 相同的字符
FString FString::RTrim(char sChar) {
    char sAns[MAX_FSTRING_LENGTH]="";
    long nLen = strlen(sString);
    FString sAns2;
    int cnt=nLen-1;
    sAns2 = sString;

    //Validate
    if (nLen == 0) return(sAns2);

    //Determine # of chars
    while (sString[cnt] == sChar && cnt>=0) cnt--;
    if (cnt >= (nLen-1)) return sAns2;

    //Get characters
    strncpy(sAns,sString,cnt+1);
    sAns2 = sAns;
}

```

```

    return(sAns2);
}

//Trim left characters——串首整理,即去掉串首与 sChar 相同的字符
FString FString::LTrim(char sChar) {
    char    sAns[MAX_FSTRING_LENGTH]="";
    long nLen = strlen(sString);
    FString sAns2;
    int cnt=0;
    sAns2 = sString;

    //Validate
    if (nLen == 0) return(sAns2);

    //Determine # of chars
    while (sString[cnt] == sChar) cnt++;
    if (cnt == 0) return sAns2;

    //Get characters
    strncpy(sAns,sString+cnt,nLen-cnt);
    sAns2 = sAns;
    return(sAns2);
}

//Returns the characters from nStart of nLength characters
//从字符串 sString 中第 nStart 个字符开始取出 nLength 个字符
FString FString::Mid(int nStart,int nLength) {
    char    sAns[MAX_FSTRING_LENGTH]="";
    FString sAns2;
    long nLen = strlen(sString);
    sAns2 = "";

    //Validate
    if (nLen == 0) return(sAns2);
    if (nLength <= 0) return(sAns2);
    if (nLength > nLen) nLength = nLen;

    //Get characters
    strncpy(sAns,sString+(nStart-1),nLength);
    sAns2 = sAns;
    return(sAns2);
}

//Returns the left nLength chars——从字符串 sString 左边取出 nLength 个字符
FString FString::Left(int nLength) {

```

```

    char    sAns[MAX_FSTRING_LENGTH] = "";
    FString sAns2;
    long nLen = strlen(sString);
    sAns2 = "";

    //Validate
    if (nLen == 0) return(sAns2);
    if (nLength <= 0) return(sAns2);
    if (nLength > nLen) nLength = nLen;

    //Get characters
    strncpy(sAns, sString, nLength);
    sAns2 = sAns;
    return(sAns2);
}

//Returns the right nLength chars——从字符串 sString 右边取出 nLength 个字符
FString FString::Right(int nLength) {
    char    sAns[MAX_FSTRING_LENGTH] = "";
    FString sAns2;
    long nLen = strlen(sString);
    sAns2 = "";

    //Validate
    if (nLen == 0) return(sAns2);
    if (nLength <= 0) return(sAns2);
    if (nLength > nLen) nLength = nLen;

    //Get characters
    strncpy(sAns, sString + (nLen - nLength), nLength);
    sAns2 = sAns;
    return(sAns2);
}

//Finds position of sSearch in string, starting with position nStart——寻找子串位置
int FString::InStr(const int nStart, const char * sSearch, const bool bCaseSensitive) {
    //Get Lengths
    int nLen1 = strlen(sSearch);
    int nLen2 = strlen(sString);
    //Other vars
    int cnt = nStart - 1;
    int cnt2 = 0;
    bool bFound;
    int nMax = (nLen2 - nLen1 + 1);

```

```

    bool bTest;

    //Validation
    if (cnt < 0) cnt = 0;
    if (cnt > (nLen2 - nLen1 + 1)) return(0);

    //Recurse String
    for (cnt=cnt; cnt<nMax; cnt++) {
        //Test first char
        bTest = ((bCaseSensitive) ? (sString[cnt] == sSearch[0]) : (tolower(sString[cnt]) ==
        tolower(sSearch[0])));
        if (bTest) { //correct
            bFound = true; //Default to 'found'
            //Recurse + test all chars in sSearch
            for (cnt2=0; cnt2<nLen1; cnt2++) {
                if (bCaseSensitive) {
                    if (! (sString[cnt2+cnt] == sSearch[cnt2])) bFound = false;
                } else {
                    if (! (tolower(sString[cnt2+cnt]) == tolower(sSearch[cnt2]))) bFound = false;
                }
            }
            if (bFound) return(cnt+1); //Found
        }
    }
    return 0; //Not Found
}

//Finds position of sSearch in string in reverse, starting at position nStart
//寻找子串的倒置串在主串中的位置
int FString::InStrRev(const int nStart, const char * sSearch, const bool bCaseSensitive) {
    //Get Lengths
    int nLen1 = strlen(sSearch);
    int nLen2 = strlen(sString);
    //Other vars
    int cnt=nStart-nLen1;
    int cnt2=0;
    bool bFound;
    bool bTest;

    //Validation
    if (cnt < 0) return(0);
    if (cnt > (nLen2 - nLen1 + 1)) cnt = (nLen2 - nLen1 + 1);

```

★ ★ ★ ★ ★

```

//Recurse String
for (cnt=cnt;cnt>=0;cnt--) {
    //Test first char
    bTest = ((bCaseSensitive) ? (sString[cnt] == sSearch[0]) :
    (tolower(sString[cnt]) == tolower(sSearch[0])));
    if (bTest) { //correct
        bFound = true; //Default to 'found'
        //Recurse + test all chars in sSearch
        for (cnt2=0; cnt2<nLen1;cnt2++) {
            if (bCaseSensitive) {
                if (! (sString[cnt2+cnt] == sSearch[cnt2])) bFound = false;
            } else {
                if (! (tolower(sString[cnt2+cnt]) == tolower(sSearch[cnt2]))) bFound = false;
            }
        }
        if (bFound) return(cnt+1); //Found
    }
}
return 0; //Not Found
}

//Format——按指定格式输出
FString FString::Format(const char * sFormat, ...){ //参数可变的函数
    char buffer[20000]="";
    FString sAns;
    sAns = "";

    if ((! sFormat)) return(sAns); // make sure string is valid——确信串是合法的

    va_list arglist; // variable argument list
    //print out the string using the variable number of arguments on stack
    va_start(arglist,sFormat);
    vsprintf(buffer,sFormat,arglist);
    va_end(arglist);

    sAns = buffer;
    return(sAns);
}

//Replaces any occurrence of sSearch with sReplace
//用子串 * sReplace 去替换主串中的所有子串 * sSearch
FString FString::Replace(const char * sSearch, const char * sReplace) {

```



```

    int nSearchLen=strlen(sSearch);
    int nPos=0;
    int nDiffLen=strlen(sReplace)-strlen(sSearch);
    FString sTemp;
    FString sLeft, sRight;

    sTemp = sString;

    //Recurse string
    nPos = sTemp.InStr(nPos+1,sSearch);
    while (nPos != 0) {
        sLeft = sTemp.Left(nPos-1);
        sRight = sTemp.Right((strlen(sTemp.sString)-nPos+1)-nSearchLen);
        sTemp = sLeft + sReplace + sRight;

        nPos = sTemp.InStr(nPos + nDiffLen + 1,sSearch);
    }

    return sTemp;
}

//Returns Length

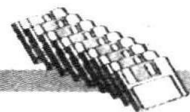
int FString::Length() {    return(strlen(sString)); }
#endif
//GFSSString.h  end

//main() begin
//----- FString Class, Greg De Haas - 27 March 2002 -----
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "GFSSString.h"
void main() {
    FString s,s2;
    s = "   abc abc";
    s += " test   ";
    printf("Trim : '%s'\n",s.Trim());
    s = s.Trim();
    printf("[3] : '%c'\n",s[3]);    //输出字符 s[3]
    printf("Replace : '%s'\n",s.Replace("c","cdef"));    //输出    s.Replace("c","cdef")的值
    printf("Format : '%s'\n",s.Format("1:%s; 2:%d","One",123));
    s = "book";    //输出 s.Format("1:%s; 2:%d","One",123)的值
    s2 = "cat";

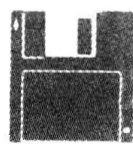
```

★★★★★

```
printf("book < 'cat': %s\n", (s < s2) ? "True" : "False");  
s = s2;  
printf("Copy 'cat' to s: %s\n", s);  
}  
//main() end
```

4 矩阵及向量处理程序



第 1 题 矩阵旋转反射

4.1.1 程序功能简介

实现矩阵的旋转反射程序。

4.1.2 课程设计要求

- (1) 将本程序用数组表示的 5×5 矩阵改为用单向链表表示的 $n \times n$ 矩阵, 其中 n 为用户可输入的任意整数, 并实现 $n \times n$ 矩阵的旋转反射。
- (2) 将所有的函数与相应的数据封装到类中。

4.1.3 课程设计说明

(1) 矩阵旋转反射的数学原理很简单, 因此程序的算法并不复杂, 主要是要解决矩阵的表示方法以及实现矩阵的转置。

(2) 采用了单向链表来表示矩阵, 对矩阵的操作就非常方便了。

(3) 要实现旋转反射则按一定的算法改变链表中节点的次序。

(4) 类 Vector 在 "Vector.h" 中声明, 在 "Vector.cpp" 中实现。

(5) 评定难易级别: B 级。

4.1.4 关键源代码注解

```
#include <iostream, h>
#include <conio, h>
void main()
{
    clrscr();
    int r,c,a[5][5],b[5][5],d[5][5];
    for (r=0;r<5;r++)
        for(c=0;c<5;c++)
            a[r][c]=r*5+c+1;
    cout<<endl<<" Array a\n\n";
    for(r=0;r<5;r++)
    {
        for (c=0;c<5;c++)
        {
            if(a[r][c]<10)
                cout<<" " << a[r][c] <<" ";
            else
                cout<<a[r][c]<<" ";
        }
        cout<<endl;
    }
}
```

```

}
int choice;
cout<< "\nPick an angle of rotation \n";
cout<< "1 - 90 degrees\n";
cout<< "2 - 180 degrees\n";
cout<< "3 - 270 degrees\n";
cin>>choice;
if (choice==1)
    { //fill array b, turn array a 90 degrees
        for (r=0;r<5;r++)
            for (c=0;c<5;c++)
                b[4-c][r]=a[r][c];
    }
if (choice==2)
    { //fill array b, turn array a 180 degrees
        for (r=0;r<5;r++)
            for (c=0;c<5;c++)
                b[4-r][4-c]=a[r][c];
    }
if (choice==3)
    {
        //fill array b, turn array a 270 degrees
        for (r=0;r<5;r++)
            for (c=0;c<5;c++)
                b[c][4-r]=a[r][c];
    }
cout<<endl<<" Array b\n\n";
for (r=0;r<5;r++)
    {
        for (c=0;c<5;c++)
            {
                if(b[r][c]<10)
                    cout<< " " << b[r][c] <<" ";
                else
                    cout<<b[r][c]<< " ";
            }
        cout<<endl;
    }
int choice2;
cout<< "\nPick an angle of reflection and point\n";
cout<< "1 - x-axis\n";
cout<< "2 - y-axis\n";
cout<< "3 - Point\n";

```

```

cin>>choice;
if (choice==1)
{    //fill array d, reflect array
    for (r=0;r<5;r++)
        for (c=0;c<5;c++)
            d[r][c]=b[4-r][c];
}
if (choice==2)
{    //fill array b, turn array a 180 degrees
    for (r=0;r<5;r++)
        for (c=0;c<5;c++)
            d[r][c]=b[r][4-c];
}
if (choice==3)
{    //fill array b, turn array a 180 degrees
    for (r=0;r<5;r++)
        for (c=0;c<5;c++)
            d[r][c]=b[4-r][4-c];
}
cout<<endl<<" Array d\n\n";
for (r=0;r<5;r++)
{
    for (c=0;c<5;c++)
    {
        if(d[r][c]<10)
            cout<<" " << d[r][c] <<" ";
        else
            cout<<d[r][c]<<" ";
    }
    cout<<endl;
}
}

```



第 2 题 矩阵乘法计算

4.2.1 程序功能简介

根据向量处理的程序实现矩阵乘法。

4.2.2 课程设计要求

- (1) 将本程序用数组表示 2×2 矩阵改为用单向链表表示的 $n \times n$ 矩阵,其中 n 为用户可输入的任意整数,并实现 $n \times n$ 矩阵的乘法。
- (2) 将所有的函数与相应的数据封装到类中。
- (3) 改写主程序,使程序简化。

4.2.3 课程设计说明

- (1) 设计在输入实数时,同时输入字符的处理程序。
- (2) 设计处理乱输入的程序。
- (3) 评定难易级别:B 级。

4.2.4 关键源代码注解

参见 4.1.4。

第3题 矩阵数乘

4.3.1 程序功能简介

根据向量处理的程序实现矩阵数乘。

4.3.2 课程设计要求

(1) 参照源程序的实现结构改用单向链表表示 $n \times n$ 矩阵, 其中 n 为用户可输入的任意整数, 并用程序实现验证 $n \times n$ 矩阵的如下定理:(数乘)

$$k(lA) = (kl)A$$

$$k(A+B) = kA + kB$$

$$(k+l)A = kA + lA$$

其中 A, B 分别为 $n \times n$ 矩阵, k, l 为两个常数。

(2) 将所有的函数与相应的数据封装到类中, 并改写主程序, 使程序简化。

4.3.3 课程设计说明

(1) 本程序应用了很多方法。比如: 模板, 链表, 运算符重载, 友元函数, 标准输入输出。

(2) 在输入数据时, 要严格按照屏幕提示进行输入, 以免出错。

(3) 评定难易级别: A 级。

4.3.4 关键源代码注解

```
//subscrpt. h 开始
// Template Numerical Toolkit (TNT) for Linear Algebra BY R. Pozo

#ifndef SUBSCRIPT_H
#define SUBSCRIPT_H
#ifndef TNT_SUBSCRIPT_TYPE
#define TNT_SUBSCRIPT_TYPE int
#endif
namespace TNT
{
    typedef TNT_SUBSCRIPT_TYPE Subscript;
}
#define TNT_BASE_OFFSET (1)
#endif
//subscrpt. h 结束
//index. h 开始
// Vector/Matrix/Array Index Module
#ifndef INDEX_H
#define INDEX_H
```



```

#include "subscript. h"
namespace TNT
{
class Index1D
{
    Subscript lbound_;
    Subscript ubound_;
public:
    Subscript lbound() const { return lbound_; }
    Subscript ubound() const { return ubound_; }
    Index1D(const Index1D &D) : lbound_(D.lbound_), ubound_(D.ubound_) {}
    Index1D(Subscript i1, Subscript i2) : lbound_(i1), ubound_(i2) {}
    Index1D & operator=(const Index1D &D)
    {
        lbound_ = D.lbound_;
        ubound_ = D.ubound_;
        return * this;
    }
};

inline Index1D operator+(const Index1D &D, Subscript i)
{
    return Index1D(i+D.lbound(), i+D.ubound());
}

inline Index1D operator+(Subscript i, const Index1D &D)
{
    return Index1D(i+D.lbound(), i+D.ubound());
}

inline Index1D operator-(Index1D &D, Subscript i)
{
    return Index1D(D.lbound()-i, D.ubound()-i);
}

inline Index1D operator-(Subscript i, Index1D &D)
{
    return Index1D(i-D.lbound(), i-D.ubound());
}

} // namespace TNT
#endif
//index. h 结束
// version. h 开始
#ifdef TNT_VERSION_H
#define TNT_VERSION_H
// Version 0.9.0
#define TNT_MAJOR_VERSION    0

```



```

#define TNT_MINOR_VERSION    9
#define TNT_SUBMINOR_VERSION 0
#endif
// version. h 结束
//tntmath. h 开始
// Header file for scalar math functions
#ifndef TNTMATH_H
#define TNTMATH_H
// conventional functions required by several matrix algorithms
namespace TNT
{
    inline double abs(double t)
    {
        return ( t > 0 ? t : -t);
    }
    inline double min(double a, double b)
    {
        return (a < b ? a : b);
    }
    inline double max(double a, double b)
    {
        return (a > b ? a : b);
    }
    inline float abs(float t)
    {
        return ( t > 0 ? t : -t);
    }
    inline float min(float a, float b)
    {
        return (a < b ? a : b);
    }
    inline float max(float a, float b)
    {
        return (a > b ? a : b);
    }
    inline double sign(double a)
    {
        return (a > 0 ? 1 : -1);
    }
    inline float sign(float a)
    {
        return (a > 0 ? 1 : -1);
    }
}

```

★★★★★

```

} /* namespace TNT */
#endif
// TNTMATH_H
//tntmath. h 结束
//tnt. h 开始
#ifndef TNT_H
#define TNT_H

//  tnt. h      TNT general header file.  Defines default types and conventions.
#include "version. h"

#include "subscript. h"

#define TNT_BOUNDS_CHECK
#ifdef TNT_NO_BOUNDS_CHECK
#undef TNT_BOUNDS_CHECK
#endif

#define TNT_USE_REGIONS

// if your system doesn't have abs() min(), and max() uncomment the following
// #define __NEED_ABS_MIN_MAX_
#include "tntmath. h"
#endif
//tnt. h 结束
// vec. h 开始
// Basic TNT  numerical vector (0-based [i] AND 1-based (i) indexing )
#ifndef VEC_H
#define VEC_H
#include "subscript. h"
#include <cstdlib>
#include <cassert>
#include <iostream>
#include <strstream>
namespace TNT
{
template <class T>
class Vector
{
public:
    typedef Subscript    size_type;
    typedef      T      value_type;
    typedef      T      element_type;

```

```

typedef      T *   pointer;
typedef      T *   iterator;
typedef      T&    reference;
typedef const T *   const_iterator;
typedef const T&    const_reference;
Subscript lbound() const { return 1;}
protected:
    T * v_;
    T * vml_;          // pointer adjustment for optimized 1—offset indexing
    Subscript n_;
    // internal helper function to create the array
    // of row pointers
    void initialize(Subscript N)
    {
        // adjust pointers so that they are 1—offset:
        // v_[] is the internal contiguous array, it is still 0—offset
        //
        assert(v_ == NULL);
        v_ = new T[N];
        assert(v_ != NULL);
        vml_ = v_ - 1;
        n_ = N;
    }
    void copy(const T * v)
    {
        Subscript N = n_;
        Subscript i;
#ifdef TNT_UNROLL_LOOPS
        Subscript Nmod4 = N & 3;
        Subscript N4 = N - Nmod4;
        for (i=0; i<N4; i+=4)
        {
            v_[i] = v[i];
            v_[i+1] = v[i+1];
            v_[i+2] = v[i+2];
            v_[i+3] = v[i+3];
        }
        for (i=N4; i< N; i++)
            v_[i] = v[i];
#else
        for (i=0; i< N; i++)
            v_[i] = v[i];
#endif
    }
#endif

```



```

}
void set(const T& val)
{
    Subscript N = n_;
    Subscript i;
#ifdef TNT_UNROLL_LOOPS
    Subscript Nmod4 = N & 3;
    Subscript N4 = N - Nmod4;
    for (i=0; i<N4; i+=4)
    {
        v_[i] = val;
        v_[i+1] = val;
        v_[i+2] = val;
        v_[i+3] = val;
    }
    for (i=N4; i< N; i++)
        v_[i] = val;
#else
    for (i=0; i< N; i++)
        v_[i] = val;
#endif
}
void destroy()
{
    /* do nothing, if no memory has been previously allocated */
    if (v_ == NULL) return ;
    /* if we are here, then matrix was previously allocated */
    delete [] (v_);
    v_ = NULL;
    vml_ = NULL;
}
public:
    // access
    iterator begin() { return v_; }
    iterator end()   { return v_ + n_; }
    const iterator begin() const { return v_; }
    const iterator end() const   { return v_ + n_; }
    // destructor
    ~Vector()
    {
        destroy();
    }
    // constructors

```

```

Vector() : v_(0), vm1_(0), n_(0) {};
Vector(const Vector<T> &A) : v_(0), vm1_(0), n_(0)
{
    initialize(A.n_);
    copy(A.v_);
}
Vector(Subscript N, const T& value = T(0)) : v_(0), vm1_(0), n_(0)
{
    initialize(N);
    set(value);
}
Vector(Subscript N, const T * v) : v_(0), vm1_(0), n_(0)
{
    initialize(N);
    copy(v);
}
Vector(Subscript N, char * s) : v_(0), vm1_(0), n_(0)
{
    initialize(N);
    std::istringstream ins(s);
    Subscript i;
    for (i=0; i<N; i++)
        ins >> v_[i];
}
// methods
Vector<T> & newsize(Subscript N)
{
    if (n_ == N) return * this;
    destroy();
    initialize(N);
    return * this;
}
// assignments
Vector<T> & operator=(const Vector<T> &A)
{
    if (v_ == A.v_)
        return * this;
    if (n_ == A.n_) // no need to re-alloc
        copy(A.v_);
    else
    {
        destroy();
        initialize(A.n_);
    }
}

```

★★★★★

```

        copy(A.v_);
    }
    return *this;
}

Vector<T>& operator=(const T& scalar)
{
    set(scalar);
    return *this;
}

Subscript dim() const
{
    return n_;
}

Subscript size() const
{
    return n_;
}

inline reference operator()(Subscript i)
{
    #ifdef TNT_BOUNDS_CHECK
    assert(1<=i);
    assert(i <= n_);
    #endif
    return vml_[i];
}

inline const_reference operator()(Subscript i) const
{
    #ifdef TNT_BOUNDS_CHECK
    assert(1<=i);
    assert(i <= n_);
    #endif
    return vml_[i];
}

inline reference operator[](Subscript i)
{
    #ifdef TNT_BOUNDS_CHECK
    assert(0<=i);
    assert(i < n_);
    #endif
    return v_[i];
}

inline const_reference operator[](Subscript i) const
{

```



```

    # ifdef TNT_BOUNDS_CHECK
    assert(0<=i);
    assert(i < n_);
    # endif
    return v_[i];
}

friend std::istream & operator>>(std::istream &s, Vector<T> &A);
};

/* ***** I/O ***** */
template <class T>
std::ostream& operator<<(std::ostream &s, const Vector<T> &A)
{
    Subscript N=A.dim();
    s << N << endl;
    for (Subscript i=0; i<N; i++)
        s << A[i] << " " << endl;
    s << endl;
    return s;
}

template <class T>
std::istream & operator>>(std::istream &s, Vector<T> &A)
{
    Subscript N;
    s >> N;
    if ( ! (N == A.n_) )
    {
        A.destroy();
        A.initialize(N);
    }
    for (Subscript i=0; i<N; i++)
        s >> A[i];
    return s;
}

// ***** [ basic matrix algorithms ] *****
template <class T>
Vector<T> operator+(const Vector<T> &A,
    const Vector<T> &B)
{
    Subscript N = A.dim();
    assert(N==B.dim());
    Vector<T> tmp(N);
    Subscript i;
    for (i=0; i<N; i++)

```

```

    tmp[i] = A[i] + B[i];
    return tmp;
}

template <class T>
Vector<T> operator+(const Vector<T> &A, const Vector<T> &B)
{
    Subscript N = A.dim();
    assert(N==B.dim());
    Vector<T> tmp(N);
    Subscript i;
    for (i=0; i<N; i++)
        tmp[i] = A[i] + B[i];
    return tmp;
}

template <class T>
Vector<T> operator-(const Vector<T> &A, const Vector<T> &B)
{
    Subscript N = A.dim();
    assert(N==B.dim());
    Vector<T> tmp(N);
    Subscript i;
    for (i=0; i<N; i++)
        tmp[i] = A[i] - B[i];
    return tmp;
}

template <class T>
Vector<T> operator*(const Vector<T> &A, const Vector<T> &B)
{
    Subscript N = A.dim();
    assert(N==B.dim());
    Vector<T> tmp(N);
    Subscript i;
    for (i=0; i<N; i++)
        tmp[i] = A[i] * B[i];
    return tmp;
}

template <class T>
T dot_prod(const Vector<T> &A, const Vector<T> &B)
{
    Subscript N = A.dim();
    assert(N == B.dim());
    Subscript i;
    T sum = 0;
    for (i=0; i<N; i++)
        sum += A[i] * B[i];
    return sum;
}

} /* namespace TNT */
#endif
// VEC_H
// vec. h 结束

//main 开始
#include "tnt. h"

```

```
#include "vec.h"
using namespace std;
using namespace TNT;
void main()
{
    Vector<double> A;                                /* 1 */
    Vector<double> B(4);                             /* 2 */
    Vector<double> C(4, 1.1);                         /* 3 */
    double t[4] = {1.1, 2.2, 3.3, 4.4};              /* 4 */
    Vector<double> D(4, &t[0]);
    Vector<double> E(D);                              /* 5 */
    Vector<double> F(4, "1.3  2.7  6.3  1.4");        /* 6 */
    cout << A << endl;
    cout << B << endl;
    cout << C << endl;
    cout << D << endl;
    cout << E << endl;
    cout << F << endl;
}
//main 结束
```



第4题 矩阵转置

4.4.1 程序功能简介

根据向量处理的程序:实现矩阵转置。

4.4.2 课程设计要求

(1) 参照 4.3.4 节的源代码,改用单向链表表示 $n \times n$ 矩阵,其中 n 为用户可输入的任意数,并用程序实现验证 $n \times n$ 矩阵的如下定理:

$$(A+B)^T = (A)^T + (B)^T$$

其中 A, B 分别为 $n \times n$ 矩阵。

(2) (合做时)将程序扩充成通用,即上式左边()中为 m 个 $n \times n$ 矩阵。

(3) 将所有的函数与相应的数据封装到类中,并改写主程序,使程序简化。

4.4.3 课程设计说明

(1) 矩阵转置的数学原理很简单,因此程序的算法并不复杂,主要是要解决矩阵的表示方法以及实现矩阵的转置。

(2) 程序中采用了单向链表来表示矩阵,对矩阵的操作就非常方便了。

(3) 要实现转置则按一定的算法改变链表中节点的次序。

(4) 类 Vector 在 "Vector. h" 中声明,在 "Vector. cpp" 中实现。

(5) 评定难易级别:A 级。

4.4.4 关键源代码注解

参见 4.3.4。

第 5 题 矩阵加法

4.5.1 程序功能简介

根据向量处理的程序:实现矩阵的加法、乘法、转置、数乘运算。

4.5.2 课程设计要求

(1) 参照 4.3.4 节的源代码,改用单向链表表示 $n \times n$ 矩阵,其中 n 为用户可输入的任意数,并用程序实现验证 $n \times n$ 矩阵的如下定理:

$$A + (B + C) = (A + B) + C$$

$$A + B = B + A$$

其中 A, B, C 分别为 $n \times n$ 矩阵。

(2) 所有的函数与相应的数据封装到类中,并改写主程序,使程序简化。

4.5.3 课程设计说明

(1) 本程序应用了很多方法。比如:模板,链表,运算符重载,友元函数,标准输入输出。

(2) 在输入数据时,要严格按照屏幕提示进行输入,以免出错。

(3) 矩阵的输出形式是以三元组的形式(行号,列号,元素),比如:(1,1,5)表示矩阵中第一行,第一列的元素值是 5。

(4) 应以大写字母表示矩阵,如矩阵 A ;以小写字母表示系数或变量。

(5) 评定难易级别:A 级。

4.5.4 关键源代码注解

参见 4.3.4。



第 6 题 矩阵乘法验证

4.6.1 程序功能简介

实现矩阵的减法、乘法、转置运算。

4.6.2 课程设计要求

(1) 参照 4.3.4 节的源代码,改用单向链表表示 $n \times n$ 矩阵,其中 n 为用户可输入的任意数,并用程序实现验证 $n \times n$ 矩阵的如下定理:

$$A(BC) = (AB)C$$

其中 A, B, C 分别为 $n \times n$ 矩阵。

(2) (合做时)将程序扩充成通用,即上式左边()中为 m 个 $n \times n$ 矩阵。

(3) 将所有的函数与相应的数据封装到类中,并改写主程序,使程序简化。

4.6.3 课程设计说明

评定难易级别:A 级。

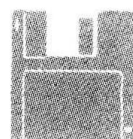
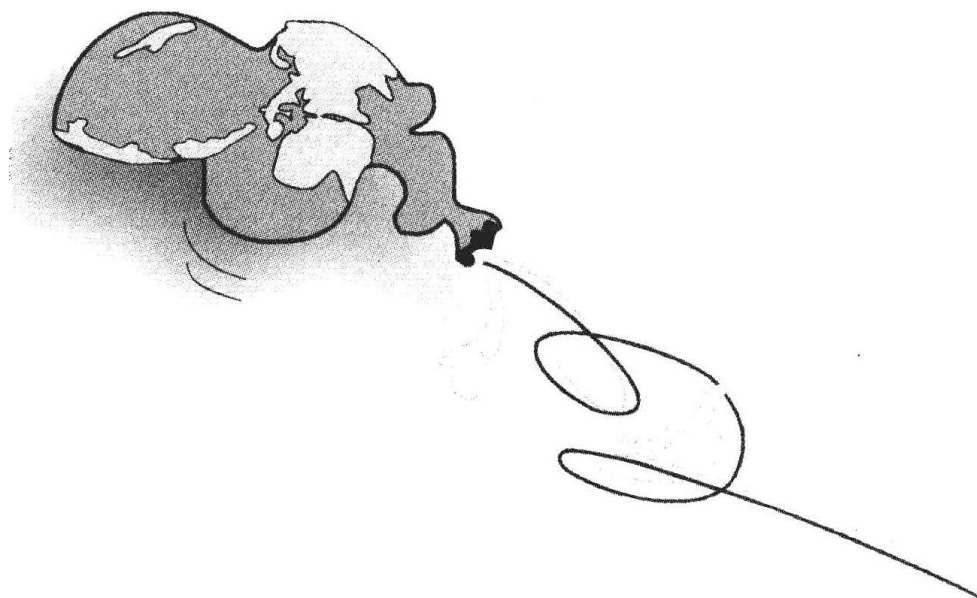
4.6.4 关键源代码注解

参见 4.3.4。



5

小型管理程序



第 1 题 工资管理

5.1.1 程序功能简介

一个员工工资管理程序,有新建、存储、显示、修改和删除功能。

5.1.2 课程设计要求

- (1) 完善帮助信息,帮助信息没有把操作的所有命令列出。
- (2) 一进入程序首先显示所有命令。
- (3) 使用 n 键新建一个数据项并询问是否要存储,按 y 键存储,其他键取消存储。
- (4) 显示、修改和删除数据项时大小写通用。
- (5) 将所有的函数与相应的数据封装到类中,并改写主程序。

5.1.3 课程设计说明

- (1) 可用 total 来存放工资总数。
- (2) n—New,新建一个数据。
- (3) s—Save,保存数据。
- (4) l—List,列出数据。
- (5) f—Find,查找数据。
- (6) e—Edit,编辑数据。
- (7) d—Delete,删除数据。
- (8) 评定难易级别:B 级。

5.1.4 关键源代码注解

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
/* structure that represents an employee record */
struct EMP {
    char name[25];
    float salary;
    int deleted;
};
void main() {
    EMP e; //Employee object for data storing & manipulation
    strcpy(e.name, "NULL"); // Default values are
    e.salary = 0;           // assigned
    e.deleted = 0;
    while(1) {
```

```

char c; // User command is recieved in this variable
printf("\nDB >"); // Command prompt
c = getch(); // Getting user input. Type 'h' for possible values
if(c == 'n') // New data is recieved int e
    printf("\nName: "); // ( but it is not saved into a file)
    scanf("%s", e.name);
    printf("Salary: ");
    scanf("%f", &e.salary);
    e.deleted = 0;
} else if(c == 's') // Saves current employee to a file
    if((strcmp(e.name, "NULL")) == 0) { // Checks if user entered new data
        printf("\nEnter employee data before trying to save");
        continue;
    }

    /* If new data was entered a file ,EMP. dat, is opened in append mode
    & employee record is written to it */
    FILE * file;
    if((file = fopen("EMP. DAT", "a")) == NULL) {
        printf("\nFailed to save data");
        continue;
    }
    fwrite((char *) &e, sizeof(e), 1, file);
    fclose(file);
    printf("\nFile saved");
} else if(c == 'l') { // If user typed 'l' records in EMP. DAT is read int e in a loop
    // & displayed to the user

    FILE * file;
    if((file = fopen("EMP. DAT", "r")) == NULL) {
        printf("\nFailed to read file");
        continue;
    }
    long curpos, length; // Variables for finding file size

    curpos = ftell(file); // Current position of filepointer is stored in curpos

    fseek(file, 0L, SEEK_END); // File pointer is moved 0 bytes from file end
    // ie, file pointer is at the file's last byte

    length = ftell(file); // Now current position is file's length

    fseek(file, curpos, SEEK_SET); // File pointer is restored to file's beginning

```

save

```

long num_recs = length / sizeof(e);    // Length of each record in the file

for(long l = 0; l < num_recs; l++) {    // Recursively retrieves & prints each record in file
    fread((char *) &e, sizeof(e), 1, file);
    printf("\nName: %s", e.name);
    printf("\nSalary: %.2f", e.salary);
}
fclose(file);
printf("\n\n\tRecords: %d", num_recs);

} else if(c == 'h') {    // Prints help
    printf("\n\n - New\n - Save\n - List\n - Quit");
} else if(c == 'q') {    // Exits while loop & program
    break;
} else if(c == 'f') {    // Searches the database for an employee based on employee name
    // entered by user

    FILE * file;
    if((file = fopen("EMP.DAT", "r")) == NULL) {    // Checks to see if file is accessible
        printf("Failed to open file");
        continue;
    }

    // Gets employee name to perform search
    printf("Enter employee name: ");
    char f[25];
    scanf("%s", f);

    // Finds file size
    long curpos, length;
    curpos = ftell(file);
    fseek(file, 0L, SEEK_END);
    length = ftell(file);
    fseek(file, curpos, SEEK_SET);
    long num_recs = length / sizeof(e);
    int found = 0;    // To check if record was found or not
    for(long l = 0; l < num_recs; l++) {
        /* Recursively checks each name if found prints details & sets found to 1 */
        fread((char *) &e, sizeof(e), 1, file);
        if((strcmp(e.name, f)) == 0) {

```

```

    printf("\nName: %s", e.name);
    printf("Salary: %f", e.salary);
    found = 1;
}
}

if(found == 0)
    printf("Record not found. ");

} else if(c == 'd') { // Deletes an employee

    FILE * file;
    FILE * temp_file;
    if((file = fopen("EMP.DAT", "r")) == NULL) {
        printf("Failed to load data");
        continue;
    }

    if((temp_file = fopen("EMP_TEMP.DAT", "w")) == NULL) {
        printf("Failed to create temp file");
        continue;
    }

    printf("Enter employee name: ");
    char f[25];
    scanf("%s", f);
    long curpos, length;
    curpos = ftell(file);
    fseek(file, 0L, SEEK_END);
    length = ftell(file);
    fseek(file, curpos, SEEK_SET);
    long num_recs = length / sizeof(e);
    int found = 0;
    for(long l = 0; l < num_recs; l++) {
        fread((char *) &e, sizeof(e), 1, file);
        if((strcmp(e.name, f)) == 0) {
            e.deleted = 1; // If record found sets it deleted flag to true
            found = 1;
        }
        fwrite((char *) &e, sizeof(e), 1, temp_file);
        // Write all records read to a temporary file
    }

    fclose(file);
    fclose(temp_file);
    if(found == 0) {

```

```

        printf("Record not found. ");
        unlink("EMP_TEMP.DAT");//If record was not found deletes temp file
                                   // & continues main command loop
        continue;
    }
    //If record was found rewrites all records from temp file to EMP.DAT
    /* where deleted flag is 0 & deletes the tem file in the end */
    file = fopen("EMP.DAT","w");
    temp_file = fopen("EMP_TEMP.DAT","r");
    curpos = ftell(temp_file);
    fseek(temp_file, 0L, SEEK_END);
    length = ftell(temp_file);
    fseek(temp_file, curpos, SEEK_SET);
    num_recs = length / sizeof(e);
    for(l = 0; l < num_recs; l++) {
        fread((char *) &e, sizeof(e), 1, temp_file);
        if(e.deleted == 0) {
            fwrite((char *) &e, sizeof(e), 1, file);
        }
    }

    fclose(file);
    fclose(temp_file);
    printf("Record deleted");
    unlink("EMP_TEMP.DAT");

} else if(c == 'e') { //Edits a record
    FILE * file;
    if((file = fopen("EMP.DAT","r")) == NULL) {
        printf("Failed to open file");
        continue;
    }
    printf("Enter employee name: ");
    char f[25];
    scanf("%s", f);
    long curpos, length;
    curpos = ftell(file);
    fseek(file, 0L, SEEK_END);
    length = ftell(file);
    fseek(file, curpos, SEEK_SET);
    long num_recs = length / sizeof(e);
    int found = 0;
    //Variables to store new name & salary
    char new_name[25];

```

```

float new_sal;
EMP new_emp;
long write_at;    //Position of record in to edit
for(long l = 0;l<num_recs;l++) {
    fread((char *) &e,sizeof(e),1,file);
    if((strcmp(e.name,f))==0)
    {    //If record exists in file get new values
        printf("Enter new name: ");
        scanf("%s",new_name);
        printf("Enter new salary: ");
        scanf("%f",&new_sal);
        //Copies values to a new EMP object
        strcpy(new_emp.name,new_name);
        new_emp.salary = new_sal;
        new_emp.deleted = 0;
        found = 1;
        write_at = l; // Position to write new record
        break;
    }
}
fclose(file);
if(found == 1) {
    //Calculates byte posiotn to write
    write_at = write_at * sizeof(new_emp);
    //Mode r+b opens a file to be writable & readable in binary
    if((file = fopen("EMP.DAT","r+b")) == NULL) {
        printf("Editing failed");
        continue;
    }
    fseek(file,write_at,SEEK_SET);
    fwrite((char *) &new_emp,sizeof(new_emp),1,file);
    fclose(file);
    printf("Record edited");
} else {
    printf("Record not found");
}

} else {
    printf("Type 'h' for valid commands ");
}
}

```

第2题 物品库存管理

5.2.1 程序功能简介

用文件系统实现的物品库存管理程序。

5.2.2 课程设计要求

- (1)在 Item 类 search()中增加按价格搜索的功能。
- (2)仿 Item 类设计客户类,增加客户管理程序。
- (3)一进入程序,首先显示汉化命令,以使用户选择。
- (4)设计输出每日进货与出货统计表。
- (5)可自行扩充本程序功能,使本程序功能更强。

5.2.3 课程设计说明

评定难易级别:B级。

5.2.4 关键源代码注解

```
// file operations
#include<iostream, h>
#include<conio, h>
#include<stdio, h>          //for gets and puts
#include<process, h>        //exit(1)
#include<fstream, h>        //file operations
#include<string, h>
#include<dos, h>

class Brand
{
    private:
        char    brnd_name[20];
        int     Brand_code;
        int     quantity;
        float    brnd_price;
    public:
        void getdata();
        void showdata();
};

class Item
{
    private:
        char Item_name[10];
```

```

    int Item_code;
    char colour[10];
    int quantity;
    Brand brnd[3];
    //Private Member functions used internally
    void getdata();
    void showdata();
public:
    void add();
        void remove();
        void search();
    void showab();
};

// GET Brand DATA
// gets data about the Brand
// it is temporary storage

void Brand::getdata()
{
    cout<<"\n\nEnter Brand Name ";
    gets(brnd_name);
    cout<<"\nEnter Brand Code ";
    cin>>Brand_code;
    cout<<"\nEnter Quantity ";
    cin>>quantity;
    cout<<"\nEnter Price";
    cin>>brnd_price;
}

// SHOW Brand DATA
// shows the data of the Brand
void Brand::showdata()
{
    cout<<"\n\nName          : "; puts(brnd_name);
    cout<<"\n\nCode          : "; cout<<Brand_code;
    cout<<"\n\nQuantity       : "; cout<<quantity;
    cout<<"\n\nPrice          : "; cout<<brnd_price;
}

// GET Item DATA

// gets data about the Item it is temporary storage

void Item::getdata()
{

```



```

        cout<<"\nEnter Item Name ";
        gets(Item_name);
        cout<<"\nEnter Item Code  ";
        cin>>Item_code;
        cout<<"\nEnter Colour ";
        gets(colour);
        cout<<"\nEnter Quantity ";
        cin>>quantity;
        cout<<"\n\nEnter Brand 1 ";
        brnd[0].getdata();
        cout<<"\n\nEnter Brand 2 ";
        brnd[1].getdata();
        cout<<"\n\nEnter Brand 3 ";
        brnd[2].getdata();
    }

// SHOW Item DATA
//shows data about the Item
void Item::showdata()
{
    cout<<"\n\n\t\tItem Data";
    cout<<"\n\nItem Name      :  ";
    puts(Item_name);
    //写一个字符串到标准的输出流(或缓冲区)——Write a string to stdout
    cout<<"\n\nItem Code      :  ";
    cout<<Item_code;
    cout<<"\n\nColour       :  ";
    puts(colour);
    cout<<"\n\nQuantity       :  ";
    cout<<quantity<<endl;
    getch();
    // clrscr();
    cout<<"\n\nBrand 1"<<endl; brnd[0].showdata();
    getch();
    //clrscr();
    cout<<"\n\nBrand 2"<<endl; brnd[1].showdata();
    getch();
    //clrscr();
    cout<<"\n\nBrand 3"<<endl; brnd[2].showdata();
}

//Add Item
/* this function first gets data into temp storage then
   saves it on the hard disk
   */

```

```

void Item::add()
{
    //clrscr();
    ofstream file( "TIS.txt", ios::out |ios::ate ); //open file for input
    if(! file)    //if file could not be opened
    {
        cout<<"Error Could Not Open File";
        getch();
        exit(1);
    }

    Item p1;
    p1.getdata();    //Get Data From User For Temp. Storage
    // write data to hard disk
    file.write(reinterpret_cast <const char * > (&p1),sizeof(Item) );
}

```

// TRAVERSE

/* this function shows the list of all the Brands */

```

void Item::showab()
{
    //clrscr();
    ifstream file( "TIS.txt", ios::in); //open file for output
    if(! file)    //if file could not be opened
    {
        cout<<"Error Could Not Open File";getch();
        exit(1);
    }

    Item p1;
    file.read(reinterpret_cast<char * >(&p1),sizeof(Item));
    //Store Data In Object
    while( ! file.eof() ) //Untill Contacts Present
    {
        //clrscr();
        p1.showdata() ; //Display on Screen
        getch();    //Wait For Keyboard Input
        file.read(reinterpret_cast<char * > (&p1),
        sizeof(Item));    //Read Next Contact
    }
}

```

// SEARCH

```

void Item::search()
{

```

```

        //clrscr();
        char name1[10];
        int code;
        int option;
        cout<<"\n(1)Search By Name\n(2)Search By Code";
        cin>>option;
        if(option==1)
        {
            cout<<"\nEnter Item name  ";
            gets(name1);
            code=0;
        }
        else
        {
            cout<<"\nEnter Code of Item To Search  ";
            cin>>code;
            strcpy(name1,"null");
        }

        fstream file( "TIS. txt" ,ios::in); //Open File For Output/Read
        if(! file) //If File Could Not Be Opened
        {
            cout<<"Error Could Not Open File";
            getch();
            exit(1);
        }

        char flag='a'; //Flag To Check If Found
        Item p1;;
        file.read(reinterpret_cast<char * >(&p1),sizeof(Item));
        //Read Data To Object
        while( ! file.eof() ) //Untill There Are Contacts
        {
            if((strcmp(p1.Item_name,name1)==0) || (p1.Item_code==code)) //Compare
            {
                //clrscr();
                p1.showdata();
                getch();
                flag='z'; //Set Flag
                break; //Break Loop
            }

            file.read(reinterpret_cast<char * > (&p1), sizeof(Item));
            //Read Next Contact
        }

        if(flag != 'z') //If Not Found

```



```

        {
            cout<<("Item Not Found");
            getch();
        }
    }

// REMOVE Item
void Item::remove()
{
    //clrscr();
    char name1[10];
    int code;
    int option;
    cout<<"\n(1)Delete By Name\n(2)Delete By Code";
    cin>>option;
    if(option==1)
    {
        cout<<"\nEnter Item name  ";
        gets(name1);
        code=0;
    }
    else
    {
        cout<<"\nEnter Code of Item To Search  ";
        cin>>code;
        strcpy(name1,"null");
    }
    ifstream file( "TIS.txt" ,ios::in );
    ofstream temp( "delete.txt", ios::trunc);
    if( ! file || ! temp)
    {
        cout<<"Could Not Open File";
        getch();
        exit(1);
    }
    file.seekg(0);
    temp.seekp(0);
    Item p1;
    file.read(reinterpret_cast<char * > (&p1),sizeof(Item));
    while( ! file.eof() )
    {
        if( (strcmp(name1,p1.Item_name)==0) || (code==p1.Item_code))
        {

```

```
file.read(reinterpret_cast<char *> (&p1), sizeof(Item));
```

```
else
```

```
{
```

```
temp.write(reinterpret_cast<char *> (&p1), sizeof(Item));
```

```
file.read(reinterpret_cast<char *> (&p1), sizeof(Item));
```

```
}
```

```
}
```

```
file.close();
```

```
temp.close();
```

```
ifstream temp1("delete.txt", ios::in);
```

```
ofstream file1("TIS.txt", ios::trunc);
```

```
temp1.seekg(0);
```

```
file1.seekp(0);
```

```
temp1.read( reinterpret_cast<char *> (&p1), sizeof(Item) );
```

```
while( ! temp1.eof() )
```

```
{
```

```
file1.write( reinterpret_cast<char *> (&p1), sizeof(Item) );
```

```
temp1.read( reinterpret_cast<char *> (&p1), sizeof(Item) );
```

```
}
```

```
temp1.close();
```

```
file1.close();
```

```
}
```

```
// MAIN FUNCTION
```

```
void main()
```

```
{
```

```
Item m;
```

```
int choice;
```

```
while(1)
```

```
{
```

```
clrscr();
```

```
cout<<"\n\tThe Inventory System (TIS)";
```

```
cout<<"\n\n(1) Add Item\n\n(2) Show All Data"
```

```
<<"\n\n(3) Search Item\n\n(4) Remove Item\n\n(5) Exit TIS";
```

```
cout<<"\n\nEnter Choice ";
```

```
cin>>choice;
```

```
switch(choice)
```

```
{
```

```
case 1;
```

```
m.add();
```

```
break;
```

★★★★★

```
case 2:
    m.showab();
    break;
case 3:
    m.search();
    break;
case 4:
    m.remove();
    break;
case 5:
{
    exit(1);
}
}
}
}
```

第3题 股票交易系统

5.3.1 程序功能简介

股票交易系统的管理程序。

5.3.2 课程设计要求

(1) 增加股票交易系统的接口程序,新增设计:

① Add_New_Stock()——增加新股票;

② Delete_Old_Stock()——删除旧股票;

③ Hang_Up_Stock()——挂起股票,停止交易;

④ Modify_Stock()——修改股票的名称、代码。

(2) 将修改后的结果存入 Stock_File.dat 中。

(3) 一进入程序,首先显示汉化命令,以便用户使用。

(4) 可自行扩充本程序功能,使本程序功能更强。

5.3.3 课程设计说明

(1) 当以管理员身份登陆系统时才可使用增加、删除、挂起、修改股票的函数。

(2) 源程序在逻辑上存在错误和缺陷。

(3) 程序中存在重复部分,可设计函数对重复部分做优化。

(4) 评定难易等级:A级。

5.3.4 关键源代码注解

```
#include <iostream. h>
#include <stdlib. h>
#include <string. h>
#include <conio. h>
#include <fstream. h>
#include <iomanip. h>
#define MAX 5
class Stock
{
    friend class Customer;
private:
    char stock_name[20];
    char stock_code[6];
    char code_for_analysis[6];
    char choice;
    int num_of_stock;
```

```

        long int stock_volume;
        long int volume_available;

public:
    Stock();
    void Interface();
    void Stock_Exchange_Market();
    void Switch_choice();
    void Input_For_New_Stock();
    void Market_Analysis();
};

class Customer
{
private:
    char customer_name[30];
    char password[10];
    char share_holding_name[30];
    char share_holding_code[5];
    long int share_holding_value;

public:
    Customer();
    void Log_in(char * ,Customer) const;
    void Register(Customer * );
    void Stock_Portfolio(Customer) const; // Portfolio———有价证券
    void Switch_function();
};

Customer::Customer()
{
    strcpy(Customer::customer_name, " ");
    strcpy(Customer::share_holding_name, " ");
    strcpy(Customer::share_holding_code, " ");
    share_holding_value=0;
}

void Customer::Log_in(char * password, Customer cust) const
{
    ifstream read_customer("customer.dat");
    while(! read_customer.eof())
    {
        read_customer.read((char * )(&cust), sizeof(cust));
        if(read_customer.fail())
        {
            cout<<"No record at all!";
            break;
        }
    }
}

```



```

        if(strcmp(password,cust. password)==0)
            Stock_Portfolio(cust);
    }
    read_customer.close();
}

void Customer::Register(Customer * cust)
{
    system("cls"); //执行系统命令:清屏=cls
    cin.get();
    cout<<"Welcome to * * * * * Stock Exchange System"<<endl;
    cout<<"~~~~~"~<<endl;
    cout<<"Enter user name: ";
    cin.getline(customer_name,30);
    cout<<"Enter your own password(less than 8 words): ";
    cin>>password;
    cin.get();
    system("cls");
    cout<<endl<<endl;
    cout<<"Your account have been registered..."<<endl;
    cout<<"Press any key to proceed..."<<endl;
    getch();
}

void Customer::Stock_Portfolio(Customer cust) const
{
    Stock share[MAX],temp;
    Customer s[MAX];
    int i;
    char ch,share_code[6];
    long int volume;
start:
    system("cls");
    cout<<endl<<endl;
    cout<<"\t\t\t\t\t Welcome Back!!!!"<<endl;
    cout<<"\t\t\t\t\t * * * * * Stock Exchange System"<<endl;
    cout<<"\t\t\t\t\t ~~~~~"~<<endl;
    cout<<"\t\t\t\t\t User: "<<cust.customer_name<<endl<<endl;
    cout<<"\t\t\t\t\t Buy Shares.....[b]"<<endl;
    cout<<"\t\t\t\t\t Sell Shares.....[s]"<<endl;
    cout<<"\t\t\t\t\t View PortFolio.....[p]"<<endl;
    cout<<"\t\t\t\t\t Log Out.....[l]"<<endl;
    cout<<endl<<"\t\t\t\t\t Enter your choice: ";
    cin>>ch;
    if((ch!= 'p')&&(ch!= 'b')&&(ch!= 's')&&(ch!= 'l'))

```

```

    goto start;
else
{
switch(ch)
{
case 'b':{
    system("cls");
    cout<<"\n\n\t\t* * * * * Stock Exchange System"<<endl;
    cout<<"\t\t~~~~~"~<<endl;
    cout<<"  Company\t\tCode\tIssued Volume\tVolume Available"<<endl;
    cout<<"  ~~~~~\t\t~~~~~\t~~~~~\t~~~~~\t"~<<endl;
    ifstream dataFile("Stock_File.dat"); //Subject to change
    dataFile.read((char * )(&share),sizeof(share));
    ifstream dataFile_for_share(cust.customer_name); //Subject to change
    dataFile_for_share.read((char * )(&s),sizeof(s)); // open file for s[x]
    for(i=0;i<MAX;++i)
    {
        share[i].Stock_Exchange_Market();
    }
    cin.get();
    cout<<endl<<"Enter the Company's code you want to purchase: ";
    cin.getline(share_code,6);
    cout<<"Enter the volume to purchase: ";
    cin>>volume;
    cin.get();
    i=0;
    while((strcmp(share[i].stock_code,share_code)==0) || (i<MAX))
    {
        if(strcmp(share[i].stock_code,share_code)==0)
        {
            if(share[i].volume_available>volume)
            {
                s[i].share_holding_value=s[i].share_holding_value+volume;// cust change to s[x]
                strncpy(s[i].share_holding_name,share[i].stock_name,30); // cust change to s[x]
                strcpy(s[i].share_holding_code,share[i].stock_code); // cust change to s[x]
                share[i].volume_available=share[i].volume_available-volume;
                ofstream dataFile (cust.customer_name,ios::binary); //Subject to change
                for(i=0;i<5;++i)
                {
                    dataFile.write((char * )(&s[i]),sizeof(s[i]));
                } // create or save file for all transactions
                ofstream file("Stock_File.dat");
                for(i=0;i<5;++i)

```

```

        {
            file.write((char * )(&share[i]), sizeof(share[i]));
        }

        break;
    }
    else
    {
        cout<<"Volume available for "<<share[i].stock_name
        <<" is less than the volume you want to purchase. "<<endl;
        cout<<"Transaction cancel. . . ";
        getch();
        break;
    }
    }
    else
    {
        i++;
        while(i==MAX)
        {
            cout<<"The code you type may be wrong. . . "<<endl;
            cout<<"Transaction cancel. . . ";
            getch();
            break;
        }
    }
    break;
}

case 's':{
    system("cls");
    ifstream dataFile_for_share(cust.customer_name);
    dataFile_for_share.read((char * )(&s), sizeof(s));    // open file for s[x]
    ifstream dataFile("Stock_File.dat");
    dataFile.read((char * )(&share), sizeof(share));
    cout<<"\t\t* * * * * Stock Exchange System"<<endl;
    cout<<"\t\t~ ~ ~ ~ ~"<<endl;
    cout<<"  Company\t\tCode\tVolume Holding"<<endl;
    cout<<"  ~ ~ ~ ~ ~\t\t~ ~ ~ ~ ~\t"<<endl;
    for(int i=0; i<MAX; i++)
    {
        cout<<"  "<<s[i].share_holding_name<<"\t\t"<<s[i].share_holding_code
        <<"\t"<<s[i].share_holding_value<<endl;
    }
}

```



```

}
cin.get();
cout<<endl<<"Enter share code to sell: ";
cin.getline(share_code,6);
cout<<"Enter the volume to sell: ";
cin>>volume;
cin.get();
i=0;

while((strcmp(share[i].stock_code,share_code)==0) || (i<MAX))
{
    if(strcmp(share[i].stock_code,share_code)==0)
    {
        if(s[i].share_holding_value>volume)
        {
            s[i].share_holding_value=s[i].share_holding_value-volume;
            // cust change to s[x]
            share[i].volume_available=share[i].volume_available+volume;
            ofstream data(cust.customer_name,ios::binary); //Subject to change
            for(i=0;i<5;++i)
            {
                data.write((char *)(&s[i]),sizeof(s[i]));
            } // create or save file for all transactions
            ofstream file("Stock_File.dat");
            for(i=0;i<5;++i)
            {
                file.write((char *)(&share[i]),sizeof(share[i]));
            }
        }
        else
        {
            cout<<"You don have so much volume to sell, ";
            cout<<"or you did not buy this share before. "<<endl;
            cout<<"Transaction cancel. . . "<<endl;
            break;
        }
    }
}
else
{
    i++;
    while(i==MAX)
    {
        cout<<"You did not buy this share or you have type in wrong share code. "<<

```

```

        endl;
        cout<<"Transcation cancel. . . ";
        break;
    }
    }
    }
    getch();
    break;
}

case 'p':{
    system("cls");
    ifstream dataFile_for_share(cust.customer_name); //Subject to change
    dataFile_for_share.read((char * )(&s),sizeof(s)); // open file for s[x]
    cout<<"\t\t* * * * * Stock Exchange System"<<endl;
    cout<<"\t\t~ ~ ~ ~ ~"<<endl;
    cout<<" Company\t\tCode\tVolume Holding"<<endl;
    cout<<" ~ ~ ~ ~ ~\t\t~ ~ ~ ~ ~\t\t~ ~ ~ ~ ~\t"<<endl;
    for(int i=0;i<MAX;i++)
    {
        cout<<" "<<s[i].share_holding_name<<"\t\t"<<
        s[i].share_holding_code<<"\t"<<s[i].share_holding_value<<endl;
    }
    getch();
    break;
}

case 't':{
    Stock s;
    s.Interface();
}

}

goto start;
}

}

Stock::Stock()
{
    num_of_stock=0;
    volume_available=stock_volume;
    strcpy(Stock::code_for_analysis,"");
}

void Stock::Interface()
{
    start:
    system("cls");

```

```

    cout<<"\n\n\t\t * * * * * Stock Exchange System"<<endl;
    cout<<"\t\t ~~~~~~" <<endl;
    cout<<"\t\t View Stock Exchange Market. .... [m]"<<endl;
    cout<<"\t\t Login for existing user. .... [i]"<<endl;
    cout<<"\t\t Register for new user. .... [r]"<<endl;
    cout<<"\t\t Market analysis. .... [v]"<<endl;
    cout<<"\t\t Quit this program. .... [q]"<<endl;
    cout<<endl<<"\t\t Enter your choice here: ";
    cin>>choice;
    if((choice! = 'm') && (choice! = 'i') && (choice! = 'r') && (choice! = 'v') && (choice! = 'a') && (choice! = 'q'))
    {
        system("cls");
        goto start;
    }
    else
        Switch_choice();
}

void Stock::Switch_choice()
{
    Stock temp,s[MAX];
    Customer cl;
    char pass_word[10];
    switch(choice)
    {
    case 'm':{
        system("cls");
        cout<<"\n\n\t\t * * * * * Stock Exchange System"<<endl;
        cout<<"\t\t ~~~~~~" <<endl;
        cout<<" Company\t\tCode\tIssued Volume\tVolume Available"<<endl;
        cout<<" ~~~~~\t\t~~~~~\t~~~~~\t~~~~~\t" <<endl;
        ifstream dataFile("Stock_File.dat"); //Subject to change
        dataFile.read((char * )(&s),sizeof(s));
        for(int x=0;x<5;++x)
        {
            s[x].Stock_Exchange_Market();
        }
        getch();
        temp.Interface();
        break;
    }
    case 'i':{
        system("cls");
        cout<<endl<<endl;

```

```

        cin.get();
        cout<<" * * * * * Stock Exchange System"<<endl;
        cout<<" ~~~~~"<<endl;
        cout<<"Account password: ";
        cin>>pass_word;
        cin.get();
        cl.Log_in(pass_word,cl);
        temp.Interface();
        break;
    }

    case 'r':{
        system("cls");
        cout<<"Open New Account"<<endl;
        cout<<" ~~~~~"<<endl;
        cl.Register(&cl);
        ofstream write_customer("customer.dat",ios::app);
        write_customer.write((char *)&cl,sizeof(cl));
        write_customer.close();
        temp.Interface();
        break;
    }

    case 'v':{
        system("cls");
        cout<<"\n\n\t\t * * * * * Stock Exchange System"<<endl;
        cout<<"\t\t ~~~~~"<<endl;
        cout<<"  Company\t\tCode\tIssued Volume\tVolume Available"<<endl;
        cout<<"  ~~~~~\t\t\t\t\t\t\t\t\t\t\t\t\t\t\t\t\t"<<endl;
        temp.Market_Analysis();
        break;
    }

    case 'q':{
        system("cls");
        cout<<"Thanks for using * * * * * Stock Exchange System!"<<endl;
        cout<<"See you again!"<<endl;
        exit(0);
    }
}

void Stock::Input_For_New_Stock()
{
    system("cls");
    cin.get();
    cout<<"Enter new stock name(XX_XXX): ";

```

```

cin.getline(stock_name,20);
cout<<"Enter stock code (capital letter & less than 5 character): ";
cin.getline(stock_code,6);
cout<<"Enter issue volume: ";
cin>>stock_volume;
cin.get();
volume_available=stock_volume;
num_of_stock++;
}

void Stock::Stock_Exchange_Market()
{cout<<"  "<<stock_name<<"\t\t"<<stock_code<<"\t"<<stock_volume
<<"\t\t"<<volume_available<<endl;
}

void Stock::Market_Analysis()
{
    Stock s[MAX];
    int i=0,flag=0;
    ifstream dataFile("Stock_File.dat"); //Subject to change
    dataFile.read((char * )(&s),sizeof(s));
    for(i=0;i<MAX;++i)
    {
        s[i].Stock_Exchange_Market();
    }
    cout<<endl;
    cin.get();
    cout<<"Type in the share code to view market analysis: ";
    cin.getline(code_for_analysis,6);
    i=0;
    while((i<MAX)&&(! flag))
    {
        if(strcmp(s[i].stock_code,code_for_analysis)==0)
        {
            getch();
            flag=1;
            break;
        }
        else
            i++;
    }
    if(i==MAX)
    {
        cout<<"No code found"<<endl;
        cout<<"Press any key to proceed..."<<endl;
        getch();
    }
}

```



```
s[i]. Interface();
}
}
system("cls");
cout<<"\t\t\t\t\t Market Analysis "<<endl;
cout<<"\t\t\t\t\t ~~~~~~"<<endl;
cout<<"\t\t\t\t\t ("<<s[i]. stock_name<<) "<<endl<<endl;
if(i==0)
{
    if(s[i]. volume_available<=3000)
    {
        cout<<"\t"<<s[i]. stock_name<<" has a very good performance, as the company "<<endl;
        cout<<"\tjust signed another 4 years contract with IOC. As Beijing just won "<<endl;
        cout<<"\ttthe bid of 2008 Olympic Games, company is expected to widen its market "<<endl;
        cout<<"\tin this country. This has attract shareholders to invest in the share. "<<endl;
    }
    else if((s[i]. volume_available>3000) && (s[i]. volume_available<=7000))
    {
        cout<<"\t"<<s[i]. stock_name<<" is reserving its streak in the moderate side due to the"<<endl
        <<"\tstagnant sales. Regional market has been rather inactive and sales"<<endl<<
        "\thave risen less than expected. The market has also been quiet due to"<<endl<<"\ttthe
        domination by banking sector rally. "<<endl;
    }
    else
    {
        cout<<" "<<s[i]. stock_name<<" this week in poor performance. "<<s[i]. stock_
        name;
        cout<<" ended weaker after McDonald's Corp. said its yearly earnings would miss Wall
        ";
        cout<<" Street estimates and a forecast of future demand from Nike Inc. disappointed
        investors. "<<endl;
        cout<<" Stock index futures sold on several occasions this week, pressured by concerns
        that the Federal Reserve could raise interest ";
        cout<<"rates as early as May. Investors are worried that if the Fed is too quick to undo some
        of last year's aggressive rate cuts, ";
        cout<<"the central bank could choke off the strength of a"<<endl;
        cout<<"rebound in corporate profits that has yet to surface. "<<endl;
    }
}
else if (i==1)
{
    if(s[i]. volume_available < 3000)
    {
```



```

cout<<"\tMomentum has picked up for "<<s[i]. stock_name<<" in the last couple of days. "
<<endl
<<"\tThis favorable earnings is due to the splitting of "<<s[i]. stock_name<<endl<<
"\tshares, thus increasing increasing the number of outstanding shares "<<endl<<
"\twwithout diluting shareholders. This sharp rise has attracted numerous" <<endl
<<"\tshareholders to invest in the share. "<<endl;
}
else if ((s[i]. volume_available < 7000) && (s[i]. volume_available >= 3000))
{
cout<<"\t"<<s[i]. stock_name<<" is reserving its streak in the moderate side due to the"<<
endl<<
"\tstagnant sales. Regional market has been rather inactive and sales"<<endl<<"\thave risen less
than expected. The market has also been quiet due to"<<endl<<
"\tthe domination by banking sector rally. "<<endl;
}
else
{
cout<<"\t"<<s[i]. stock_name<<" close out with a whimper, indicating weak revenue and "
<<endl<<"\tearnings. Sales have been rather poor and weakness in the regional "<<endl<<
"\tmarket is expected to continue weighing down the stock with growth "<<endl<<"\tremaining
at the downside. "<<endl;
}
}
else if (i==2)
{
if(s[i]. volume_available < 3000)
{
cout<<"\t"<<s[i]. stock_name<<" has continued its bearish run and has obtained solid gain "
<<endl<<"\tfrom its sales. Increase in local demand has contributed much to the"<<endl<<
"\trise in the company's sales. For the past few months, new branches"<<endl<<
"\twere blooming like mushrooms in various places, denoting much"<<endl<<
"\tpositive growth. "<<endl;
}
else if ((s[i]. volume_available < 7000) && (s[i]. volume_available >= 3000))
{
cout<<"\t"<<s[i]. stock_name<<" stock is maintaining its stability with its market rising and"
<<endl<<"\tdropping with little distinction, showing impressive resiliency. "<<endl<<
"\tSpeculation has it that this share is in a well position to rise. "<<endl;
}
else
{
cout<<"\t"<<s[i]. stock_name<<" stock continued to flounder after Food Maker Service
Rating"<<endl<<"\tannounced a negative statement on the company's services. The stock"<<

```

```

endl<<    "\tis experiencing some tired blown and showing more hang-tough"<<endl<<
"\ttendencies. Recovery is only expected if it's led by favorable"<<endl<<
"\twindows dressing activities. "<<endl;
}
}
else if (i==3)
{
    if(s[i]. volume_available < 3000)
    {
        cout<<"\tWith fresh leads, "<<s[i]. stock_name<<" are in the position of favorable "<<endl
        << "\tearnings. The company has also gained good reputation from its fast, "<<endl<<
        "\tefficient and friendly services, not to mention their excellent quality"<<endl<<
        "\tproducts. In short, the market is in its bearish mood. "<<endl;
    }
    else if ((s[i]. volume_available < 7000) && (s[i]. volume_available >= 3000))
    {
        cout<<"\t"<<s[i]. stock_name<<" closed in a stable manner, sustaining its point-sized.
        "<<endl<< "\tRevenues and earnings from the company has been dormant but in a "<<endl<<
        "\trather positive mode. The share has, in short, been showing quite "<<endl<<
        "\tremarkable flexibility. "<<endl;
    }
    else
    {
        cout<<"\t"<<s[i]. stock_name<<" share is in a bad prospect, with its regional market "<<endl
        << "\tdemand experiencing offset decline. The slowing down of this stock has "<<endl<<
        "\tmade itself one of the point-sized losers falling back with extremely "<<endl<<
        "\tlow capital. Furthermore, there was no fresh positive news to "<<endl<<
        "\tboost sentiment. "<<endl;
    }
}
else if (i==4)
{
    if(s[i]. volume_available < 3000)
    {
        cout<<"\tThe share of "<<s[i]. stock_name<<" is running its bearish race, leading with
        "<<endl<< "\texcellent growth in its sales. "<<s[i]. stock_name<<" too has gain investors'
        "<<endl<< "\tconfidence with the launch of its recent multi-million MOBILE COFFEE "<<endl
        << "\tBEAN project. It is now marked as one of the most active stock in the "<<endl
        << "\tmarket. "<<endl;
    }
    else if ((s[i]. volume_available < 7000) && (s[i]. volume_available >= 3000))
    {
        cout<<"\tTaking a slow stride, "<<s[i]. stock_name<<" is in its inert mood, still "<<endl

```

★ ★ ★ ★ ★

```
<<< "\tmaintaining its volume. The share has able to reserve its winning "<<<endl<<<
"\tstreak though, through some street sponsorship and favorable budgets. "<<<endl<<<
"\tHowever, the market is to be inactive with the economic direction " <<<endl<<< "\
tunderpinned. "<<<endl;
}
else
{
cout<<< "\t"<<<s[i]. stock_name<<<" is showing less favorable earnings in its volume and is not"
<<<endl<<< "\tsitting well in the market. The drastic revert of the stock is the"<<<endl<<<
"\tresult of the competition given by other companies of the same nature. "<<<endl<<<
"\tThe stock is now in a dire position to recover anytime soon. "<<<endl;
}
}
getch();
s[i]. Interface();
}

void main()
{
    Stock s[MAX],temp;
    // The followings only be activated to renew the stocks.
    // ~~~~~
    temp. Interface();
}
```

第4题 个人财政支出管理

5.4.1 程序功能简介

个人财政支出管理程序,完成收入、支出、盈余的记录和管理。

5.4.2 课程设计要求

- (1) 增加函数,完成财政账目的管理功能。
- (2) 将所有的函数与相应的数据封装到类中,并改写主程序,使程序简化。
- (3) 可自行增加程序的功能。
- (4) 设计测试主程序,在主程序中完成测试。

5.4.3 课程设计说明

- (1) SetGoal()函数账目中有问题,改正。
- (2) 可将整个记录装入一个字符串,在两条记录间用特殊符号隔开。
- (3) 查找相应记录时,可采用逐个字节比较的方法。
- (4) 评定难易级别:B级。

5.4.4 关键源代码注解

```

/* The Code from web site : www. technocode. scriptmania. com. */
#include <iostream. h>
#include <fstream. h>
#include <conio. h>
#include <iomanip. h>

void Deposit();
void Withdraw();
void ViewTotal();
void SetGoal();           //Declaring all my functions
void depositDisplay();
void mainDisplay();
void withdrawDisplay();
void setgoalDisplay();
void viewtotalDisplay();

int main(){
    char select;

    //clrscr();           // if add
    mainDisplay();       //Title that displays heading with background color
    cout<<"\n\n\t<1> Deposit"<<endl;

```

★ ★ ★ ★ ★

```

cout<<"\n\t<2> Withdraw"<<endl;
cout<<"\n\t<3> View Total"<<endl;
cout<<"\n\t<4> Set Goal"<<endl;
cout<<"\n\t<0> Exit"<<endl;
cout<<"\n\n\tEnter selection: ";
cin>>select;
if(select == '1'){Deposit();}else{
if(select == '2'){Withdraw();}else{
if(select == '3'){ViewTotal();}else{
if(select == '4'){SetGoal();}else{
if(select == '0')
{
//exit(1);
goto mml;
}
else{
main();
}
}}}}
mml:    return 0;
}

void Deposit(){    //Depositing amounts into program
    double dAmt = 0;
    char ans;
    double num = 0;
//Gets any amount that is in the Deposit file to add later on
    ifstream get("Deposit. txt");
    get>>num;
    get.close();
    depositDisplay();    //My splash display screen
    cout<<"\n\n\tEnter deposit amount: $ ";
    cin>>dAmt;
//Checks if amount entered is no a negative # or 0 and then continues
    if(dAmt <= 0){
        cout<<"\n\n\n\n\n"<<endl;
        //textcolor(RED);    //if add ?
        printf("                That was an invalid amount. . .");
        //textcolor(LIGHTGRAY);    //if add ?
        getch();
        main();
    }else{
        cout<<"\n\n\tAmount entered <"<<dAmt<<">. "<<endl;

```



```

cout<<"\n\tContinue with deposit <y/n>: ";
cin>>ans;

if(ans == 'y' || 'Y'){
    dAmt = dAmt + num;    //add amount entered by what was retrieved earlier
    ofstream save("Deposit.txt");
    save<<dAmt<<endl;
    save.close();
    //clrscr();
    cout<<"\n\n\n\n\n"<<endl;
    //textcolor(LIGHTBLUE); //changes color of text //if add ?
    printf("                Your deposit was successful. . .");
    //textcolor(LIGHTGRAY); //if add ?
    getch();
    main();
}else{
    main();
}
main();
}

```

```

}

void Withdraw(){
    double wAmt = 0;
    char ans;
    double num = 0;

    ifstream get("Withdraw.txt");
    get>>num;
    get.close();
    withdrawDisplay();
    cout<<"\n\n\tEnter withdraw amount: $ ";
    cin>>wAmt;
    if(wAmt <= 0){
        cout<<"\n\n\n\n\n"<<endl;
        //textcolor(RED) //if add ?
        printf("                That was an invalid amount. . .");
        //textcolor(LIGHTGRAY); //if add ?
        getch();
        main();
    }else{
        cout<<"\n\n\tAmount entered <"<<wAmt<<">. "<<endl;
    }
}

```

```
cout<<"\n\tContinue with withdrawal <y/n>: ";
cin>>ans;
```

```
if(ans == 'y' || 'Y'){
    wAmt = wAmt + num;
    ofstream save("Withdraw.txt");
    save<<wAmt<<endl;
    save.close();
    //clrscr();//if add ?
    cout<<"\n\n\n\n\n"<<endl;
    //textcolor(LIGHTBLUE);//if add ?
    cprintf("                Your withdrawal was successful. . .");
    //textcolor(LIGHTGRAY);//if add ?
    getch();
    main();
} else{
    main();
}
main();
}
```

```
void SetGoal(){
    double setAmt = 0;
    setgoalDisplay();
    cout<<"\n\n\tSet Goal amount: ";
    cin>>setAmt;
    ofstream set("Goal.txt");
    set<<setAmt<<endl;
    set.close();
    //clrscr(); //if add ?
    cout<<"\n\n\n\n\n"<<endl;
    //textcolor(LIGHTBLUE);//if add ?
    cprintf("                Goal amount was set. . .");
    //textcolor(LIGHTGRAY);//if add ?
    getch();
    main();
}
```

```
void ViewTotal(){
    double dAmt = 0;
    double wAmt = 0;
    double gAmt = 0;
```



```

double balance = 0;
double newbalance = 0;
ifstream getDeposit("Deposit.txt");
getDeposit>>dAmt;
getDeposit.close();
ifstream getWithdraw("Withdraw.txt");
getWithdraw>>wAmt;
getWithdraw.close();
ifstream getGoal("Goal.txt");
getGoal>>gAmt;
getGoal.close();
//Withdrawal amount is minus from Deposit amount and put into balance
balance = dAmt - wAmt;
viewtotalDisplay();
cout<<"\n\n"<<endl;
printf("                                Total Amounts");
cout<<" "<<endl;
printf("                                =====");
cout<<"\n\n"<<endl;
printf("Deposit Total Withdraw Total  Balance Total ");
cout<<"\n"<<endl;
cout<<setw(15)<<dAmt<<setw(25)<<wAmt<<setw(20)<<balance<<endl;
//setw(15) sets the margins between the text.
newbalance = gAmt - balance;
cout<<"\n\n\n"<<endl;
printf("                                Total Balance");
cout<<" "<<endl;
printf("                                =====");
cout<<"\n\n"<<endl;
printf("Set Goal Total Balance Total  Balance Left ");
cout<<"\n"<<endl;
cout<<setw(15)<<gAmt<<setw(25)<<balance<<setw(23)<<newbalance<<endl;
getch();
main();
}

//===== These are all my color displays for my headings=====

void depositDisplay(){

    printf("DEPOSIT  ");

}

void mainDisplay(){
    printf("Money Manager  ");

```

★★★★★

```
}  
void withdrawDisplay(){  
    cprintf("WITHDRAW  ");  
}  
void setgoalDisplay(){  
    cprintf("SET GOAL ");  
}  
void viewtotalDisplay()  
{  
    cprintf("VIEW TOTALS ");  
}
```

第5题 电话簿管理

5.5.1 程序功能简介

一个基本的电话簿管理程序,具有插入、删除、显示、修改和查询联系人电话号码的功能。

5.5.2 课程设计要求

- (1) 增加记录修改功能(姓、名和电话号码)。
- (2) 增加显示功能(对链表提供逐屏显示选择而且可以由用户确定每屏显示的数目)。
- (3) 汉化菜单和帮助。
- (4) 程序的结构改写,采用双链表结构,用类和对象重建系统。

5.5.3 课程设计说明

- (1) 程序采用双向链表数据结构实现。
- (2) 用类来实现数据的封装,改变这个系统结构是比较容易实现的。
- (3) 评定难易级别:A级。

5.5.4 关键源代码注解

```
//PhoneBook 1.0 By Mark Miller
//compiler directives
#include<fstream.h>
#include<conio.h>
#include<iostream.h>
#include<iomanip.h>
#include<string.h>
#include<stdlib.h>
//global structures and variables
struct friend_node //结构体类型定义,包括:姓、名和电话号码
{
    char last_name[20];
    char first_name[15];
    char phone_num[12];
    friend_node * next;
};
friend_node * head_ptr; //全程变量,链头指针
friend_node * current_ptr; //全程变量,用于指明当前在链表中的位置
//function prototypes
void handle_choice(int choice); //函数原形说明
void add_record();
void insert_node(friend_node * new_rec_ptr);
```

```

friend_node * position_insertion_point(char lastname[20]);
void make_node_new_head(friend_node * new_rec_ptr);
void add_node_to_end(friend_node * new_rec_ptr);
void move_current_to_end();
void display_list();
void delete_record();
void delete_head_of_list();
void delete_end_of_list(friend_node * previous_ptr);
void delete_from_middle_of_list(friend_node * previous_ptr);
int verify_delete();
void delete_node(friend_node * previous_ptr);
void delete_list();
void search_by_lastname();
void write_list_to_file();
void load_list_from_file();
void help_me();
char pause;
//main function
int main()
{
    cout << "Welcome to PhoneBook 1.0 for dos and freedos\n";
    cout << "Press Enter To Continue\n";
    cin.get(pause);
    system("cls"); //执行系统命令:cls—清屏
    int choice;
    head_ptr = NULL;      // Initialize head pointer to NULL
    load_list_from_file(); // Load data from the disk file into linked list
    do
    { // Display menu. —— 主菜单显示
        cout << "1 — Add record\n";
        cout << "2 — Display all records\n";
        cout << "3 — Search for friend by last name\n";
        cout << "4 — Delete record\n";
        cout << "5 — Help\n";
        cout << "6 — Exit program\n";
        cout << "Enter choice: ";
        cin >> choice;
        handle_choice(choice); // Call function to direct flow based on choice
    } while(choice != 6); // Repeat menu until user chooses to exit
    return 0;
} // end of main function
// Function to direct program flow based on user's choice
void handle_choice(int choice) //根据用户选择(choice)调用对应处函数

```

```

{
switch(choice)    // choice is passed into the function by value
{
case 1:    // If choice was to add a record to the database,
    add_record();    // call function to add a record to the linked list
    break;
case 2:    // If choice was to display all records in the database,
    display_list();    // call function to display all records in the linked list
    break;
case 3:    // If choice was to search for a record in the database,
    search_by_lastname(); // call function to search for record by last name
    break;
case 4:    // If choice was to delete a record in the database,
    delete_record();    // call a function that searched for record by last name and deletes it
    break;
case 5:    // If choice is for help
    help_me();
    break;
case 6:    // If choice was to exit, save database to a file and
    write_list_to_file();    // delete the list from memory
    if(head_ptr != NULL)
    {
        delete_list();
    }
    break;
default : // If any other (invalid) choice was entered, display error message
    cout << "Invalid choice\n";
    break;
}
} // end of function handle_choice
// Function to add record to the linked list.
void add_record()    // 在链表中增加一个记录
{
    friend_node * new_rec_ptr;    // Declare temporary pointer for the new node
    new_rec_ptr = new friend_node;    // Allocate memory for a new node and
                                        // initialize pointer to point to it
    if(new_rec_ptr != NULL)    // If no error allocating memory, get data
    {
        system("cls");    // and insert node
        // Get name and phone number from the user.
        cin.ignore(20,'\n');
        cout << "First Name: ";
        cin.get(new_rec_ptr->first_name,15);
        cin.ignore(20,'\n');
    }
}

```

```

    cout << "Last Name: ";
    cin.get(new_rec_ptr->last_name,20);
    cin.ignore(20, '\n');
    cout << "Phone Number: ";
    cin.get(new_rec_ptr->phone_num,15);
    cin.ignore(20, '\n');
    insert_node(new_rec_ptr);
}

else // If error occurred allocating memory, display warning
{
    // and do not create node
    cout << "WARNING: Memory error. New record cannot be added.\n";
    system("cls");
} // end of function add_record

// Function to insert new node into correct position in list
void insert_node(friend_node * new_rec_ptr)
//将一个由 new_rec_ptr 指向的新节点插入链表中
{
    system("cls");
    friend_node * before_ptr;
    friend_node * after_ptr;
    if(head_ptr == NULL)
    {
        // If no nodes exist, make the node
        new_rec_ptr->next = NULL; // the head
        head_ptr = new_rec_ptr;
    }
    else
    {
        if(strcmp(new_rec_ptr->last_name, head_ptr->last_name) < 0)
        {
            // If new record comes before head
            make_node_new_head(new_rec_ptr); // make it the new head
        }
        else
        {
            // Else, determine where the new node
            // should be inserted
            current_ptr = position_insertion_point(new_rec_ptr->last_name);
            before_ptr = current_ptr; // Use pointers to keep track of nodes
            after_ptr = current_ptr->next; // on each side of the insertion point
            if(after_ptr == NULL) // If after_ptr is NULL, the node needs to be
            {
                // added to the end of the list
                add_node_to_end(new_rec_ptr);
            }
            else
            {
                // Else add the node between the nodes pointed to
                // by before_ptr and after_ptr
                before_ptr->next = new_rec_ptr;
            }
        }
    }
}

```

```

        new_rec_ptr->next = after_ptr;
    }
}

} // end of function insert_node
// Function that positions current_ptr at the node before the position
// where the new node should be inserted
friend_node * position_insertion_point(char lastname[20])
{
    // 根据姓氏, 返回其在链表中的正确位置, 新节点即将插入此点
    char temp_name[20];
    friend_node * temp_ptr;
    int tempint;
    if(head_ptr->next != NULL) // If more than one node exists, search the
    {
        // list for the correct insertion point
        current_ptr = head_ptr;
        temp_ptr = current_ptr->next;
        strcpy(temp_name, temp_ptr->last_name);
        // Loop until the proper insertion point is located
        tempint = strcmp(lastname, temp_name);
        while((tempint > 0) && (current_ptr->next != NULL))
        {
            current_ptr = temp_ptr;
            // check to see if the current node is the last node
            if(current_ptr->next != NULL)
            {
                temp_ptr = current_ptr->next;
                strcpy(temp_name, temp_ptr->last_name);
                tempint = strcmp(lastname, temp_name);
            }
        }
    }
    else // If only one node exists in the list, current_ptr is the same
    {
        // as head_ptr. New node will be added to the end of the list
        current_ptr = head_ptr;
    }
    return(current_ptr);
} // end of function position_insertion_point
// Function that makes the node pointed to by new_rec_ptr the new
// head of the linked list. It handles the special case of inserting at
// the front of the list
void make_node_new_head(friend_node * new_rec_ptr)
{
    friend_node * temp_ptr; // temporary pointer to keep track of the head

```



```

temp_ptr = head_ptr;      // Set temp_ptr to point at the current head
new_rec_ptr->next = temp_ptr;  // Make new nodes next pointer point to
head_ptr = new_rec_ptr;    // current head and make new node the head
}    // end of function make_node_new_head

// Function that adds a node to the end of the linked list. It handles
// the special case of inserting at the end of the list
void add_node_to_end(friend_node * new_rec_ptr)
{
    new_rec_ptr->next = NULL;    // Set next node pointer of new node to NULL
    move_current_to_end();      // Make sure current_ptr is at end of list
    current_ptr->next = new_rec_ptr;  // Place new node at the end of the list
}    // end of function add_node_to_end

// Function that moves current_ptr to end of the linked list
void move_current_to_end()
{
    current_ptr = head_ptr;    // Move temp_ptr to head of the list
    while(current_ptr->next != NULL)
    {
        // Traverse list until NULL is reached
        current_ptr = current_ptr->next;
    }
}    // end of function move_current_to_end

// Function that displays entire linked list
void display_list()
{
    char fullname[36];        // used to combine names into one array

    current_ptr = head_ptr;    // Move current_ptr to head of list
    if(current_ptr != NULL)
    {
        cout << endl;
        cout << "Name"                                "Phone Number\n";
        cout << "-----\n";
        do
        {
            strcpy(fullname, "");    // Clear fullname
            strcat(fullname, current_ptr->last_name);    // Put last name, then a comma,
            strcat(fullname, ", ");    // then the first name into fullname
            strcat(fullname, current_ptr->first_name);
            cout.setf(ios::left);
            cout << setw(36) << fullname;
            cout.unsetf(ios::left);
            cout.setf(ios::right);

```



```

        cout << setw(12) << current_ptr->phone_num << endl;
        current_ptr = current_ptr->next;        // Set current_ptr to next node
        cout << endl;
    } while(current_ptr != NULL);    // Loop until end of list
    cout << "Press Enter to continue \n";
    cin.get(pause);
    cin.ignore(1, pause);
    system("cls"); }
else    // If list is empty, display message
{
    cout << "\nNO RECORDS TO DISPLAY\n";
}

}    // end of function display_list

// Function that searches linked list for the first occurrence of a given
// last name and displays the record to the screen
void search_by_lastname()
{
    system("cls");
    char search_string[20];    // Character array for last name to search for
    current_ptr = head_ptr;    // Move current_ptr to head of list to begin search
    cin.ignore(20, '\n');
    cout << "\nEnter the last name for which you want to search: ";
    cin.get(search_string, 20);
    cin.ignore(20, '\n');
    // Loop until search_string is found or end of list is reached.
    while((current_ptr != NULL) &&
        (strcmp(current_ptr->last_name, search_string) != 0))
    {
        current_ptr = current_ptr->next;
    }
    if(current_ptr != NULL) // If current_ptr is not NULL, then match was found
    {
        cout << "\nRECORD FOUND\n";
        cout << current_ptr->first_name << " "
            << current_ptr->last_name << endl;
        cout << current_ptr->phone_num << endl;
    }
    else
    {
        cout << "NO MATCH FOUND\n";
        cout << "Press Enter to Continue\n";
        cin.get(pause);
        system("cls");
    }
}

```

```

    }
} // end of function search_by_lastname

// Function that deletes individual nodes from the linked list
void delete_record()
{
    system("cls");
    char search_string[20];
    friend_node * previous_ptr;
    previous_ptr = NULL; // Initialize previous_ptr to NULL
    current_ptr = head_ptr; // Move current_ptr to head of list to begin search
    cin.ignore(20, '\n');
    cout << "\nEnter the last name of the friend you want to delete: ";
    cin.get(search_string, 20);
    cin.ignore(20, '\n');
    // Loop to find matching record.
    while((current_ptr != NULL) &&
        (strcmp(current_ptr->last_name, search_string) != 0))
    {
        previous_ptr = current_ptr; // A pointer must be maintained that
        current_ptr = current_ptr->next; // points to the node before the node
    } // to be deleted

    if(current_ptr != NULL) // If current_ptr is not NULL, then match was found
    {
        cout << "\nRECORD FOUND\n";
        cout << current_ptr->first_name << " "
            << current_ptr->last_name << endl;
        cout << current_ptr->phone_num << endl;
        if(verify_delete()) // Ask user if he/she wants to delete the record
        {
            // If user wants to delete the record, delete the node
            delete_node(previous_ptr); // that follows the one pointed to by previous_ptr
            cout << "\nRECORD DELETED\n";
        }
        else // Otherwise, do nothing
        {
            cout << "\nRECORD NOT DELETED\n";
        }
    }
    else // If no match for the record found, display message
    {
        cout << "\nNO MATCH FOUND. NO RECORD DELETED.\n";
    }
    system("cls");
} // end of function delete_record

```

```
//Function that delete's the file for reset
```

```
void help_me()
```

```
{
```

```
help:
```

```
int choice2;
```

```
system("cls");
```

```
cout << "Welcome to help please choose help type\n";
```

```
cout << "1: What happened to my records\n";
```

```
cout << "2: How do I clear all records\n";
```

```
cout << "3: When will there be a new version\n";
```

```
cout << "4: What will be in the new versions\n";
```

```
cout << "5: Exit\n";
```

```
cin >> choice2;
```

```
switch(choice2) //Menu for help
```

```
{
```

```
case 1:
```

```
cout << "Check the directory with the executable for a file named Friends.dat\n";
```

```
cout << "If its not there its gone forever if renamed rename it to Friends.dat\n";
```

```
cout << "Press enter to contiune\n";
```

```
pause=getch();
```

```
system("cls");
```

```
goto help;
```

```
break;
```

```
case 2:
```

```
cout << "Delete The file Friends.dat\n";
```

```
cout << "Press Enter to contiune\n";
```

```
pause=getch();
```

```
system("cls");
```

```
goto help;
```

```
break;
```

```
case 3:
```

```
cout << "When I get time\n";
```

```
cout << "Press Enter to contiune\n";
```

```
pause=getch();
```

```
system("cls");
```

```
goto help;
```

```
break;
```

```
case 4:
```

```
cout << "I hope to add the following\n";
```

```
cout << "Multi User's and passwords\n";
```

```
cout << "Encryption \n";
```

```
cout << "Clear all records command\n";
```

```
cout << "Bug fixes\n";
```

```

    cout << "Press Enter to contiune\n";
    pause=getch();
    system("cls");
    goto help;
    break;
default:
    cout << "Press Enter To exit";
    cout << "Press Enter to contiune\n";
    break;
}
} // end of function delete_all
// Function to ask user to verify intention to delete the node
int verify_delete()
{
    char YesNo;
    cout << "\nAre you sure (Y/N) ";
    cin >> YesNo;
    if((YesNo == 'Y') || (YesNo == 'y'))
    {
        return(1); // Return TRUE if user want to delete
    }
    else
    {
        return(0); // Return FALSE if user does not want to delete
    }
} // end of function verify_delete
// Function that deletes node pointed to by current_ptr.
void delete_node(friend_node * previous_ptr)
{
    if(current_ptr == head_ptr) // If node to be deleted is the head of the
    {
        // list, call a special function that
        delete_head_of_list(); // deletes the first node in the list
    }
    else
    {
        // Otherwise:
        if(current_ptr->next == NULL) // If node to be deleted is at the
        {
            // end of the list, call a special
            delete_end_of_list(previous_ptr); // function to delete that node
        }
        else // Otherwise
        {
            // Delete the node from the
            delete_from_middle_of_list(previous_ptr); // middle of the list using
        }
        // a function that does that
    }
}

```

```

    }
    } // end of function delete_node
//Function that deletes the head of the list
void delete_head_of_list()
{
    current_ptr = head_ptr;    // Make current_ptr point to the head of the list
    if(head_ptr->next != NULL)
    {
        // If more than one node is in the list,
        head_ptr = current_ptr->next;    // make second node in list the new head
    }
    else
        // Otherwise, just set head_ptr to NULL
        // to signal that the list is empty
        head_ptr = NULL;
    delete current_ptr;    // Deallocate memory used by the deleted node
} // end of function delete_head_of_list
// Function that deletes the last node of the linked list.
void delete_end_of_list(friend_node * previous_ptr)
{
    delete current_ptr;    // Deallocate memory used by the deleted node
    previous_ptr->next = NULL;    // Make node before deleted node the end of list
    current_ptr = head_ptr;    // Set current_ptr to head to give it a value
} // end of function delete_end_of_list
// Function that deletes a node from the middle of the list
void delete_from_middle_of_list(friend_node * previous_ptr)
{
    // Set pointers of the nodes before and after the node to be deleted to
    // skip the node that is to be deleted
    previous_ptr->next = current_ptr->next;
    delete current_ptr;    // Deallocate memory used by the deleted node
    current_ptr = head_ptr;    // Set current_ptr to head to give it a value
} // end of function delete_from_middle_of_list
// Function that frees the memory used by the linked list
void delete_list()
{
    friend_node * temp_ptr;    // pointer used for temporary storage
    current_ptr = head_ptr;    // Move current_ptr to head of the list
    do    // Traverse list, deleting as we go
    {
        temp_ptr = current_ptr->next;    // Set temporary pointer to point to the remainder of the list
        delete current_ptr;    // Delete current node
        current_ptr = temp_ptr;    // Set current_ptr to next node after the deleted one
    } while(temp_ptr != NULL);
}

```

```

} // end of function delete_list

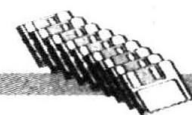
// Function to write linked list data to the data file
void write_list_to_file()
{
    ofstream outfile; // output file pointer
    outfile.open("FRIENDS.DAT",ios::out); // Open file for output
    if (outfile) // If no error occurred while opening the file,
    {
        // it is okay to write the data to the file
        current_ptr = head_ptr; // Set current_ptr to head of list
        if(head_ptr != NULL) // If the list is not empty, begin
        {
            // writing data to the file
            do // Traverse list until the end is reached
            {
                // Write the nodes data to the file
                outfile << current_ptr->last_name << endl;
                outfile << current_ptr->first_name << endl;
                outfile << current_ptr->phone_num << endl;
                current_ptr = current_ptr->next; // Move current_ptr to next node
            } while(current_ptr != NULL); // Loop until end of list is reached
        }

        // The word END OF FILE are written to the end of the file to make it
        // easy to locate the end of the file when the data is read back in
        outfile << "END OF FILE" << endl;
        outfile.close(); // Close the file
    }
    else // If an error occurs while opening the file, display a message
    {
        cout << "Error opening file.\n";
    }
} // end of function write_list_to_file

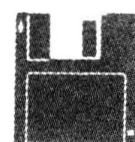
// Function to load the linked list from the data file
void load_list_from_file() // 从数据文件 FRIENDS.DAT 中读取数据重建链表处理函数
{
    friend_node *new_rec_ptr;
    ifstream infile; // input file pointer
    int end_loop = 0;
    infile.open("FRIENDS.DAT",ios::in); // Open file for input
    if (infile) // If no error occurred while opening file
    {
        // input the data from the file
        do
        {
            new_rec_ptr = new friend_node; // Allocate memory for a node
            if(new_rec_ptr != NULL) // Check for allocation error

```

```
{
    // Get the next last name from the file
    infile.get(new_rec_ptr->last_name, 20);
    infile.ignore(20, '\n');
    // If the end of the file has not yet been reached, get other data
    if((strcmp(new_rec_ptr->last_name, "") != 0) &&
        (strcmp(new_rec_ptr->last_name, "END OF FILE") != 0))
    {
        infile.get(new_rec_ptr->first_name, 15);
        infile.ignore(20, '\n');
        infile.get(new_rec_ptr->phone_num, 15);
        infile.ignore(20, '\n');
        insert_node(new_rec_ptr);
    }
    else // If end of file has been reached, delete the most recently
        { // created node and set the flag that ends the loop
            delete new_rec_ptr;
            end_loop = 1;
        }
}
else // If a memory allocation error occurs, display a message and
{ // set the flag that ends the loop
    cout << "WARNING: Memory error. Load from disk was unsuccessful.\n";
    end_loop = 1;
}
} while(end_loop == 0); // Loop until the end_loop flag becomes TRUE
infile.close(); // Close the file
}
else // If error occurred opening file, display message
{
    cout << "No usable data file located. List is empty.\n";
}
} // end of function load_list_from_file
```



6 时间和日期处理程序





第 1 题 时间和日期

6.1.1 程序功能简介

一个时间和日期的实用程序。可以自己设置时间和日期并得到是星期几;可以比较日期的大小,也可以比较时间大小,可以两个日期相加和相减,可以实现日期和天数的加和减,可以实现小时和分钟的加和减。

6.1.2 课程设计要求

- (1)测试所有类的成员函数,时间类和日期类的初始化,具体值的设置,比较大小。
- (2)时间类和日期类多种格式输入和输出。
- (3)在日期类 cDate_t 中添加成员重载函数+,如得到新日期,应考虑闰年处理。
- (4)在日期类 cDate_t 中添加成员重载函数-,如得到新日期,应考虑闰年处理。
- (5)在时间类 cTime_t 中添加成员重载函数+,时间对象加上分钟后得到新日期。
- (6)在时间类 cTime_t 中添加成员重载函数-,时间对象加上分钟后得到新日期。

6.1.3 课程设计说明

- (1)将时间对象加上分钟后得出一个新的时间,只在 24 小时内考虑。
- (2)程序中要求测试的功能太多,参数过多,可利用文件进行参数设置。
- (3)可进行分屏,分步测试。
- (4)评定难易级别:A 级。

6.1.4 关键源代码注解

```
// TIME & DATE written : Amit Friedmann
// cDate_t.h Begin

#include <iostream.h>
#include <time.h>
#ifndef cDate_t_h
#define cDate_t_h
#define N 3
class cDate_t
{
    enum Days {Saturday, Sunday, Monday, Tuesday, Wensday, Thursday, Friday };
    enum Months {None, January, February, Mars, April, May, June, July, August,
                 September, October, November, December };

    static int format;
    static char Mon[][10];
    static char DDays[][10];
```

```

    int Day;
    int Month;
    int Year;
    struct tm * CurrentTime;
    time_t time_date;
    bool Error;
public:
    cDate_t();
    cDate_t(const cDate_t &D);
    cDate_t(int year,int month,int day);
    ~cDate_t(){}
    const cDate_t& operator = (const cDate_t& D);
    inline void SetDay (int day) {Day=day;}
    inline void SetMon (int mon) {Month=mon;}
    inline void SetYear (int year) {Year=year;}
    void print() const;
    inline int GetDayOfMonth () const {return Day;}
    inline int GetMonth () const {return Month;}
    inline int GetYear () const {return Year;}
    int GetDayOfYear(int year,int month,int day);
    int GetDaysInMonth(int month,int year);
    int GetDayOfWeek();
    bool IsLeapYear(int year);
    char * GetDayOfWeekName();
    char * GetNameOfMonth();
    int RetriveDay(int Days,int year);
    int RetriveMonth(int Days,int year);
    bool operator < (const cDate_t& D)const;
    bool operator <= (const cDate_t& D)const;
    bool operator > (const cDate_t& D)const;
    bool operator >= (const cDate_t& D)const;
    bool operator == (const cDate_t& D)const;
    bool operator != (const cDate_t& D)const;
    const cDate_t operator + (const cDate_t& D) ;
    const cDate_t operator - (const cDate_t& D) ;
    void operator ++ ();
    void operator -- ();
    static void cDate_t::ChangeFormat() //implementation of static function must be in header
{
    switch(format)
    {
        case 1:    format=2;    break;
        case 2:    format=3;    break;
    }
}

```

```

        case 3:          format=1;
        }

    }

    friend ostream& operator << (ostream &out,const cDate_t &D) ;
    friend istream& operator >> (istream &in, cDate_t &D) ;
    void SetError() {Error=false;}
    friend class cTDmanage;
};

#endif
//cDate_t.h End
//cTime_t.h Begin
#include <iostream.h>
#include <time.h>
#ifndef cTime_t_h
#define cTime_t_h
class cTime_t
{
    static int FlagMoreDay;
    static int FlagLessDay;
    static int format;
    int Seconds;
    int Minutes;
    int Hours;
    struct tm * CurrentTime;
    time_t time_date;
public:
    cTime_t();
    cTime_t(const cTime_t &T);
    cTime_t(int hour,int min=0,int sec=0);Seconds(sec),Minutes(min),Hours(hour) {};
    ~cTime_t(){};

    const cTime_t& operator = (const cTime_t& T);
    inline void SetSec (int sec) {Seconds=sec;}
    inline void SetMin (int min) {Minutes=min;}
    inline void SetHour (int hour) {Hours=hour;}
    void print() const;
    inline int GetSec () const {return Seconds;}
    inline int GetMin () const {return Minutes;}
    inline int GetHour () const {return Hours;}
    bool operator < (const cTime_t& T)const;
    bool operator <= (const cTime_t& T)const;
    bool operator > (const cTime_t& T)const;
    bool operator >= (const cTime_t& T)const;
    bool operator == (const cTime_t& T)const;

```

```

    bool operator != (const cTime_t& T) const;
    const cTime_t operator + (const cTime_t& T) const;
    const cTime_t operator - (const cTime_t& T) const;
    void operator ++ ();
    void operator -- ();
    static void ChangeFormat() { format = (format==1)? 2 : 1 ; } //format can be only 1 or 2
    friend ostream& operator << (ostream &out, const cTime_t &T) ;
    friend istream& operator >> (istream &in, cTime_t &T) ;
    friend class cTDmanage;
};
#endif
//cTime_t.h End
//error.h Begin
#include <iostream.h>
#ifdef _error_h
#define _error_h
void Error1();
void Error2();
void Error3();
void Error4();
void Error5();
#endif
//error.h End
//TDmanage.h Begin
#include <iostream.h>
#include <time.h>
#include "cDate_t.h"
#include "ctime_t.h"
#ifdef TDmanage_h
#define TDmanage_h
class cTDmanage
{
    cTime_t T;
    cDate_t D;
public:
    cTDmanage () {}
    cTDmanage (const cDate_t &DD); D(DD) {}
    cTDmanage (const cTime_t &TT); T(TT) {}
    cTDmanage (const cDate_t &DD, const cTime_t &TT); D(DD), T(TT) {}
    cTDmanage (int hour, int minute, int second); T(hour, minute, second) {}
    cTDmanage (int hour, int minute, int second, int year, int month, int day); T(hour, minute, second), D
    (year, month, day) {}
    void operator = (const cTDmanage& M) { T=M.T ; D=M.D ; }

```

```

//option to put all get and set function of date and time class
int GetDDayOfWeek() { return D. GetDayOfWeek(); }
bool IsDLeapYear (int year) { return D. IsLeapYear(year); }
char * GetDDayOfWeekName() { return D. GetDayOfWeekName(); }
char * GetDNameOfMonth() { return D. GetNameOfMonth(); }
int RetriveDDay(int Days,int year) { return D. RetriveDay(Days,year); }
int RetriveDMonth(int Days,int year) { return D. RetriveMonth(Days,year); }
//if you want to do operations only on 1 of the classes
const cTime_t& GetTimeObj () { return T; }
const cDate_t& GetDateObj () { return D; }
void print() const;
//operator:
bool operator < (const cTDmanage& M)const {return ( (T<M. T) && (D<M. D) ); }
bool operator <= (const cTDmanage& M)const {return ( (T<=M. T) && (D<=M. D) ); }
bool operator > (const cTDmanage& M)const {return ( (T>M. T) && (D>M. D) ); }
bool operator >= (const cTDmanage& M)const {return ( (T>=M. T) && (D>=M. D) ); }
bool operator == (const cTDmanage& M)const {return ( (T==M. T) && (D==M. D) ); }
bool operator != (const cTDmanage& M)const {return ( (T!=M. T) && (D!=M. D) ); }
const cTDmanage operator + (const cTDmanage& M) ;
const cTDmanage operator - (const cTDmanage& M) ;
//using Date and Time ++ operator:
void operator ++ () { AddDay() ; AddSecond() ; }
void AddDay() { ++D; }
void AddSecond() { ++T; }
void ChangeDateFormat() { D. ChangeFormat(); }
void ChangeTimeFormat() { T. ChangeFormat(); }
friend ostream& operator << (ostream &out,const cTDmanage &M) ;
friend istream& operator >> (istream &in, cTDmanage &M) ;
};

#endif
//TDmanage.h End
//cDate_t.cpp Begin
//implementation of cDate_t.h

#include "cDate_t.h"
#include "error.h"
int cDate_t::format=1;
char cDate_t::Mon[][10] = {"Janaury","February","Mars","April","May",
    "Juny","July","August","September","October","November","December" };
char cDate_t::DDays[][10] = {"Saturday","Sunday","Monday","Tuesday",
    "Wensday","Thursday","Friday" };
cDate_t::cDate_t() //empty ctor

```

```

{
    SetError();
    time_date=time(0);
    CurrentTime=localtime(&time_date); //get data from computer about today date
    Day=CurrentTime->tm_mday;
    Month=CurrentTime->tm_mon+1;
    Year=CurrentTime->tm_year+1900;
}

//-----
cDate_t::cDate_t(int year,int month,int day):Year(year),Month(month),Day(day) //ctor
{
    SetError();
    if (Year<1900) //bad data
    {
        Error1();
        Error=true;
    }
}

//-----
cDate_t::cDate_t(const cDate_t &D) //copy ctor
{
    SetError();
    Day=D. Day;
    Month=D. Month;
    Year=D. Year;
    if (Year<1900)//bad data
    {
        Error1();
        Error=true;
    }
}

//-----
const cDate_t& cDate_t::operator = (const cDate_t& D) //operator = function
//get same data object and copy to THIS class all fields
{
    Day=D. Day;
    Month=D. Month;
    Year=D. Year;
    if (Year<1900)//bad data from the other object
    {
        Error1();
        Error=true;
    }
}

```

```

else
    SetError();
return * this;
}

//-----

void cDate_t::print() const //print function
//prints all relevent date fields
{
    char str[N]; //string to store 3 first letters of month
    for(int i=0;i<N;++i)
        str[i]=Mon[Month-1][i]; //copt from static member
    str[i]='\0';
    if(Error)//check for bad data
    {
        Error2();
        return;
    }

    switch(format) //kind of printing
    {
    case 1: //European
        cout<<"The date is:  "<< Day<<"/"<<Month<<"/"<<Year<<". "<<endl;
        break;
    case 2: //American
        cout<<"The date is:  "<<Month<<"/"<<Day<<"/"<<Year<<". "<<endl;
        break;
    case 3: //with string instead of month
        cout<<"The date is:  "<<Day<<"/"<<str<<"/"<<Year<<". "<<endl;
        break;
    default:
        cout<<"The format is not ok"<<endl;
    }
}

//-----

bool cDate_t::operator < (const cDate_t& D)const // operator < function
// return true if THIS object is < then the second
{
    if (Year<D.Year)    return true;
    if(Year==D.Year)    {
        if(Month<D.Month)    return true;
        if(Month==D.Month)    return (Day<D.Day);
    }
}

```

```

    return false;
}
//-----
bool cDate_t::operator <= (const cDate_t &D)const // operator <= function
// return true if THIS object is <= then the second
{
    if (Year<D. Year)    return true;
    if(Year==D. Year)
    {
        if(Month<D. Month)    return true;
        if(Month==D. Month)    return (Day<=D. Day);
    }
    return false;
}
//-----
bool cDate_t::operator > (const cDate_t& D)const //operator > function
// return true if THIS object is > then the second
{
    if (Year>D. Year)    return true;
    if(Year==D. Year) {
if(Month>D. Month)    return true;
        if(Month==D. Month)    return (Day>D. Day);
    }
    return false;
}
//-----
bool cDate_t::operator >= (const cDate_t &D)const //operator >=
// return true if THIS object is >= then the second
{
    if (Year>D. Year)    return true;
    if(Year==D. Year){
        if(Month>D. Month)    return true;
        if(Month==D. Month)    return (Day>=D. Day);
    }
    return false;
}
//-----
bool cDate_t::operator == (const cDate_t &D)const//operator == function
//return true if THIS object and the second one are equal in all their date data member
{
    return ( (Year==D. Year) && (Month==D. Month) && (Day==D. Day) );
}
//-----

```



```

bool cDate_t::operator != (const cDate_t &D)const//operator != function
//return true if THIS object and the second one are not equal in all their date data member
{
    return ! ( (Year==D. Year) && (Month==D. Month) && (Day==D. Day) );
}
//-----
const cDate_t cDate_t::operator + (const cDate_t &D)//operator + function
{
    int YearTemp,MonthTemp,DayTemp,NewDays; //variables to store to new date object
    if(Error || D. Error) // if no less then 1 object have error data
    {
        Error3(); // print orror messege
        return (cDate_t (-1,-1,-1) ); //return error class
    }
    YearTemp=Year+D. Year; //get the sum of years
    NewDays=GetDayOfYear(Year,Month,Day) + GetDayOfYear(D. Year,D. Month,D. Day);
    //get sum of days
    if(NewDays > ((IsLeapYear(YearTemp))? 366 : 365)) //if the sum is more thrn in 1 year
    {
        NewDays=NewDays - ((IsLeapYear(YearTemp))? 366 : 365); //cut 1 year days
        YearTemp+=1; //and add a year
    }
    DayTemp=RetriveDay(NewDays,YearTemp); //retrive the current day in the specific month
    MonthTemp=RetriveMonth(NewDays,YearTemp); //same with month
    return (cDate_t (YearTemp,MonthTemp,DayTemp) );
}
// return a local object , created only for return
}
//-----
const cDate_t cDate_t::operator - (const cDate_t &D) //operator - function
{
    int YearTemp,MonthTemp,DayTemp,NewDays,Temp1,Temp2;
    //variables to store to new date object
    if(Error || D. Error)// if no less then 1 object have error data
    {
        Error4();// print orror messege
        return (cDate_t (-1,-1,-1) ); //return error class
    }
    YearTemp=Year-D. Year; //substraction of THIS and second year
    if(YearTemp<1900) // error
    {
        Error1();//print error messege
        return (cDate_t (-1,-1,-1) ); //return error class
    }
}

```

```

Temp1=GetDayOfYear(Year,Month,Day); // get days of year of THIS object
if(Temp1<0) //error
{
    Error5();//print error messege
    return (cDate_t (-1,-1,-1) );//return error class
}
Temp2=GetDayOfYear(D. Year,D. Month,D. Day);// get days of year of second object
if(Temp2<0)//error
{
    Error5();//print error messege
    return (cDate_t (-1,-1,-1) );//return error class
}
NewDays= Temp1 - Temp2; //if the data ok get their subtraction
if(NewDays<0) // the subtraction is less then zero
{
    NewDays = NewDays + ((IsLeapYear(YearTemp)) ? 366 : 365);
// set days for previous year
    YearTemp-=1; //sub 1 year
}
DayTemp=RetriveDay(NewDays,YearTemp); //retrive the current day in the specific month
MonthTemp=RetriveMonth(NewDays,YearTemp);//same with month
return (cDate_t (YearTemp,MonthTemp,DayTemp) );//return local class
}
//-----
int cDate_t::GetDayOfYear(int year,int month,int day) //get day of year function
{
    int NumOfDay=0; //variable to store number of days until the current month
    int DaysIn1Month=0; // get days in each month to add to NumOfDay
    while(month!= 1 && DaysIn1Month!= -1) //loop for all the month except the current one
    {
        DaysIn1Month=GetDaysInMonth(month,year); //for checking
        NumOfDay+=DaysIn1Month;
        month--;
    }
    if(DaysIn1Month== -1) return -1; //error
    else
        return NumOfDay+day; //return number of days until this month + days from this month
}
//-----
int cDate_t::GetDaysInMonth(int month,int year) // get days in month function
{
    switch(month)
    {

```

```

        case January: case Mars: case May: case July: case August: case October:
        case December:      return 31;
        case April: case June: case September: case November:      return 30;
        case February:  if(IsLeapYear(year))      return 29;
                          else      return 28;
    }
    return -1; //error (should never visit this line!)
}
//-----
int cDate_t::GetDayOfWeek() //starting point : 1/1/1900 was Monday (Saturday==0, Sunday==1,...)
{
    int weekday=2 ; //in 1/1/1900
    for(int TheYear=1900; TheYear<Year; ++TheYear) //loop to know the day in 1/1/this year
    {
        if(IsLeapYear(TheYear))
            weekday=(weekday+366)%7;
        else
            weekday=(weekday+365)%7;
    }
    //until here we found the correct day in 1/1/this year
    for(int TheMonth=1; TheMonth<Month; ++TheMonth)
        weekday=(weekday+GetDaysInMonth(TheMonth, Year))%7;
    //until here we found the correct day in 1/this month
    return (weekday+Day)%7-1; //the correct day in THIS day...
}
//-----
bool cDate_t::IsLeapYear(int year) // is leap year function
{
    return (! (year%4)) && (year%100) || (! (year%400));
}
//-----
char * cDate_t::GetDayOfWeekName() // get day of week function
{
    int Weekday=GetDayOfWeek();
    return DDays[Weekday]; // use static DDays to identify the day
}
//-----
char * cDate_t::GetNameOfMonth() // get name of month function
{
    return Mon[Month-1]; // use static Mon to identify the month
}
//-----

```



```

int cDate_t::RetriveDay(int Days,int year) // retriive day function
// do the oppiste thing — find the day in a month when know the days from the start of year
{
    int day=Days; //variable to store days
    int leap=(IsLeapYear(year)); // know if the current year is leaped
    int M = (leap)? 29 : 28;
    for(int mon=1;mon<=12;++mon) //loop of month
    {
        if ( (day<=31 && (mon==1 || mon==3 || mon==5 || mon==7 || mon==8 || mon==10 |
            | mon==12)) || (day<=30 && (mon==4 || mon==6
            || mon==9 || mon==11))
            || (day<=M && mon==2) )
            return day;
        else
            day-=GetDaysInMonth(mon,year);
    }
    return -1; // if there was an error
}

//-----
int cDate_t::RetriveMonth(int Days,int year) // retriive month function
//same as retriive day just returning the month
{
    int day=Days;
    int leap=(IsLeapYear(year));
    int M = (leap)? 29 : 28;
    for(int mon=1;mon<=12;++mon) //loop of month
    {
        if( (day<=31 && (mon==1 || mon==3 || mon==5 || mon==7 || mon==8 || mon==10 |
            | mon==12 )) || (day<=30 && (mon==4 || mon==6
            || mon==9 || mon==11))
            || (day<=M && mon==2) )
            return mon;
        else
            day-=GetDaysInMonth(mon,year);
    }
    return -1;
}

//-----
void cDate_t::operator ++ () // operator ++ function
{
    if( (Day==GetDaysInMonth(Month,Year)) && (Month==12) )
// if reached the end of the year
{

```

```

        Day=1;
        Month=1;
        ++Year;
    }
    else if( (Day==GetDaysInMonth(Month,Year)) )
// if reached the end of the month (but not year)
    {
        Day=1;
        ++Month;
    }
    else
        ++Day; // add 1 day
}
//-----

```

```

void cDate_t::operator -- () //operator -- function
{
    if( (Day==1) && (Month==1) ) // start of year
    {
        Day=31; //in december there r always 31 days
        Month=12;
        --Year;
    }
    else if( Day==1 ) //start of month (but not year)
    {
        --Month;
        Day=GetDaysInMonth(Month,Year);
    }
    else
        --Day;
}

```

```
//cDate_t.cpp end
```

```
//cTime_t.cpp
```

```
//implementation of cTime_t
```

```
#include "cTime_t.h"
```

```
int cTime_t::format=1;
```

```
int cTime_t::FlagMoreDay=0; // default values for static data members
```

```
int cTime_t::FlagLessDay=0;
```

```
//-----
```

```
cTime_t::cTime_t() //empty ctor
```

```
{
```

```

time_date=time(0);
CurrentTime=localtime(&time_date);
Seconds=CurrentTime->tm_sec;
Minutes=CurrentTime->tm_min;
Hours=CurrentTime->tm_hour;
}

//-----
cTime_t::cTime_t(const cTime_t &T) //copy ctor
{
    Seconds=T.Seconds;
    Minutes=T.Minutes;
    Hours=T.Hours;
}

//-----
const cTime_t& cTime_t::operator = (const cTime_t& T) //operator = function
{
    Seconds=T.Seconds;
    Minutes=T.Minutes; // copy relevent fields
    Hours=T.Hours;
    return *this;
}

//-----
void cTime_t::print() const //print function
{
    switch(format)
    {
    case 1:
        if(Seconds<10 && Minutes<10)
            cout<<"The time is: " << Hours<<":0" << Minutes<<":0" << Seconds<<". " << endl;
        else if(Seconds<10)
            cout<<"The time is: " << Hours<<": " << Minutes<<":0" << Seconds<<". " << endl;
        else if(Minutes<10)
            cout<<"The time is: " << Hours<<":0" << Minutes<<": " << Seconds<<". " << endl;
        else
            cout<<"The time is: " << Hours<<": " << Minutes<<": " << Seconds<<". " << endl;
        break;
    case 2:
        if(Hours<=12)
        {
            if(Seconds<10 && Minutes<10)

```

```

cout<<"The time is:  "<<Hours<<":0"<<Minutes<<":0"<<Seconds
<< " AM."<<endl;
else if(Minutes<10)
cout<<"The time is:  "<<Hours<<":0"<<Minutes<<":"<<Seconds<< " AM."<<
endl;
else if(Seconds<10)
cout<<"The time is:  "<<Hours<<":"<<Minutes<<":0"<<Seconds<< " AM."<<
endl;
else
cout<<"The time is:  "<<Hours<<":"<<Minutes<<":"<<Seconds<< " AM."<<
endl;
}
else
{
    if (Seconds<10 && Minutes<10)
cout<<"The time is:  "<<Hours-12<<":0"<<Minutes<<":0"<<Seconds
<< " PM."<<endl;
    else if (Minutes<10)
cout<<"The time is:  "<<Hours-12<<":0"<<Minutes<<":"<<Seconds
<< " PM."<<endl;
    else if (Seconds<10)
cout<<"The time is:  "<<Hours-12<<":"<<Minutes<<":0"<<Seconds<< " PM."<<endl;
    else
cout<<"The time is:  "<<Hours-12<<":"<<Minutes<<":"<<Seconds<< " PM."<<endl;
}
break;
default:
cout<<"The format is not ok"<<endl; //error (if format value wasn't one of the above
}
}
//-----
bool cTime_t::operator < (const cTime_t& T)const //operator < function
{
    if (Hours<T. Hours)          return true;
    if(Hours==T. Hours)
    {
        if(Minutes<T. Minutes)    return true;
        if(Minutes==T. Minutes)    return (Seconds<T. Seconds);
    }
    return false;
}
//-----
bool cTime_t::operator <= (const cTime_t &T)const //operator <= function

```

```

{
    if (Hours<T. Hours)        return true;
    if(Hours==T. Hours)
    {
        if(Minutes<T. Minutes)    return true;
        if(Minutes==T. Minutes)    return (Seconds<=T. Seconds);
    }
    return false;
}

//-----
bool cTime_t::operator > (const cTime_t& T)const //operator > function
{
    if (Hours>T. Hours)        return true;
    if(Hours==T. Hours)
    {
        if(Minutes>T. Minutes)    return true;
        if(Minutes==T. Minutes)
            return (Seconds>T. Seconds);
    }
    return false;
}

//-----
bool cTime_t::operator >= (const cTime_t& T)const //operator >= function
{
    if (Hours>T. Hours)
        return true;
    if(Hours==T. Hours)
    {
        if(Minutes>T. Minutes)
            return true;
        if(Minutes==T. Minutes)
            return (Seconds>=T. Seconds);
    }
    return false;
}

//-----
bool cTime_t::operator == (const cTime_t &T)const //operator == function
{
    return ( (Hours==T. Hours) && (Minutes==T. Minutes) && (Seconds==T. Seconds) );
}

//-----
bool cTime_t::operator != (const cTime_t &T)const//operator != function
{

```



```

return ! ( (Hours==T. Hours) && (Minutes==T. Minutes) && (Seconds==T. Seconds) );
}

//-----
const cTime_t cTime_t::operator + (const cTime_t &T) const //operator + function
{
    int HourTemp,MinuteTemp,SecondTemp;//define 3 temp variables to get time data
    SecondTemp=Seconds+T. Seconds;
    if(SecondTemp>=60)//more thn 1 minute
    {
        SecondTemp-=60;
        MinuteTemp=Minutes+T. Minutes+1;//so add to minute
    }
    else
        MinuteTemp=Minutes+T. Minutes;
    if(MinuteTemp>=60)//more then 1 hour
    {
        MinuteTemp-=60;
        HourTemp=Hours+T. Hours+1;//add to hour
    }
    else
        HourTemp=Hours+T. Hours;
    if( HourTemp>=24)
    {
        FlagMoreDay=1; //to add day to date class
        HourTemp-=24;
    }
    return (cTime_t (HourTemp,MinuteTemp,SecondTemp) ); //return local time class
}

//-----
const cTime_t cTime_t::operator - (const cTime_t &T) const //operaor - function
{
    int HourTemp,MinuteTemp,SecondTemp;//define 3 temp variables to get time data
    HourTemp = Hours-T. Hours;
    if( HourTemp<0)//T class hour was bigger then THIS class
    {
        FlagLessDay=1; //to cut 1 day form date class
        HourTemp+=24;// add 24 hours to previous day
    }
    MinuteTemp=Minutes-T. Minutes;
    if(MinuteTemp<0)//same for minutes
    {
        MinuteTemp+=60;
        -- HourTemp;
    }
}

```

```

}
SecondTemp=Seconds-T.Seconds;
if(SecondTemp<0)//same for seconds
{
    SecondTemp+=60;
    --MinuteTemp;
}
return ( cTime_t ( HourTemp,MinuteTemp,SecondTemp ) );//return local class
}
//-----

void cTime_t::operator ++ () //operator ++ function
{
    if ( (Seconds==59) && (Minutes==59) && (Hours==23) ) // end of a day
    {
        FlagMoreDay=1;//to add day to day class
        Seconds=0;
        Minutes=0;
        Hours=0;
    }
    else if( (Seconds==59) && (Minutes==59) )//end of hour
    {
        Seconds=0;
        Minutes=0;
        ++Hours;
    }
    else if( Seconds==59 )//end of minute
    {
        Seconds=0;
        ++Minutes;
    }
    else
        ++Seconds;
}
//-----

void cTime_t::operator -- () //operator -- function
{
    if ( (Seconds==0) && (Minutes==0) && (Hours==0) )//start of day
    {
        FlagLessDay=1;//to substruct 1 day from day class
        Seconds=59;
        Minutes=59;
        Hours=23;
    }
}

```

```

    }
    else if( (Seconds==0) && (Minutes==0) )//start of hour
    {
        Seconds=59;
        Minutes=59;
        --Hours;
    }
    else if( Seconds==0 )//start of minute
    {
        Seconds=59;
        --Minutes;
    }
    else
        --Seconds;
}
//cTime_t.cpp end

//error.cpp
//implementation of error.h
#include "error.h"
void Error1() // used for setting class with c'tor or operator =
{
    cout<<"You tried to set a year that is less then 1900. which is illegal"<<endl;
}

void Error2() //used for trying to print an error class
{
    cout<<"There are error/s in your date (year less then 1900). Can't print date."<<endl;
}

void Error3() //used for operator +
{
    cout<<"There are error/s in your date (year less then 1900). Can't use + of date."<<endl;
}

void Error4() //used for operaor -
{
    cout<<"There are error/s in your date (year less then 1900). Can't use - of date."<<endl;
}

void Error5() //used for operator - and GetDayOfYear function
{

```

```

    cout<<"There are error/s in your date (year less then 1900). Can't get day of year."<<endl;
}
//error. cpp end

//TD. cpp

#include "error. h"
#include "TDmanage. h"
#include <iostream. h>
#include <string. h>
#include <stdio. h>
//main;
void main()
{
    char DayName[10];
    char MonthName[10];
    bool Test;
    int c;
    printf("Time Date — Exercise 1\n");
    printf("===== \n");
    // 1
    printf(" * * making default object (M1), get current computer time * * \n");
    cTDmanage M1;
    M1. print();
    putchar('\n');
    // 2
    printf(" * * get current day in default object * * \n");
    strcpy(DayName, M1. GetDDayOfWeekName());
    printf("%s\n", DayName);
    putchar('\n');
    // 3
    printf(" * * initialize a new object (M2) with date and time data and print it * * \n");
    cTDmanage M2(3,7,47,1904,1,27);
    M2. print();
    putchar('\n');
    // 4
    printf(" * * making a new object (M3) that will get the data of M1+M2 and print it * * \n");
    cTDmanage M3=M1+M2;
    M3. print();
    putchar('\n');
    // 5
    printf(" * * printing M3 in 2 others formats * * \n");
    M3. ChangeDateFormat();
}

```

```

M3. print();
M3. ChangeDateFormat();
M3. print();
putchar('\n');
printf("Press a Enter to continue...");
c=getchar();
printf("-----\n");
//6
printf(" * * making 2 'good' object (M4,M5) * *\n");
cTDmanage M4 (19,0,0,2000,2,1);
M4. print();
cTDmanage M5(18,0,0,1999,1,2);
M5. print();
putchar('\n');
//7
printf(" * * make new object (M6) and try to do M4-M5 and to print it * *\n");
cTDmanage M6=M4-M5;
M6. print();
putchar('\n');
//8
printf(" * * give M6 the data of M4 with operator = and print it (make him 'good') * *\n");
M6=M4;
M6. print();
putchar('\n');
//9
printf(" * * make object (M7) from classes time&date and print before/after using ++ * *\n");
cTDmanage M7 (cDate_t(1950,12,31),cTime_t(15,55,38));
M7. print();
++ M7;
M7. print();
putchar('\n');
printf("Press a Enter to continue...");
c=getchar();
printf("-----\n");
//10
printf(" * * print name of month of M7 * *\n");
strcpy(MonthName,M7. GetDNameOfMonth());
printf("%s\n",MonthName);
putchar('\n');
//11
printf(" * * check if M7 (1/1/1951,15:55:39) > M5 (2/1/1999,18:00:00) * *\n");
Test=M7>M5;
if(Test)

```

```

        printf("M7 is greater\n");
    else
        printf("M5 is greater\n");
    putchar('\n');
    //12
    printf(" * * make new object M8 equal to M7 and check it with operator == * * \n");
    cTDmanage M8(15,55,39,1951,1,1);
    Test=(M7==M8);
    if(Test)
        printf("M7 and M8 are equal\n");
    else
        printf("M7 and M8 are not equal\n");
    printf("\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\nThis are 12 examples of the program. More explanation in the
    note. txt file\n");
}
//TD. cpp end

//TDglobals. cpp Begin

#include <iostream. h>
#include "cTime_t. h"
#include "cDate_t. h"
#include "TDmanage. h"
#include "error. h"
//global functions:
//-----
//cTime_t global functions:
ostream&. operator << (ostream &out,const cTime_t &T) //operator << for class time function
{
    switch(cTime_t::format)
    {
        case 1:
            if(T.Seconds<10 && T.Minutes<10)
                out<<"The time is:  "<< T.Hours<<":0"<<T.Minutes<<":0"<<T.Seconds<<". "<<<endl;
            else if (T.Seconds<10)
                out<<"The time is:  "<< T.Hours<<": "<<T.Minutes<<":0"<<T.Seconds<<". "<<<endl;
            else if(T.Minutes<10)
                out<<"The time is:  "<< T.Hours<<":0"<<T.Minutes<<": "<<T.Seconds<<". "<<<endl;
            else
                out<<"The time is:  "<< T.Hours<<": "<<T.Minutes<<": "<<T.Seconds<<". "<<<endl;
            break;
        case 2:
            if(T.Hours<=12)

```

```

{
    if(T.Seconds<10 && T.Minutes<10)
        out<<"The time is:  "<<T.Hours<<":0"<<T.Minutes<<":0"<<T.Seconds<< "
        AM."<<endl;
    else if(T.Seconds<10)
        out<<"The time is:  "<<T.Hours<<": "<<T.Minutes<<":0"<<T.Seconds<< "
        AM."<<endl;
        else if(T.Minutes<10)
            out<<"The time is:  "<< T.Hours<<":0"<<T.Minutes<<": "<<T.Seconds
            <<"AM."<<endl;
        else
            out<<"The time is:  "<<T.Hours<<": "<<T.Minutes<<": "<<T.Seconds<<
            " AM."<<endl;
    }
    else
    {
        if(T.Seconds<10 && T.Minutes<10)
            out<<"The time is:  "<<T.Hours-12<<":0"<<T.Minutes<<":0"<<T.Seconds
            << " PM."<<endl;
        else if(T.Seconds<10)
            out<<"The time is:  "<<T.Hours-12<<": "<<T.Minutes<<":0"<<T.Seconds
            << " PM."<<endl;
        else if(T.Minutes<10)
            out<<"The time is:  "<< T.Hours<<":0"<<T.Minutes<<": "<<T.Seconds
            <<"PM."<<endl;
        else
            out<<"The time is:  "<<T.Hours-12<<": "<<T.Minutes<<": "<<T.Seconds<< "
            PM."<<endl;
    }
    break;
default:
    out<<"The format is not ok"<<endl;
}

return out;
}

istream&. operator >> (istream &in, cTime_t &T)//operator >> for class time function
{
    cout<<"Enter hour:";
    in>>T.Hours;
    cout<<"Enter Minutes:";
    in>>T.Minutes;
    cout<<"Enter Seconds:";
    in>>T.Seconds;
}

```

```

return in;
}

//cDate_t global functions:
ostream& operator << (ostream &out, const cDate_t &D)
//operator << for class date function
{
    if(D. Error)
    {
        Error2();        return out;
    }
    switch(cDate_t::format)
    {
    case 1: //European
        out<<"The date is: " << D. Day<<"/" <<D. Month<<"/" <<D. Year
        << ". " <<endl;
        break;
    case 2: //American
        out<<"The date is: " <<D. Month<<"/" <<D. Day<<"/" <<D. Year
        << ". " <<endl;
        break;
    case 3: //with string instead of month
        out<<"The date is: " <<D. Day<<"/" <<D. Mon[D. Month-1]<<"/" <<D. Year
        << ". " <<endl;
        break;
    default:
        out<<"The format is not ok" <<endl;
    }
    return out;
}

istream& operator >> (istream &in, cDate_t &D)//operator >> for class date function
{
    cout<<"Enter year:";
    in>>D. Year;
    cout<<"Enter month (1-12):";
    in>>D. Month;
    cout<<"Enter day (1-31):";
    in>>D. Day;
    if (D. Year<1900)
    {
        Error1();
        D. Error=true;
    }
}

```



```

        else
            D. Error=false;
        return in;
    }

ostream& operator << (ostream &out, const cTDmanage &M)
//operator << for class cTDmanage function
{
    out<<M. D<<" and "<<M. T<<endl;
    return out;
}

istream& operator >> (istream &in, cTDmanage &M)
//operator >> for class cTDmanage function
{
    in>>M. D;
    in>>M. T;
    return in;
}

//TDglobals. cpp End

//TDmanage. cpp
#include "TDmanage. h"
void cTDmanage::print() const
{
    D. print();
    T. print();
}

const cTDmanage cTDmanage::operator + (const cTDmanage& M)
{
    cTime_t TT = T + M. T;
    cDate_t DD = D + M. D;
    if(cTime_t::FlagMoreDay==1) ++DD. Day;
    cTime_t::FlagMoreDay=0;
    return ( cTDmanage ( DD, TT) );
}

const cTDmanage cTDmanage::operator - (const cTDmanage& M)
{
    cTime_t TT = T - M. T;
    cDate_t DD = D - M. D;
    if(cTime_t::FlagLessDay==1) --DD. Day;
    cTime_t::FlagLessDay=0;
    return ( cTDmanage ( DD, TT) );
}

//TDmanage. cpp End

```



第 2 题 日期格式处理

6.2.1 程序功能简介

一个处理日期格式的程序。

6.2.2 课程设计要求

- (1) 解决程序运行中会出现非法操作的问题。
- (2) 在源程序基础上增加一个中文格式,使之能够实现中文日期的处理功能。
- (3) 增加可以从文件中输入日期的功能。
- (4) 如有可扩充的好主意,请自行设计到程序中。

6.2.3 课程设计说明

评定难易级别:B 级。

6.2.4 关键源代码注解

```
// dateformatter. h
// defines a date datatype class.    Author: Vijay Mathew Pandyalakal.    Date: 10/02/2001

#ifndef DATEFORMATTER_H
#define DATEFORMATTER_H
#include <time. h>
#include <string>
using namespace std;
class DateFormatter {
private:
    int day;
    int mon;
    int year;
public:
    DateFormatter();
    DateFormatter(int day,int mon,int year);
    DateFormatter(char * date);
    inline int GetDay() { return day; } // Returns Day Value
    inline int GetMonth() { return mon; } // Returns Month
    inline int GetYear() { return year; } // Returns Year
    inline void SetDay(int d) { day = d; } // Sets new value for day
    inline void SetMonth(int m) { mon = m; } // Sets new value for month
    inline void SetYear(int y) { year = y; } // Sets new value for year
    inline bool IsLeapYear() { // Checks if current year is leap
```



```

        float rem = year%4;
        if(rem == 0.0) {
            return true;
        }else {
            return false;
        }
    }

    void AddDays(int days); // Adds a number of days to the current date
    void IncrementMonth(); // Increments current date by one month
    char * GetDate();
    int FindDifferenceInMonths(DateFormatter); // finds diff between 2 dates in months
    int FindDifferenceInDays(DateFormatter); // finds diff between 2 dates in days

    /* Static function that returns system date in format DD-MON-YYYY * */
    static char * Today() {
        const time_t curr = time(0);
        tm * dt = localtime(&curr);
        int d = dt->tm_mday;
        int y = (dt->tm_year+1900);
        int m = dt->tm_mon;
        char mn[4];
        strcpy(mn, FindMonth(m));
        char ret[25];
        strcpy(ret, FormatDate(d, mn, y));
        return ret;
    }

    /* */
    char * GetFormattedDate();
    int operator < (DateFormatter df);
    int operator > (DateFormatter df);
    int operator == (DateFormatter df);
private:
    static char * FindMonth(int m) {
        if(m == 0) {
            return "JAN";
        }else if(m == 1) {
            return "FEB";
        }else if(m == 2) {
            return "MAR";
        }else if(m == 3) {
            return "APR";
        }else if(m == 4) {
            return "MAY";
        }
    }

```

```
    }else if(m == 5) {
        return "JUN";
    }else if(m == 6) {
        return "JUL";
    }else if(m == 7) {
        return "AUG";
    }else if(m == 8) {
        return "SEP";
    }else if(m == 9) {
        return "OCT";
    }else if(m == 10) {
        return "NOV";
    }else if(m == 11) {
        return "DEC";
    }else {
        return "NUL";
    }
}

int FindMonthInNumber(string m);
static char * FormatDate(int d,char * m,int y) {
    char * dy = new char[25];
    char * yr = new char[25];
    dy = itoa(d,dy,10);
    yr = itoa(y,yr,10);
    strcat(dy,"-");
    strcat(dy,m);
    strcat(dy,"-");
    strcat(dy,yr);
    return dy;
}

string to_lower(string s);
int FindDaysInMonth(int m) {
    if(m == 0) {
        return 31;
    }else if(m == 1) {
        if(IsLeapYear()) {
            return 29;
        }else {
            return 28;
        }
    }else if(m == 2) {
        return 31;
    }else if(m == 3) {
```

```

        return 30;
    } else if(m == 4) {
        return 31;
    } else if(m == 5) {
        return 30;
    } else if(m == 6) {
        return 31;
    } else if(m == 7) {
        return 31;
    } else if(m == 8) {
        return 30;
    } else if(m == 9) {
        return 31;
    } else if(m == 10) {
        return 30;
    } else if(m == 11) {
        return 31;
    }
    return -1;
}

};

#endif

// end of dateformatter.h

// dateformatter.cpp. implements DateFormatter class. Author: Vijay Mathew Pandyalakal
// Date: 12/23/2001 11:36:48 AM

#include <stdlib.h>
#include "DateFormatter.h"

//~~~~~ function default constructor ~~~~~
// Default constructor that creates a DateFormatter Object with system date arguments: 0 0
// returns 0
DateFormatter::DateFormatter() {
    const time_t curr = time(0);
    tm *dt = localtime(&curr);
    this->day = dt->tm_mday;
    this->mon = dt->tm_mon;
    this->year = (dt->tm_year+1900);
}

//~~~~~ end of function default constructor ~~~~~
//~~~~~ function overloaded constructor ~~~~~

```

```

// Constructor that creates a DateFormatter Object with arguments as date values
// arguments: int,int,int
// arg1 is day,arg2 month & arg3 is year
// returns 0
DateFormatter::DateFormatter(int day,int mon,int year) {
    this->day = day;
    this->mon = mon;
    this->year = year;
}
//~~~~~ end of function overloaded constructor~~~~~
//~~~~~ function overloaded constructor - 2 ~~~~~
// This constructor takes a string in the format DD-MON-YYYY or DD-MM-YY & creates a date from // that
// arguments: char *
// arg is a string in the format dd-mon-yyyy
// returns 0
DateFormatter::DateFormatter(char * date) {
    if(strlen(date) > 8) {
        string s1 = date;
        int p = s1.find_first_of('-');
        string day;
        string month;
        string year;
        if(p == 2) {
            day = s1.substr(0,2);
            month = s1.substr(3,3);
            year = s1.substr(7,4);
        }else if(p==1){
            day = s1.substr(0,1);
            month = s1.substr(2,3);
            year = s1.substr(6,4);
        }
        char cday[5];
        strcpy(cday,day.c_str());
        char cyear[5];
        strcpy(cyear,year.c_str());
        this->day = atoi(cday);
        this->mon = FindMonthInNumber(month);
        this->year = atoi(cyear);
    }else if(strlen(date) == 8 || strlen(date) == 7 || strlen(date) == 6) {
        int len = strlen(date);
        string str = date;
        string day;
        string month;
    }
}

```

```
string year;
int p1 = str.find_first_of('-');
if(p1 <= 0) p1 = str.find_first_of('/');
int p2 = str.find_last_of('-');
if(p2 <= 0) p2 = str.find_last_of('/');
if(len == 8) {
    day = str.substr(0,2);
    month = str.substr(3,2);
    year = str.substr(6,2);
}else if(len == 7) {
    if(p1 == 1) {
        day = str.substr(0,1);
        if(p2 == 3) {
            month = str.substr(2,1);
            year = str.substr(4,2);
        }else {
            month = str.substr(2,2);
            year = str.substr(5,2);
        }
    }else {
        day = str.substr(0,2);
        if(p2 == 3) {
            month = str.substr(2,1);
            year = str.substr(4,2);
        }else {
            month = str.substr(3,2);
            year = str.substr(5,2);
        }
    }
}else if(len == 6) {
    day = str.substr(0,1);
    month = str.substr(2,1);
    year = str.substr(4,2);
}
int m = atoi((month.c_str()));
m -= 1;
int yr = atoi((year.c_str()));
char y[5];
if(yr >= 0 && yr <= 70) {
    strcpy(y,"20");
    strcat(y,year.c_str());
} else {
    strcpy(y,"19");
```

```

    strcat(y, year.c_str());
}
this->day = atoi((day.c_str()));
this->mon = m;
this->year = atoi(y);
}
}
//~~~~~ end of function overloaded constructor — 2~~~~~
//~~~~~ function FindMonthInNumber ~~~~~
// Private function that take a string in the format MON (like JAN) & returns
// arguments: string
// arg specifies a month in format mmm
// returns the month in a number (0-11)
int DateFormatter::FindMonthInNumber(string m) {
    if(to_lower(m) == "jan") {
        return 0;
    } else if(to_lower(m) == "feb") {
        return 1;
    } else if(to_lower(m) == "mar") {
        return 2;
    } else if(to_lower(m) == "apr") {
        return 3;
    } else if(to_lower(m) == "may") {
        return 4;
    } else if(to_lower(m) == "jun") {
        return 5;
    } else if(to_lower(m) == "jul") {
        return 6;
    } else if(to_lower(m) == "aug") {
        return 7;
    } else if(to_lower(m) == "sep") {
        return 8;
    } else if(to_lower(m) == "oct") {
        return 9;
    } else if(to_lower(m) == "nov") {
        return 10;
    } else if(to_lower(m) == "dec") {
        return 11;
    } else {
        return -1;
    }
}
//~~~~~ end of function FindMonthInNumber~~~~~

```



```

//~~~~~ function to_lower ~~~~~
// Utility function that converts a string to lower case
// arguments: string
// arg is the string to be changed to lower case
// returns lower cased string
string DateFormatter::to_lower(string s) {
    int len = s.length();
    for(int i=0;i<len;i++) {
        if(s[i] >= 'A' && s[i] <= 'Z') {
            s[i] = tolower(s[i]);
        }
    }
    return s;
}

//~~~~~ end of function to_lower~~~~~
/* * PUBLIC FUNCTIONS * */
//~~~~~ function GetDate ~~~~~
// Returns date in format DD/MM/YYYY
// arguments: 0
// 0
// returns date as a string in the format dd/mm/yyyy
char * DateFormatter::GetDate() {
    char * d = new char[25];
    char * m = new char[5];
    char * y = new char[5];
    d = itoa(day,d,10);
    m = itoa((mon+1),m,10);
    y = itoa(year,y,10);
    strcat(d,"/");
    strcat(d,m);
    strcat(d,"/");
    strcat(d,y);
    char ret[25];
    strcpy(ret,d);
    delete[] d;
    delete[] m;
    delete[] y;
    return ret;
}

//~~~~~ end of function GetDate~~~~~

//~~~~~ function GetFormattedDate ~~~~~
// Returns date in format DD-MON-YYYY

```

```

// arguments: 0
// 0
// returns date as string

char * DateFormatter::GetFormattedDate() {
    char mn[5];
    strcpy(mn, FindMonth(mon));
    char ret[25];
    strcpy(ret, FormatDate(day, mn, year));
    return ret;
}
//~~~~~ end of function GetFormattedDate~~~~~

//~~~~~ function AddDays ~~~~~
// adds a number of days to the date
// arguments: int
// arg is the no. of days to add
// returns void
void DateFormatter::AddDays(int days) {
    if(days <= 0) return;
    int curr_day = this->day;
    int curr_mon = this->mon;
    int days_in_mon = FindDaysInMonth(curr_mon);
    if(this->mon == 11 && days_in_mon == curr_day) {
        this->year++;
    }
    if(days_in_mon == curr_day) {
        this->mon++;
        if(this->mon > 11) {
            this->mon = 0;
        }
    }
    if(days <= days_in_mon) {
        this->day += days;
        if(this->day > days_in_mon) {
            this->day = 1;
        }
        return;
    } else {
        int days_to_add = days_in_mon - curr_day;
        this->day += days_to_add;
        if(this->day > days_in_mon) {
            this->day = 1;
        }
    }
}

```

```

    }
    int rem_days = days - days_to_add;
    AddDays(rem_days);
    return;
}
//~~~~~ end of function AddDays~~~~~

```

```

//~~~~~ function IncrementMonth ~~~~~
// increments by 1 month
// arguments:
//
// returns void
void DateFormatter::IncrementMonth() {
    mon++;
    if(mon > 11) {
        mon = 0;
        year++;
    }
}
//~~~~~ end of function IncrementMonth~~~~~

```

```

//~~~~~ function FindDifferenceInMonths ~~~~~
// find diff between 2 dates in months
// arguments: DateFormatter 2nd_date
//
// returns no of months
int DateFormatter::FindDifferenceInMonths(DateFormatter dft) {
    int diff = 0;
    if(dft > *this) {
        return -1;
    } else if(dft == *this) {
        return 0;
    } else {
        while(1) {
            dft.IncrementMonth();
            diff++;
            if((dft.GetYear() == this->year) && (dft.GetMonth() == this->mon)) {
                break;
            }
        }
    }
    if(this->GetDay() < dft.GetDay()) {

```

```

        diff--;
    }
    return diff;
}
//~~~~~ end of function FindDifferenceInMonths~~~~~

//~~~~~ function FindDifferenceInDays ~~~~~
// finds diff of two dates in days
// arguments: DateFormatter
//
// returns No of days, -1 if argument date is less than this
int DateFormatter::FindDifferenceInDays(DateFormatter dft) {
    if( * this < dft) {
        return -1;
    }else if( * this == dft) {
        return 0;
    }else {
        int ret = 0;
        while(1) {
            if(dft == * this) {
                break;
            }
            dft.AddDays(1);
            ret++;
        }
        return ret;
    }
}
//~~~~~ end of function FindDifferenceInMonths~~~~~

/* Overloaded comparison operators */

int DateFormatter::operator < (DateFormatter df) {
    if(df.year > this->year) {
        return 1;
    }else if(df.year == this->year && df.mon > this->mon) {
        return 1;
    }else if(df.mon == this->mon && df.day > this->day) {
        return 1;
    }else {
        return 0;
    }
}

```

```

int DateFormatter::operator > (DateFormatter df) {
    if(df. year < this->year) {
        return 1;
    }else if(df. year == this->year && df. mon < this->mon) {
        return 1;
    }else if(df. mon == this->mon && df. day < this->day) {
        return 1;
    }else {
        return 0;
    }
}

```

```

int DateFormatter::operator == (DateFormatter df) {
    if( (df. year == this->year)
        && (df. mon == this->mon) && (df. day == this->day)) {
        return 1;
    }else {
        return 0;
    }
}

```

```

// ~~~~~
// end of dateformatter. cpp
// ~~~~~

```

```

#include <iostream. h>
#include "dateformatter. h"

```

```

int main() {

```

```

    //Dateformatter class supports 2 formats of dates :- DD-MON-YYYY & DD/MM/YY
    DateFormatter dft1("1-MAR-2001");
    DateFormatter dft2("15/4/02");
    char tmp[25];

```

```

    //Prints First & second dates
    strcpy(tmp,dft1. GetFormattedDate());
    cout<<"First date : "<<tmp<<endl;
    strcpy(tmp,dft2. GetFormattedDate());
    cout<<"Second date: "<<tmp<<endl;

```

```

    //Prints difference in days

```



```
cout<<"Diif in days: "<<dft2. FindDifferenceInDays(dft1)<<endl;
//Prints difference in months
cout<<"Diif in months: "<<dft2. FindDifferenceInMonths(dft1)<<endl;

// Prints current date
strcpy(tmp,dft1. Today());
cout<<"Today is: "<<tmp<<endl;

// Adds 35 days to dft2
dft2. AddDays(35);

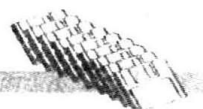
//& prints its current value in the format DD-MON-YYYY
strcpy(tmp,dft2. GetFormattedDate());
cout<<"Added 35 days to date 2: "<<tmp<<endl;

//Increments dft1 by 1 month
dft1. IncrementMonth();

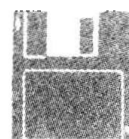
//& prints its current value in the format DD/MM/YY
strcpy(tmp,dft1. GetDate());
cout<<"Incremented first date's month: "<<tmp<<endl;

//Performs comparison operations on the two DateFormatter objects
//& prints result
cout<<"Date1 < Date2: "<<(dft1 < dft2)<<endl;
cout<<"Date1 == Date2: "<<(dft1 == dft2)<<endl;
cout<<"Date1 > Date2: "<<(dft1 > dft2)<<endl;

return 0;
}
```

7 多媒体应用程序



第 1 题 屏保程序

7.1.1 程序功能简介

一个屏保程序,可以显示时钟或礼花。

7.1.2 课程设计要求

(1) 可以随时任意修改时钟显示的时间,并可以设定报时功能,时钟到设定的时间即自动响铃。

(2) 改进礼花程序,使之随机变换多种颜色和形状。

(3) 设定屏幕锁定功能,任按一键后要求输入密码(自定),密码符合后才能返回,否则继续执行屏保程序。

7.1.3 课程设计说明

(1) 参考 graphics.h 中的作图功能。

(2) 评定难易级别:A 级。

7.1.4 关键源代码注解

```
#include <stdio.h>
#include <conio.h>
#include <dos.h>
#include <graphics.h>
#include <math.h>
#include <bios.h>
#include <stdlib.h>
void Clock_f() /* 时钟 */
{
    int gd=DETECT, gm, no, rad=210;
    struct time t;
    float theta, x, y, x1, y1, theta1, theta2;
    char des[40], ch;
    initgraph(&gd, &gm, "");
    setcolor(14);
    circle(320, 240, 180);
    circle(320, 240, 15);
    setcolor(BLACK);
    circle(320, 240, 170);
    circle(320, 240, 165);
    circle(320, 240, 158);
    circle(320, 240, 16);
```



```

setcolor(11);
for(theta=0.0,no=0;no<60;no++,theta+=.1047)
{
    if(no%5==0)
    {
        x=320+158*cos(theta);
        y=240+158*sin(theta);
    }
    else
    {
        x=320+165*cos(theta);
        y=240+165*sin(theta);
    }
    x1=320+170*cos(theta);
    y1=240+170*sin(theta);
    line(x,y,x1,y1);
}
settextjustify(CENTER_TEXT,CENTER_TEXT);
setcolor(YELLOW);
for(no=1,theta=5.236;no<=12;no++,theta+=.524)
{
    x=320+rad*cos(theta);
    y=240+rad*sin(theta);
    settextstyle(GOTHIC_FONT,HORIZ_DIR,5);
    sprintf(des,"%d",no);
    outtextxy(x,y,des);
}
gettime(&t);
theta1=(.0175*6*t.ti_min)-1.5708;
theta2=(.0175*30*(t.ti_hour%12+t.ti_min/60.0))-1.5708;
while(bioskey(1)==0)
{
    gettime(&t);
    theta=(.0175*6*t.ti_sec)-1.5708;
    x=320+15*cos(theta);
    y=240+15*sin(theta);
    x1=320+150*cos(theta);
    y1=240+150*sin(theta);
    setcolor(11);
    setlinestyle(SOLID_LINE,1,1);
    line(x,y,x1,y1);
    sound(7000);
    delay(100);
}

```

```

        nosound();
        delay(900);
        setcolor(BLACK);
        line(x,y,x1,y1);
        setlinestyle(SOLID_LINE,1,3);
        x=320+15*cos(theta1);
        y=240+15*sin(theta1);
        x1=320+150*cos(theta1);
        y1=240+150*sin(theta1);
        line(x,y,x1,y1);
        x=320+15*cos(theta2);
        y=240+15*sin(theta2);
        x1=320+100*cos(theta2);
        y1=240+100*sin(theta2);
        line(x,y,x1,y1);
        theta1=(.0175*6*(t.ti_min+t.ti_sec/60.0))-1.5708;
        x=320+15*cos(theta1);
        y=240+15*sin(theta1);
        x1=320+150*cos(theta1);
        y1=240+150*sin(theta1);
        setcolor(14);
        line(x,y,x1,y1);
        circle(320,240,15);
        theta2=(.0175*30*(t.ti_hour%12+t.ti_min/60.0))-1.5708;
        x=320+15*cos(theta2);
        y=240+15*sin(theta2);
        x1=320+100*cos(theta2);
        y1=240+100*sin(theta2);
        line(x,y,x1,y1);
    }
    getch();
    closegraph();
}

void Boom_f(void)/* 礼花 */
{
    int gd=DETECT, gm;
    int maxx=0, maxy=0, midx=0, midy=0, a=0, b=0, x=0, y=0, k=0;
    initgraph(&gd, &gm, "");
    maxx=getmaxx(), maxy=getmaxy(), midx=maxx/2, midy=maxy/2;
    randomize();
    while(bioskey(1)==0)
    {

```



```

    x=60;
    y=50;
    for(b=-100;b<=100;b+=10)
    {
        k=random(15);
        if(k==0)k=1;
        for(a=0;a<=50;a++)
        {
            putpixel((midx+a)+b, (midy+a)+b, k);
            putpixel((midx+a)+b, (midy-a)+b, k);
            putpixel((midx-a)+b, (midy+a)+b, k);
            putpixel((midx-a)+b, (midy-a)+b, k);
            putpixel((midx)+b, (midy+a)+b, k+8);
            putpixel((midx)+b, (midy-a)+b, k+8);
            putpixel((midx+a)+b, (midy)+b, k+8);
            putpixel((midx-a)+b, (midy)+b, k+8);

            putpixel((midx+a)+b+x, (midy+a)+b+y, k);
            putpixel((midx+a)+b+x, (midy-a)+b+y, k);
            putpixel((midx-a)+b+x, (midy+a)+b+y, k);
            putpixel((midx-a)+b+x, (midy-a)+b+y, k);
            putpixel((midx)+b+x, (midy+a)+b+y, k+8);
            putpixel((midx)+b+x, (midy-a)+b+y, k+8);
            putpixel((midx+a)+b+x, (midy)+b+y, k+8);
            putpixel((midx-a)+b+x, (midy)+b+y, k+8);

            putpixel((midx+a)+b-x, (midy+a)+b-y, k);
            putpixel((midx+a)+b-x, (midy-a)+b-y, k);
            putpixel((midx-a)+b-x, (midy+a)+b-y, k);
            putpixel((midx-a)+b-x, (midy-a)+b-y, k);
            putpixel((midx)+b-x, (midy+a)+b-y, k+8);
            putpixel((midx)+b-x, (midy-a)+b-y, k+8);
            putpixel((midx+a)+b-x, (midy)+b-y, k+8);
            putpixel((midx-a)+b-x, (midy)+b-y, k+8);

            putpixel((midx+a)+b+x, (midy+a)+b-y, k);
            putpixel((midx+a)+b+x, (midy-a)+b-y, k);
            putpixel((midx-a)+b+x, (midy+a)+b-y, k);
            putpixel((midx-a)+b+x, (midy-a)+b-y, k);
            putpixel((midx)+b+x, (midy+a)+b-y, k+8);
            putpixel((midx)+b+x, (midy-a)+b-y, k+8);
            putpixel((midx+a)+b+x, (midy)+b-y, k+8);
            putpixel((midx-a)+b+x, (midy)+b-y, k+8);
        }
    }

```

```
putpixel((midx-a)+b+x, (midy)+b-y, k+8);
```

```
putpixel((midx+a)+b-x, (midy+a)+b+y, k);
```

```
putpixel((midx+a)+b-x, (midy-a)+b+y, k);
```

```
putpixel((midx-a)+b-x, (midy+a)+b+y, k);
```

```
putpixel((midx-a)+b-x, (midy-a)+b+y, k);
```

```
putpixel((midx)+b-x, (midy+a)+b+y, k+8);
```

```
putpixel((midx)+b-x, (midy-a)+b+y, k+8);
```

```
putpixel((midx+a)+b-x, (midy)+b+y, k+8);
```

```
putpixel((midx-a)+b-x, (midy)+b+y, k+8);
```

```
/* beginning if(a<36) */
```

```
circle((midx+a)+b, (midy+a)+b, k);
```

```
circle((midx+a)+b, (midy-a)+b, k);
```

```
circle((midx-a)+b, (midy+a)+b, k);
```

```
circle((midx-a)+b, (midy-a)+b, k);
```

```
circle((midx)+b, (midy+a)+b, k+8);
```

```
circle((midx)+b, (midy-a)+b, k+8);
```

```
circle((midx+a)+b, (midy)+b, k+8);
```

```
circle((midx-a)+b, (midy)+b, k+8);
```

```
circle((midx+a)+b+x, (midy+a)+b+y, k);
```

```
circle((midx+a)+b+x, (midy-a)+b+y, k);
```

```
circle((midx-a)+b+x, (midy+a)+b+y, k);
```

```
circle((midx-a)+b+x, (midy-a)+b+y, k);
```

```
circle((midx)+b+x, (midy+a)+b+y, k+8);
```

```
circle((midx)+b+x, (midy-a)+b+y, k+8);
```

```
circle((midx+a)+b+x, (midy)+b+y, k+8);
```

```
circle((midx-a)+b+x, (midy)+b+y, k+8);
```

```
circle((midx+a)+b-x, (midy+a)+b-y, k);
```

```
circle((midx+a)+b-x, (midy-a)+b-y, k);
```

```
circle((midx-a)+b-x, (midy+a)+b-y, k);
```

```
circle((midx-a)+b-x, (midy-a)+b-y, k);
```

```
circle((midx)+b-x, (midy+a)+b-y, k+8);
```

```
circle((midx)+b-x, (midy-a)+b-y, k+8);
```

```
circle((midx+a)+b-x, (midy)+b-y, k+8);
```

```
circle((midx-a)+b-x, (midy)+b-y, k+8);
```

```
circle((midx+a)+b+x, (midy+a)+b-y, k);
```

```
circle((midx+a)+b+x, (midy-a)+b-y, k);
```

```
circle((midx-a)+b+x, (midy+a)+b-y, k);
```

```
circle((midx-a)+b+x, (midy-a)+b-y, k);
```



```

circle((midx)+b+x, (midy+a)+b-y, k+8);
circle((midx)+b+x, (midy-a)+b-y, k+8);
circle((midx+a)+b+x, (midy)+b-y, k+8);
circle((midx-a)+b+x, (midy)+b-y, k+8);

circle((midx+a)+b-x, (midy+a)+b+y, k);
circle((midx+a)+b-x, (midy-a)+b+y, k);
circle((midx-a)+b-x, (midy+a)+b+y, k);
circle((midx-a)+b-x, (midy-a)+b+y, k);
circle((midx)+b-x, (midy+a)+b+y, k+8);
circle((midx)+b-x, (midy-a)+b+y, k+8);
circle((midx+a)+b-x, (midy)+b+y, k+8);
circle((midx-a)+b-x, (midy)+b+y, k+8);
delay(25);
cleardevice();
}
}
}
closegraph();
}

void main(void)
{
    int nChoice;
    do
    {
        printf("\n\t1. CLOCK\n");
        printf("\t2. BOOM\n");
        printf("\t3. EXIT\n");
        scanf("%d", &nChoice);
        switch(nChoice)
        {
            case 1:
                Clock_f();
                break;
            case 2:
                Boom_f();
                break;
            case 3:
                break;
            default:
                printf("input again\n");
        }
    } while(nChoice != 3);
}

```

第2题 牛顿法图解方程

7.2.1 程序功能简介

利用牛顿切线法求解固定方程 $x^2 - 5x + 6 = 0$, 并图示解方程过程。

7.2.2 课程设计要求

仿照源程序用弦截法求解任意系数之方程, 要求如下:

(1) 可以求解一元 n 次方程的实数解, 输入 $ax^n + bx^{(n-1)} + \dots$ 的形式, 在程序中识别 n 及各项系数。

(2) 输入 x 的初值范围, 输入后要判断范围内是否有实数解。

(3) 画出弦截线及多次迭代的过程, 要求有必要的坐标说明。

(4) 输出结果。

(5) 可以从数据文件中读取方程并将方程解输出到该文件中。

7.2.3 课程设计说明

(1) 参考 graphics.h 中的作图功能。

(2) 评定难易级别: A 级。

7.2.4 关键源代码注解

```
#include<graphics.h>
#include<math.h>
#include<stdio.h>
float flRatioX, flRatioY, flX0=10, flY0=10, flXmin, flXmax, flYmin, flYmax;
double Cla_f(double flX) /* 方程 */
{
    return flX * flX - 5 * flX + 6;
}

double Cald_f(double flX) /* 方程求导 */
{
    return 2 * flX - 5;
}
float Xcoor_f(float flX) /* 已知 x 求解 x 在显示器上的坐标 */
{
    return flX0 + (flX - flXmin) * flRatioX;
}
float Ycoor_f(float y) /* 已知 y 求解 y 在显示器上的坐标 */
{
    return 459 - (flY0 + (y - flYmin) * flRatioY);
}
main()
{
    int gdriver=DETECT, gmode;
    float flX;
```

```

char szStr[100];
float flXstart, flYstart, flX1, flX2;
float flXlength, flYlength;
printf("Input the range of x:");
scanf("%f%f", &flXmin, &flXmax);
printf("input start x:");
scanf("%f", &flXstart);
flYstart = Cla_f(flXstart);
initgraph(&gdriver, &gmode, ""); /* 设置图形格式 */
setbkcolor(1); /* 背景色 */
setcolor(12); /* 画笔色 */
rectangle(0, 0, 639, 479);
setlinestyle(0, 0, 2); /* 线宽 */
flXlength = flXmax - flXmin; /* x 的范围 */
flRatioX = 619 / flXlength; /* x 放大的比率 */
flYmin = flYmax = Cla_f(flXmin);
for(flX = flXmin; flX <= flXmax; flX = flX + 0.01)
    if(flYmin > Cla_f(flX))
        flYmin = Cla_f(flX);
    else
        if(flYmax < Cla_f(flX))
            flYmax = Cla_f(flX);
flYlength = flYmax - flYmin; /* y 的范围 */
flRatioY = 459 / flYlength; /* y 放大的比率 */
moveto(Xcoor_f(flXmin), Ycoor_f(Cla_f(flXmin)));
for(flX = flXmin; flX <= flXmax; flX = flX + (flXmax - flXmin) / 100) /* 画方程曲线 */
{
    lineto(Xcoor_f(flX), Ycoor_f(Cla_f(flX)));
}
setcolor(9);
line(Xcoor_f(0), 10, Xcoor_f(0), 469);
line(10, Ycoor_f(0), 629, Ycoor_f(0));
outtextxy(Xcoor_f(0) + 5, Ycoor_f(0) + 5, "(0,0)"); /* 画原点坐标 */
sprintf(szStr, "(%.1f,0)", flXmin);
outtextxy(Xcoor_f(flXmin), Ycoor_f(0) + 5, szStr);
sprintf(szStr, "(%.1f,0)", flXmax);
outtextxy(Xcoor_f(flXmax) - 20, Ycoor_f(0) + 5, szStr); /* 初值说明 */
flX1 = flX2 = flXstart;
setcolor(15);
outtextxy(Xcoor_f(flX2) + 5, Ycoor_f(0) + 5, "start");
do
{
    setcolor(10);

```



```
line(Xcoor_f(flX1), Ycoor_f(0), Xcoor_f(flX1), Ycoor_f(Cla_f(flX1))); /* 垂直于 x 轴的直线 */
flX2 = flX1;
flX1 = flX2 - Cla_f(flX2) / Cald_f(flX2);
setcolor(14);
line(Xcoor_f(flX2), Ycoor_f(Cla_f(flX2)), Xcoor_f(flX1), Ycoor_f(0)); /* 切线 */
} while (fabs(Cla_f(flX1)) > 0.01);
sprintf(szStr, "%.2f", flX1);
setcolor(15);
outtextxy(Xcoor_f(flX2) + 5, Ycoor_f(0) + 5, szStr); /* 标注方程解 */
getch();
closegraph();
```



第 3 题 图形运动程序

7.3.1 程序功能简介

本题由两个程序组成:小车运动与小球碰撞。

7.3.2 课程设计要求

将这两个程序组合成一个程序,要求如下:

- (1) 小车运动方式为复制屏幕图像方式(用 putimage() 函数)。
- (2) 小车沿固定轨迹来回运动。
- (3) 小球随机运动,当小球与小车碰撞后,小球向相反方向随机运动,碰到屏幕边缘后运动方向依然是随机的。

7.3.3 课程设计说明

- (1) 参考 graphics.h 中的作图功能。
- (2) 评定难易级别:A 级。

7.3.4 关键源代码注解

```
//小车运动
#include "stdio.h"
#include "graphics.h"
#include "dos.h"
void main()
{
    int gd=DETECT,i=0,j,k=0;
    initgraph(&gd,&gd,"\\tc\\bgi");
    while(! kbhit())/* 不按键返回 0 */
    {
        for( j=0;j<2;j++)
        {
            for(i=0;i<500;i+=5)
            {
                setcolor(RED);
                line(118+i,65+j*300,160+i,65+j*300);
                line(120+i,70+j*300,160+i,70+j*300);
                line(60+i,100+j*300,220+i,100+j*300);
                line(200+i,100+j*300,160+i,70+j*300);
                line(205+i,100+j*300,160+i,65+j*300);
                line(80+i,100+j*300,120+i,70+j*300);
                line(75+i,100+j*300,118+i,65+j*300);
```

```

        setfillstyle(SOLID_FILL, RED);
        floodfill(77+i, 99+j*300, RED);
        line(135+i, 70+j*300, 135+i, 100+j*300);
        line(145+i, 70+j*300, 145+i, 100+j*300);
        setfillstyle(SOLID_FILL, RED);
        floodfill(137+i, 75+j*300, RED);

        setfillstyle(SOLID_FILL, WHITE);
        floodfill(130+i, 75+j*300, RED);
        setfillstyle(SOLID_FILL, WHITE);
        floodfill(150+i, 75+j*300, RED);
    /* rectangle(left, top, right, bottom); */
    /*   rectangle(80+i, 90+j*300, 100+i, 110+j*300); */
        line(150+i, 90+j*300, 160+i, 80+j*300);
        line(154+i, 90+j*300, 164+i, 80+j*300);
        line(158+i, 90+j*300, 168+i, 80+j*300);
        line(110+i, 90+j*300, 120+i, 80+j*300);
        line(114+i, 90+j*300, 124+i, 80+j*300);
        line(118+i, 90+j*300, 128+i, 80+j*300);
        line(20+i, 140+j*300, 260+i, 140+j*300);
    /* arc(midx, midy, stangle, endangle, radius); */
        arc(200+i, 140+j*300, 0, 177, 25);
        arc(80+i, 140+j*300, 0, 177, 25);
        arc(60+i, 140+j*300, 90, 178, 40);
        arc(220+i, 140+j*300, 0, 90, 40);
        setfillstyle(SOLID_FILL, RED);
        floodfill(100+i, 110+j*300, RED);
    /* 边框黑色 */
        setcolor(0);
        line(139+i, 70+j*300, 139+i, 220+j*300);
        line(140+i, 70+j*300, 140+i, 220+j*300);
        line(141+i, 70+j*300, 141+i, 220+j*300);
        setcolor(RED);
    /* 车灯 */
        setcolor(YELLOW);
        circle(250+i, 130+j*300, 5);
        line(253+i, 125+j*300, 312+i, 113+j*300);
        line(253+i, 135+j*300, 310+i, 143+j*300);
        arc(315+i, 128+j*300, 250, 100, 15);
    /* EMPTY_FILL, SOLID_FILL, LINE_FILL, LTSLASH_FILL, SLASH_FILL,
       BKSLASH_FILL, LTBKSLASH_FILL, HATCH_FILL, XHATCH_FILL,
       INTERLEAVE_FILL, WIDE_DOT_FILL, CLOSE_DOT_FILL, */
        setfillstyle(LTBKSLASH_FILL, YELLOW);

```

✧ ✧ ✧ ✧ ✧

```

floodfill(280+i,130+j*300,YELLOW);
/* 排气管 */
rectangle(15+i,137+j*300,20+i,140+j*300);
setcolor(BROWN);
circle(15+i-k,135+j*300,k);
setfillstyle(INTERLEAVE_FILL,BROWN);
floodfill(15+i-k,135+j*300,BROWN);
if(k>20)      k=2;
else          k+=5;
setfillstyle(SOLID_FILL,15);
floodfill(190+i,120+j*300,RED);
floodfill(80+i,120+j*300,RED);
/* 轮胎 */
setcolor(DARKGRAY);
circle(200+i,140+j*300,18);
circle(80+i,140+j*300,18);
setfillstyle(SOLID_FILL,DARKGRAY);
floodfill(190+i,140+j*300,DARKGRAY);
floodfill(80+i,140+j*300,DARKGRAY);
setcolor(RED);
circle(200+i,140+j*300,10);
circle(80+i,140+j*300,10);
setfillstyle(SOLID_FILL,RED);
floodfill(205+i,140+j*300,RED);
floodfill(85+i,140+j*300,RED);
clearviewport();/* 清除窗口信息 */
if(kbhit()) break;
}
}

```

```

}
closegraph();
}

```

//模拟小球碰撞

#include<graphics.h>

#include<stdlib.h>

main()

```

{
    int i,gdriver,gmode, size;
    void * buf;
    gdriver=DETECT;
    initgraph(&gdriver,&gmode,"");
    setbkcolor(BLUE);
    cleardevice();/* 清除屏幕 */

```

```
setcolor(LIGHTRED);
setlinestyle(0,0,1);
setfillstyle(1,10);
circle(100,200,30);
floodfill(100,200,12);
size=imagesize(69,169,131,231);/* 返回存储屏幕所需的字节数 */
buf=malloc(size);/* 动态分配内存空间 */
getimage(69,169,131,231,buf);/* 保存指定位置的小球图 */
putimage(500,169,buf,COPY_PUT);/* 在另一位置上复制小球 */
while(! kbhit())
{
    for(i=0;i<185;i++)
    {
        delay(400);/* 延迟时间,使运动变慢 */
        putimage(70+i,170,buf,COPY_PUT);/* 左边球右移 */
        putimage(500-i,170,buf,COPY_PUT);/* 右边球左移 */
    }
    for(i=0;i<185;i++)
    {
        delay(400);
        putimage(255-i,170,buf,COPY_PUT);
        putimage(315+i,170,buf,COPY_PUT);
    }
}
getch();
closegraph();
}
```



第 4 题 特殊图像效果

7.4.1 程序功能简介

特殊图像效果,可以对 512×512 的 256 色位图进行柔化、锐化、浮雕效果的处理。

7.4.2 课程设计要求

- (1) 扩展程序功能,可以根据所输入的图像而对任意大小的位图进行处理。
- (2) 对图像进行任意位置及大小的马赛克处理。
- (3) 对图像进行加强柔化及锐化效果的处理。
- (4) 对图像进行扩散效果的处理(即将像素周围的某点值随机赋给该像素)。

7.4.3 课程设计说明

(1) 参考位图图像(*.bmp)的组成结构,图像中的像素点的值实际上是调色板数组中的元素序号。

(2) 评定难易级别:A 级。

7.4.4 关键源代码注解

```
#include "stdio.h"
#include "alloc.h"
#include "dos.h"
#include "graphics.h"
#include "math.h"

typedef struct
{
    int  bfType;    /* 类型标志,总是 BM */
    long bfSize;    /* 文件大小 */
    int  bfReserved1;
    int  bfReserved2;
    long bfOffBits; /* 位图点阵偏移量 */
} HEAD;

typedef struct
{
    long biSize;    /* 结构体字节总数 */
    long biWidth;   /* 图像宽度 */
    long biHeight;  /* 图像高度 */
    int  biPlanes;  /* 必须为 1 */
    int  biBitCount; /* 每个像素所占二进制位数,可能是 1,4,8 或 24 */
    long biCompress; /* 压缩方式 */
```

```

    long biSizeImage;    /* 像素点阵大小 */
    long biXPelsPerMeter; /* 水平像素数 */
    long biYPelsPerMeter; /* 垂直像素数 */
    long biClrUsed;      /* 使用的颜色数 */
    long biClrImportant; /* 重要颜色数 */
}INFO;

typedef struct
{
    unsigned char  rgbBlue; /* 蓝色所占比重 */
    unsigned char  rgbGreen; /* 绿色所占比重 */
    unsigned char  rgbRed; /* 红色所占比重 */
    unsigned char  rgbReserved; /* 保留字节 */
}RGBQUAD;

int Match_f(int nRed,int nGreen,int nBlue); /* 寻找与像素匹配的调色板的序号 */
Sharp_f(FILE * fpi, FILE * fpo); /* 锐化处理 */
Emboss_f(FILE * fpi, FILE * fpo); /* 浮雕效果处理 */
Smooth_f(FILE * fpi, FILE * fpo); /* 柔化效果处理 */
Initial_f(int nChoice); /* 初始化图像文件 */

RGBQUAD  straPla[256]; /* 256 色调色板 */
HEAD  strHead;
INFO strInfo;
unsigned int nWidth=512,nDepth=512; /* 处理 512×512 位图 */
unsigned char naImage[64][512];

int Match_f(int nRed,int nGreen,int nBlue) /* 寻找与像素匹配的调色板的序号 */
{
    long int nMax,nLength;
    int nNum=0,nCounti;
    unsigned int nImageRed,nImageGreen, nImageBlue;
    nImageRed=(unsigned int)(straPla[0].rgbRed);
    nImageGreen=(unsigned int)(straPla[0].rgbGreen);
    nImageBlue=(unsigned int)(straPla[0].rgbBlue);

    nMax=(nRed-nImageRed)*(nRed-nImageRed)+(nGreen-nImageGreen)*(nGreen-nImageGreen)
    +(nBlue-nImageBlue)*(nBlue-nImageBlue);
    for(nCounti=1;nCounti<256;nCounti++)
    {
        nImageRed=(unsigned int)(straPla[nCounti].rgbRed);
        nImageGreen=(unsigned int)(straPla[nCounti].rgbGreen);
        nImageBlue=(unsigned int)(straPla[nCounti].rgbBlue);
        nLength=(nRed-nImageRed)*(nRed-nImageRed)+(nGreen-nImageGreen)*(nGreen-n

```



```

nImageGreen) + (nBlue - nImageBlue) * (nBlue - nImageBlue);
if (nMax > nLength)
{
    nMax = nLength;
    nNum = nCounti;
    if (nMax == 0)
        break;
}
}
return nNum;
}

```

Sharp_f(FILE * fpi, FILE * fpo) /* 锐化处理 */

```

{
    int nCounti, nCountj, nCountk, nCountl;
    int nRed, nBlue, nGreen;
    int nDx = 1, nDy = 1;
    unsigned int nPixel1, nPixel2, nPixel;
    fread((char *) &strHead, 1, sizeof(strHead), fpi);
    fread((char *) &strInfo, 1, sizeof(strInfo), fpi);
    for (nCounti = 0; nCounti < 256; nCounti++)
        fread((char *) &straPla[nCounti], 1, sizeof(RGBQUAD), fpi);
    fwrite((char *) &strHead, 1, sizeof(strHead), fpo);
    fwrite((char *) &strInfo, 1, sizeof(strInfo), fpo);
    for (nCounti = 0; nCounti < 256; nCounti++)
        fwrite((char *) &straPla[nCounti], 1, sizeof(RGBQUAD), fpo);
    for (nCounti = 0; nCounti < 8; nCounti++)
    {
        for (nCountj = 0; nCountj < 64; nCountj++)
            fread(&naImage[nCountj][0], 1, nWidth, fpi);
        for (nCountk = 1; nCountk < 64 - 2; nCountk++)
            for (nCountl = 1; nCountl < nWidth - 2; nCountl++)
            {
                nPixel1 = (unsigned char) (naImage[nCountk][nCountl]);
                nPixel2 = (unsigned char) (naImage[nCountk - nDx][nCountl - nDy]);
                nRed = straPla[nPixel1].rgbRed + 0.5 * (straPla[nPixel1].rgbRed - straPla[nPixel2].rgbRed);
                /* 0.5 为锐化系数 */
                nGreen = straPla[nPixel1].rgbGreen + 0.5 * (straPla[nPixel1].rgbGreen - straPla[nPixel2].rgbGreen);
                nBlue = straPla[nPixel1].rgbBlue + 0.5 * (straPla[nPixel1].rgbBlue - straPla[nPixel2].rgbBlue);
                if (nRed > 255)
                    nRed = 255;
                if (nRed < 0)

```



```

        nRed=0;
        if(nGreen>255)
            nGreen=255;
        if(nGreen<0)
            nGreen=0;
        if(nBlue>255)
            nBlue=255;
        if(nBlue<0)
            nBlue=0;
        nPixel=Match_f(nRed,nGreen,nBlue);
        naImage[nCountk][nCountl]=nPixel;
    }
    for(nCountj=0;nCountj<64;nCountj++)
        fwrite(&naImage[nCountj][0],1,nWidth,fpo);
}

```

Emboss_f(FILE * fpi, FILE * fpo) /* 浮雕效果处理 */

```

{
    int nCounti,nCountj,nCountk,nCountl;
    int nRed,nBlue,nGreen;
    int nDx=1,nDy=1;
    unsigned int nPixel1,nPixel2,nPixel;
    fread((char *)&strHead,1,sizeof(strHead),fpi);
    fread((char *)&strInfo,1,sizeof(strInfo),fpi);
    for(nCounti=0;nCounti<256;nCounti++)
        fread((char *)&straPla[nCounti],1,sizeof(RGBQUAD),fpi);
    fwrite((char *)&strHead,1,sizeof(strHead),fpo);
    fwrite((char *)&strInfo,1,sizeof(strInfo),fpo);
    for(nCounti=0;nCounti<256;nCounti++)
        fwrite((char *)&straPla[nCounti],1,sizeof(RGBQUAD),fpo);
    for(nCounti=0;nCounti<8;nCounti++)
    {
        for(nCountj=0;nCountj<64;nCountj++)
            fread(&naImage[nCountj][0],1,nWidth,fpi);
        for(nCountk=0;nCountk<64-1;nCountk++)
        {
            for(nCountl=0;nCountl<nWidth-1;nCountl++)
            {
                nPixel1=(unsigned char)(naImage[nCountk][nCountl]);
                nPixel2=(unsigned char)(naImage[nCountk+nDx][nCountl+nDy]);
                nRed=fabs(straPla[nPixel1].rgbRed-straPla[nPixel2].rgbRed+128);
            }
        }
    }
}

```

```

        nGreen=fabs(straPla[nPixel1].rgbGreen-straPla[nPixel2].rgbGreen+128);
        nBlue=fabs(straPla[nPixel1].rgbBlue-straPla[nPixel2].rgbBlue+128);
        nPixel=Match_f(nRed,nGreen,nBlue);
        naImage[nCountk][nCountl]=nPixel;
    }
}
for(nCountl=0;nCountl<nWidth-1;nCountl++)
{
    nPixel1=(unsigned char)(naImage[63][nCountl]);
    nPixel2=(unsigned char)(naImage[63][nCountl+nDy]);
    nRed=fabs(straPla[nPixel1].rgbRed-straPla[nPixel2].rgbRed+128);
    nGreen=fabs(straPla[nPixel1].rgbGreen-straPla[nPixel2].rgbGreen+128);
    nBlue=fabs(straPla[nPixel1].rgbBlue-straPla[nPixel2].rgbBlue+128);
    nPixel=Match_f(nRed,nGreen,nBlue);
    naImage[63][nCountl]=nPixel;
}
for(nCountj=0;nCountj<64;nCountj++)
    fwrite(&naImage[nCountj][0],1,nWidth,fpo);
}
}

```

Smooth_f(FILE * fpi, FILE * fpo) /* 柔化效果处理 */

```

{
    int nCounti,nCountj,nCountk,nCountl,nCountm,nCountn,nRed,nGreen,nBlue;
    unsigned nPixel;
    fread((char *)&strHead,1,sizeof(strHead),fpi);
    fread((char *)&strInfo,1,sizeof(strInfo),fpi);
    for(nCounti=0;nCounti<256;nCounti++)
        fread((char *)&straPla[nCounti],1,sizeof(RGBQUAD),fpi);
    fwrite((char *)&strHead,1,sizeof(strHead),fpo);
    fwrite((char *)&strInfo,1,sizeof(strInfo),fpo);
    for(nCounti=0;nCounti<256;nCounti++)
        fwrite((char *)&straPla[nCounti],1,sizeof(RGBQUAD),fpo);
    for(nCounti=0;nCounti<8;nCounti++)
    {
        for(nCountj=0;nCountj<64;nCountj++)
            fread(&naImage[nCountj][0],1,nWidth,fpi);
        for(nCountk=1;nCountk<64-2;nCountk++)
        {
            for(nCountl=1;nCountl<nWidth-2;nCountl++)
            {
                nRed=0,nGreen=0,nBlue=0;
                for(nCountm=0;nCountm<3;nCountm++) /* 用 9 个点的均值柔化 */

```

```

        for(nCountn=0;nCountn<3;nCountn++)
        {
            nPixel=(unsigned char)(naImage[nCountk-1+nCountm][nCountl-1+nCountn]);
            nRed+=straPla[nPixel].rgbRed;
            nGreen+=straPla[nPixel].rgbGreen;
            nBlue+=straPla[nPixel].rgbBlue;
        }
        nRed=nRed/9;
        nGreen=nGreen/9;
        nBlue=nBlue/9;
        nPixel=Match_f(nRed,nGreen,nBlue);
        naImage[nCountk][nCountl]=nPixel;
    }
}
for(nCountj=0;nCountj<64;nCountj++)
    fwrite(&naImage[nCountj][0],1,nWidth,fpo);
}
}

Initial_f(int nChoice)
{
    char szFilena1[30],szFilena2[30];
    FILE * fpi, * fpo;
    printf("Input the bmp source file: ");
    scanf("%s",szFilena1);
    printf("Input the bmp termini file: ");
    scanf("%s", szFilena2);
    if((fpi=fopen(szFilena1,"rb"))==NULL)
    {
        printf("OPEN SOURCE FILE ERROR\n");
        exit(0);
    }
    if((fpo=fopen(szFilena2,"wb"))==NULL)
    {
        printf("OPEN TERMINI FILE ERROR\n");
        exit(0);
    }
    if(nChoice==3)
        Emboss_f(fpi,fpo);
    else if(nChoice==2)
        Sharp_f(fpi,fpo);
    else if(nChoice==1)
        Smooth_f(fpi,fpo);
    fclose(fpi);
}

```



```
fclose(fpo);
}

void main()
{
    int nChoice;
    do
    {
        printf("\n\t1. Smooth\n");
        printf("\n\t2. Sharp\n");
        printf("\n\t3. Emboss\n");
        printf("\n\t4. Exit\n");
        scanf("%d", &nChoice);
        switch(nChoice)
        {
            case 1:
            case 2:
            case 3:
                Initial_f(nChoice);
                break;
            case 4:
                break;
            default:
                printf("Input again! \n");
                break;
        }
    } while(nChoice != 4);
}
```



第5题 窗口菜单显示

7.5.1 程序功能简介

窗口菜单显示程序,可以显示两级菜单。

7.5.2 课程设计要求

(1) 将 File 菜单改为 Animation,功能是显示小球运动,并设置子菜单,可以选择小球的运动速度。

(2) 将 Edit 菜单改为 Music,功能是播放音乐,并设置子菜单,可以选择播放的曲目(至少两个)。

(3) 主菜单有三项,除上述两个外,增加 Help,显示帮助信息,所有菜单由 Alt+热键打开。

(4) 每项子菜单中都有退出键,可以返回主界面。

7.5.3 课程设计说明

(1) window(x1,y1,x2,y2);水平为 x,垂直为 y,范围 x2-x1, y2-y1。

(2) gettext(x1,y1,x2,y2,buf):将屏幕指定区域内文本存入空间,行数×列数×2,行数=y2-y1+1,列数=x2-x1+1。

(3) 参考 Turbo C 中关于屏幕输出的函数。

(4) 评定难易级别:A 级。

7.5.4 关键源代码注解

```
SoundSing_f();
Blound_f();
int Changedirnx_f(int);
int Changedirny_f(int);

#include<conio.h>
#include<graphics.h>
#include<alloc.h>
#include<dos.h>
SoundSing_f()/* 音乐:两只老虎 */
{
    unsigned naSing[]={262,250,294,250,330,250,262,250,
        262,250,294,250,330,250,262,250,
        330,250,349,250,392,500,
        330,250,349,250,392,500,
        392,125,440,125,392,125,349,125,330,250,262,250,
```

```

392,125,440,125,392,125,349,125,330,250,262,250,
294,250,196,250,262,500,
294,250,196,250,262,500,0,0};
unsigned int * naPoint;
naPoint=naSing;
while( * naPoint)
{
    sound(( * naPoint++));
    delay(200 * ( * naPoint++));
    nosound();
}

}

Blound_f(){

int gd=DETECT,gm;
int nArea,nStartx,nStarty,nMinx=0,nMaxx=630,nMiny=0,nMaxy=470,nCurx=300,nCury=30;
char * chpBuff;
int nChangeX,nChangeY,nIni=0;

initgraph(&gd,&gm,"");
cleardevice();

setcolor(3);
outtextxy(225,185,"BOUNCING BALL");
setcolor(14);
outtextxy(240,220,"Programming : Aisharya");
getch();

cleardevice();
setcolor(7);
rectangle(2.2,630,479);
rectangle(3.3,629,478);
setcolor(13);
circle(7,7,3);setfillstyle(SOLID_FILL,13);floodfill(7,7,13);
nArea=imagesize(4.4,11,11);
chpBuff=malloc(nArea);
getimage(4,4,11,11,chpBuff);
setcolor(0);
circle(7,7,3);setfillstyle(SOLID_FILL,0);floodfill(7,7,0);

while(! kbhit()){
delay(200);if(nIni!=0)putimage(nStartx,nStarty,chpBuff,XOR_PUT);

```

```

nStartx=nCurx;nStarty=nCury; nIni=1;
putimage(nCurx,nCury, chpBuff, XOR_PUT);
if(nCurx>nMaxx || nCurx<nMinx)nChangeX=Changedirnx_f(nCurx);
if(nCury>nMaxy || nCury<nMiny)nChangeY=Changedirny_f(nCury);
if(nChangeX==1)nCurx++;else nCurx--;
if(nChangeY==1)nCury++;else nCury--;
restorecrtmode();closegraph();
printf("Thank You. \n");
getch();}

```

```

Changedirnx_f(int nCurx){
int nChangeX;
if(nCurx>630)nChangeX=0;
if(nCurx<0)nChangeX=1;
sound(100);delay(80);nosound();
return(nChangeX);}

```

```

Changedirny_f(int nCury){
int nChangeY;
if(nCury>460)nChangeY=0;
if(nCury<0)nChangeY=1;
sound(100);delay(80);nosound();
return(nChangeY);}

```

```

main()
{
int nCount, nKey0,nKey,nKey1,nValue,nTest;
char * szMainMenu[]={ "File", "Edit", "Run", "Compile",
"Project", "Options", "Debug", "Break/watch" };
char * szSubMenu[]={ "MOVEBALL",
" SOUND ",
" New ",
" Save ",
" Write to ",
" Directory ",
" Change dir ",
" Os shell ",
" Quit Alt-X "
};

```

```

char naBuf[19*14*2], naBufOne[19*2];/* 缓冲区,保存窗口大小;行×列×2 */
window(1,1,80,25);
textbackground(BLUE);
clrscr();
window(1,1,80,1);

```



```
textbackground(WHITE);
textcolor(BLACK);
clrscr();
window(1,1,80,1);
for(nCount=0;nCount<8;nCount++)/* 显示主菜单 */
    cprintf("  %s",szMainMenu[nCount]);
while(1)
{
    nKey=0;
    while(bioskey(1)==0);
    nKey=bioskey(0);
    nKey=nKey&~0xff? 0: nKey>>8;
    if(nKey==45) exit(0);
    if(nKey==33) /* Alt+F */
    {
        textbackground(BLACK);
        textcolor(WHITE);
        gotoxy(3,1); /* 反白显示主菜单选项 */
        cprintf("%s",szMainMenu[0]);
        gettext(3,1,18,14,naBuf);/* 第二行第三列开始,存 11 行 16 列 */
        window(3,2,18,12);/* 开辟窗口 */
        textbackground(WHITE);
        textcolor(BLACK);
        clrscr();
        window(3,2,18,13); /* 多开辟一行,最后字符有回车符 */
        gotoxy(1,1); /* 画方框 */
        putchar(0xda);
        for(nCount=2;nCount<16;nCount++)
            putchar(0xc4);
        putchar(0xbf);
        for(nCount=2;nCount<11;nCount++)
        {
            gotoxy(1,nCount);
            putchar(0xb3);
            gotoxy(16,nCount);
            putchar(0xb3);
        }
        gotoxy(1,11);
        putchar(0xc0);
        for(nCount=2;nCount<16;nCount++)
            putchar(0xc4);
        putchar(0xd9);
        for(nCount=1;nCount<10;nCount++)/* 显示子菜单 */
```




```

{
    gotoxy(2,nCount+1);
    cprintf("%s",szSubMenu[nCount-1]);
}

gettext(2,2,18,3,naBufOne); /* 存一行 */
textbackground(BLACK);
textcolor(WHITE);
gotoxy(2,2);
cprintf("%s",szSubMenu[0]); /* 反白显示子菜单 */
nValue=2;
nKey0=0;
while(nKey0!=27&& nKey0!=45&& nKey0!=13) /* 输入为 Alt+X、回车或 ESC 时退出 */
{
    while(bioskey(1)==0);
    nKey0=nKey1=bioskey(0);
    nKey0=nKey0&0xff;
    nKey1=nKey1&0xff? 0:nKey1>>8;
    if(nKey1==72|nKey1==80) /* 为上下箭头键 */
    {
        puttext(2,nValue,18,nValue+1,naBufOne);
        if(nKey1==72)nValue=(nValue==2)? 10:nValue-1;
        if(nKey1==80)nValue=(nValue==10)? 2:nValue+1;
        gettext(2,nValue,18,nValue+1,naBufOne);
        textbackground(BLACK);
        textcolor(WHITE);
        gotoxy(2,nValue);
        cprintf("%s",szSubMenu[nValue-2]);
    }
    nTest=nValue-1;
}
if(nKey1==45) exit(0);
if(nKey0==13)
{
    switch(nValue-1)
    {
        case 1:
            Blound_f(); /* 显示运动小球 */
            main();
        case 2:
            SoundSing_f(); /* 音乐 */
            main();
        case 9:
            exit(0);
    }
}

```

★★★★★

default:

break;

}

}

else

{

window(1,1,80,1);

puttext(3,1,18,14,naBuf);

textbackground(WHITE);

textcolor(BLACK);

gotoxy(3,1);

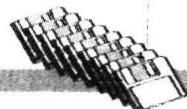
cprintf("%s",szMainMenu[0]);

}

}

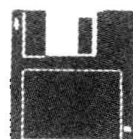
}

}



8

数据结构应用程序



第 1 题 矩阵链表堆栈

8.1.1 程序功能简介

一个测试矩阵、链表、堆栈类功能的程序。

8.1.2 课程设计要求

- (1) 设计出插入、删除数据的测试程序。
- (2) 设计出压栈、弹出数据的测试程序。
- (3) 设计出测试结果的输出程序。

8.1.3 课程设计说明

- (1) 链表查找要从头到尾进行。
- (2) 栈是先存进去的后取出来。
- (3) 评定难易级别:B 级。

8.1.4 关键源代码注解

```
// structs.h Begin
#ifndef _STRUCTS_H
#define _STRUCTS_H
#include<iostream.h>
#include<stdlib.h>
typedef enum Kind {Array, Pointers} Kind;
typedef int Data;
class UList    //基类
{
public:
    virtual int Size()=0;
    virtual bool Insert(const Data&,int)=0;
    virtual bool Remove(int)=0;
    virtual int Find(const Data&)=0;
    virtual bool Get(int, Data&)=0;
    virtual void Print()const=0;
};

class PList:public UList    //链表操作类
{
    class Node{
        Data item;
        Node * next;
```



```

public:
    Node(const Data &dat); item(dat), next(NULL){}
    Node(const Node &nod); item(nod.item), next(NULL){}
    friend class PList;
};

Node * begin;
Node * end;
int num;
public:
    PList(): begin(NULL), end(NULL), num(0){}
    ~PList(){
        Node * tmp = begin;
        while(num > 0){
            begin = begin->next;
            delete tmp;
            tmp = begin;
        }
    }
    int Size(){ return num; }
    bool Insert(const Data&, int);
    bool Remove(int);
    int Find(const Data&);
    bool Get(int, Data&);
    void Print() const;
};

class AList: public UList    //矩阵操作类
{
    Data * arr;              // holds the dynamic Array
    int num;                  // cells filled so far
public:
    static const int MAX_SIZE;    // max_size beginning value
    AList(): arr(new Data[MAX_SIZE]), num(0){}
    ~AList(){ delete[] arr; }
    int Size(){ return num; }
    bool Insert(const Data&, int);
    bool Remove(int);
    int Find(const Data&);
    bool Get(int, Data&);
    void Print() const;
};

UList * init_list(Kind);

class UStack    //堆栈操作类

```



```

{
    UList * stk;
public:
    UStack(Kind k) : stk(init_list(k)){};
    bool Push(const Data&);
    bool Top(Data&) const;
    bool Pop(Data&);
    void Print()const{stk->Print();return;};
};
#endif
// structs.h end
//structs.cpp Begin
#include"structs.h"
const int AList::MAX_SIZE = 4;
UList * init_list(Kind k) //初始化函数
{
    UList * tmp;
    switch(k){
        case Array:
            tmp=new AList;
            break;
        case Pointers:
            tmp=new PList;
            break;
        default:
            cout<<endl<<"unknown option"<<endl;
            return NULL;
    }
    return tmp;
}

bool PList::Insert(const Data &x,int place) //链表插入操作
{
    Node * tmp;
    if ((place<=0)|| (place>num+1))
        return false;
    if (place==1) // insert at the begining of the list
    {
        tmp=begin;
        begin=new Node(x);
        begin->next=tmp;
        if (num==0) // in case is the only item in the list
        {
            end=begin;

```

★★★★★

```

        end->next=NULL;
    }
}

else if (place==num+1)           // insert at the end of list
{
    tmp=end;
    end=new Node(x);
    end->next=NULL;
    tmp->next=end;
}
else
{
    Node * holder;
    tmp=begin;
    for(int i=1;i<place-1;++i)    // forward till list(place-1)
        tmp=tmp->next;
    holder=tmp->next;
    tmp->next=new Node(x);
    tmp->next->next=holder;
}
++num;
return true;
}

bool PList::Remove(int place)    //链表移出操作
{
    Node * tmp=begin;
    if((place<=0)|| (place>num+1))
        return false;
    if(place==1)                 // remove the last item
    {
        begin=begin->next;
        delete tmp;
    }
    else if(place==num)          // remove the last item
    {
        for(int i=1;i<place-1;++i)
            tmp=tmp->next;
        delete end;
        end=tmp;
        end->next=NULL;
    }
    else
    {

```



```

        Node * holder;
        for(int i=1;i<place-1;++i)
            tmp=tmp->next;
        holder=tmp->next;
        tmp->next=holder->next;
        delete holder;
    }
    if(num==1)
        end=begin=NULL;
    --num;
    return true;
}

int PList::Find(const Data &x)    //链表节点查找
{
    Node * tmp=begin;
    int i=1;
    while((tmp!=NULL)&&(tmp->item!=x))
    {
        ++i;
        tmp=tmp->next;
    }
    if (tmp==NULL)
        return 0;
    return i;
}

bool PList::Get(int place, Data &ret)    //得到节点数据
{
    Node * tmp=begin;
    if((place<=0)|| (place>num+1))
        return false;
    for(int i=1;i<place;++i)
        tmp = tmp->next;
    ret = tmp->item;
    return true;
}

void PList::Print(void) const{    //输出链表
    if (num==0) return;
    cout<<"head";
    for(Node * tmp=begin;tmp!=NULL;tmp=tmp->next){
        cout<<"->"<<tmp->item;
    }
    cout<<endl;
    return;
}

```



```

}
bool AList::Insert(const Data &x,int place=1)    //矩阵插入一元素
{
    if (num==MAX_SIZE)        // if the array is full
        return false;
    if((place<=0)|| (place>num+1))
        return false;
    for(int i=num;i>=place-1;--i)
        arr[i+1]=arr[i];
    arr[place-1]=x;
    ++num;
    return true;
}
bool AList::Remove(int place)    //矩阵移出一元素
{
    if((place<=0)|| (place>num+1))
        return false;
    for(int i=place-1;i<num;++i)
        arr[i]=arr[i+1];
    --num;
    return true;
}
int AList::Find(const Data &x)    //矩阵查找一元素
{
    int i=0;
    while((i<num)&&(arr[i]!=x))    ++i;
    if (i==num) i=-1;                // we got to the end of the array
    return i+1;
}
bool AList::Get(int place, Data &ret)    //得到元素值
{
    if((place<=0)|| (place>num+1))
        return false;
    ret = arr[place-1];
    return true;
}
void AList::Print()const    //输出矩阵
{
    cout<<"head";
    for(int i=0;i<num;++i)    cout<<"-"<<arr[i];
    cout<<endl;
    return;
}

```

```

bool UStack::Push(const Data &dat) //压栈操作
{
    return stk->Insert(dat,stk->Size()+1);
}

bool UStack::Top(Data &ret)const //栈顶操作
{
    int n=stk->Size();
    if (n<=0)    return false;
    (void)stk->Get(n, ret);
    return true;
}

bool UStack::Pop(Data &ret) //出栈操作
{
    int n=stk->Size();
    if (n<=0)    return false;
    (void)stk->Get(n, ret);
    (void)stk->Remove(n);
    return true;
}

//structs. cpp End
//main. cpp Begin—主函数开始
// all the test functions return false if one of the tests fail or true if all the tests succeeded
// the function reports failures to the standard error output device
#include "structs. h"

bool Test_insert (Kind k)
{
    bool res = true;
    UList * tmp=init_list(k);
    if (! tmp->Insert(5,1))
    {
        cerr << "Failure on Insert test no. 1 — insert into a valid cell number" << endl;
        res = false;
    }
    else
        cout << "Success on Insert test no. 1 — insert into a valid cell number" << endl;
    if (tmp->Insert(3,tmp->Size()+2))
    {
        cerr << "Failure on Insert test no. 2 — insert into upper cell number" << endl;
        res = false;
    }
    else
        cout << "Success on Insert test no. 2 — insert into upper cell number" << endl;
    if (tmp->Insert(3,0))

```

```

{
    cerr << "Failure on Insert test no. 3 - insert into lower cell number" << endl;
    res = false;
}
else
    cout << "Success on Insert test no. 3 - insert into lower cell number" << endl;
if (k == Array)
{
    bool ares = true;
    for(int i=0; i<AList::MAX_SIZE+1; ++i)
    {
        ares = tmp->Insert(i, 1);
    }
    if (ares)
    {
        cerr << "Failure on Insert test no. 4 - Array is full" << endl;
        res = false;
    }
    else
        cout << "Success on Insert test no. 4 - Array is full" << endl;
}
return res;
}

bool Test_remove (Kind k)
{
    bool res = true;
    UList * tmp = init_list(k);
    for (int i=0; i<10; ++i)    // filling the list
    {
        (void)tmp->Insert(i, 1);
    }
    if (! tmp->Remove(5))
    {
        cerr << "Failure on Remove test no. 1 - remove from valid cell number" << endl;
        res = false;
    }
    else
        cout << "Success on Remove test no. 1 - remove from valid cell number" << endl;
    if (tmp->Remove(12))
    {
        cerr << "Failure on Remove test no. 2 - remove from upper cell number" << endl;
        res = false;
    }
}

```

```

else
    cout << "Success on Remove test no. 2 — remove from upper cell number" << endl;
if (tmp->Remove(0))
{
    cerr << "Failure on Remove test no. 3 — remove from lower cell number" << endl;
    res = false;
}
else
    cout << "Success on Remove test no. 3 — remove from lower cell number" << endl;
return res;
}

bool Test_push (Kind k)
{
    bool res = true;
    UStack * tmp = new UStack(k);
    if (! tmp->Push(7))
    {
        cerr << "Failure on Push test no. 1 — Normal push" << endl;
        res = false;
    }
    else
        cout << "Success on Push test no. 1 — Normal push" << endl;
    if (k == Array)
    {
        bool ares = true;
        for(int i=0; i<AList::MAX_SIZE+1; ++i)
        {
            ares = tmp->Push(i);
        }
        if (ares)
        {
            cerr << "Failure on Push test no. 2 — Array is full" << endl;
            res = false;
        }
        else
            cout << "Success on Push test no. 2 — Array is full" << endl;
    }
    return res;
}

bool Test_pop (Kind k)
{
    bool res = true;
    UStack * tmp = new UStack(k);

```



```

Data ret;    // for testing purposes
(void)tmp->Push(2);
if (! tmp->Pop(ret))
{
    cerr << "Failure on Pop test no. 1 - Normal pop" << endl;
    res = false;
}
else
    cout << "Success on Pop test no. 1 - Normal pop" << endl;
if (tmp->Pop(ret))
{
    cerr << "Failure on Pop test no. 2 - Pop from empty stack" << endl;
    res = false;
}
else
    cout << "Success on Pop test no. 2 - Pop from empty stack" << endl;
return res;
}

int main()
{
    while (1)
    {
        int ch;
        Kind k;
        cout<<"choose Kind"<<endl<<"1. Array"<<endl<<"2. Pointers"
            <<endl<<"any other no. to exit"<<endl;
        cin>>ch;
        switch(ch){
        case 1:
            k=Array;
            break;
        case 2:
            k=Pointers;
            break;
        default:
            cout<<"bye bye"<<endl;
            exit(0);
        }

        cout<<"choose test"<<endl<<"1. insert (LIST)"<<endl<<"2. remove (LIST)"<<endl
            <<"3. push (STACK)"<<endl<<"4. pop (STACK)"<<endl<<"any other no. to exit"<<endl;
        cin>>ch;
        switch(ch){
        case 1:

```

```
        if(Test_insert(k))
            cout<<endl<<"insert test seccessful"<<endl;
        break;
    case 2:
        if(Test_remove(k))
            cout<<endl<<"remove test seccessful"<<endl;
        break;
    case 3:
        if(Test_push(k))
            cout<<endl<<"push test seccessful"<<endl;
        break;
    case 4:
        if(Test_pop(k))
            cout<<endl<<"pop test seccessful"<<endl;
        break;
    default:
        cout<<"goodbye"<<endl;
        exit(0);
    }
    cout << endl << endl;
}

return 0;
}

//main. cpp end
```



第 2 题 双向链表

8.2.1 程序功能简介

双向链表。

8.2.2 课程设计要求

(1) 扩充双向链表的功能,增加排序,根据顺序增加节点,根据顺序查找节点并删除的功能。

(2) 将节点改成通讯录中的一个记录,添加年龄、姓名、电话等信息,并能根据不同的信息进行快速向前或向后查找,链表根据姓名排序。

(3) 根据名字查找记录时支持通配符 * 和?,即 * 通配任意字符和字符串,? 通配一个字符,字符不分大小写。

(4) 可以修改某个节点,即修改其中的年龄和电话字段,修改时,新旧记录要同时显示在屏幕上,经“确认”后新记录才可替代旧记录,否则取消修改操作。

(5) 将通讯录以文件的形式存在磁盘上,每次操作时将通讯录调出,操作完毕后存盘。

8.2.3 课程设计说明

(1) 可设计数字键选择相应的操作。

(2) 可将记录存入 record. dat,再从 record. dat 中重新加载。

(3) 评定难易级别:A 级。

8.2.4 关键源代码注解

```
// Author : Vinayak Marwah
```

```
#ifndef DoubleLinkedList_H
```

```
#define DoubleLinkedList_H
```

```
template< class NodeType > class Node;      /* * Generic Class To Define A Node . * */
```

```
template< class NodeType > class DoubleLinkedList;
```

```
/* * Generic Class To Define A Double Link List . * */
```

```
template< class NodeType >
```

```
class Node
```

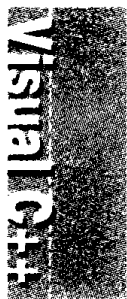
```
{
```

```
    friend class DoubleLinkedList< NodeType >;
```

```
/* * Class DoubleLinkedList Made A Friend To Access Private Data Members Of Class Node . * */
```

```
private :
```

```
    NodeType Data;      /* Data Item Of A Node , Of Return Type NodeType . * */
```




```

Node< NodeType > * NextNode;    /* Pointer Towards The Succeeding Node . */
Node< NodeType > * PreviousNode; /* Pointer Towards The Preceding Node . */
public :
    Node();                    /* Default Constructor , Initialize A Node . */
    Node( NodeType &Value );
        /* Constructor Accepting One Argument To Assign Data To The Node . */
    ~Node();                   /* Destructor , To Destroy A Node . */
};
/*****
template< class NodeType >
class DoubleLinkedList
{
private :
    Node< NodeType > * FirstNode;    /* Pointer Towards The First Node Of The List . */
    Node< NodeType > * RearNode;    /* Pointer Towards The Last Node Of The List */
public :
    DoubleLinkedList();    /* Default Constructor , Initiates A Double Linked List . */
    ~DoubleLinkedList(); /* Destructor , Destroy The Double Linked List . */
    bool IsEmpty();    /* Function To Assert Wether The List Is Emty Or Not . */
    void InsertAtFront ( NodeType &Value );
        /* Function To Insert A Node , At The Begining Of The List . */
    void InsertAtRear ( NodeType &Value );
        /* Function To Insert A Node , At The End Of The List . */
    void InsertAtMiddle( NodeType &Value );
        /* Function To Insert A Node , In The Middle Of The List . */
    bool RemoveFromFront();
        /* Function To Remove A Node , From The Beginning Of The List . */
    bool RemoveFromRear();
        /* Function To Remove A Node , From The End Of The List . */
    bool RemoveFromMiddle();
        /* Function To Remove A Node , From The Middle Of The List . */
    void TraverseForward();
        /* Function To Traverse & Print , A Linked List From Top To Bottom . */
    void TraverseBackwards ();
        /* Function To Traverse & Print , A Linked List From Bottom To Top . */
    int LenghtOfDoubleLinkedList ();
        /* Function To Traverse & Calculate , The Number Of Nodes In A List . */
    Node< NodeType > * CreateNode( NodeType &Value );
        /* Function To Create & Assign Data To A New Node . */
};
*****/
#endif

```



```

#include <iostream.h>
#include <conio.h>
#include "Double Link List.Cpp"

void main()
{
    DoubleLinkedList< int > List;
    int Value = 0 ,Option = 0;
    do
    {
        cout<<"\n\t\t Driver To Test  Link List "
            <<"\n\t\t\t Menu . "
            <<"\n\t\t\t\t _____ "
            <<"\n 1.) Insert At Front Of A Double Linked List . "
            <<"\n 2.) Insert At Rear Of A Double Linked List . "
            <<"\n 3.) Insert In Middle Of A Double Linked List . "
            <<"\n 4.) Remove From Front Of A Double Linked List. "
            <<"\n 5.) Remove From Rear Of A Double Linked List. "
            <<"\n 6.) Remove From Middle Of The Double Linked List . "
            <<"\n 7.) Traverse Forward A Double Linked List . "
            <<"\n 8.) Traverse Backwards A Double Linked List . "
            <<"\n 9.) Lenght Of The Double Linked List . "
            <<"\n 10.) End Double Linked List Processing & Exit . "
            <<"\n      Enter Your Choice : \t";
        cin>>Option;
        switch(Option)
        {
            case 1 :
            {
                cout<<"\n Enter Data : ";
                cin>>Value;
                List.InsertAtFront(Value);
                break;
            }
            case 2 :
            {
                cout<<"\n Enter Data : ";
                cin>>Value;
                List.InsertAtRear(Value);
                break;
            }
            case 3:
            {

```

```
        cout<<"\n Enter Data : ";
        cin>>Value;
        List.InsertAtMiddle(Value);
        break;
    }
    case 4 :
    {
        List.RemoveFromFront();
        break;
    }
    case 5 :
    {
        List.RemoveFromRear();
        break;
    }
    case 6:
    {
        List.RemoveFromMiddle();
        break;
    }

    case 7 :
    {
        List.TraverseForward();
        break;
    }
    case 8 :
    {
        List.TraverseBackwards();
        break;
    }
    case 9 :
    {
        cout<<"\n The Lenght Of A Double Linked List "
            <<"\n (ie Number Of Nodes In A Double Linked List ) : "
            <<List.LenghtOfDoubleLinkList();
        break;
    }
    case 10 :
    {
        break;
    }
    default :
```

```

        }
        while( Option != 10 );
    }

#include <iostream. h>
#include <conio. h>
#include <assert. h>
#include "Double Link List. H"
template< class NodeType >
Node< NodeType >::Node()
:Data( NULL ),NextNode( NULL ),PreviousNode( NULL ) {}
/* * * * * * */
template< class NodeType >
Node< NodeType >::Node( NodeType &Value )
:Data( Value ),NextNode( NULL ),PreviousNode( NULL )
{
    cout<<"\n A Node Created Successfully . ";
}
/* * * * * * */
template< class NodeType >
Node< NodeType >::~~Node()
{
    cout<<"\n A Node Destroyed Successfully . ";
}
/* * * * * * */
template< class NodeType >
DoubleLinkedList< NodeType >::DoubleLinkedList()
:FirstNode( NULL ),RearNode( NULL )
{
    cout<<"\n A Double Linked List Created Successfully . ";
}
/* * * * * * */
template< class NodeType >
DoubleLinkedList< NodeType >::~~DoubleLinkedList()
{
    cout<<"\n Mission : Destroy A Double Linked List . \n";
    Node< NodeType > * CurrentNode = FirstNode, * TempNode ;

```

```

while( CurrentNode != NULL )
{
    TempNode = CurrentNode;
    CurrentNode = CurrentNode->NextNode;
    cout<<TempNode->Data<<" , ";
    delete TempNode;
}

cout<<"\n Status : Mission Successfull . "
    <<"\n A Double Linked List Destroyed Successfully . "
    <<"\n Press Any Key To Continue ";
getch();
}

/*****
template< class NodeType >
bool DoubleLinkedList< NodeType >::IsEmpty()
{
    if( FirstNode == NULL )
    {
        cout<<"\n No Double Linked List Exists . ";
        return true;
    }
    else
        return false;
}

/*****
template< class NodeType >
Node< NodeType > * DoubleLinkedList< NodeType >::CreateNode( NodeType &Value )
{
    Node< NodeType > * NewNode = new Node< NodeType >( Value );
    assert( NewNode != NULL );
    return NewNode ;
}

/*****
template< class NodeType >
void DoubleLinkedList< NodeType >::InsertAtFront( NodeType &Value )
{
    cout<<"\n Mission : Insertion Of A Node At The Front Of The Linked List .\n";
    Node< NodeType > * NewNode = CreateNode(Value);
    if ( IsEmpty() )
    {
        cout<<"\n Mission : Generating A New Double Link List . ";
        FirstNode = RearNode = NewNode ;
    }
}

```

```

else
{
    FirstNode->PreviousNode = NewNode;
    NewNode->NextNode = FirstNode;
    FirstNode = NewNode;
    FirstNode->PreviousNode = NULL;
}

cout<<FirstNode->Data;
cout<<"\n Status : Mission SuccessFull . "
    <<"\n Node Inserted At Front Successfully . "
    <<"\n Press Any Key To Continue . ";
getch();
}

/*****
template< class NodeType >
void DoubleLinkedList< NodeType >::InsertAtMiddle( NodeType &Value )
{
    cout<<"\n Operation : Insertion Of A Node In The Middle Of The Linked List . \n";
    Node< NodeType > * NewNode = CreateNode(Value);
    if ( IsEmpty() )
    {
        cout<<"\n Mission : Generating A New Double Link List . ";
        FirstNode = RearNode = NewNode ;
    }
    else
    {
        Node< NodeType > * CurrentNode = FirstNode, * TempNode;
        int Middle = (LenghtOfDoubleLinkedList()/2), Position = 0;
        while( Position != Middle )
        {
            CurrentNode = CurrentNode->NextNode;
            Position++;
        }
        TempNode = CurrentNode;
        NewNode->PreviousNode = CurrentNode->PreviousNode;
        NewNode->NextNode = CurrentNode;
        CurrentNode = CurrentNode->PreviousNode;
        CurrentNode->NextNode = NewNode ;
        TempNode->PreviousNode = NewNode;
    }
    cout<<NewNode->Data;
    cout<<"\n Status : Mission SuccessFull . "
        <<"\n Node Inserted In Middle Successfully . "

```

```

        <<"\n Press Any Key To Continue . ";
        getch();
    }

    / * * * * * /
    template< class NodeType >
    void DoubleLinkedList< NodeType >::InsertAtRear( NodeType &Value )
    {
        cout<<"\n Operation : Insertion Of A Node At The End Of The Linked List . \n";
        Node< NodeType > * NewNode = CreateNode(Value);
        if ( IsEmpty() )
        {
            cout<<"\n Mission : Generating A New Double Link List . ";
            FirstNode = RearNode = NewNode ;
        }
        else
        {
            NewNode->PreviousNode = RearNode;
            RearNode->NextNode = NewNode;
            RearNode = NewNode;
            RearNode->NextNode = NULL;
        }
        cout<<RearNode->Data;
        cout<<"\n Status : Mission SuccessFull . "
            <<"\n Node Inserted At Rear Successfully . "
            <<"\n Press Any Key To Continue . ";
        getch();
    }

    / * * * * * /
    template< class NodeType >
    bool DoubleLinkedList< NodeType >::RemoveFromFront()
    {
        cout<<"\n Operation : Removal Of A Node From Front Of A Double Linked List . \n";
        if( IsEmpty() )
        {
            cout<<"\n Invalid Operation . "
                <<"\n Status : Mission Failed . "
                <<"\n Press Any Key To Continue . ";
            getch();
            return false;
        }
        else
        {
            Node< NodeType > * CurrentNode = FirstNode;

```

```

    if( FirstNode == RearNode )
        FirstNode = RearNode = NULL;
    else
    {
        cout<<"\n Data In The First Node Is : ";
        cout<<CurrentNode->Data;
        FirstNode = FirstNode->NextNode;
        FirstNode->PreviousNode = NULL ;
    }
    delete CurrentNode;
    cout<<"\n Status : Mission SuccessFull . "
        <<"\n Node Removed From Front Of The Doubled Linked List Successfully ";
    return true;
}
cout<<"\n Press Any Key To Continue . ";
getch();
}
/ * * * * *
template< class NodeType >
bool DoubleLinkedList< NodeType >::RemoveFromMiddle()
{
    cout<<"\n Operation : Insertion Of A Node In The Middle Of The Linked List . \n";
    if( IsEmpty() )
    {
        cout<<"\n Invalid Operation . "
            <<"\n Status : Mission Failed . "
            <<"\n Press Any Key To Continue . ";
        getch();

        return false;
    }
    else
    {
        Node< NodeType > * CurrentNode = FirstNode, * TempNode, * TEmpNextNode;
        int Middle = (LenghtOfDoubleLinkedList()/2), Position = 0;
        while( Position != Middle )
        {
            CurrentNode = CurrentNode->NextNode;
            Position++;
        }
        TempNode = CurrentNode ;
        CurrentNode = CurrentNode->NextNode ;
        CurrentNode->PreviousNode = TempNode->PreviousNode ;
    }
}

```



```

TempNode = TempNode->PreviousNode ;
TempNode->NextNode = CurrentNode ;
cout<<TempNode->Data;
delete TempNode;
}

cout<<"\n Status : Mission SuccessFull ."
<<"\n Node Removed From Middle Successfully . "
<<"\n Press Any Key To Continue . ";
getch();
return true;
}

/*****
template< class NodeType >
bool DoubleLinkedList< NodeType >::RemoveFromRear()
{
    cout<<"\n Operation : Removal Of A Node From Rear Of A Double Linked List .\n";
    if( IsEmpty() )
    {
        cout<<"\n Invalid Operation . "
        <<"\n Status : Mission Failed . "
        <<"\n Press Any Key To Continue . ";
        getch();

        return false;
    }
    else
    {
        Node< NodeType > * TempNode = RearNode;
        if( FirstNode == RearNode )
            FirstNode = RearNode = NULL;
        else
            RearNode = RearNode->PreviousNode;
        RearNode->NextNode = NULL;
        cout<<TempNode->Data;
        delete TempNode;
        cout<<"\n Status : Mission SuccessFull ."
        <<"\n Node Removed From Rear Of The Double Linked List Successfully . ";
        return true;
    }
    cout<<"\n Press Any Key To Continue . ";
    getch();
}

*****/

```

```

template< class NodeType >
void DoubleLinkedList< NodeType >:: TraverseForward()
{
    Node< NodeType > * CurrentNode = FirstNode ;
    cout<<"\n Mission : Traversing & Printing A Double Linked List . "
        <<"\n          In Forward Direction . "
        <<"\n The Double Linked List Follows : \n";
    while ( CurrentNode != NULL)
    {
        cout<<CurrentNode->Data<<" , ";
        CurrentNode = CurrentNode->NextNode;
    }
    cout<<"\n Status : Operation Successfull . "
        <<"\n Traversed & Printed Double Linked List Successfully . "
        <<"\n Press Any Key To Continue . ";
    getch();
}

/ * * * * *

template< class NodeType >
void DoubleLinkedList< NodeType >:: TraverseBackwards()
{
    Node< NodeType > * CurrentNode = RearNode ;
    cout<<"\n Mission : Traversing & Printing A Double Linked List . "
        <<"\n          In Backward Direction "
        <<"\n The Double Linked List Follows : \n";
    while ( CurrentNode != NULL)
    {
        cout<<CurrentNode->Data<<" , ";
        CurrentNode = CurrentNode->PreviousNode;
    }
    cout<<"\n Status : Operation Successfull . "
        <<"\n Traversed & Printed Double Linked List Successfully . "
        <<"\n Press Any Key To Continue . ";
    getch();
}

/ * * * * *

template< class NodeType >
int DoubleLinkedList< NodeType >:: LenghtOfDoubleLinkedList()
{
    int NosOfNodes = 0 ;
    Node< NodeType > * CurrentNode = FirstNode ;
    cout<<"\n Mission : Traverse & Calculate The Number Of Nodes In A Double Linked List . "<<"\n The
    Double Linked List Follows : \n";
}

```



```
while ( CurrentNode != NULL)
{
    NosOfNodes++;
    CurrentNode = CurrentNode->NextNode;
}

cout<<"\n Status : Operation Successfull . "
<<"\n Traversed & Calculated Number Of Nodes In A Double Linked List Successfully . " <<"\n
Press Any Key To Continue . ";
getch();
return NosOfNodes;
}
```



第3题 二叉树后序遍历

8.3.1 程序功能简介

二叉树的创建、后序遍历、计算并输出。程序根据一表达式建立对应二叉树，并计算表达式的值。

8.3.2 课程设计要求

- (1) 对二叉树上的节点数据由大到小排序。
- (2) 对二叉树上的节点数据由小到大排序。
- (3) 用堆栈实现计算二叉树节点的个数。
- (4) 用递归实现打印各节点的中间计算结果。

8.3.3 课程设计说明

(1) 程序在处理表达式的时候用了一个符号栈和一个数据栈，在每次符号进栈时和栈顶的符号比较优先级，优先级高则进栈，否则栈顶符号出栈运算，依此类推，直至处理完所有的输入。

(2) 评定难易等级：A 级。

8.3.4 关键源代码注解

```
// FileName: tree.h By: Musawir Ali Shah
```

```
#ifndef _TREE_H_
```

```
#define _TREE_H_
```

```
#include <iostream>
```

```
#include <string>
```

```
#include <math.h>
```

```
#include <sstream>
```

```
using namespace std;
```

```
// Overloaded function, checks a string for an operator
```

```
bool IsOperator(string mystring)
```

```
{
```

```
    if(mystring == "-" || mystring == "+" || mystring == "/" || mystring == "*" || mystring == "~")
```

```
        return(true);
```

```
    else
```

```
        return(false);
```

```
}
```

```
// Overloaded function, checks a character for an operator
```

```
bool IsOperator(char ops)
```

```
{
    if(ops == '+' || ops == '-' || ops == '*' || ops == '/' || ops == '~ || ops == '(' || ops ==
        ')')
        return(true);
    else
        return(false);
}
```

```
//Binary Tree Definition and Implementation
```

```
class node_type
```

```
{
public:
    string data;
    node_type * left_child;
    node_type * right_child;
    node_type(string k)
    {
        data=k;
        left_child = NULL;
        right_child = NULL;
    }
};
```

```
class binary_tree
```

```
{
public:
    node_type * root;
    void print(node_type * r); // Postfix traversal
    binary_tree(void) { root = NULL;}
    void print(void) {print (root);}
    void evaluate()
    {
        evaluate(root);
    }
    void evaluate(node_type * prt) //recursive tree evaluation
    {
        if(IsOperator(prt->data) && ! IsOperator(prt->left_child->data) && ! IsOperator(prt->
            >right_child->data))
        {
            int num = 0;
```

```

    int num1=atoi(prt->left_child->data.c_str());
    int num2=atoi(prt->right_child->data.c_str());
    if(prt->data == "+")
        num = num1 + num2;
    else if(prt->data == "-")
        num = num1 - num2;
    else if(prt->data == "*")
        num = num1 * num2;
    else if(prt->data == "/")
        num = num1 / num2;
    else if(prt->data == "^")
        num = pow(num1,num2);
    stringstream bob;
    bob << num;
    string suzzy(bob.str());
    prt->data = suzzy;
    prt->left_child = NULL;
    prt->right_child = NULL;
}

else if(prt->left_child == NULL && prt->right_child == NULL);
else
{
    evaluate(prt->left_child);
    evaluate(prt->right_child);
    evaluate(prt);
}
}

void clear_help(node_type * rt)    // destructor
{
    if( rt != NULL )
    {
        clear_help( rt->left_child);
        clear_help( rt->right_child);
        delete rt;
    }
}

void clear()
{
    clear_help(root);
}

```

};

node_type * build_node(string x) //build a new node for the tree

{

node_type * new_node;

new_node = new node_type(x);

return(new_node);

}

void binary_tree::print(node_type * p) //postfix traversal

{

if(p != NULL)

{

print(p->left_child);

print(p->right_child);

cout << p->data << " ";

}

}

bool IsOperand(char ch) //Checks for character to be an operand

{

if ((ch >= '0') && (ch <= '9'))

return true;

else

return false;

}

//Checks Precedence, returns true if A is higher precedence over B

bool TakesPrecedence(char OperatorA, char OperatorB)

{

if (OperatorA == '(')

return false;

else if (OperatorB == '(')

return false;

else if (OperatorB == ')')

return true;

else if ((OperatorA == '~') && (OperatorB == '~'))

return false;

else if (OperatorA == '~')

return true;

else if (OperatorB == '~')

return false;

else if ((OperatorA == '*' || OperatorA == '/')



```

return true;
else if ((OperatorB == '*' || (OperatorB == '/'))
return false;
else
return true;
}

```

/* Creates a copy of a tree passes the roots of the 2 trees, r1 being the root of the tree to be copied to and r2 being the root of the tree being copied. */

```

void copy(node_type * &r1, node_type * r2)
{
    if(r2 == NULL)
        r1 = NULL;
    else
    {
        if(r1 == NULL)
        {
            r1 = build_node(r2->data);
            copy(r1->left_child, r2->left_child);
            copy(r1->right_child, r2->right_child);
        }
        else
        {
            r1 = build_node(r2->data);
            copy(r1->left_child, r2->left_child);
            copy(r1->right_child, r2->right_child);
        }
    }
}

```

```
#endif
```

```
// FileName: treemain. cpp . By: Musawir Ali Shah
```

```
#include <iostream>
```

```
#include <string>
```

```
#include <stack>
```

```
#include "tree. h"
```

```
/* * * * * * Checks if expression is ok * * * * * */
```

```
bool isok(string exp)
```

```

{
    char check;
    int error=0;

```



```
int lb=0;
int rb=0;
for(int m=0;m < exp.size(); m++)
{
    check = exp[m];
    if(IsOperand(check))
    {

    else if(IsOperator(check))
    {
        if(check == ')')
        {
            rb++;
            if(IsOperator(exp[m+1]) && (exp[m+1] == '+' || exp[m+1] == '-' || exp[m+1] == '*'
            || exp[m+1] == '/' || exp[m+1] == '~ || exp[m+1] == '^'))
            {
                m++;
                if(exp[m] == ')')
                    rb++;
            }
            else if(IsOperator(exp[m+1]))
                error++;
        }
        else if(check == '(')
        {
            lb++;
            if(IsOperator(exp[m+1]) && exp[m+1] == '(')
            {
                m++;
                lb++;
            }
            else if(IsOperator(exp[m+1]))
                error++;
        }
    }
    else
    {
        if(IsOperator(exp[m+1]) && exp[m+1] == '(')
        {
            m++;
            lb++;
        }
        else if(IsOperator(exp[m+1]))
            error++;
    }
}
```



```

        }
    }
    else
        error++;
}

if(error == 0 && lb == rb)
    return(true);
else
    return(false);
}

/*****

void main()
{
    binary_tree etree;
    stack<binary_tree> NodeStack;
    stack<char> OpStack;
    string infix;
    char choice = 'y';
    char c;

    while(choice == 'y' || choice == 'Y')
    {
        cout << "\n\nIntroduce the Expression (no spaces): ";
        cin >> infix;
        cout << "-----" << endl;
        cout << "Expression:   " << infix << endl;

        if(isok(infix))
        {
            for(int i=0; i < infix.size(); i++)
            {
                c = infix[i];
                if(IsOperand(c)) //if operand, create tree and push into NodeStack
                {
                    string tempstring;
                    tempstring = tempstring + c;
                    if(i+1 < infix.size() && IsOperand(infix[i+1]))
                    {
                        while(i+1 < infix.size() && IsOperand(infix[i+1]))
                        {
                            tempstring = tempstring + infix[++i];

```

```

    }
}

binary_tree temp;
temp.root = build_node(tempstring);
NodeStack.push(temp);
}

//Else if Operator, check precedence and push into OpStack
else if(c == '+' || c == '-' || c == '*' || c == '/' || c == '~')
{
    if(OpStack.empty())
        OpStack.push(c);
    else if(OpStack.top() == '(')
        OpStack.push(c);
    else if(TakesPrecedence(c, OpStack.top()))
        OpStack.push(c);
    else
    {
        while(! OpStack.empty() && TakesPrecedence(OpStack.top(), c))
        {
            binary_tree temp_tree;
            string thisstring="";
            thisstring = thisstring + OpStack.top();
            OpStack.pop();
            etree.root = build_node(thisstring);
            copy(temp_tree.root, NodeStack.top().root);
            NodeStack.pop();
            etree.root->right_child = temp_tree.root;
            temp_tree.root = NULL;
            copy(temp_tree.root, NodeStack.top().root);
            etree.root->left_child = temp_tree.root;
            NodeStack.pop();
            temp_tree.root = NULL;
            copy(temp_tree.root, etree.root);
            NodeStack.push(temp_tree);
            etree.root = NULL;
        }
        OpStack.push(c);
    }
}

else if(c == '(')
    OpStack.push(c);
else if(c == ')')
{

```

```

    while(OpStack.top() != '(')
    {
        binary_tree temp_tree;
        string thisstring="";
        thisstring = thisstring + OpStack.top();
        OpStack.pop();
        etree.root = build_node(thisstring);
        copy(temp_tree.root, NodeStack.top().root);
        NodeStack.pop();
        etree.root->right_child = temp_tree.root;
        temp_tree.root = NULL;
        copy(temp_tree.root, NodeStack.top().root);
        etree.root->left_child = temp_tree.root;
        NodeStack.pop();
        temp_tree.root = NULL;
        copy(temp_tree.root, etree.root);
        NodeStack.push(temp_tree);
        etree.root = NULL;
    }
    OpStack.pop();
}
} // end of for loop
while(! OpStack.empty()) //While OpStack not empty, pop that stack
    //and create tree
{
    binary_tree temp_tree;
    string thisstring="";
    thisstring = thisstring + OpStack.top();
    OpStack.pop();
    etree.root = build_node(thisstring);
    copy(temp_tree.root, NodeStack.top().root);
    NodeStack.pop();
    etree.root->right_child = temp_tree.root;
    temp_tree.root = NULL;
    copy(temp_tree.root, NodeStack.top().root);
    etree.root->left_child = temp_tree.root;
    NodeStack.pop();
    temp_tree.root = NULL;
    copy(temp_tree.root, etree.root);
    NodeStack.push(temp_tree);
    if(! OpStack.empty())
    {
        etree.root = NULL;
    }
}

```

```

    }

    cout << "Postfix traversal: ";
    etree.print();
    cout << endl;
    etree.evaluate();
    cout << "Result: " << etree.root->data << endl;
    cout << "-----" << endl;
    cout << "\n\nRun Program again? Enter <y/n> : ";
    cin >> choice;
}
else
{
    cout << " * * * * * " << endl;
    cout << "ERROR: Invalid Expression" << endl;
    cout << " * * * * * " << endl;
    cout << "\n\nRun Program again? Enter <y/n> : ";
    cin >> choice;
}
}

} //end of main

```

第4题 二叉树操作

8.4.1 程序功能简介

一个基本的二叉树程序。实现二叉树节点数据的插入、删除、查找和打印输出等功能。

8.4.2 课程设计要求

- (1) 设计对二叉树上节点中的数据进行由大到小排序的函数。
- (2) 设计对二叉树上节点中的数据进行由小到大排序的函数。
- (3) 选做:设计计算二叉树中节点上数据的平均数值的函数。
- (4) 用类来实现封装。

8.4.3 课程设计说明

(1) 将要操作的数据按以下格式输入到文档中: 1 表示插入数据, 2 表示删除数据, 3 表示查找数据, 4 表示找前续数据, 5 表示找后继数据, 6 表示最小值, 7 表示最大值, 8 表示输出二叉树, 9 表示输出平均值, 10 表示顺序输出二叉树。

(2) 评定难易级别: A 级。

8.4.4 关键源代码注解

```
// By Dror Helper. last updated 23. 1. 2002
//Rbtree.h Begin
#ifndef _RBTREE_H
#define _RBTREE_H
#include<iostream.h>
#include<stdlib.h>
template<class T>
class RBtree
{
public:
    enum Color{Red,Black};
private:
    class RBnode
    {
        T value;
        Color color;
        RBnode * p;
        RBnode * left;
        RBnode * right;
        RBnode():value(0),color(Red),p(NULL),left(NULL),right(NULL){};
        RBnode(const T &val):value(val),color(Red),p(NULL),left(NULL),right(NULL){};
    };
};
```

```

friend class RBtree<T>;

public:
    void Print(int level)
    {
        for(int i=0;i<level;++i)           // puts spaces
            cout<<" ";
        cout<<"| "<<value;                // formatted print — |value(color)
        if (color==Black)
            cout<<"(B)";
        else
            cout<<"(R)";
        return;
    }
};

RBnode * ROOT;
RBnode * NIL;
int alive;           // for debugging purposes only — not really needed

public:
    RBtree():ROOT(new RBnode()),NIL(ROOT),alive(0){NIL->left=NIL->right=NIL->p=
    NIL;NIL->color=Black;}
    RBtree(const T &val):ROOT(new RBnode(val)),NIL(new RBnode()),alive(1)
    {
        NIL->color=Black;
        NIL->left=NIL->right=NIL->p=NIL;
        ROOT->left=ROOT->right=ROOT->p=NIL;
    }
    ~RBtree(){if(ROOT!=NIL) DeleteAll(ROOT);delete NIL;}
    void Insert(const T&);           // Inserting a new value to tree
    void Delete(const T&);           // Deleteing an onl value form tree
    RBnode * RBFind(const T &val)    // finding value in tree if found
    // returns node otherwise return NIL
    {
        RBnode * tmp=ROOT;
        while((tmp!=NIL)&&(tmp->value!=val))
            tmp=tmp->val<tmp->value ? tmp->left : tmp->right;
        return tmp;
    }
    bool Find(const T &val)          // the same as above but returns true if found false if not
    {
        RBnode * tmp=RBFind(val);
        return (tmp!=NIL);
    }
    RBnode * RBMax(RBnode * start){RBnode * tmp=start;while(tmp->right!=NIL)tmp=tmp->

```

```

right;return tmp;}

//the max node in the tree
RBnode * RBMin(RBnode * start){RBnode * tmp=start;while(tmp->left!=NIL)tmp=tmp->
left;return tmp;}

// the min node in the tree
RBnode * RBPred(RBnode * start)
{
    RBnode * tmp=start;
    if (tmp->left!=NIL)
        tmp=RBMax(tmp->left);
    else
    {
        RBnode * ptmp=tmp;
        tmp=tmp->p;
        while((tmp!=NIL)&&(ptmp!=tmp->right))
        {
            ptmp=tmp;
            tmp=tmp->p;
        }
    }
    return tmp;
}

RBnode * RBSucc(RBnode * start)
{
    RBnode * tmp=start;
    if (tmp->right!=NIL)
        tmp=RBMin(tmp->right);
    else
    {
        RBnode * ptmp=tmp;
        tmp=tmp->p;
        while((tmp!=NIL)&&(ptmp!=tmp->left))
        {
            ptmp=tmp;
            tmp=tmp->p;
        }
    }
    return tmp;
}

void Print();                // prints a RBtree
void RotateLeft(RBnode * );  // used in inset & delete functions
void RotateRight(RBnode * );
int NodeInsert(RBnode * );   // puts a node in the tree

```



```

void FixDel(RBnode * );           // fixing the Delete proc
T Succ(const T &val)             // printing the successor of certain value
{
    RBnode * tmp=RBFind(val);
    if (tmp==NIL)
    {
        cout<<"value not found";
        return 0;
    }
    else
        tmp=RBSucc(tmp);
    return tmp->value;
}

T Pred(const T &val)             // the same for predecessor
{
    RBnode * tmp=RBFind(val);
    if (tmp==NIL)
    {
        cout<<"value not found";
        return 0;
    }
    else
        tmp=RPred(tmp);
    return tmp->value;
}

T Max(const T &val){RBnode * tmp=RBMax(ROOT);return tmp->value;} //prints max
T Min(const T &val){RBnode * tmp=RBMin(ROOT);return tmp->value;} //prints min
void RBPrint(RBnode * ,int);      // req func for printing RBtree
void DeleteAll(RBnode * );       // deletes all of the RBtree (except the NIL)
};

template<class T>
void RBtree<T>::Insert(const T &val){
    RBnode * x=new RBnode(val), * y;
    if(NodeInsert(x)==1)
    {
        cout<<val<<" already exist in tree"<<endl;
        return;
    }
    while ((x!=ROOT)&&(x->p->color==Red))
    {
        if (x->p==x->p->p->left)           // if x parent is left son

```



```

{
    y=x->p->p->right;           // checking x uncle
    if (y->color==Red)           // if red case 1
    {
        x->p->color=Black;
        y->color=Black;
        x->p->p->color=Red;
        x=x->p->p;
    }
    else
    {
        if (x==x->p->right)       //if x right the case 2
        {
            x=x->p;
            RotateLeft(x);
        }
        x->p->color=Black;         // case 3
        x->p->p->color=Red;
        RotateRight(x->p->p);
    }
}
else
{
    y=x->p->p->left;               // checking x uncle
    if (y->color==Red)           // if red case 4
    {
        x->p->color=Black;
        y->color=Black;
        x->p->p->color=Red;
        x=x->p->p;
    }
    else
    {
        if (x==x->p->left)        //if x right the case 5
        {
            x=x->p;
            RotateRight(x);
        }
        x->p->color=Black;         // case 6
        x->p->p->color=Red;
        RotateLeft(x->p->p);
    }
}
}

```

```

    }
    ROOT->color=Black;
    return;
}

template<class T>
void RBtree<T>::Delete(const T &val)
{
    RBnode *z, *y, *x;
    z=RBFind(val);
    if(z==NIL)
    {
        cout<<val<<" not found"<<endl;
        return;
    }
    --alive;
    if ((z->left == NIL) || (z->right==NIL))    // deleting from leaves
        y=z;
    else
        y=RBSucc(z);
    if (y->left!=NIL)
        x=y->left;
    else
        x=y->right;
        x->p=y->p;
    if (y->p==NIL)
    {
        ROOT=x;
        x->p=NIL;
    }
    else
    {
        if (y==y->p->left)
            y->p->left=x;
        else
            y->p->right=x;
    }
    if (y!=z)    // if we changed the successor with z
        z->value=y->value;
    if (y->color==Black)
        FixDel(x);    // if we deleted a Black node we have to rearrange the tree
    delete y;
    if (x==NIL)
        x->p=NIL;
}

```

```

        return;
    }

template<class T>
void RBtree<T>::RotateLeft(RBnode * x)
{
    RBnode * y=x->right;
    x->right=y->left;
    if (y->left!=NIL)
        y->left->p=x;
    y->p=x->p;
    if (x->p==NIL)
        ROOT=y;
    else if (x==x->p->left)
        x->p->left=y;
    else
        x->p->right=y;
    y->left=x;
    x->p=y;
    return;
}

template<class T>
void RBtree<T>::RotateRight(RBnode * x)
{
    RBnode * y=x->left;
    x->left=y->right;
    if (y->right!=NIL)
        y->right->p=x;
    y->p=x->p;
    if (x->p==NIL)
        ROOT=y;
    else if (x==x->p->right)
        x->p->right=y;
    else
        x->p->left=y;
    y->right=x;
    x->p=y;
    return;
}

template<class T>
int RBtree<T>::NodeInsert(RBnode * x)
{
    RBnode * tmp1=ROOT, * tmp2=tmp1;
    if (ROOT==NIL)

```

```

    {
        // in case that x is the 1st node
        ROOT = x;
        x->left = x->right = x->p = NIL;
        ++alive;
        return 0;
    }
    while((tmp1 != NIL) && (tmp1->value != x->value))
    {
        tmp2 = tmp1;
        tmp1 = x->value > tmp1->value ? tmp1->right : tmp1->left;
    }
    if (tmp1 == NIL) {
        // if the item not in tree - add
        if (x->value > tmp2->value)
            tmp2->right = x;
        else
            tmp2->left = x;
        x->p = tmp2;
        x->left = x->right = NIL;
        ++alive;
        return 0;
    }
    return 1;
}

template<class T>
void RBtree<T>::FixDel(RBnode * x)
{
    RBnode * w;
    while ((x != ROOT) && (x->color == Black))
    {
        if (x == x->p->left) {
            // if x is the right son
            w = x->p->right;
            if (w->color == Red) {
                // case 1
                w->color = Black;
                x->p->color = Red;
                RotateLeft(x->p);
                w = x->p->right;
            }
            if ((w->left->color == Black) && (w->right->color == Black))
            {
                w->color = Red;
                // case 2
                x = x->p;
            }
        } else {
            if (w->right->color == Black)

```

```

    {
        w->left->color=Black;           // case 3
        w->color=Red;
        RotateRight(w);
        w=x->p->right;
    }
    w->color=x->p->color;                 // case 4
    x->p->color=Black;
    w->right->color=Black;
    RotateLeft(x->p);
    x=ROOT;
}
else
{
    // if x is the left son
    w=x->p->left;
    if (w->color==Red)                  // case 5
    {
        w->color=Black;
        x->p->color=Red;
        RotateRight(x->p);
        w=x->p->left;
    }
    if ((w->right->color==Black)&&(w->left->color==Black))
    {
        w->color=Red;                  // case 6
        x=x->p;
    }
    else
    {
        if (w->left->color==Black)      // case 7
        {
            w->right->color=Black;
            w->color=Red;
            RotateLeft(w);
            w=x->p->left;
        }
        w->color=x->p->color;            // case 8
        x->p->color=Black;
        w->left->color=Black;
        RotateRight(x->p);
        x=ROOT;
    }
}

```

```

    }
    }
    x->color=Black;
    return;
}

template<class T>
void RBtree<T>::Print()
{
    int depth=-1;
    RBnode * x=ROOT;
    RBPrint(x,depth);
    return;
}

// the printing will be done so that the root will be on the left
// and the tree will expend to the right
template<class T>
void RBtree<T>::RBPrint(RBnode * x,int level)
{
    if (x==NIL) return;
    ++level;
    RBPrint(x->right,level);
    x->Print(level);
    cout<<"---|"<<endl;
    RBPrint(x->left,level);
    return;
}

template<class T>
void RBtree<T>::DeleteAll(RBnode * x)
{
    if(x==NIL) return;
    DeleteAll(x->left);
    DeleteAll(x->right);
    delete x;
};

#endif

//Rbtree. h end

#include<iostream. h>           // Dror Helper I. D. 034354712
#include<fstream. h>           // sample main for RB tree
#include"RBtree. h"

int main()
{
    ifstream data("data4. txt", ios::in|ios::nocreate);

```

```

if(data.bad())
{
    cout<<endl<<"file not found"<<endl;
    return 1;
}

int choice,num;
RBtree<int> tr;
data>>choice;
while(! data.eof())
{
    if(choice<6)          // the func 6 to 8 don't need arguments
        data>>num;
    switch(choice)
    {
        case(1):cout<<"Insert - "<<num<<endl;
            tr.Insert(num);
            break;
        case(2):cout<<"Delete - "<<num<<endl;
            tr.Delete(num);
            break;
        case(3):cout<<"Find - "<<num<<endl;
            if(tr.Find(num))
                cout<<num<<" exist in tree"<<endl;
            else
                cout<<num<<" doesn't exist in tree"<<endl;
            break;
        case(4):cout<<"the successor of "<<num<<" is:"<<endl;
            cout<<tr.Succ(num);
            cout<<endl;
            break;
        case(5):cout<<"the predecessor of "<<num<<" is:"<<endl;
            cout<<tr.Pred(num);
            cout<<endl;
            break;
        case(6):cout<<"the min value of the tree is: ";
            cout<<tr.Min(num);
            cout<<endl;
            break;
        case(7):cout<<"the max value of the tree is: ";
            cout<<tr.Max(num);
            cout<<endl;
            break;
        case(8):cout<<"printing tree"<<endl;
    }
}

```



```
        tr. Print();  
        break;  
default: cout<<endl<<"the choice number is out of range"<<endl;  
        break;  
}  
data>>choice;  
}  
return 0;
```

第5题 模板实现排行榜管理

8.5.1 程序功能简介

竞赛排行榜管理程序。该程序能够管理各类比赛的成绩,将成绩录入,或将弃权者的记录删去,并能根据不同比赛的要求,按由低到高或由高到低来输出排行榜。可以用人名或成绩来查找信息。

8.5.2 课程设计要求

(1) 用模板类构建一棵二叉树,表示游戏挑战大赛积分排行榜。当出现一挑战者,其成绩能加到数据库中;也可将弃权者的记录从数据库中删除。

(2) 能显示分数的排行榜。

(3) 统计平均得分。

(4) 用分数查找人名。

(5) 用人名查找分数。

8.5.3 课程设计说明

(1) 排行榜的所有功能包含在模板类中(TREESTRUCT)。

(2) 在主函数中定义一个模板类对象,并测试所有功能。

(3) 评定难易级别:A级。

8.5.4 关键源代码注解

```
//Coded by Joe Konney II
//Templated binary tree class
#include <iostream.h>
#include <string.h>
template< class T>
struct TreeNode
{
    TreeNode * right;
    TreeNode * left;
    T data;
};
template< class T >
class TREESTRUCT
{
public:
    TREESTRUCT() { root = NULL;} //initialize root to NULL
    ~TREESTRUCT()
```

```

        { makeDeleteSubTree(root); root = NULL;} //delete all nodes from memory
void removeSubTree(T); //removes the tree/subtree
void deleteSubTree(T, TreeNode<T> * &);
    //helper function to removeSubTree(), searches for the tree/subtree
void makeDeleteSubTree(TreeNode<T> * &); //helper function to removeSubTree(),
    //destroy the tree/subtree given the root pointer of a tree/subtree
void insertNode(T); //inserts a new node given a value
void remove(T); //remove a node given its value
void deleteNode(T, TreeNode<T> * &); //helper function to remove(), finds pointer
void makeDeletion(TreeNode<T> * &); //helper function to remove(),
    //deletes node, repoints pointers
void showNode(TreeNode<T> * ); //shows a node given a pointer to it
void showNodesInOrder() //shows entire tree in order
    { showNodesInOrder(root); }
void showNodesInOrder(TreeNode<T> * ); //shows tree/subtree in order
//given pointer to the root of a tree/subtree
void showNodesPreOrder() //shows entire tree in preorder
    { showNodesPreOrder(root); }
void showNodesPreOrder(TreeNode<T> * ); //shows tree/subtree in preorder
//given pointer to the root of a tree/subtree
void showNodesPostOrder() //shows entire tree in postorder
    { showNodesPostOrder(root); }
void showNodesPostOrder(TreeNode<T> * ); //shows tree/subtree in postorder
//given pointer to the root of a tree/subtree
void showRoot() //shows root of the tree
    { cout<<"ROOT: "<<root->data<<endl; }
TreeNode<T> * searchNodes(T);
//searches for a value in the tree, returns its pointer location
TreeNode<T> * getRoot(); //returns the pointer to the root of the tree
T getRootData();
private:
    TreeNode<T> * root; //pointer to the root of the tree
};
template< class T>
T TREESTRUCT<T>::getRootData()
{
    return root->data;
}
template< class T>
void TREESTRUCT<T>::remove(T data)
{
    deleteNode(data, root);
}

```

```

template< class T>
void TREESTRUCT<T>::deleteNode(T num, TreeNode<T> * &nodePtr)
{
    if(num < nodePtr->data)
        deleteNode(num, nodePtr->left);
    else if(num > nodePtr->data)
        deleteNode(num, nodePtr->right);
    else
        makeDeletion(nodePtr);
}

template< class T>
void TREESTRUCT<T>::makeDeletion(TreeNode<T> * &nodePtr)
{
    TreeNode<T> * tempNodePtr;
    cout << "\t\tDELETING " << nodePtr->data << endl;
    if(nodePtr == NULL)
        cout << "Cannot delete empty node!" << endl;
    else if(nodePtr->right == NULL)
    {
        tempNodePtr = nodePtr;
        nodePtr = nodePtr->left;
        delete tempNodePtr;
    }
    else if(nodePtr->left == NULL)
    {
        tempNodePtr = nodePtr;
        nodePtr = nodePtr->right;
        delete tempNodePtr;
    }
    else
    {
        tempNodePtr = nodePtr->right;
        while(tempNodePtr->left)
            tempNodePtr = tempNodePtr->left;
        tempNodePtr->left = nodePtr->left;
        tempNodePtr = nodePtr;
        nodePtr = nodePtr->right;
        delete tempNodePtr;
    }
}

template< class T>
TreeNode<T> * TREESTRUCT<T>::searchNodes(T search)

```

```

{
    TreeNode<T> * tRoot = root;
    if(tRoot == NULL)
        return 0;
    if(tRoot->data == search)
        return tRoot;
    while(tRoot != NULL)
    {
        if(search < tRoot->data)
            tRoot = tRoot->left;
        else if(search > tRoot->data)
            tRoot = tRoot->right;
        else if(search == tRoot->data)
            return tRoot;
        else
            return 0;
    }
    return 0;
}

template< class T>
void TREESTRUCT<T>::showNode(TreeNode<T> * tNode)
{
    if(tNode)
        cout << tNode->data << endl;
    else
        cout << "Query is not in binary tree." << endl;
}

template< class T>
void TREESTRUCT<T>::removeSubTree(T data)
{
    deleteSubTree(data, root);
}

template< class T>
void TREESTRUCT<T>::deleteSubTree(T data, TreeNode<T> * &nodePtr)
{
    if(data < nodePtr->data)
        deleteSubTree(data, nodePtr->left);
    else if(data > nodePtr->data)
        deleteSubTree(data, nodePtr->right);
    else

```

```
{
    makeDeleteSubTree(nodePtr->left);
    makeDeleteSubTree(nodePtr->right);
    nodePtr = NULL;
}

template< class T>
void TREESTRUCT<T>::makeDeleteSubTree(TreeNode<T> * &tRoot)
{
    if(tRoot == NULL) return;
    makeDeleteSubTree(tRoot->left);
    makeDeleteSubTree(tRoot->right);
    cout << "\t\tDELETING SUBTREE NODE " << tRoot->data << endl;
    delete tRoot;
}

template< class T>
void TREESTRUCT<T>::showNodesInOrder(TreeNode<T> * tRoot)
{
    if(tRoot == NULL)
        return;
    showNodesInOrder(tRoot->left);
    cout << tRoot->data << endl;
    showNodesInOrder(tRoot->right);
}

template< class T>
void TREESTRUCT<T>::showNodesPreOrder(TreeNode<T> * tRoot)
{
    if(tRoot == NULL) return;
    cout << tRoot->data << endl;
    showNodesInOrder(tRoot->left);
    showNodesInOrder(tRoot->right);
}

template< class T>
void TREESTRUCT<T>::showNodesPostOrder(TreeNode<T> * tRoot)
{
    if(tRoot == NULL) return;
    showNodesInOrder(tRoot->left);
    showNodesInOrder(tRoot->right);
    cout << tRoot->data << endl;
}
```

```
template< class T>
TreeNode<T> * TREESTRUCT<T>::getRoot()
{
    return root;
}

template< class T >
void TREESTRUCT<T>::insertNode(T data)
{
    TreeNode<T> * newNode = new TreeNode<T>;
    TreeNode<T> * nodePtr;
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    if(! root)
        root = newNode;
    else
    {
        nodePtr = root;
        while(nodePtr != NULL)
        {
            if(newNode->data < nodePtr->data)
            {
                if(nodePtr->left)
                    nodePtr = nodePtr->left;
                else
                {
                    nodePtr->left = newNode;
                    break;
                }
            }
            else if(newNode->data > nodePtr->data)
            {
                if(nodePtr->right)
                    nodePtr = nodePtr->right;
                else
                {
                    nodePtr->right = newNode;
                    break;
                }
            }
            else
            {

```

```

    cout << "Duplicate value(" << data << ") found in tree";
    break;
}
}
}
}

```

//Visual representation of binary tree when all nodes have been inserted:

```

//          5
//        /  \
//       /    \
//      /      \
//     -5        11
//      \        /
//       0        9
//        \      /
//         1      7
//          \    / \
//           4   6  8
//            /
//           3

```

```
void main()
```

```

{
    TREESTRUCT<int> mytree;
    mytree.insertNode(5);
    mytree.insertNode(-5);
    mytree.insertNode(0);
    mytree.insertNode(1);
    mytree.insertNode(4);
    mytree.insertNode(3);
    mytree.insertNode(-1);
    mytree.insertNode(11);
    mytree.insertNode(9);
    mytree.insertNode(7);
    mytree.insertNode(8);
    mytree.insertNode(6);
    mytree.showRoot();
    cout << "SHOW: " << endl;
    mytree.showNodesInOrder();
    mytree.remove(4);
    mytree.showRoot();
    mytree.removeSubTree(7);
}

```



```
cout << "SHOW: " << endl;
mytree.showNodesInOrder();
mytree.remove(5);
cout << "SHOW: " << endl;
mytree.showNodesInOrder();
mytree.showRoot();
cout << "SUCCESS! GOING TO DECONSTRUCTOR!" << endl;
```

```
}
```

第 6 题 模板实现电话簿管理

8.6.1 程序功能简介

模板实现电话簿管理。

8.6.2 课程设计要求

- (1) 设计由文件建立链表的函数。
- (2) 设计由链表建立文件的函数。
- (3) 在主程序中设计一个电话簿管理系统。
- (4) 将链表改为双向链表节点结构。

8.6.3 课程设计说明

- (1) 改为双向链表时,要注意指针的变化。
- (2) 系统读数据时在一行结束自动加上“\0”,若两次读入的数据放在同一行,下次调用 getline() 函数时只能读到“\0”前的数据,后面的数据无法读入。
- (3) 当找到若干节点时,若要全部删除每次调用 Remove() 时,一定要注意每删除一个节点后,其他节点的位置会改变。
- (4) 评定难易级别:A 级。

8.6.4 关键源代码注解

```
#ifndef LINKEDLIST
#define LINKEDLIST
#include <fstream.h>
#include <assert.h>

template <class T> //模版,使用方法见 VC++ 手册
class LList
{
protected:
    struct Node
    {
        T Data; //自定义的节点数据类型
        Node * Next;
        Node()
        {
            Next = 0;
        }
        Node(T element, Node * ptr = 0)
        {
```

```

        Data = element;
        Next = ptr;
    }
    //friend ostream& operator<<(ostream&s, Node n){ s<< n. Data; return s;}
} * First, * Last;    //是指向节点(Node 类型)的指针

```

```

int NumberNodes;    //链表的节点计数器

```

```

public:

```

```

    LList(){ First=0; Last=0; NumberNodes=0;}

```

```

    ~LList();

```

```

    //copy operation

```

```

    LList(const LList& L);    // 复制链表

```

```

    int Length();

```

```

    //subscripting

```

```

    T& operator[](int i);    // 返回第 i 个节点的内容

```

```

    void Print();

```

```

    const T& operator[](int i) const;

```

```

    //T& operator[](int i) ;

```

```

    //Insertion

```

```

    void Insert(const T& el);

```

```

    //Append

```

```

    void Append(const LList<T>& L);

```

```

    //Removal

```

```

    void Remove(int location);

```

```

    //Search

```

```

    int Search(const T& value) const;

```

```

    //output

```

```

    friend ostream& operator<<(int s, const LList<T>& L) ;

```

```

};

```

```

template <class T>

```

```

LList<T>::~~LList()

```

```

{

```

```

    Node * p=First;

```

```

    while( p != 0)

```

```

    {

```

```

        First=p->Next;

```

```

        delete p;

```

```

        p=First;

```

```

    }

```

```

    NumberNodes=0;

```

```

    Last=0;

```

```

}

```

```

template <class T>
int LList<T>::Length() { return  NumberNodes;}

template <class T>
LList<T>::LList(const LList& L)
{
    Node * Old=L. First, * New=0;
    NumberNodes=L. NumberNodes;
    if(NumberNodes==0) First=0;
    else
    {
        New= new Node(Old->Data);
        assert(New !=0);           //判断 New 指针是否不空? assert()在 assert. h 中
        Old=Old->Next;
        First=New;
    }
    while(Old !=0)
    {
        New->Next= new Node(Old->Data);
        assert(New->Next !=0);
        New= New->Next;
        Old= Old->Next;
    }
    Last=New;
}

template <class T>
ostream& operator<<(ostream & s,const LList<T>& L)
{
    int i,M=L. Length();
    for(i=0;i<M-1;i++)
    {
        s << L[i] << ", ";
    }
    s << L[M-1] ;
    return s;
}

template <class T>
void LList<T>::Print()
{
    Node * p=First;

```

```

        while (p != Last)
        {
            cout << p->Data << " ";
            p = p->Next;
        }
        cout << p->Data << endl;
    }

    template <class T>
    T& LList<T>::operator[](int i)
    {
        assert(i >= 0);
        assert(i < NumberNodes);
        Node * p = First;
        for(int k=0; k<i; k++)
        {
            p = p->Next;
        }
        return p->Data;
    }

    template <class T>
    const T& LList<T>::operator[](int i) const
    {
        assert(i >= 0);
        assert(i < NumberNodes);
        Node * p = First;
        for(int k=0; k<i; k++)
        {
            p = p->Next;
        }
        return p->Data;
    }

    template <class T>
    void LList<T>::Insert(const T& el)
    {
        Node * newnode = new Node(el);
        assert(newnode != 0);
        newnode->Next = First;
        First = newnode;
        if(NumberNodes == 0) Last = newnode;
        NumberNodes++;
    }

    template <class T>

```

```

void LList<T>::Remove(int location)
{
    assert(location >= 0);
    assert(location < NumberNodes);
    Node * p;
    if(location == 0)
    {
        p = First;
        First = p->Next;
        p->Next = 0;
    }
    else
    {
        Node * pp = First;
        for(int i = 0; i < location - 1; i++) pp = pp->Next;
        p = pp->Next;
        pp->Next = p->Next;
        p->Next = 0;
        if(location == NumberNodes - 1) Last = pp;
    }
    delete p;
    NumberNodes--;
}

template<class T>
int LList<T>::Search(const T& value) const
{
    Node * p = First;
    for(int i = 0; i < NumberNodes; i++)
    {
        if(p->Data == value) return i;
        else p = p->Next;
    }
    return -1;
}

template<class T>
void LList<T>::Append(const LList<T>& L)
{
    Node * p = L.First;
    while(p != 0)
    {
        Insert(p->Data);
        p = p->Next;
    }
}

```

```

    }
}

#endif

/* This stack is implemented on top of LList above. */

#ifndef STACK
#define STACK
template <class T>
class Stack: private LList<T>      //定义栈类
{
public:
    Stack(); LList<T>(){}
    Stack(const Stack<T>& S): LList<T>(S){}
    int Empty() const{ return (Length()==0); }      //清空栈操作成员作函数说明
    void Push(T& el){ Insert(el); }
    T Pop()
    {
        assert(Length(>0);      //判断栈(即链表)是否不空
        T el = First->Data;
        Remove(0);
        return el;
    }
    Print(){ LList<T>::Print(); }
};

#endif

main()
{
    LList<int> L,M;      //定义两个链表 L,T;数据成员为整型变量
    L.Insert(2);
    L.Insert(3);
    for(int i=0;i<10;i++) L.Insert(i);
    L.Print();
    cout << endl;
    L.Remove(0);
    L.Print();
    cout << endl;
    L.Remove(1);      //删除 L 链表的第二个点
    L.Print();
}

```

附录 1 变量命名应遵守的原则

建议变量命名遵守表 1 和表 2 的前缀约定。

表 1 常用的变量前缀

前缀	含义	前缀	含义
a	数组	l	长整型
ch	字符	lp	长指针
cb	字节计数	n	整型
dw	无符号长型	np	近指针
h	16 位的句柄	pt	点结构(x,y)
hdc	设备描述表的句柄	r	矩形结构
hwnd	窗口的句柄	sz	以空字符为结束符的字符串
i	索引	w	无符号整型

表 2 常用的变量前缀

前缀	含义	前缀	含义
fl	float	np	整型指针
dl	double	chp	字符型指针
str	结构体	flp	float 指针
strp	结构体指针	dlp	double 指针
na	整型数组	fpinfile	输入文件指针
sz	字符型数组	fpoutfile	输出文件指针

附录2 VC++编程应遵守的原则

1. 编程命名原则

关于宏、常量、变量和函数名等的前缀建议参考附录1。命名要本着清楚、简单的原则,名字尽可能精确地表达其内容。避免用模棱两可、晦涩或不标准的缩写,避免使用会引起误解的词汇。变量名、类和对象名应是名词,实现行为的类成员函数名应是动词。

2. 使用宏的原则

尽量用常量替代实现类似功能的宏,尽量用 typedef 代替宏定义新类型,尽量用 inline 函数代替实现类似功能的宏。若宏值多于一项,一定要使用括号,宏的每个参数都要括起来。不要在公共头文件中定义宏,不要用宏改写语言。

3. 使用类型的原则

慎用无符号类型、浮点数类型、union 类型。慎用位操作,用 enum 取代(一组相关的)常量,(尽量)用引用取代指针。

4. 使用函数的原则

一定要做到先定义后使用。重复使用的代码可用函数代替。函数原型声明要放在一个头文件中,函数无参数一定要用 void 标注,函数的基本类型参数应传值,函数的非基本类型参数应先考虑传递引用,后考虑传递指针。避免使用参数不确定的函数,避免函数的参数过多。定义清楚返回类型,尽量保证函数只有惟一 return 语句。返回指针类型的函数应该用 NULL 表示失败。

5. 使用条件编译的原则

若使用 #if 或 #ifdef,不要遗漏 #else,对复杂的编译条件用括号使其清晰,条件编译和普通条件语句不要混合使用。

6. 使用类的原则

类应是描述一组对象的集合,避免为每个类成员提供访问函数。不要在类定义时,实现类中的函数体部分。努力使类中的函数尽可能少。类中的函数声明时定义参数的缺省值。

7. 使用继承的原则

限制继承的层数,慎用多重继承。基类析构函数首选是虚函数。不要重新定义继承来的非虚函数,不要重新定义缺省参数值,不要将基类强制转换成派生类。

8. 使用异常处理的原则

确保代码在异常出现时能正确处理。抛出的异常最好是一个对象,捕捉异常时用引用。

9. 使用代码格式的原则

水平缩进每次用两个空格。在引用操作符前后加空格,在单目操作符和其操作对象间不要加空格,在“::”前后不要加空格,函数名和左括号之间不要加空格。

函数间要用空行分开。一条语句占一行,每一行不超过 78 个字符。所有 switch 语句要用 default 分支。变量定义集中放置在一起,每个变量占一行,按字母顺序排列。

10. 使用注释的原则

注释要简单、清楚、切中要害。注释用词要精确,不能有二义性,注释中的术语要通用。注释清楚基本类型变量及扩充类型的变量;注释清楚每个引用的头文件。

注释清楚函数要实现的功能,注释清楚函数实现的方法(或称实现算法),注释清楚函数体中的每个变量,每行程序。

11. 兼容性原则

遵守 ANSI C 和 ISO C++ 国际标准。确保类型转换不会丢失信息,注意运算溢出问题。注意引用公共库的兼容性,一定不要重新实现标准库函数。将所有 #include 的文件名视为大小写有区别的。确保 main() 函数总是返回一个整型。

12. 性能原则

不要用移位代替乘除运算。避免在循环体内部定义对象,尽量减少对象拷贝、临时对象、对象数组。

使用前缀 ++ 和 -- 比使用后缀 ++ 和 -- 效率高,使用非递归方法比使用递归方法效率高,使用 inline 函数比使用普通函数效率高。

参 考 文 献

主要书籍:

- 1 陈世忠编著. C++编程规范. 北京:人民邮电出版社,2002
- 2 董士海主编. 图像格式编程指南. 北京:清华大学出版社,1994
- 3 陆润民. C语言绘图教程. 北京:清华大学出版社,1996
- 4 谭浩强. C程序设计. 北京:清华大学出版社,2000
- 5 王强如,李兰友编. C语言绘图与计算机仿真技术. 北京:北京航空航天大学出版社,1996
- 6 张岳新. Visual C++程序设计基础. 苏州:苏州大学出版社,2000

主要网站:

<http://www.yahoo.com/>
<http://www.webwareindex.com/>
<http://cplus.about.com/cs/>
<http://www.digitalmars.com/>
<http://www.cuj.com/>
<http://www.thefreecountry.com/>
<http://msdn.microsoft.com/>
<http://www.javacode.net/>
<http://www.pscore.com/>
<http://directory.google.com/>

Images have been losslessly embedded. Information about the original file can be found in PDF attachments. Some stats (more in the PDF attachments):

```
{
  "filename": "MTI5ODI4NjQuemlw",
  "filename_decoded": "12982864.zip",
  "filesize": 16833734,
  "md5": "4901d3465f732981f480ae6bc083b4a3",
  "header_md5": "6f82d84a9f82c0793d8e7ec5193287b0",
  "sha1": "f57a111cb3396d6d3b25cfabf1b573c4eb9dcc34",
  "sha256": "b4fd6b0932146257b46ee6873026bf5f767f9c691462635e57f5d64c47e48a6b",
  "crc32": 2220090536,
  "zip_password": "",
  "uncompressed_size": 17918480,
  "pdg_dir_name": "",
  "pdg_main_pages_found": 332,
  "pdg_main_pages_max": 332,
  "total_pages": 342,
  "total_pixels": 2177666240,
  "pdf_generation_missing_pages": false
}
```